

NatureDSP Baseband

**Digital Signal Processing
for BBE32EP cores with VFPU**

Library Reference

API Revision 1.54
June, 2017

IntegrIT, Limited
<http://www.integrit.com/>
support@integrit.com
Tel: +7 495 545 46 42

IMPORTANT NOTICE

Copyright (c) 2017 by Cadence Design Systems, Inc.

These coded instructions, statements, and computer programs (“Cadence Libraries”) are the copyrighted works of Cadence Design Systems Inc. Cadence IP is licensed for use with Cadence processor cores only and must not be used for any other processors and platforms. Your use of the Cadence Libraries is subject to the terms of the license agreement you have entered into with Cadence Design Systems, or a sublicense granted to you by a direct Cadence licensee.

Copyright (C) 2009-2017 IntegrIT, Limited.

This library contains copyrighted materials, trade secrets and other proprietary information of IntegrIT, Ltd. This software is licensed for use with Cadence processor cores only and must not be used for any other processors and platforms. The license to use these sources was given to Cadence, Inc. under Terms and Condition of a Software License Agreement between Cadence, Inc. and IntegrIT, Ltd.

ALL RIGHTS RESERVED

Table of Contents

Preface	7
About this Manual	7
About this release	7
Supported Targets	7
Notations	7
Abbreviations	7
1 General Library Organization	9
1.1 Headers	9
1.2 Static Variables and Usage of C Standard Libraries	9
1.3 Types	9
1.4 Fractional Formats	10
1.5 Compiler Requirements	10
1.6 Call Conventions	10
1.7 Overflow Control	11
1.8 Exceptions and Processor Control Registers	11
1.9 Special Numbers	11
1.10 Endianness	11
1.11 Support of Different ISA Capabilities	12
1.12 Performance Issues	12
1.12.1 Data Alignment	12
1.12.2 Argument Overlapping	12
1.12.3 Object Model	12
1.12.4 Scratch Memory	13
2 Reference	14
2.1 FIR Filters and Related Functions	14
2.1.1 Data Layout and Representation	14
2.1.2 Block FIR Filter	15
2.1.3 Block FIR Filter with Symmetric Response	16
2.1.4 Decimating Block Complex FIR Filter	17
2.1.5 Interpolating Block Complex FIR Filter	18
2.1.6 Convolution	20
2.1.7 Correlation	22
2.1.8 Autocorrelation	23
2.1.9 Despreading	24
2.1.10 Blockwise Adaptive LMS Algorithm	25
2.2 IIR Filters	27
2.2.1 Bi-Quad Block IIR	27
2.2.2 Streaming Bi-Quad Real Block IIR	29
2.2.3 Lattice Block IIR	31

2.2.4	Streaming Lattice Block IIR	33
2.2.5	Matlab Code for Conversion of SOS Matrices to IIR Parameters	34
2.3	Math functions	36
2.3.1	Reciprocal	38
2.3.2	Square Root	39
2.3.3	Reciprocal Square Root	40
2.3.4	Pseudo Floating Point Reciprocal	41
2.3.5	Pseudo Floating Point Inverse Square Root	41
2.3.6	Division	42
2.3.7	Remainder	43
2.3.8	Logarithm	44
2.3.9	Antilogarithm and Exponential	45
2.3.10	Modify the Exponent of a Floating-Point Number	45
2.3.11	Raise To a Power	46
2.3.12	Sine/Cosine	47
2.3.13	Tangent	48
2.3.14	Cotangent	49
2.3.15	Arcsine	50
2.3.16	Arccosine	51
2.3.17	Full Arctangent	52
2.3.18	Arctangent	53
2.3.19	Hyperbolic Tangent	54
2.3.20	Hyperbolic Sine	55
2.3.21	Hyperbolic Cosine	55
2.3.22	Integer to Floating Value Conversion	56
2.3.23	Floating To Integer Value Conversion	57
2.3.24	Fixed Point to Half_precision Floating Point Conversion	58
2.3.25	Half-Precision Floating Point to Fixed Point Conversion	59
2.3.26	Floating-point Floor	59
2.3.27	Floating-point Ceil	60
2.3.28	Absolute Value	60
2.3.29	Copy Sign	61
2.3.30	Clipping	62
2.3.31	Count One Bits in a Word	62
2.3.32	Average of Two Arguments	63
2.4	Complex Math Functions	65
2.4.1	Complex Division	65
2.4.2	Polar to Cartesian Conversion	66
2.4.3	Cartesian to Polar Conversion	67
2.4.4	Complex Vector Magnitude	68
2.4.5	Conjugate of Complex Number	69
2.4.6	Phase Angle of Complex Number	69
2.4.7	Complex Exponential	70
2.4.8	Complex Exponential with Linearly Evolving Phase	71
2.4.9	Correction of Vector Slope	72
2.5	Vector Operations	74
2.5.1	Real/Complex Vector Product	74
2.5.2	Real/Complex Vector Sum	74
2.5.3	Sum of Squares of a Vector	75
2.5.4	Square Root of Sum of Squares	76

2.5.5	Vector Normalization	76
2.5.6	Common Exponent.....	77
2.5.7	Mean of Vector Elements	77
2.5.8	Dual Peak Search	78
2.5.9	Search for 8 Top Values.....	79
2.5.10	Find Values above Threshold	80
2.5.11	Move the Elements at Given Indices	80
2.5.12	Remove the Elements at Given Indices.....	81
2.5.13	Angle Unwrapping	82
2.6	Matrix Operations.....	82
2.6.1	General Definitions and Matrix Representation.....	82
2.6.2	Real Matrix/Vector Multiply	83
2.6.3	Complex Matrix Multiply	85
2.6.4	Real Matrix-Complex Matrix/Vector Multiply	87
2.6.5	Real Matrix Transpose	89
2.6.6	Complex Matrix Transpose.....	90
2.6.7	Matrix Hermitian Product.....	91
2.6.8	Streaming to Block Order Conversion.....	93
2.6.9	Block to Streaming Order Conversion.....	94
2.6.10	Interleaving.....	95
2.6.11	Deinterleaving.....	97
2.7	Matrix Decomposition and Inversion Routines.....	98
2.7.1	Introduction	98
2.7.2	QR decomposition by Householder reflections	103
2.7.3	Rotation by Housholder Vectors	106
2.7.4	QR-based Back Substitution.....	109
2.7.5	Cholesky MMSE Solver.....	114
2.7.6	Cholesky Decomposition for Pseudo-inversion	116
2.7.7	Cholesky Forward Recursion for Pseudo-inversion.....	120
2.7.8	Cholesky Backward Recursion for Pseudo-inversion	125
2.7.9	Direct Matrix Inversion.....	129
2.7.10	Matrix Inversion by Gauss-Jordan Algorithm	131
2.7.11	LU Decomposition	132
2.7.12	Matrix Determinant	133
2.7.13	Singular Value Decomposition	135
2.7.14	Eigenvalues and Eigenvectors	137
2.8	Communications	138
2.8.1	CRC Calculation.....	138
2.8.2	Slicer Hard: QPSK, QAM16, QAM64	139
2.8.3	QAM Mapper: QPSK, QAM16, QAM64.....	140
2.8.4	Universal Grey Coding/QAM Mapping	140
2.8.5	Soft Demapper for QPSK and QAM Constellations.....	141
2.8.6	Bit Segmentation	142
2.8.7	Bit Packing	143
2.8.8	Convolution Encoder	143
2.8.9	Zadoff-Chu Sequence	144
2.8.10	LTE Pseudo-random Sequence Generation and Scrambling.....	145
2.8.11	SFD	146
2.9	Fast Fourier Transforms	147

2.9.1	General Definitions and Notes	147	
2.9.2	FFT on Complex Data	147	
2.9.3	FFT on Real Data	149	
2.9.4	Inverse FFT on Complex Data	150	
2.9.5	Inverse FFT Forming Real Data	151	
2.9.6	Blockwise FFT on Complex Data	152	
2.9.7	Blockwise Inverse FFT on Complex Data	153	
2.9.8	Blockwise FFT on Real Data	154	
2.9.9	Blockwise Inverse FFT Forming Real Data	155	
2.10	Fitting and interpolation routines	156	
2.10.1	Polynomial Fitting	156	
2.10.2	Interpolation	158	
2.10.3	Tap-based Interpolation	160	
2.10.4	Find Roots of a Polynomial	160	
2.11	Identification Routines	161	
2.11.1	Library Version Request	161	
2.11.2	Library API Version Request	161	
2.11.3	Library API Capability Request	161	
3	Test Environment and Examples	163	
3.1	Supported Use Environment, Configurations, and Targets	163	
3.2	Building the Library under Xplorer	163	
3.2.1	Importing the workspaces in Xtensa Xplorer	163	
3.2.2	Building and running tests	163	
3.3	Other platforms supported for building the NatureDSP Baseband Library and the Testdriver		163
4	Customer Support	164	

Preface

About this Manual

Welcome to the **NatureDSP Baseband Processing Library**, or “NatureDSP Baseband library” for short. The library is a collection of a number of highly optimized DSP functions for the DSP targets.

This source code library includes C-callable functions (ANSI-C language compatible) for general signal processing (filtering, correlation, and convolution), math, matrix, and vector functions specially optimized for high-density signal processing.

About this release

The release consists of a testframe work and a library. This is the first release of the BBE32EP that supports optimized single precision floating point functions in addition to the previously existing fixed point functions for general signal processing and communications applications.

Supported Targets

This library supports Tensilica BBE32EP targets. The library is conditionalized so that configurations lacking certain options are supported. Certain functions that only work with those options will only be compiled on configurations supporting those options. This is handled automatically through compiler #define controls.

Notations

This document uses the following conventions:

1. Program listings, program examples, interactive displays, filenames, variables and other software elements are shown in a special typeface (Courier)
2. Tables use smaller fonts

Abbreviations

API	Application program interface
DSP	Digital signal processing
FFT	Fast Fourier transform
FIR	Finite impulse response
IDE	Integrated development environment
IFFT	Inverse Fast Fourier transform
IIR	Infinite impulse response
IR	Impulse response
LMS	Least mean squares
LSB	least significant bit
MSB	most significant bit
QPSK	Quadrature phase shift keying
QAM	Quadrature amplitude modulation

SFD	Space frequency detector
ULP	unit of least precision

1 General Library Organization

1.1 Headers

The library is delivered in several packages. The API for each package is defined in the appropriate header file which describes particular functions in the package. When the appropriate `#include` preprocessor directive is included in your source, the compiler uses the prototypes to check that each function is called with the correct arguments.

<code>./library/include/NatureDSP_types.h</code>	Declarations of basic data types and compiler auto detection	1.3
<code>./library/include/NatureDSP_Baseband.h</code>	Declarations of all library functions	
<code>./library/include/NatureDSP_Baseband_fir.h</code>	FIR Filters and Related Functions	2.1
<code>./library/include/NatureDSP_Baseband_iir.h</code>	IIR Filters	2.2
<code>./library/include/NatureDSP_Baseband_math.h</code>	Math Functions	2.3
<code>./library/include/NatureDSP_Baseband_complex.h</code>	Complex Math Functions	2.4
<code>./library/include/NatureDSP_Baseband_vector.h</code>	Vector Operations	2.5
<code>./library/include/NatureDSP_Baseband_matop.h</code>	Matrix Operations	2.6
<code>./library/include/NatureDSP_Baseband_matinv.h</code>	Matrix Decomposition and Inversion Functions	2.7
<code>./library/include/NatureDSP_Baseband_comm.h</code>	Communications	2.8
<code>./library/include/NatureDSP_Baseband_fft.h</code>	FFT Routines	2.9
<code>./library/include/NatureDSP_Baseband_fit.h</code>	Fitting and Interpolation Routines	2.10
<code>./library/include/NatureDSP_Baseband_id.h</code>	Identification functions	2.11

1.2 Static Variables and Usage of C Standard Libraries

All library functions are re-entrant. The library does not require floating point support from the compiler or tool chain.

1.3 Types

The library uses standardized ANSI C types with defined length:

Data type	Meaning	Alignment, bytes
<code>int8_t</code>	8-bit signed value	1
<code>uint8_t</code>	8-bit unsigned value	1
<code>int16_t</code>	16-bit signed value	2
<code>uint16_t</code>	16-bit unsigned value	2
<code>int32_t</code>	32-bit signed value	4
<code>uint32_t</code>	32-bit unsigned value	4
<code>float32_t</code>	single precision floating point value	4
<code>complex_fract16</code>	Complex 16-bit fixed point value	4
<code>complex_fract32</code>	Complex 32-bit fixed point value	8
<code>complex_float</code>	Complex 32-bit single precision floating point value	8

It is assumed throughout this document that constant pointers passed through function arguments point at read-only data.

Complex numbers are represented as interleaved real/imaginary pairs where the real part goes first. Under xcc compiler, types `complex_float` is equivalent to the standard C99 type `complex float`.

As a general rule, all variables of given data type must be aligned on `sizeof` of this data type, i.e. pointers to `int16_t` must be aligned on 16-bit boundary, pointers to `complex_fract32` must be aligned on 64-bit boundary.

However, some library routines may require specific data alignment to provide the better performance.

1.4 Fractional Formats

Unless specifically noted, fixed point library functions use a Q_{15} format, in other words, $Q_{0.15}$.

In a $Q_{m.n}$ format, there are m bits used to represent the 2's complement integer portion of the number, and n bits used to represent the 2's complement fractional portion. $m+n+1$ bits are needed to store a general $Q_{m.n}$ number. The extra bit is needed to store the sign of the number in the most-significant bit position. The representable integer range is specified by $[-2^m, 2^m-1]$, and the finest fractional resolution is 2^{-n} . Typically, m from the Q notation is omitted (because the total length is defined of the data type used for operand) and it is simply written as Q_m .

For example, the most commonly used format is Q_{15} , which means that a 16-bit word is used to express a signed number between positive and negative 1. Thus, minimum value -32768 represents $-1.Q_{15}$ and maximum value 32767 represents $0.999969482421875Q_{15}$.

Floating point library functions use IEEE754 compatible floating point data.

Other typical cases used in the library are in the following table:

Format	Range	Minimum value	Maximum value
$Q_{0.15}$	-1 ... 0.999969	-32768	32767
$Q_{6.9}$	-64 ... 63,998	-32768	32767
$Q_{1.30}$	-2 ... 1.9999999991	-2147483648	2147483647
$Q_{0.30}$	-1 ... 0.9999999995	-2147483648	2147483647

The most-significant binary digit is interpreted as the sign bit in any Q format number. Thus, in the Q_{15} format, the decimal point is placed immediately to the right of the sign bit. The fractional portion to the right of the sign bit is stored in regular 2's complement format.

1.5 Compiler Requirements

When building the library source files or library-dependent modules, it is assumed that the target is a Tensilica processor implementing the Xtensa Instruction Set Architecture. This is accomplished through checking the `__XTENSA__` preprocessor symbol performed in `NatureDSP_types.h`. This symbol is automatically defined by the compiler. The library is delivered as an Xtensa Xplorer workspace, along with a second workspace containing demonstrations and tests of the various library routines. The library is intended to be used with Xtensa Xplorer. When the workspaces are imported, all relevant compiler options are automatically set, although the user can modify them if needed.

1.6 Call Conventions

The library uses ANSI-C call conventions.

1.7 Overflow Control

If not specially noted, the library does not check the real dynamic range of input data, so it is the user's responsibility to select parameters and the scale of input data according to each specific case. The library uses BBE32EP registers of 40-bit width for data accumulation during processing so BBE32EP will saturate the accumulated results automatically when it is stored to the memory. However, overflow of 40-bit accumulators in intermediate computations may result in improper results.

The user is expected to conform to the range requirements if specified and take care to restrict the input range in such a way that the outputs do not overflow.

Special considerations for overflow apply to the FFT routines, these are discussed in the relevant section.

1.8 Exceptions and Processor Control Registers

Except for some mathematical routines, compatible with IEEE-754 and C99 standards (see para 2.3), all library functions do not touch global `errno` variable and do not modify the FPU enabled bits. FPU flags may be set during the execution of the routines. It is up to the caller to decide how to proceed given the flags.

Example of use cases are:

- The caller could enable floating point control bits before calling functions. This would result in an external signal that indicates an exceptional condition has occurred. We expect the customer to use that signal to control an external interrupt – thus enabling an imprecise interrupt.
- The caller could zero the status flags before a function and check them when the function returns to see if any exceptional conditions occurred.

1.9 Special Numbers

The IEEE754 standard specifies some special values, and their representation: positive infinity ($+\infty$ or `+Inf`), negative infinity ($-\infty$ or `-Inf`), a negative zero (-0) distinct from ordinary ("positive") zero ($+0$), and "not a number" values (`NaNs`). In general, the following rules are applied:

- negative zero is treated as usual negative number
- the result of operations under `NaN` is `NaN`
- operations with infinity return `NaN` except for few routines which require to interpret only the sign of infinity
- If a result depends on several values (E.g. in filters and correlations), and one or more of them is `NaN` or `Inf`, the propagation of those special values is complicated. The library routines will propagate the value in a way that minimizes cycles and code size. A special value will still appear in the output.
- outputs for mathematical functions for special numbers on their inputs follows ISO/IEC 9899 if not explicitly mentioned

1.10 Endianness

The library supports little-endian mode only, since BBE32EP is little endian only.

1.11 Support of Different ISA Capabilities

BBE32EP core is very flexible and some parts of its functionality might not be supported by specific core build. At the compile time, library recognizes if required capability present or not and omits unsupported functions from the generating code. Library includes dedicated function allowing to detect if specific library function has been omitted.

1.12 Performance Issues

1.12.1 Data Alignment

Real-time performance of all functions depends on the fulfillment of special restrictions applied to input arguments. Typically, for maximum performance, users must use **aligned data arrays** for storing input and output arguments. The BBE32EP core is optimized for 256-bit aligned data accesses. By aligned arrays, we mean aligned on a 256-bit (32-byte) boundary. In some cases, the number of data for specific functions cannot be selected arbitrarily and should be divisible by a small integer (2, 4, 8 or 16).

Specific requirements are given for each function in its API description.

Data alignment may be achieved by several methods, including:

3. Placing the data into special data sections and making alignment at link-time
4. Dynamically allocating arrays of slighter bigger size and align pointers
5. Using the `aligned(32)` specifier in array declarations or equivalent macro `ALIGNED_ARRAY()` from library header.

In the runtime, library functions do not check if alignment requirements are satisfied and if not satisfied, it may result in undefined behavior and results. However, these restrictions are checked if library is built and run in the debug mode (with `xcc` or `gcc`) or under Visual Studio with `cstubs` (see para **Error! Reference source not found.**); in this case, library uses standardized `assert()` function or checking if alignment restriction is satisfied or not.

Test examples use the last two methods and you may use the `ALIGNED_ARRAY()` macro to create aligned arrays of the necessary size.

1.12.2 Argument Overlapping

Generally, input and output arguments of functions are assumed to be non-overlapping arrays; otherwise, the proper operation of functions is not guaranteed.

1.12.3 Object Model

Effective use of BBE32EP requires specific layouts for data/coefficients storage; there is no single method that is applicable for all cases. Hence, many functions (especially for filtering and FFT) require special initialization before processing. This is normally done once at the initialization time and does not affect the real-time performance. The sequence consists of three stages

- Call the `<obj>_alloc()` function with parameters that define the block size, filter length, etc. This function/macro returns the size of memory that must be allocated for the object for that set of specific parameters.
- Allocate the memory using various mechanisms. This can be done dynamically if the `<obj>_alloc()` function is used or even statically if the `<obj>_ALLOC()` macro is used with arguments known at compile-time.

- Pass the pointer to allocated memory to the function `<obj>_init`. This cleans up that memory block, reorders filter coefficients appropriately, etc. and returns the handle to the object. This handle will be used later for data processing by this given object, .i.e. block filtering.

Here we denote the symbolic name of object as `<obj>`. For example, corresponding functions for block FIR filtering will be named as:

<code>bkfir_alloc()</code>	Request the memory size for object
<code>bkfir_init()</code>	Initialize the object
<code>bkfir_process()</code>	Make filtering of block

1.12.4 *Scratch Memory*

Some functions require scratch memory area for saving temporary data during the execution. It can be shared between objects if they are called in the same processing thread. Scratch memory area must always be aligned on a 32-bytes boundary. For each algorithm that requires a scratch area, its minimum size (either in bytes or in 16-bit words) is specified in such a way that both static and dynamic allocation are possible. Most often the required scratch size is specified through a macro that takes a number of algorithm parameters. For example, one might use the following wrapping function to perform a QR decomposition (see 2.7.2) with the scratch area being allocated on the stack:

```
#define M 3
#define N 3
#define L 8

int run_qrmxns_MNL( int16_t * Q, int16_t * R )
{
    // Allocate the scratch memory. Macro returns required size in bytes
    int8_t scratch[QRSCRATCH(M,N,L)] __attribute__((aligned(32)))
    // Invoke the QR decomposition routine
    return ( qrmxns( scratch, Q, R, M, N, L ) )
}
```

2 Reference

2.1 FIR Filters and Related Functions

2.1.1 Data Layout and Representation

All FIR filtering APIs excepting correlation/convolution, autocorrelation and blockwise LMS algorithm require instantiation. In particular, filter objects encapsulate the delay line buffer, which is organized in such a way that advanced processor capabilities (e.g. circular data addressing) are efficiently utilized. When allocating and initializing a filter instance through `xfir_alloc()` and `xfir_init()` function calls, the user has to specify the input/output samples block length N . On the data processing stage the user application sequentially calls an `xfir_process()` function, providing it with a block of N input samples on each call. `xfir_process()` function updates the internal delay line with input samples, and computes N filter output samples, which are returned to the calling application via the output data buffer argument.

In the LMS algorithm, the IR is represented in the Q31 (Q1.30) format, in other cases it is represented in Q15. If not explicitly stated, the format of input and output samples is unchanged. That is, if input samples come in Q12, the outputs will be in Q12 as well.

Complex data is represented as an interleaved sequence of real and imaginary values.

All FIR filtering functions excluding autocorrelation require object allocation and setting the size of the block for processing. The FIR object stores the delay line in a format providing the most efficient use of BBE32EP capabilities. Some objects provide special access to the delay line to minimize unneeded data moves.

Usually, input and output samples should be aligned on a 32-bytes boundary.

For fixed point implementations, internal FIR computation, wider 40-bit registers are used to prevent intermediate overflow.

2.1.2 Block FIR Filter

Description	Computes a real/complex FIR filter (direct-form) using real/complex IR stored in vector h . The input is stored in vector x . The filter output result is stored in vector y . The filter calculates N output samples using M coefficients and requires last $M+N-1$ samples in the delay line. API defines filters for: - real data, real coefficients - complex data, complex coefficients - complex data, real coefficients																						
Representation	16-bit data, 16-bit coefficients, 16-bit outputs, everything in Q15 floating point data, floating point coefficients, floating point outputs																						
Algorithm	$y_n = \sum_{m=0}^{M-1} h_{M-1-m} x_{n+m}, n = \overline{0 \dots N-1}$ NOTE: This is formal description of algorithm, in reality processing is done using circular buffers, so user application is not responsible for management of delay lines																						
Object allocation	<pre>size_t bkfir_alloc (int M) size_t bkfirf_alloc(int M) size_t cxfir_alloc (int M) size_t cxfirf_alloc(int M) size_t rcfir_alloc (int M) size_t rcfirf_alloc(int M)</pre> <table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td colspan="4">Input</td></tr><tr><td>int</td><td>M</td><td></td><td>length of filter</td></tr></table> Returns: size of memory in bytes to be allocated			Type	Name	Size	Description	Input				int	M		length of filter								
Type	Name	Size	Description																				
Input																							
int	M		length of filter																				
Object initialization	<pre>bkfir_handle_t bkfir_init (void * objmem, int M, const int16_t * h) bkfirf_handle_t bkfirf_init(void * objmem, int M, const float32_t * h) cxfir_handle_t cxfir_init (void * objmem, int M, const complex_fract16* h) cxfirf_handle_t cxfirf_init(void * objmem, int M, const complex_float * h) rcfir_handle_t rcfir_init (void * objmem, int M, const int16_t * h) rcfirf_handle_t rcfirf_init(void * objmem, int M, const float32_t * h)</pre> <table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td colspan="4">Input</td></tr><tr><td>void*</td><td>objmem</td><td></td><td>allocated memory block</td></tr><tr><td>int16_t, float32_t, complex_fract16, complex float</td><td>h</td><td>M</td><td>filter coefficients; $h[0]$ is to be multiplied with the newest sample , Q15 or floating point</td></tr><tr><td>int</td><td>M</td><td></td><td>length of filter</td></tr></table> Returns: handle to the object			Type	Name	Size	Description	Input				void*	objmem		allocated memory block	int16_t, float32_t, complex_fract16, complex float	h	M	filter coefficients; $h[0]$ is to be multiplied with the newest sample , Q15 or floating point	int	M		length of filter
Type	Name	Size	Description																				
Input																							
void*	objmem		allocated memory block																				
int16_t, float32_t, complex_fract16, complex float	h	M	filter coefficients; $h[0]$ is to be multiplied with the newest sample , Q15 or floating point																				
int	M		length of filter																				
Update the delay line and compute filter output	<pre>void bkfir_process (bkfir_handle_t handle, int16_t * y, const int16_t * x, int N) void bkfirf_process(bkfirf_handle_t handle, float32_t * y, const float32_t * x, int N) void cxfir_process (cxfir_handle_t handle, complex_fract16 * y, const complex_fract16 * x, int N) void cxfirf_process(cxfirf_handle_t handle, complex_float * y, const complex_float * x, int N) void rcfir_process (rcfir_handle_t handle, complex_fract16 * y, const complex_fract16 * x, int N) void rcfirf_process(rcfirf_handle_t handle, complex_float * y, const complex_float * x, int N)</pre> <table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr></table>			Type	Name	Size	Description																
Type	Name	Size	Description																				

Input				
int16_t, float32_t, complex_fract16, complex_float	x	N	input samples , Q15 or floating point	
int	N		length of sample block	
Output				
int16_t, float32_t, complex_fract16, complex_float	y	N	output samples , Q15 or floating point	
Returns: none				
Restrictions	x, y	not overlapping		
	x, y	aligned on a 32-bytes boundary		
	M	2, 4, 8, or multiple of 16		
	N	multiple of 16 (bkfir), 8 (bkfirf, cxfir, rcfir) or 4 (cxfirf, rcfirf)		
Notes	Depending on available ISA options, some combinations of filter parameters would not be supported. In that case, initialization function returns <code>NULL</code> handle			

2.1.3 Block FIR Filter with Symmetric Response

Description	Computes a real FIR filter (direct-form) using real symmetric IR stored in the vector <i>h</i> . The real/complex data input is stored in the vector <i>x</i> . The filter output result is stored in the vector <i>y</i> . The filter calculates <i>N</i> output samples using $(M+1)/2$ coefficients and requires last $M+N-1$ samples in the delay line.																						
Representation	16-bit data, 16-bit coefficients, 16-bit outputs, everything in Q15																						
Algorithm	$y_n = \sum_{m=0}^{M-1} h_{M-1-m} x_{n+m}, n = 0 \dots N-1$ NOTE: This is formal description of algorithm, in reality processing is done using circular buffers, so user application is not responsible for management of delay lines																						
Object allocation	<pre>size_t srfir_alloc(int M) size_t srcfir_alloc(int M)</pre> <table> <tr> <th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr> <tr> <td colspan="4">Input</td></tr> <tr> <td>int</td><td>M</td><td></td><td>length of filter</td></tr> </table> Returns: size of memory in bytes to be allocated			Type	Name	Size	Description	Input				int	M		length of filter								
Type	Name	Size	Description																				
Input																							
int	M		length of filter																				
Object initialization	<pre>srfir_handle_t srfir_init (void * objmem, int M, const int16_t * h) srcfir_handle_t srcfir_init (void * objmem, int M, const int16_t * h)</pre> <table> <tr> <th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr> <tr> <td colspan="4">Input</td></tr> <tr> <td>void*</td><td>objmem</td><td></td><td>allocated memory block</td></tr> <tr> <td>int16_t</td><td>h</td><td>$(M+1)/2$</td><td>half of filter coefficients; <i>h</i>[0] is to be multiplied with the newest sample, Q15</td></tr> <tr> <td>int</td><td>M</td><td></td><td>length of filter</td></tr> </table> Returns: handle to the object			Type	Name	Size	Description	Input				void*	objmem		allocated memory block	int16_t	h	$(M+1)/2$	half of filter coefficients; <i>h</i> [0] is to be multiplied with the newest sample, Q15	int	M		length of filter
Type	Name	Size	Description																				
Input																							
void*	objmem		allocated memory block																				
int16_t	h	$(M+1)/2$	half of filter coefficients; <i>h</i> [0] is to be multiplied with the newest sample, Q15																				
int	M		length of filter																				

**Update the delay line
and compute filter
output**

```
void srfir_process ( srfir_handle_t handle,
                    int16_t * y, const int16_t * x, int N )
void srcfir_process ( srcfir_handle_t handle,
                    complex_fract16 * y, const complex_fract16 * x, int N )
```

Type	Name	Size	Description
Input			
int16_t, complex_fract16	x	N	input samples, Q15
int	N		length of sample block
Output			
int16_t, complex_fract16	y	N	output samples, Q15

Returns: none

Restrictions

x, y not overlapping
 x, y aligned on a 32-bytes boundary
 N for `srfir` - multiples of 16
 for `srcfir` - multiples of 8
 M for `srfir` - 2, 4, 8 or multiple of 16
 for `srcfir` - 17, 33 or multiple of 16

Notes

Depending on available ISA options, some combinations of filter parameters would not be supported. In that case initialization function returns `NULL` handle

2.1.4 Decimating Block Complex FIR Filter

Description

Computes a complex FIR filter (direct-form) with decimation using real IR stored in vector `h`. The complex data input is stored in vector `x`. The filter output result is stored in vector `y`. The filter calculates `N` output samples using `M` coefficients and requires last `D*N+M-1` samples in the delay line.

NOTE:

To avoid aliasing, the IR should be synthesized in such a way that filter pass band is limited by input sample frequency divided by 2^D .

Representation

16-bit data, 16-bit coefficients, 16-bit outputs, everything in Q15
 floating point data, floating point coefficients, floating point outputs

Algorithm

$$y_n = \sum_{m=0}^{M-1} h_{M-1-m} x_{D \cdot n + m}, n = 0 \dots N-1$$

NOTE:

This is formal description of algorithm, in reality processing is done using circular buffers, so user application is not responsible for management of delay lines

Object allocation

```
size_t firdec_alloc(int D, int M)
size_t firdecf_alloc(int D, int M)
```

Type	Name	Size	Description
Input			
int	D		decimation factor
int	M		length of filter

Returns: size of memory in bytes to be allocated

Object initialization

```

firdec_handle_t firdec_init(void * objmem,
                           int D, int M, const int16_t * h)
firdecf_handle_t firdecf_init(void * objmem,
                              int D, int M, const float32_t * h)

```

Type	Name	Size	Description
Input			
void*	objmem		allocated memory block
int16_t, float32_t	h	M	filter coefficients; $h[0]$ is to be multiplied with the newest sample, Q15 or floating point
int	D		decimation factor
int	M		length of filter

Returns: handle to the object

Update the delay line and compute decimator output

```

void firdec_process(firdec_handle_t handle,
                   complex_fract16 * y, const complex_fract16 * x, int N )
void firdecf_process(firdecf_handle_t handle,
                    complex_float * y, const complex_float * x, int N )

```

Type	Name	Size	Description
Input			
complex_fract16, complex float	x	N*D	input complex samples, Q15 or floating point
int	N		length of output sample block
Output			
complex_fract16, complex float	y	N	output complex samples, Q15 or floating point

Returns: none

Restrictions

x, h, y not overlapping
 x, h aligned on a 32-bytes boundary
 N Multiple of 8 (*firdec*) or 4 (*firdecf*)
 M 2,4,8 or a positive multiple of 16 for $D=2,3,4$; or a positive multiple of 16 for $D>4$
 D >1

Notes

Depending on available ISA options, some combinations of filter parameters would not be supported. In that case, *firdec[f]_init* will return NULL handle

2.1.5 Interpolating Block Complex FIR Filter**Description**

Computes a complex FIR filter (direct-form) with interpolation using real IR stored in vector h . The complex data input is stored in vector x . The filter output result is stored in vector y . The filter calculates $N \cdot D$ complex output samples using $M \cdot D$ coefficients and requires last $N+M-1$ samples in the delay line.

Representation

16-bit data, 16-bit coefficients, 16-bit outputs, everything in Q15
floating point data, floating point coefficients, floating point outputs

Algorithm

$$y_{n \cdot D + d} = D \cdot \sum_{m=0}^{M-1} h_{D(M-1-m)+d} x_{n+m}, n = \overline{0 \dots N-1}, d = \overline{0 \dots D-1},$$

NOTE:

This is formal description of algorithm, in reality processing is done using circular buffers, so user application is not responsible for management of delay lines

Object allocation

```
size_t firinterp_alloc(int D, int M)
size_t firinterp_f_alloc(int D, int M)
```

Type	Name	Size	Description
Input			
int	D		interpolation ratio
int	M		length of subfilter. Total length of filter is $M \times D$

Returns: size of memory in bytes to be allocated

Object initialization

```
firinterp_handle_t firinterp_init(void * objmem,
                                int D, int M, const int16_t* h)
firinterp_f_handle_t firinterp_f_init(void * objmem,
                                      int D, int M, const float32_t* h)
```

Type	Name	Size	Description
Input			
void*	objmem		allocated memory block
int16_t, float32_t	h	$M \times D$	filter coefficients; $h[0]$ is to be multiplied with the newest sample, Q15 or floating point
int	D		interpolation ratio
int	M		length of subfilter. Total length of filter is $M \times D$

Returns: handle to the object

Update the delay line and compute interpolator output

```
void firinterp_process(firinterp_handle_t handle,
                      complex_fract16* y, const complex_fract16* x, int N)
void firinterp_f_process(firinterp_f_handle_t handle,
                        complex_float * y, const complex_float * x, int N)
```

Type	Name	Size	Description
Input			
complex_fract16, complex_float	x	N	input complex samples, Q15 or floating point
int	N		length of input sample block
Output			
complex_fract16, complex_float	y	$N \times D$	output complex samples, Q15 or floating point

Returns: none

Restrictions

x, y not overlapping
 x, y aligned on a 32-bytes boundary
 N Multiple of 8 (`firinterp`) or 4 (`firinterp_f`)
 M 2,4,8 or a positive multiple of 16 for $D=2,3,4,6,12$; or a positive multiple of 8 for other D
 M, D >1

Notes

Depending on available ISA options, some combinations of filter parameters would not be supported. In that case, `firinterp[f]_init` will return `NULL` handle

2.1.6 Convolution

Description Performs circular or linear convolution between real or complex vectors x (of length N) and y (of length M) resulting in vector r of N (circular) or $N+M-1$ (linear) real or complex values.

MATLAB code for circular convolution:

```
for n=1:N
    r(n) = 2*sum(x(1+mod(n-(1:M),N)).*y(1:M));
end
```

MATLAB code for linear convolution:

```
for n=1:N+M-1
    ix = max(n-N+1,1):min(n,M);
    r(n) = 2*sum(x(n+1-ix).*y(ix));
end
```

Representation 16-bit inputs, Q15, 32-bit outputs, Q31
floating point inputs, floating point outputs

Algorithm

$$r_k = \sum_{m=0}^{M-1} x_{\text{mod}(k-m,N)} y_m, k = 0 \dots (N-1) \text{ (circular convolution)}$$

$$r_k = \sum_{j=\max(k-M+1,0)}^{\min(N-1,k)} x_j y_{k-j}, k = 0 \dots (M+N-1) \text{ (linear convolution)}$$

Prototype

```
void fir_convol_circ (complex_fract32* r,
                    const complex_fract16 * x, const complex_fract16* y, int N, int M)
void fir_convolf_circ( complex_float * r,
                    const complex_float * x, const complex_float * y, int N, int M)
void fir_convol_lin (complex_fract32 * r,
                    const complex_fract16 * x, const complex_fract16* y, int N, int M)
void fir_convolf_lin (complex_float * r,
                    const complex_float * x, const complex_float * y, int N, int M)
void fir_rconvolf_circ( float32_t * r,
                    const float32_t * x, const float32_t * y, int N, int M)
void fir_rconvolf_lin ( float32_t * r,
                    const float32_t * x, const float32_t * y, int N, int M)
```

Function name	Description
fir_convol_circ, fir_convolf_circ	circular convolution, complex data
fir_convol_lin, fir_convolf_lin	linear convolution, complex data
fir_rconvolf_circ	circular convolution, real data
fir_rconvolf_lin	linear convolution, real data

Arguments	Type	Name	Size	Description
Input				
	complex_fract16, complex_float, float32_t	x	N	Left-hand data sequence (Q15 or floating point)
	complex_fract16, complex_float, float32_t	y	M	Right-hand data sequence (Q15 or floating point)
	int	N		length of x
	int	M		length of y
Output				
	complex_fract32, complex_float, float32_t	r	N for circular convolution (N+M-1) for linear	output data, Q31 or floating point
Returned value:	none			
Restrictions	x, y, r	not overlapping		
	x, y, r	should be aligned on 32-bytes boundary		
	N, M	multiples of 8 (fir_convolve_circ, fir_convolve_lin, fir_rconvolve_circ, fir_rconvolve_lin) or 4 (fir_convolve_circ, fir_convolve_lin)		
	N >= M	must be greater than or equal to M		

2.1.7 Correlation

Description	Estimates the circular/linear cross-correlation between complex or real vectors x (of length N) and y (of length M) resulting in vector r . It is similar to correlation but y is read in opposite direction.													
Representation	16-bit inputs, Q15, 32-bit outputs, Q31 floating point inputs, floating point outputs													
Algorithm	$r_k = \sum_{m=0}^{M-1} x_{\text{mod}(k+m,N)} y_m^*, k = 0 \dots (N-1) \text{ (circular correlation)}$ $r_k = \sum_{j=\max(k-M-1,0)}^{\min(N-1,k)} x_j y_{M-1-(k-j)}^*, k = 0 \dots (M+N-1) \text{ (linear correlation)}$													
Prototype	<pre>void fir_xcorr_circ(complex_fract32* r, const complex_fract16* x,const complex_fract16* y, int N, int M) void fir_xcorr_circ(complex_float * r, const complex_float * x,const complex_float * y, int N, int M) void fir_xcorr_lin (complex_fract32 * r, const complex_fract16* x,const complex_fract16* y, int N, int M) void fir_xcorr_lin (complex_float * r, const complex_float * x, const complex_float * y, int N, int M) void fir_rxcorr_circ(float32_t * r, const float32_t * x, const float32_t * y, int N, int M) void fir_rxcorr_circ(float32_t * r, const float32_t * x, const float32_t * y, int N, int M) void fir_rxcorr_lin (float32_t * r, const float32_t * x, const float32_t * y, int N, int M)</pre> <table><tr><th>Function name</th><th>Description</th></tr><tr><td>fir_xcorr_circ, fir_xcorr_circ</td><td>circular correlation, complex data</td></tr><tr><td>fir_xcorr_lin, fir_xcorr_lin</td><td>linear correlation, complex data</td></tr><tr><td>fir_rxcorr_circ</td><td>circular correlation, real data</td></tr><tr><td>fir_rxcorr_lin</td><td>linear correlation, real data</td></tr></table>				Function name	Description	fir_xcorr_circ, fir_xcorr_circ	circular correlation, complex data	fir_xcorr_lin, fir_xcorr_lin	linear correlation, complex data	fir_rxcorr_circ	circular correlation, real data	fir_rxcorr_lin	linear correlation, real data
Function name	Description													
fir_xcorr_circ, fir_xcorr_circ	circular correlation, complex data													
fir_xcorr_lin, fir_xcorr_lin	linear correlation, complex data													
fir_rxcorr_circ	circular correlation, real data													
fir_rxcorr_lin	linear correlation, real data													
Arguments	Type	Name	Size	Description										
	Input													
	complex_fract16, complex_float, float32_t	x	N	input data (Q15 or floating point)										
	complex_fract16, complex_float, float32_t	y	M	input data (Q15 or floating point)										
	int	N		length of x										
	int	M		length of y										
	Output													
	complex_fract32, complex_float, float32_t	r	N for circular correlation (N+M-1) for linear	output data, Q31 or floating point										
	Returned value:	none												
Restrictions	x, y, r x, y, r N, M $N \geq M$	not overlapping should be aligned on 32-bytes boundary multiples of 8 (fir_xcorr_circ, fir_xcorr_lin, fir_rxcorr_circ, fir_rxcorr_lin) or 4 (fir_xcorr_circ, fir_xcorr_lin) must be greater than or equal to M												
Note	fir_xcorr[f]_lin() returns the same output as MATLAB xcorr() but omits first $N-M$ zeros													

2.1.8 Autocorrelation

Description	Estimates autocorrelation for a complex or real data vector x (positive side only). Returns autocorrelation of length N . For an input vector of N complex or real samples $x[0..N-1]$ the computation follows the pseudo-code given below: <pre>for (n=0; n<N; n++) { r[n] = 0; for (m=0; m<N-n; m++) { r[n] += x[n+m] * conj(x[m]); } }</pre>															
Representation	16-bit data, Q15, 32-bit outputs, Q31 floating point inputs, floating point outputs															
Algorithm	$r_k = \sum_{k=0}^{N-k-1} x_{n+k} x_k^*, k = \overline{0...(N-1)}$															
Prototype	<pre>void fir_acorr (complex_fract32 * r, const complex_fract16 * x, int N) void firf_acorr (complex_float * r, const complex_float * x, int N) void fir_racorr(float32_t * r, const float32_t * x, int N)</pre> <table><tr><th>Function name</th><th colspan="3">Description</th></tr><tr><td>fir_acorr, firf_acorr</td><td colspan="3">autocorrelation, complex data</td></tr><tr><td>fir_racorr</td><td colspan="3">autocorrelation, real data</td></tr></table>				Function name	Description			fir_acorr, firf_acorr	autocorrelation, complex data			fir_racorr	autocorrelation, real data		
Function name	Description															
fir_acorr, firf_acorr	autocorrelation, complex data															
fir_racorr	autocorrelation, real data															
Arguments	Type		Name	Size	Description											
	Input															
	complex_fract16, complex_float, float32_t	x		N	input data (Q15 or floating point)											
	int	N			length of x											
	Output															
	complex_fract32, complex_float, float32_t	r		N	output data, Q31 or floating point											
Returned value:	none															
Restrictions	x, r x, r N	not overlapping should be aligned on 32-bytes boundary multiple of 8 (fir_acorr, fir_racorr) or 4 (fir_acorr)														

2.1.9 Despreding

Description

Performs a special kind of correlation utilized in Direct-Sequence Spread Spectrum (DSSS) modulation technique, with Spreading Factor (SF) 4 or 8. Despreding functions multiply the input sequence of 16-bit complex samples $x[]$ by a coded pseudo-noise sequence of 2-bit QPSK symbols $pn[]$, then reduce the product sequence by adding together 4 or 8 contiguous products. Resulting sums are shifted to the right by rsh bit positions, saturated to -32768 or 32767 and stored to the output array $y[]$.

For the coded sequence multiplicand $pn[]$, there are two variants of 2-bit QPSK codeset identified with the `qpsk_type` argument:

Dibit	QPKS symbol for <code>qpsk_type == 0</code>	QPKS symbol for <code>qpsk_type == 1</code>
00	$1 + j$	1
01	$-1 + j$	$-j$
10	$1 - j$	-1
11	$-1 - j$	j

First 8 consecutive dibits of the coded sequence are concatenated in a 16-bit word $pn[0]$, with the very first dibit going into 2 LSBs of $pn[0]$ and the last dibit - into 2 MSBs. The next 8 dibits are stored in $pn[1]$, and so on.

Representation

16-bit data, Q15, 16-bit outputs (flexible representation)

Algorithm

$$r_n^{(l)} = \left| \frac{1}{2^{rsh}} \sum_{m=0}^{M-1} x_{n+(m-M+1)D} y_m^{(l)*} \right|^2, n = \overline{0 \dots N-1}, l = \overline{0 \dots L-1}$$

Prototypes

```
void despread4( complex_fract16 * y,
               const complex_fract16 * x, const int16_t * pn,
               int N, int qpsk_type, int rsh )

void despread8( complex_fract16 * y,
               const complex_fract16 * x, const int16_t * pn,
               int N, int qpsk_type, int rsh )
```

Spreading Factor SF	Function name
4	<code>despread4</code>
8	<code>despread8</code>

Type	Name	Size	Description
Input:			
<code>complex_fract16</code>	<code>x</code>	<code>N</code>	Input signal, 16-bit complex samples
<code>int16_t</code>	<code>pn</code>	<code>N/8</code>	Pseudo-noise coded sequence of 2-bit QPSK symbols
<code>int</code>	<code>N</code>		
<code>int</code>	<code>qpsk_type</code>		QPSK codeset selection, 0 or 1
<code>int</code>	<code>rsh</code>		Right shift for scaling (≥ 0)
Output:			
<code>complex_fract16</code>	<code>y</code>	<code>N/SF</code>	Output signal, 16-bit complex samples. Fixed point position for the output signal is $Q_x - rsh$, where Q_x is the fixed point position for the input signal.

Returned value:

none

Restrictions	SF	4 or 8
	N	should be a multiple of 8
	rsh	should not be negative
	x, y, pn	should be aligned on 32-bytes boundary

2.1.10 Blockwise Adaptive LMS Algorithm

Description	Blockwise LMS algorithm performs filtering of complex input samples, computation of error over a block of reference samples and makes blockwise update of IR to minimize the error output. Algorithm includes FIR filtering, calculation of correlation between the error output and reference signal and IR taps update based on that correlation. NOTES: 1. For $N=1$ this algorithm is equivalent to standard LMS, however a bigger block size reduces the computational overhead keeping adaptation properties. 2. Right selection of N depends on the change rate of impulse response. However, on static or slow varying channels convergence rate depends on selected μ and M, but not on N. 3. Computation of filter output is done using only higher 16-bit words of IR coefficients (in fact converting them to Q15). However, when it performs coefficient update, it uses full Q31 accuracy. 4. Functions use BBE32EP 40-bit accumulators for all computational steps. Thus, the user needs to avoid very long blocks to prevent overflows during IR update step. Typically, N should not exceed 128 to guarantee 40-bit range.																				
Representation	16-bit data, Q15, 32-bit impulse response, Q31																				
Algorithm	$b = \frac{\mu}{norm}$ $e_n = r_n - \sum_{m=0}^{M-1} h_{M-1-m}^* x_{m+n}, n = \overline{0 \dots N-1}$ $h_{M-1-m} = h_{M-1-m} + b \cdot \sum_{n=0}^{N-1} x_{n+m} e_n^*, m = \overline{0 \dots M-1}$																				
Prototype	<pre>void fun (void *pScr, complex_fract16 * e, complex_fract32 * h, const complex_fract16 * r, const complex_fract16 * x, int32_t norm, int16_t mu, int N, int M) void funMs (void *pScr, complex_fract16 * e, complex_fract32 * h, const complex_fract16 * r, const complex_fract16 * x, int32_t norm, int16_t mu, int N)</pre>																				
<table><tr><th>Function Name</th><th>M</th><th>Prototype</th><th>Scratch size, bytes</th></tr><tr><td>fir_blms</td><td>any</td><td>fun</td><td rowspan="5">FIR_BLMS_SCRATCH (M, N)</td></tr><tr><td>fir_blms4</td><td>4</td><td>funMs</td></tr><tr><td>fir_blms8</td><td>8</td><td>funMs</td></tr><tr><td>fir_blms16</td><td>16</td><td>funMs</td></tr><tr><td>fir_blms32</td><td>32</td><td>funMs</td></tr></table>		Function Name	M	Prototype	Scratch size, bytes	fir_blms	any	fun	FIR_BLMS_SCRATCH (M, N)	fir_blms4	4	funMs	fir_blms8	8	funMs	fir_blms16	16	funMs	fir_blms32	32	funMs
Function Name	M	Prototype	Scratch size, bytes																		
fir_blms	any	fun	FIR_BLMS_SCRATCH (M, N)																		
fir_blms4	4	funMs																			
fir_blms8	8	funMs																			
fir_blms16	16	funMs																			
fir_blms32	32	funMs																			

Arguments	Type	Name	Size	Description
Input				
	complex_fract32	h	M	impulse response, Q31
	complex_fract16	r	N	reference (near end) complex data vector. First in time value is in $r[0]$, Q15
	complex_fract16	x	(N+M-1)	input (far end) complex data vector. First in time value is in $x[0]$, Q15
	int32_t	norm		normalization factor: power of signal multiplied by N, Q31
	int16_t	mu		adaptation coefficient in Q15 (LMS step)
	int	N		length of data block
	int	M		length of h
Output				
	complex_fract16	e	N	estimated error, Q15
	complex_fract32	h	M	updated impulse response, Q31
Returned value:	none			
Restrictions	h, x, r, e not overlapping x, e, h, r aligned on a 32-bytes boundary M multiple of 8 (for <code>fir_blms()</code>) N multiple of 8			

2.2 IIR Filters

IIR routines allow to pass complex or real data through biquad and lattice IIR structures. Two APIs available:

- usual API allowing to pass one data stream through the filter
- streaming API where filter processes a number of data streams in parallel.

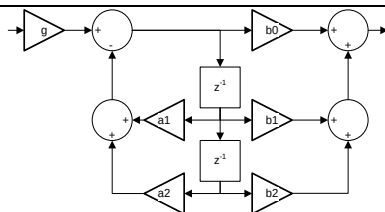
Streaming API is more computationally effective but requires special layout of data. Number of streams should be a multiple of:

- 8 for complex 16-bit data
- 16 for real 16-bit data
- 4 for complex floating point data
- 8 for real floating point data

Streaming functions require special (streaming) data layout where data come in blocks of length L : first block corresponds to the first sample from all L streams, the second block contains second samples from all L streams and so on. Also, data blocks should be aligned on 32-bytes boundary.

2.2.1 Bi-Quad Block IIR

Description	Computes a real or complex IIR filter (cascaded IIR direct form II using five coefficients per bi-quad + gain term). The real or complex data inputs are stored in vector x. The filter output result is stored in vector y. The filter calculates N output samples using SOS and G matrices. NOTES: 1. Bi-quad coefficients may be derived from standard SOS and G matrices generated by MATLAB, however its scale factors may require some tuning to find a compromise between quantization noise and possible overflows. 2. Two algorithms are implemented. The Fast version is much faster, while Low_Noise provides better accuracy in terms of quantization noise and lower rounding errors. For each section the user has to provide two feedback tap coefficients (a_1, a_2), three feedforward tap coefficients (b_0, b_1, b_2). For fixed point API the input gain term (g) is added to parameters of each section to guarantee no overflow in the intermediate computations (see 2.2.5). To scale the filter output, a total gain ($gain$) is applied to output samples produced by the last biquad section. Implementation of biquad series for fixed-point data format has two flavors: ▪ Low_Noise. Performs 32x16-bit multiplications and utilizes the scratch memory to keep intermediate results in order to lower the quantization noise and reduce round-off errors ▪ Fast. Makes use of native 16x16-bit multipliers and thus appears faster than the Low Noise variant at the cost of deteriorated accuracy Due to the performance reasons, implementation may introduce additional algorithmic delay (group delay) to the output data
Representation	16-bit data, Q15, 16-bit outputs, 32-bit intermediate (<code>BQRIIR_OPT_LOW_NOISE</code>) 16-bit data, Q15, 16-bit outputs, 16-bit intermediate (<code>BQRIIR_OPT_FAST</code>) floating point inputs, floating point out
Algorithm	Algorithm consists of applying sequentially M times bi-quad block filter with the structure shown below and at the last stage scale factor g is formed from bi-quad scale factor and total gain

**Object Allocation**

```

size_t bqriir_alloc(int M, int opt)
size_t bqriirf_alloc( int M )
size_t bqciir_alloc ( int M, int opt )
size_t bqciirf_alloc( int M )

```

Type	Name	Size	Description
Input:			
int	M		Number of bi-quad sections
int	opt		Fixed-point implementation variant selector, BQRIIR_OPT_LOW_NOISE= BQCIIR_OPT_LOW_NOISE=1 or BQRIIR_OPT_FAST= BQCIIR_OPT_FAST=0

Returns: Size of memory in bytes to be allocated

Object Initialization

```

bqriir_handle_t bqriir_init(void * objmem,
                           int M, int opt,
                           const int16_t * coef,
                           int16_t gain)
bqriirf_handle_t bqriirf_init( void * objmem, int M,
                              const float32_t * coef, float32_t gain )
bqciir_handle_t bqciir_init ( void * objmem, int M, int opt,
                              const int16_t * coef, int16_t gain )
bqciirf_handle_t bqciirf_init( void * objmem, int M,
                              const float32_t * coef, float32_t gain )

```

Type	Name	Size	Description
Input:			
void*	objmem		Allocated memory block
int	M		Number of bi-quad sections
int	opt		BQCIIR_OPT_LOW_NOISE, BQRIIR_OPT_LOW_NOISE or BQCIIR_OPT_FAST, BQRIIR_OPT_FAST
int16_t	coef	M*6	For fixed point API: Filter coefficients stored in blocks of 6 numbers: g b0 b1 b2 a1 a2; Q14
float32_t	coef	M*5	For floating point API: Filter coefficients stored in blocks of 5 numbers: b0 b1 b2 a1 a2
int16_t, float32_t	gain		total gain, Q8 or floating point

Returns: handle to the object

Process Samples

```

void bqriir_process( bqriir_handle_t _bqriir,
                    int16_t * r, const int16_t *x, void * scr, int N)
void bqriirf_process( bqriirf_handle_t bqriirf,
                     float32_t * r, const float32_t * x, int N)
void bqciir_process ( bqciir_handle_t _bqciir,
                     complex_fract16 * r, const complex_fract16 *x,
                     int16_t * scr, int N )
void bqciirf_process( bqciirf_handle_t _bqciirf,
                     complex_float * r, const complex_float *x, int N )

```

Initialization functions	Allocation functions	Processing functions	Description
bqriir_alloc, bqriirf_alloc	bqriir_init, bqriirf_init	bqriir_process, bqriirf_process	real data

	bqciir_alloc, bqciirf_alloc	bqciir_init, bqciirf_init	bqciir_process, bqciirf_process	complex data
--	--------------------------------	------------------------------	------------------------------------	--------------

Type	Name	Size	Description
Input:			
int16_t, float32_t, complex_fract16, complex float	x	N	Input samples, Q15 or floating point
int	N		Length of input sample block
	C		1 for real data, 2 – for complex
Output:			
int16_t, float32_t, complex_fract16, complex float	r	N	Output data, Q15 or floating point
Temporary:			
void	scr		Scratch memory area; required for a low-noise fixed point variant only. Size in bytes is $4 * C * N + 32$

Returned value	none
Restrictions	x, r – aligned on 4 byte boundary scr – whenever supplied, must be aligned on a 32-bytes boundary M – number of biquads should be less or equal to 32

2.2.2 Streaming Bi-Quad Real Block IIR

Description	Computes a real IIR filter (cascaded IIR direct form II using five coefficients per bi-quad + gain term) for several streams of data. The filter output result is stored in vector <i>r</i>. The filter calculates <i>N</i> output samples for each <i>L</i> streams using SOS and G matrices. Detailed description of filter structure is given in 2.2.1. Complex biquad IIR filtering in streaming format are supported directly via corresponding real functions because real and imaginary parts might be considered as separate real streams. Due to the performance reasons, implementation may introduce additional algorithmic delay (group delay) to the output data. Streaming lattice block real IIR filter may be applied to complex data stored in streaming format, which is equivalent to passing a complex signal through a complex IIR with real coefficients. The number of real data streams <i>L</i> as it is seen by the filter implementation must be two times the number of actual complex streams.
Representation	16-bit data, Q15, 16-bit outputs, 32-bit intermediate (BQRIIRS_OPT_LOW_NOISE) 16-bit data, Q15, 16-bit outputs, 16-bit intermediate (BQRIIRS_OPT_FAST) floating point inputs, floating point out
Algorithm	Algorithm consists of applying sequentially <i>M</i> times bi-quad block filter with the structure shown below and at the last stage scale factor <i>g</i> is formed from bi-quad scale factor and total gain

Object Allocation

```
size_t bqriirs_alloc(int L, int M, int opt)
size_t bqriirsf_alloc( int L, int M )
```

Type	Name	Size	Description
Input:			
int	L		Number of streams, should be a multiple of 32
int	M		Number of bi-quad sections
int	opt		Fixed-point implementation variant selector, BQRIIRS_OPT_LOW_NOISE or BQRIIRS_OPT_FAST

Returns: Size of memory in bytes to be allocated

Object Initialization

```
bqriirs_handle_t bqriirs_init(void * objmem,
                             int L, int M, int opt,
                             const int16_t * coef, int16_t gain)
bqriirsf_handle_t bqriirsf_init( void * objmem, int L, int M,
                                const float32_t * coef, float32_t gain )
```

Type	Name	Size	Description
Input:			
void*	objmem		Allocated memory block
int	L		Number of streams
int	M		Number of bi-quad sections
int	opt		Fixed-point implementation variant selector, BQRIIRS_OPT_LOW_NOISE or BQRIIRS_OPT_FAST
int16_t	coef	M*6	For fixed point API: Filter coefficients stored in blocks of 6 numbers: g b0 b1 b2 a1 a2; Q14
float32_t	coef	M*5	For floating point API: Filter coefficients stored in blocks of 5 numbers: b0 b1 b2 a1 a2
int16_t, float32_t	gain		total gain, Q8 or floating point

Returns: handle to the object

Process Samples

```
void bqriirs_process( bqriirs_handle_t _bqriir,
                     int16_t * r, const int16_t * x, void * scr, int N)
void bqriirsf_process( bqriirsf_handle_t _bqriirsf,
                      float32_t * r, const float32_t * x, int N )
```

Type	Name	Size	Description
Input:			
int16_t, float32_t	x	N*L	Input samples stored in streaming format, Q15 or floating point
int	N		Length of input sample block
Output:			
int16_t, float32_t	r	N*L	Output data stored in streaming format, Q15 or floating point
Temporary:			
void	scr		Scratch memory area; required only for a low-noise variant. Size in bytes is 4*N*L

Returned value

none

Restrictions

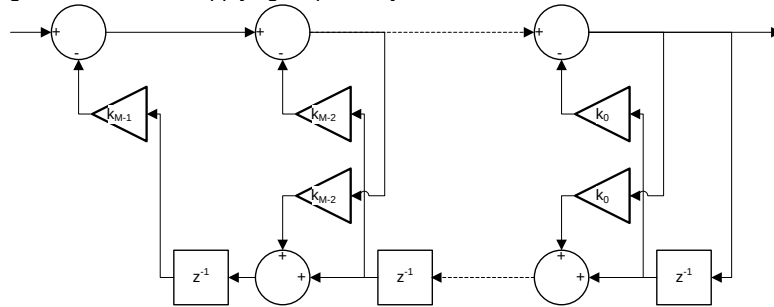
x, r – must be aligned on 32-byte boundary
scr – whenever supplied, must be aligned on a 32-bytes boundary
L – Must be a multiple of 16 (bqriirs) or 8 (bqriirsf)
M – number of biquads should be less or equal to 32

2.2.3 Lattice Block IIR

Description	Computes a real or complex cascaded lattice autoregressive IIR filter using reflection coefficients stored in vector k. The data input is stored in vector x. The filter output result is stored in vector r. Implementation of lattice filter for fixed-point data format has two flavors: - Low_Noise. Performs 32x16-bit multiplications in the feedback branch to attain a precise output sample; feedforward products that update the delay line are still produced by 16x16-bit multiplications, but the full 32-bit result is preserved for each delay element to be used on future iterations. - Fast. Makes use of native 16x16-bit multipliers and allocates 16 bits for a delay element
--------------------	---

Representation	16-bit data, Q15, 16-bit outputs, 32-bit intermediate (LATR_OPT_LOW_NOISE/LATC_OPT_LOW_NOISE) 16-bit data, Q15, 16-bit outputs, 16-bit intermediate (LATR_OPT_FAST/LATC_OPT_FAST) floating point inputs, floating point out
-----------------------	--

Algorithm	Algorithm consists of applying sequentially M times IIR sections with the structure shown below
------------------	---



Object Allocation	<pre> size_t latr_alloc(int N, int M, int opt) size_t latrf_alloc(int M) size_t latc_alloc (int M, int opt) size_t latcf_alloc(int M) </pre>
--------------------------	--

Type	Name	Size	Description
Input:			
int	M		Filter order
int	opt		Fixed-point implementation variant selector, LATR_OPT_LOW_NOISE or LATR_OPT_FAST

Returns: Size of memory in bytes to be allocated

Object Initialization

```

latr_handle_t latr_init(void * objmem,
                      int M, int prec,
                      const int16_t * k)
latrf_handle_t latrf_init( void * objmem, int M,
                          const float32_t * coef, float32_t gain )
latc_handle_t latc_init ( void * objmem, int M, int opt,
                          const int16_t * coef, int16_t gain )
latcf_handle_t latcf_init( void * objmem, int M,
                          const float32_t * coef, float32_t gain )

```

Type	Name	Size	Description
Input:			
void*	objmem		Allocated memory block
int	M		Filter order
int	opt		Fixed-point implementation variant selector, LATR_OPT_LOW_NOISE/LATC_OPT_LOW_NOISE or LATR_OPT_FAST/LATC_OPT_FAST
int16_t, float32_t	k	M	Reflection coefficients, Q15 or floating point

Returns: Handle to the object

Process Samples

```

void latr_process (latr_handle_t handle,
                  int16_t * r,
                  const int16_t * x, int N)
void latrf_process( latrf_handle_t _latrf,
                  float32_t * r, const float32_t * x, int N )
void latc_process ( latc_handle_t _latc,
                  complex_fract16 * r, const complex_fract16 * x, int N )
void latcf_process( latcf_handle_t _latcf,
                  complex float * r, const complex float * x, int N )

```

Initialization functions	Allocation functions	Processing functions	Description
latr_alloc, latrf_alloc	latr_init, latrf_init	latr_process, latrf_process	real data
latc_alloc, latcf_alloc	latc_init, latcf_init	latc_process, latcf_process	complex data

Type	Name	Size	Description
Output:			
int16_t, float32_t, complex_fract16, complex float	r	N	Output data, Q15 or floating point
int	N		Length of input sample block
Input:			
int16_t, float32_t, complex_fract16, complex float	x	N	Input samples, Q15 or floating point

Returned Value

none

Restrictions

input/output complex data arrays must be aligned by 4 (latc_process) or 8 (latcf_process) bytes

filter order must not exceed 8: $M \leq 8$

Performance Restrictions:

Basically, there are two hard-coded filter implementations: order 4 and order 8 lattice structures, so that any user-defined structure is reduced to one of these variants.

2.2.4 Streaming Lattice Block IIR

Description Computes a real or complex cascaded lattice autoregressive IIR filter using reflection coefficients stored in vector k . The data input is stored in vector x . The filter output result is stored in vector y . Filter makes filtration of L data streams in parallel effectively utilizing SIMD nature of BBE architecture.

Implementation of lattice filter for fixed-point data format has two flavors:

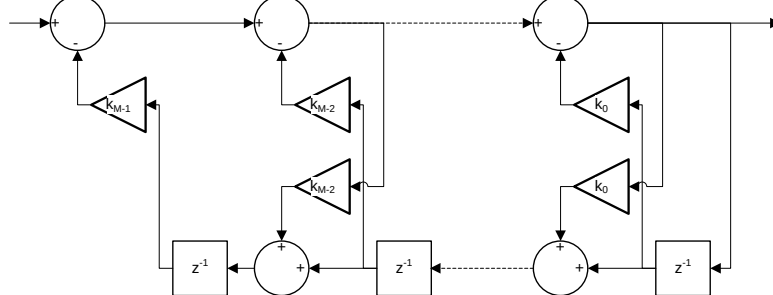
- **Low_Noise.** Performs 32x16-bit multiplications in the feedback branch to attain a precise output sample; feedforward products that update the delay line are still produced by 16x16-bit multiplications, but the full 32-bit result is preserved for each delay element to be used on future iterations.
- **Fast.** Makes use of native 16x16-bit multipliers and allocates 16 bits for a delay element

Note:

Streaming lattice block real IIR filter may be applied to complex data stored in streaming format, which is equivalent to passing a complex signal through a complex IIR with real coefficients. The number of real data streams L as it is seen by the filter implementation must be two times the number of actual complex streams.

Representation 16-bit data, Q15, 16-bit outputs, 32-bit intermediate (LATRS_OPT_LOW_NOISE)
16-bit data, Q15, 16-bit outputs, 16-bit intermediate (LATRS_OPT_FAST)
floating point inputs, floating point out

Algorithm Algorithm consists of applying sequentially M times IIR sections with the structure shown below



Object Allocation `size_t latrs_alloc(int L, int M, int prec)`
`size_t latrsf_alloc( int L, int M )`

Type	Name	Size	Description
Input:			
int	L		number of streams
int	M		Filter order
int	opt		Fixed-point implementation variant selector, LATRS OPT LOW NOISE OR LATRS OPT FAST

Returns: Size of memory in bytes to be allocated

Object Initialization

```
latrs_handle_t latrs_init(void * objmem,
                        int L, int M, int opt,
                        const int16_t * k)
latrsf_handle_t latrsf_init( void * objmem, int L, int M,
                        const float32_t * coef, float32_t gain )
```

Type	Name	Size	Description
Input:			
void*	objmem		Allocated memory block
int	L		Number of streams
int	M		Filter order
int	opt		Fixed-point implementation variant selector, LATRS_OPT_LOW_NOISE or LATRS_OPT_FAST
int16_t, float32_t	k	M	Reflection coefficients, Q15 or floating point

Returns: Handle to the object

Process Samples

```
void latrs_process (latrs_handle_t handle,
                  int16_t * r,
                  const int16_t * x)
void latrsf_process( latrsf_handle_t _latrsf,
                  float32_t * r, const float32_t * x, int N )
```

Type	Name	Size	Description
Output:			
int16_t, float32_t	r	N*L	Output data, Q15 or floating point
int	N		Length of input sample block
Input:			
int16_t, float32_t	x	N*L	Input samples, Q15 or floating point

Returns: None

Returned Value

none

Restrictions

x, r should be aligned on a 32-bytes boundary
M <=8
L number of streams must be a multiple of 16 (latrs) or 8 (latrsf)

Performance Restrictions:

2.2.5 Matlab Code for Conversion of SOS Matrices to IIR Parameters

Below is example Matlab code for conversion of SOS matrices generated by Matlab or another filter synthesis tools to the format of biquad filter functions (para 2.2.1, 2.2.2) for 16-bit fixed point representation. This code computes gains of each stage to minimize potential overflow in the intermediate stage. It assumes total nominal passband gain of 0 dB.

```
%-----
% convert SOS+G to coefficients of IIR filter
% (bqriir/bqciir DF2 functions)
% parameters:
% SOS,G      - SOS matrix and gain vector G
% Fs         - sample rate
% nfft       - FFT length for analysis
% output:
% coef       - vector with coefficients, Q14
```

```

% gain      - biquad gains, Q15
% scale     - final scale factor (Q8)
%-----
function [coef,gain,scale]=cvtSos_bqriir16x16_df2(SOS,G,Fs,nfft)

sz=size(SOS);
M=sz(1);
coef=[];
f=(0:nfft-1)/nfft*(Fs/2);
tf0=sos2freqz(SOS,G,nfft);

Gtotal=prod(G);
G=ones(1,M+1);

% define gain for each stage to provide maximim of tf for
% intermediate outputs <=0.5
% note: for DF2 structure we have to check 2 points:
% b0,b1,b2,a1,a2 and 1,0,0,a1,a2
for m=1:M
    tf=sos2freqz(SOS(1:m,:),[G(1:m) 1],nfft);
    tfmax0=max(abs(tf));
    tf=sos2freqz([SOS(1:m-1,:);1 0 0 1 SOS(m,5:6)], [G(1:m) 1],nfft);
    tfmax1=max(abs(tf));
    tfmax = max(tfmax0,tfmax1);
    G(m)=min(2-(2^-14),0.5/tfmax);
end

% define last stage shift
dg=Gtotal/prod(G);
scale=ceil(dg*256)/256;
dg=dg-scale;
% correct coefficient of the last stage
G(M)=G(M)+dg;
% output b,a
coef=SOS; coef(:,4)=[]; coef=reshape(coef.',1,numel(coef));
% and convert coefficients to given format
coef = int16(round(16384.*coef));
gain = int16(round(16384.*G(1:M)));
scale= int16(round(256.*scale));

% check results and plot final filter response
sos=reshape(double(coef),5,M).'/16384;
sos=[sos(:,1:3) ones(M,1) sos(:,4:5)];
g=double(gain)/16384;
g(M+1)=double(scale)/256;
tf=sos2freqz(sos,g,nfft);
plot(f,20*log10(abs(tf)),f,20*log10(abs(tf0))); ylim([-80 0]); title('transfer function, dB'); grid
on;

% convert SOS/G to frequency response
function [tf]=sos2freqz(SOS,G,nfft)
sz=size(SOS);
M=sz(1);
[b,a]=sos2tf(SOS(1,:),[G(1) G(end)]);
tf=freqz(b,a,nfft);
for m=2:M
    [b,a]=sos2tf(SOS(m,:),[G(m) 1]);
    tf=tf.*freqz(b,a,nfft);
end

```

2.3 Math functions

Vector mathematic functions include vectorized mathematics, trigonometric and transcendental functions.

Typically, they require non-overlapping arguments. Most functions require aligned inputs/outputs and number of data which is a multiple of 16 real 16-bit data or 8 for real floating point data.

A number of library functions supersede standard floating-point mathematical functions defined in `<math.h>`, as listed below:

ANSI function	Scalar function	Vectorized function	reference
<code>acosf</code>	<code>sacosf</code>	<code>vacosf, vfastacosf</code>	2.3.16
<code>asinf</code>	<code>sasinf</code>	<code>vasinf, vfastasinf</code>	2.3.15
<code>atanf</code>	<code>satanf</code>	<code>vatanf, vfastatanf</code>	2.3.18
<code>atan2f</code>	<code>satan2f</code>	<code>vatan2f, vfastatan2f</code>	2.3.17
<code>cosf</code>	<code>scosf</code>	<code>vcosf, vfastcosf</code>	2.3.12
<code>sinf</code>	<code>ssinef</code>	<code>vsinef, vfastsinef</code>	2.3.12
<code>tanf</code>	<code>stanf</code>	<code>vtanf, vfasttanf</code>	2.3.13
<code>cotf</code>	<code>scotf</code>	<code>vcotf, vfastcotf</code>	2.3.14
<code>coshf</code>	<code>scoshf</code>	<code>vcoshf, vfastcoshf</code>	2.3.21
<code>sinhf</code>	<code>ssinhf</code>	<code>vsinhf, vfastsinhf</code>	2.3.20
<code>tanhf</code>	<code>stanhf</code>	<code>vtanhf, vfasttanhf</code>	2.3.19
<code>floorf</code>	<code>sfloorf</code>	<code>vfloorf</code>	2.3.26
<code>ceilf</code>	<code>sceilf</code>	<code>vceilf</code>	2.3.27
<code>fmodf</code>	<code>sfmodf</code>	<code>vfmodf</code>	2.3.7
<code>ldexpf</code>	<code>sldexpf</code>	<code>vldexpf</code>	2.3.10
<code>logf</code>	<code>slognf</code>	<code>vlognf</code>	2.3.8
<code>log10f</code>	<code>slog10f</code>	<code>vlog10f</code>	2.3.8
<code>expf</code>	<code>salognf</code>	<code>valognf</code>	2.3.9
<code>alog10f</code>	<code>salog10f</code>	<code>valog10f</code>	2.3.9
<code>powf</code>	<code>spowf</code>	<code>vpowf</code>	2.3.11
<code>copysignf</code>	<code>scopysignf</code>	<code>vcopysignf</code>	2.3.29
<code>sqrtf</code>	<code>ssqrtf</code>	<code>vsqrtf, vfastsqrtf</code>	2.3.2

All these functions conform to ISO/IEC 9899 standard (commonly referred to as C99) in respect to function semantics, parameters and return value specification, however have slightly different name, so might be used together with functions from standard C-library supported by the core. These floating-point mathematical functions and their vectorized counterparts handle error conditions in a way that differs from general DSP Library approach as stated in 1.8. Aforementioned functions follow the next basic rules:

- Each function executes as if it were a single operation, and may generate any of “invalid”, “overflow” or “divide-by-zero” floating-point exceptions only to reflect the result of that operation.
- A domain error occurs if input argument(s) fall out of the function domain as defined in function specification. In such a case, the function assigns `EDOM` to the integer expression `errno`¹, raises the “invalid” floating-point exception, and returns a quiet NaN.
- NaN as an input argument is a special kind of domain error. Namely, the integer expression `errno` acquires `EDOM` and returned value is a quiet NaN, but the function raises the “invalid” floating-point exception only if the input argument is a *signaling* NaN.
- A pole error occurs if the mathematical function has an exact infinite result as the finite input argument(s) are approached in the limit (for example, `cotf(0.0f)`). The specification of each function lists all potential pole errors. On a pole error, the function assigns `ERANGE` to the integer expression `errno`, raises the “divide-by-zero” floating-point exception and returns the properly signed infinity value.
- A floating-point result overflows if the magnitude of the mathematical result is finite but so large that the target floating-point type cannot represent the mathematical result without extraordinary round-off error (for example, `expf(100.0f)`). If a function detects a floating-point result overflow, it assigns `ERANGE` to the integer expression `errno`, raises the “overflow” floating-point exception and returns the properly signed infinity value.

Vectorized functions obey the same rules of error condition treatment, with the following clarifications:

- When a vectorized floating-point function is applied to data vector(s), it raises the same set of floating-point exceptions as if the corresponding scalar function was sequentially invoked for each slice of input data that comprise the data vector(s).
- `ERANGE` and `EDOM` values are mutually exclusive in respect to evaluation result of the integer expression `errno`. By design, the vectorized functions give the `EDOM` value precedence over the `ERANGE` value. That is, if a vectorized function detects both a domain error and a pole error (or a result overflow) for distinct slices of input data vector(s), then the expression `errno` evaluates to `EDOM` after the function returns. This behavior does not depend on the order the errors actually occurred.

Accuracy for each floating point function is defined specifically for each one but, as general rule, all of them provide accuracy not worse than 2 ULP in specified domain.

For applications where computing speed is critical, the library defines a set of ‘fast’ vectorized floating point functions which use simpler approximations, but still keep reasonable level of accuracy. These ‘fast’ versions differ from normal variants as following:

- accuracy is worse by 1 ULP comparing with non-fast versions
- domain is narrower (for example, for trigonometric functions it is $\pm 256\pi$, not $\pm 32768\pi$)
- functions do not handle inputs which are outside of domain, do not define specific return values for such cases (i.e. for negative inputs for logarithm) or just may return results with worse accuracy
- usually, these functions do not handle infinities

¹ `errno` and its values (`EDOM`, `ERANGE`) are described in ISO/IEC 9899

- functions do not make special handling of signaling NaNs; all NaNs are treated as inputs outside the domain and return value is not defined
- functions do not touch `errno` and may generate some exceptions if input is outside domain

2.3.1 Reciprocal

Description	Evaluate the reciprocal of input value <code>x</code>. Fixed-point routines compute reciprocals for Q15 input data, and return the fractional and exponential parts of the result. Since the reciprocal of a 16-bit Q15 is at least 1.0 in magnitude, functions return fractional part <code>frac</code> in Q(15-<code>exp</code>) format, where <code>exp</code> is the exponential part of the respective result. Full result can be restored in 32-bit Q15 format by sign extending the fractional part to 32 bits and shifting it to the left by <code>exp</code> bit positions. Scalar fixed-point function returns packed 32-bit result, where exponential part resides in 16 MSBs and fractional part is located in 16 LSBs. There are some special cases (non-fast versions) <table><tr><th>x</th><th>result</th><th>API</th></tr><tr><td>$\pm\infty$</td><td>± 0</td><td rowspan="2">floating point API</td></tr><tr><td>± 0</td><td>$\pm\infty$</td></tr><tr><td>0</td><td>not defined</td><td>fixed point API</td></tr></table> Input domain for 'fast' version <code>vfastrecipf()</code>: $x >2.94e-39$, $x <\text{Inf}$ The output value is not defined outside of this range.				x	result	API	$\pm\infty$	± 0	floating point API	± 0	$\pm\infty$	0	not defined	fixed point API																					
x	result	API																																		
$\pm\infty$	± 0	floating point API																																		
± 0	$\pm\infty$																																			
0	not defined	fixed point API																																		
Representation	16-bit inputs, 16-bit fractional part, 16-bit exponent floating point input, floating point output																																			
Accuracy	For fixed point API, accuracy is measured as accuracy of fractional part (mantissa): $1 \cdot (3 \cdot 10^{-5})$ 1 ULP for <code>vrecipf()</code> , <code>srecipf()</code> 2 ULP for <code>vfastrecipf()</code>																																			
Algorithm	for fxd point API point API: $\text{frac}_n \cdot 2^{\text{exp}_n} = 1/x_n, n = \overline{0 \dots N-1}$ for floating point API: $y_n = 1/x_n, n = \overline{0 \dots N-1}$																																			
Prototype	<pre>void vrecipl6 (int16_t * fract, int16_t * exp, const int16_t * x, int N) void vrecipf (float32_t * y, const float32_t * x, int N) void vfastrecipf (float32_t * y, const float32_t * x, int N)</pre>																																			
Arguments	<table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td colspan="4">Input:</td></tr><tr><td>int16_t, float32_t</td><td>x</td><td>N</td><td>Input data</td></tr><tr><td>int</td><td>N</td><td></td><td>Length of vectors</td></tr><tr><td colspan="4">Output:</td></tr><tr><td>int16_t</td><td>frac</td><td>N</td><td>Fractional part of result, Q(15-<code>exp</code>), fixed point API</td></tr><tr><td>int16_t</td><td>exp</td><td>N</td><td>Exponent of result (1...16) , fixed point API</td></tr><tr><td>float32_t</td><td>y</td><td>N</td><td>reciprocal, floating point API</td></tr></table>	Type	Name	Size	Description	Input:				int16_t, float32_t	x	N	Input data	int	N		Length of vectors	Output:				int16_t	frac	N	Fractional part of result, Q(15- <code>exp</code>), fixed point API	int16_t	exp	N	Exponent of result (1...16) , fixed point API	float32_t	y	N	reciprocal, floating point API			
Type	Name	Size	Description																																	
Input:																																				
int16_t, float32_t	x	N	Input data																																	
int	N		Length of vectors																																	
Output:																																				
int16_t	frac	N	Fractional part of result, Q(15- <code>exp</code>), fixed point API																																	
int16_t	exp	N	Exponent of result (1...16) , fixed point API																																	
float32_t	y	N	reciprocal, floating point API																																	
Returned value	none																																			
Restrictions	<table><tr><td>x, fract, exp</td><td rowspan="3">aligned on a 32-bytes boundary not overlapping multiple of 16 (<code>vrecipl6</code>) or 8 (<code>vrecipf</code>, <code>vfastrecipf</code>)</td></tr><tr><td>x, fract, exp</td></tr><tr><td>N</td></tr></table>	x, fract, exp	aligned on a 32-bytes boundary not overlapping multiple of 16 (<code>vrecipl6</code>) or 8 (<code>vrecipf</code> , <code>vfastrecipf</code>)	x, fract, exp	N																															
x, fract, exp	aligned on a 32-bytes boundary not overlapping multiple of 16 (<code>vrecipl6</code>) or 8 (<code>vrecipf</code> , <code>vfastrecipf</code>)																																			
x, fract, exp																																				
N																																				
Scalar Version																																				
Prototype	<pre>uint32_t srecipl6 (int16_t x) float32_t srecipf (float32_t x)</pre>																																			

Arguments	Type	Name	Description
	Input:		
	int16_t, float32_t	x	Input data, Q15
Returned value	For fixed point API, packed value where exponent is in the higher 16-bit part and fractional part is in the lower 16-bit part. For floating point API, result is just reciprocal value.		

2.3.2 Square Root

Description	These functions compute the positive square root of input vector elements. Notes (for non-fast versions): 1. Square root functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 2. For a negative input value, functions raise the "invalid" floating-point exception, assign <code>EDOM</code> to <code>errno</code> and set the respective output value to <code>NaN</code> . 3. Negative zero (<code>-0</code>) is considered as a valid input, the result is also <code>-0</code> . Input domain for 'fast' version <code>vfastsqrtf()</code> : <code>x</code> $\geq +0$, <code>x</code> $< \text{Inf}$ The output value is not defined outside of this range.			
Representation	floating point			
Accuracy	2 ULP for <code>vsqrtf()</code> , <code>ssqrtf()</code> 3 ULP for <code>vfastsqrtf()</code>			
Algorithm	$y_n = \sqrt{x_n}, n = 0 \dots N - 1$			
Prototypes	<pre>void vsqrtf (float32_t * y, const float32_t * x, int N) void vfastsqrtf (float32_t * y, const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	<code>float32_t</code>	<code>x</code>	<code>N</code>	Input data, floating point
	<code>int</code>	<code>N</code>		Length of vectors
	Output:			
	<code>float32_t</code>	<code>y</code>	<code>N</code>	Result, floating point
Returned value	none			
Restrictions	<code>x, y</code> aligned on a 32-bytes boundary <code>x, y</code> not overlapping <code>N</code> multiple of 8			
Scalar Versions				
Prototypes	<code>float32_t ssqrtf (float32_t x)</code>			
Arguments	Type	Name	Description	
	Input:			
	<code>float32_t</code>	<code>x</code>	Input data, floating point	
Returned Value	Result			

2.3.3 Reciprocal Square Root

Description	Routine calculates $1/\sqrt{x}$. For 32-bit signed fixed-point format, the number of fractional bits for input data Q_x is user-defined, provided that it is even. Fixed-point format Q_y for output data is $Q_y = 31 - Q_x/2$. For example, if $Q_x = 30$ then $Q_y = 31 - 30/2 = 16$. Notes (non-fast versions): 1. Fixed-point functions return <code>MIN_INT32 (0x80000000)</code> for a negative or zero input value. 2. Floating-point reciprocal square root conforms to IEEE-754 Std <code>rsqrt</code> operation in respect to signaling error conditions by means of floating-point exceptions. 3. Floating-point reciprocal square root limits the range of allowable input values, as follows: A) If $x < 0$, functions raise the "invalid" floating-point exception, assign <code>EDOM</code> to <code>errno</code> and set output value y to NaN. B) If $x = \pm 0$, functions set output value y to <code>±HUGE_VALF</code> , raise the "divide by zero" floating-point exception, and assign <code>ERANGE</code> to <code>errno</code> . Input domain for 'fast' version <code>vfastrsqrtf()</code> $x > +0, x < \text{Inf}$ The output value is not defined outside of this range.																											
Representation	32-bit normalized input/output floating point																											
Accuracy	For fixed point API: relative accuracy: $1.1 \cdot 10^{-4}$ 2 ULP for <code>vrsqrtf()</code> , <code>srsqrtf()</code> 3 ULP for <code>vfastrsqrtf()</code>																											
Algorithm	$y_n = 1/\sqrt{x_n}, n = 0 \dots N - 1$																											
Prototypes	<pre>void vrsqrt (int32_t * y, const int32_t * x, int N) void vrsqrtf (float32_t * y, const float32_t * x, int N) void vfastrsqrtf (float32_t * y, const float32_t * x, int N)</pre>																											
Arguments	<table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td colspan="4">Input:</td></tr><tr><td>int32_t, float32_t</td><td>x</td><td>N</td><td>Input data, Q_x or floating point</td></tr><tr><td>int</td><td>N</td><td></td><td>Length of vectors</td></tr><tr><td colspan="4">Output:</td></tr><tr><td>int32_t, float32_t</td><td>y</td><td>N</td><td>Result, Q_y or floating point</td></tr></table>	Type	Name	Size	Description	Input:				int32_t, float32_t	x	N	Input data, Q_x or floating point	int	N		Length of vectors	Output:				int32_t, float32_t	y	N	Result, Q_y or floating point			
Type	Name	Size	Description																									
Input:																												
int32_t, float32_t	x	N	Input data, Q_x or floating point																									
int	N		Length of vectors																									
Output:																												
int32_t, float32_t	y	N	Result, Q_y or floating point																									
Returned value	none																											
Restrictions	x, y aligned on a 32-bytes boundary x, y not overlapping N multiple of 16 (<code>vrsqrt</code>) or 8 (<code>vrsqrtf</code> , <code>vfastrsqrtf</code>)																											
Scalar Versions																												
Prototypes	<pre>int32_t srsqrt(int32_t x) float32_t srsqrtf (float32_t x)</pre>																											
Arguments	<table><tr><th>Type</th><th>Name</th><th>Description</th></tr><tr><td colspan="3">Input:</td></tr><tr><td>int32_t, float32_t</td><td>x</td><td>Input data, Q_x or floating point</td></tr></table>	Type	Name	Description	Input:			int32_t, float32_t	x	Input data, Q_x or floating point																		
Type	Name	Description																										
Input:																												
int32_t, float32_t	x	Input data, Q_x or floating point																										
Returned Value	Result																											

2.3.4 Pseudo Floating Point Reciprocal²

Description	Calculates $y=1/x$; input and output values are represented by 7-bit signed exponent and 16-bit signed mantissa, $Q(15+exp)$. If the magnitude of result is so large that the result cannot be represented in the pseudo-floating point format, then the function returns 32767/-32768,-64 (+/-1.8e19) depending on the sign bit of the input mantissa.			
Representation	input/output: pseudo floating point pairs of 7-bit signed exponent and 16-bit signed mantissa			
Accuracy	Relative accuracy: 7.7e-3 (~7 bits of precision) in worst case			
Algorithm	$frac_n \cdot 2^{exp_n} = 1/x_n, n = 0 \dots N-1$			
Prototype	<pre>void vfastrecip16 (int16_t * ymant, int16_t * yexp, const int16_t * xmant, const int16_t * xexp, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t	xmant	N	Mantissa of input values, -32768..32767
	int16_t	xexp	N	Exponent of input values, -64..63
	int	N		Length of vectors
	Output:			
	int16_t	ymant	N	Mantissa of output values, -32768..32767
	int16_t	yexp	N	Exponent of output values, -64..63
Returned value	none			
Restrictions	xmant, ymant, xexp, yexp aligned on a 32-bytes boundary xmant, ymant, xexp, yexp not overlapping N multiple of 16			

2.3.5 Pseudo Floating Point Inverse Square Root³

Description	Calculates $y=1/\sqrt{x}$; input and output values are represented by 16-bit signed mantissa and 7-bit signed exponent, $Q(15+exp)$. Returns -32768,0 (-1.0) for input values with negative mantissa. For zero mantissa the result is 32767,-64 (~1.8e19).			
Representation	input/output: pseudo floating point pairs of 7-bit signed exponent and 16-bit signed mantissa			
Accuracy	Relative accuracy: 2.0e-3 (~9 bits of precision) in the worst case			
Algorithm	$y_n = 1/\sqrt{x_n}, n = 0 \dots N-1$			
Prototypes	<pre>void vfastrsqrt (int16_t * ymant, int16_t * yexp, const int16_t * xmant, const int16_t * xexp, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t	xmant	N	Mantissa of input values, -32768..32767
	int16_t	xexp	N	Exponent of input values, -64..62
	int	N		Length of vectors
	Output:			
	int16_t	ymant	N	Mantissa of output values, -32768,16384..3276
	int16_t	yexp	N	Exponent of output values, -64,-40..31
Returned value	none			

²Requires BBE32EP Advanced precision option

³Requires BBE32EP Advanced precision option

Restrictions	xmant, ymant, xexp, yexp	aligned on a 32-bytes boundary
	xmant, ymant, xexp, yexp	not overlapping
	N	multiple of 16

2.3.6 Division

Description	Fixed-point function returns the fractional and exponential portion of the division result. Fixed-point format for the fractional part is 16-bit Q(15-exp), where exp denotes the respective exponential value. Full division result can be restored in 48-bit Q31 format by sign extending the fractional part to 64 bits and shifting it to the left by 16+exp bit positions. Scalar fixed-point function returns packed 32-bit result, where exponential part resides in 16 MSBs and fractional part is located in 16 LSBs.																																				
	Special cases:																																				
	<table><tr><th>x</th><th>y</th><th>Result</th><th>API</th></tr><tr><td>±0</td><td>±0</td><td>NaN</td><td rowspan="3">floating point API</td></tr><tr><td>x</td><td>±∞</td><td>0</td></tr><tr><td>±∞</td><td>y</td><td></td></tr><tr><td>±∞</td><td>±∞</td><td>NaN</td><td rowspan="2">fixed point API</td></tr><tr><td>0</td><td>0</td><td>not defined</td></tr></table>	x	y	Result	API	±0	±0	NaN	floating point API	x	±∞	0	±∞	y		±∞	±∞	NaN	fixed point API	0	0	not defined															
x	y	Result	API																																		
±0	±0	NaN	floating point API																																		
x	±∞	0																																			
±∞	y																																				
±∞	±∞	NaN	fixed point API																																		
0	0	not defined																																			
Representation	inputs in Q15, output in mantissa/exponent pairs floating point input, floating point output																																				
Accuracy	For fixed point API: 1 LSB of fractional part, that is 1.22*10 ⁻⁴ of relative error in worst case For floating point API: 1 ULP																																				
Algorithm	$frac_n \cdot 2^{exp_n} = x_n / y_n, n = \overline{0...N-1}$ $z_n = x_n / y_n, n = \overline{0...N-1}$																																				
Prototype	<pre>void vdivide (int16_t * fract, int16_t * exp, const int16_t * x, const int16_t * y, int N) void vdividef (float32_t * z, const float32_t * x, const float32_t * y, int N);</pre>																																				
Arguments	<table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td>Input:</td><td></td><td></td><td></td></tr><tr><td>int16_t, float32_t</td><td>x</td><td>N</td><td>Numerator, Q15 or floating point</td></tr><tr><td>int16_t, float32_t</td><td>y</td><td>N</td><td>denominator, Q15 or floating point</td></tr><tr><td>int</td><td>N</td><td></td><td>length of vectors</td></tr><tr><td>Output:</td><td></td><td></td><td></td></tr><tr><td>int16_t</td><td>fract</td><td>N</td><td>Fractional part of results, Q(15-exp); if non-zero, then 8192<= fract <=32768, , fixed point API</td></tr><tr><td>int16_t</td><td>exp</td><td>N</td><td>Exponential part of results, -14..16, fixed point API</td></tr><tr><td>float32_t</td><td>z</td><td>N</td><td>Result. floating point API</td></tr></table>	Type	Name	Size	Description	Input:				int16_t, float32_t	x	N	Numerator, Q15 or floating point	int16_t, float32_t	y	N	denominator, Q15 or floating point	int	N		length of vectors	Output:				int16_t	fract	N	Fractional part of results, Q(15-exp); if non-zero, then 8192<= fract <=32768, , fixed point API	int16_t	exp	N	Exponential part of results, -14..16, fixed point API	float32_t	z	N	Result. floating point API
Type	Name	Size	Description																																		
Input:																																					
int16_t, float32_t	x	N	Numerator, Q15 or floating point																																		
int16_t, float32_t	y	N	denominator, Q15 or floating point																																		
int	N		length of vectors																																		
Output:																																					
int16_t	fract	N	Fractional part of results, Q(15-exp); if non-zero, then 8192<= fract <=32768, , fixed point API																																		
int16_t	exp	N	Exponential part of results, -14..16, fixed point API																																		
float32_t	z	N	Result. floating point API																																		
Returned value	none																																				
Restrictions	x, y, z, fract, exp x, y, z, fract, exp N	aligned on a 32-bytes boundary not overlapping Multiple of 16 (vdivide) or 8 (vdividef)																																			
Scalar Version																																					
Prototype	uint32_t sdivide (int16_t x, int16_t y) float32_t sdividef(float32_t x, float32_t y)																																				

Arguments	Type	Name	Description
	Input:		
	int16_t, float32_t	x	Numerator, Q15 or floating point
	int16_t, float32_t	y	Denominator, Q15 or floating point
Returned Value	Scalar fixed point function returns packed 32-bit value where exponent is in the higher 16-bit part and fractional part resides in the lower 16-bit part		

2.3.7 Remainder

Description	These functions compute the floating-point remainder that results from dividing the first argument by the second argument. The result is less than the second argument and has the same sign as the first argument. Notes: 1. Modulus functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 2. Modulus functions limit the range of allowable input values, as follows: A) If $ x/y \geq 2^{24}$, then the respective result z is set to NaN B) If $x == \pm\infty$ and/or $y == \pm 0$, functions set output value z to NaN, raise the "invalid" floating-point exception, and assign <code>EDOM</code> to <code>errno</code> . Special cases:																											
	<table><tr><th>x</th><th>y</th><th>Result</th><th>Note</th></tr><tr><td>± 0</td><td>y</td><td>± 0</td><td>$y \neq 0$</td></tr><tr><td>x</td><td>$\pm\infty$</td><td>x</td><td>$x \neq \pm\infty$</td></tr><tr><td>$\pm\infty$</td><td>y</td><td>NaN</td><td>any y</td></tr><tr><td>x</td><td>± 0</td><td>NaN</td><td>any x</td></tr><tr><td>x</td><td>y</td><td>NaN</td><td>$x/y \geq 2^{24}$</td></tr></table>	x	y	Result	Note	± 0	y	± 0	$y \neq 0$	x	$\pm\infty$	x	$x \neq \pm\infty$	$\pm\infty$	y	NaN	any y	x	± 0	NaN	any x	x	y	NaN	$ x/y \geq 2^{24}$			
x	y	Result	Note																									
± 0	y	± 0	$y \neq 0$																									
x	$\pm\infty$	x	$x \neq \pm\infty$																									
$\pm\infty$	y	NaN	any y																									
x	± 0	NaN	any x																									
x	y	NaN	$ x/y \geq 2^{24}$																									
Representation	floating point input, floating point output																											
Accuracy	2 ULP																											
Algorithm	$z_n = remainder(x_n, y_n), n = 0 \dots N - 1$																											
Prototype	<pre>void vfmodf (float32_t * z, const float32_t * x, const float32_t * y, int N)</pre>																											
Arguments	Type	Name	Size	Description																								
	Input:																											
	float32_t	x	N	Numerator																								
	float32_t	y	N	denominator																								
	int	N		length of vectors																								
	Output:																											
	float32_t	z	N	remainder																								
Returned value	none																											
Restrictions	x, y, z x, y, z N		aligned on a 32-bytes boundary not overlapping multiple of 8																									
Scalar Version																												
Prototype	float32_t sfmodf (float32_t x, float32_t y)																											
Arguments	Type	Name	Description																									
	Input:																											

	int16_t, float32_t	x	Numerator
	int16_t, float32_t	y	Denominator
Returned Value	remainder		

2.3.8 Logarithm

Description	Different types of logarithm (base-2, natural, base-10).			
	For fixed point API, results are represented in the Q11 format, with 0x8000 (-32768) being returned for a negative of zero argument. Functions assume Q16.15 for the input data format.			
	Examples:			
	log2 (65536) -> 2048 [1.0]			
	log2 (2147483647) -> 32767 [15.9995]			
log2 (1) -> -30720 [-15.0]				
Notes:				
1. Fixed-point Functions return -32768 for a negative or zero input.				
2. Floating-point functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions.				
3. Floating point functions limit the range of allowable input values:				
A) If $x < 0$, the result is set to NaN, <code>errno</code> is assigned the value <code>EDOM</code> , and "invalid" floating-point exception is raised				
B) If $x == 0$, the result is set to minus infinity, <code>errno</code> is assigned the value <code>ERANGE</code> , and "divide-by-zero" floating-point exception is raised				
Representation	input in Q16.15, output in Q4.11 floating point input, floating point output			
Accuracy	1 LSB for the fixed-point functions 2 ULP for the floating-point functions			
Algorithm	$z_n = \log_K x_n, n = \overline{0 \dots N-1}, K = 2, e, 10$			
Prototypes	void vlog2 (int16_t * y, const int32_t * x, int N) void vlogn (int16_t * y, const int32_t * x, int N) void vlog10(int16_t * y, const int32_t * x, int N) void vlog2f (float32_t * y, const float32_t * x, int N) void vlognf (float32_t * y, const float32_t * x, int N) void vlog10f (float32_t * y, const float32_t * x, int N)			
Arguments	Type	Name	Size	Description
	Input:			
	int32_t, float32_t	x	N	Input data, Q15 or floating point
	int	N		Length of vectors
	Output:			
	int16_t, float32_t	y	N	Result, Q11 or floating point
Returned value	none			
Restrictions	x, y aligned on a 32-bytes boundary			
	x, y not overlapping			
	N multiple of 16 (vlog2,vlogn,vlog10) or 8 (vlog2f,vlognf,vlog10f)			
Scalar Versions				
Prototypes	int16_t slog2 (int32_t x) int16_t slogn (int32_t x) int16_t slog10(int32_t x) float32_t slog2f (float32_t x) float32_t slognf (float32_t x) float32_t slog10f (float32_t x)			

Arguments	Type	Name	Description
	Input:		
	int32_t, float32_t	x	Input data, Q15 or floating point
Returned Value	Result, Q11 or floating point		

2.3.9 Antilogarithm and Exponential

Description	These functions compute base-10 or natural antilogarithm (exponential) of input data. Note: Antilogarithm functions conform to ANSI C requirements on standard math library functions in respect to treatment of errno and floating-point exceptions.			
Representation	floating point input, floating point output			
Accuracy	2 ULP for the floating-point functions			
Algorithm	$z_n = K^{x_n}, n = 0 \dots N-1, K = e, 10$			
Prototypes	<pre>void valog10f (float32_t * y, const float32_t * x, int N) void valognf (float32_t * y, const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	int	N		Length of vectors
	Output:			
	float32_t	y	N	Result
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	N	multiple of 8		
Scalar Versions				
Prototypes	<pre>float32_t salog10f (float32_t x) float32_t salognf (float32_t x)</pre>			
Arguments	Type	Name	Description	
	Input:			
	float32_t	x	Input data	
Returned Value	Result			

2.3.10 Modify the Exponent of a Floating-Point Number

Description	These functions multiply input values by 2^n , where n is an exponent adjustment term. If the result overflows, functions return <code>HUGE_VALF</code> with the proper sign. If, on the contrary, the result underflows, functions return zero with proper sign. Note: Exponent modification functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions.			
Representation	floating point input, floating point output			
Accuracy	exact			
Algorithm	$z_m = x_m \cdot 2^{n_m}, m = 0 \dots N-1$			
Prototype	<pre>void vldexpf (float32_t* y, const float32_t* x, const int32_t* n, int N)</pre>			

Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	int32_t	n	N	Exponent adjustment terms
	int	N		Length of vectors
	Output:			
	float32_t	y	N	Result
Returned value	none			
Restrictions	x, y, n	aligned on a 32-bytes boundary		
	x, y, n	not overlapping		
	N	multiple of 8		
Scalar Versions				
Prototype	float32_t sldexpf (float32_t x, int32_t n)			
Arguments	Type	Name	Description	
	Input:			
	float32_t	x	Input data	
	int32_t	n	Exponent adjustment term	
Returned Value	Result			

2.3.11 Raise To a Power

Description	These functions compute the value of the first argument x raised to the power of the second argument y. Notes: 1. Raise to a power functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 2. If $x < 0$ is finite, y is finite and not an integer value, then the respective result z is set to NaN, <code>errno</code> is assigned the value <code>EDOM</code>, and the "invalid" floating-point exception is raised. Special cases: <table> <tr> <th>x</th><th>y</th><th>Result</th><th>Extra conditions</th></tr> <tr> <td>± 0</td><td>y</td><td>$\pm \infty$</td><td>odd $y < 0$</td></tr> <tr> <td>± 0</td><td>y</td><td>$+\infty$</td><td>even $y < 0$</td></tr> <tr> <td>± 0</td><td>y</td><td>± 0</td><td>odd $y > 0$</td></tr> <tr> <td>± 0</td><td>y</td><td>0</td><td>even $y > 0$</td></tr> <tr> <td>± 1</td><td>$\pm \infty$</td><td>1</td><td></td></tr> <tr> <td>1</td><td>y</td><td>1</td><td>any y including NaN</td></tr> <tr> <td>x</td><td>± 0</td><td>1</td><td>any x including NaN</td></tr> <tr> <td>x</td><td>y</td><td>NaN</td><td>finite $x < 0$ and finite non-integer y (see note 2 above)</td></tr> <tr> <td>x</td><td>$-\infty$</td><td>$+\infty$</td><td>$x < 1$</td></tr> <tr> <td>x</td><td>$-\infty$</td><td>0</td><td>$x > 1$</td></tr> <tr> <td>x</td><td>$+\infty$</td><td>0</td><td>$x < 1$</td></tr> <tr> <td>x</td><td>$+\infty$</td><td>$+\infty$</td><td>$x > 1$</td></tr> <tr> <td>$-\infty$</td><td>y</td><td>-0</td><td>y an odd integer < 0</td></tr> <tr> <td>$-\infty$</td><td>y</td><td>0</td><td>$y < 0$ and not an odd integer</td></tr> <tr> <td>$-\infty$</td><td>y</td><td>$-\infty$</td><td>y an odd integer > 0</td></tr> <tr> <td>$-\infty$</td><td>y</td><td>$+\infty$</td><td>$y > 0$ and not an odd integer</td></tr> <tr> <td>$+\infty$</td><td>y</td><td>0</td><td>$y < 0$</td></tr> <tr> <td>$+\infty$</td><td>y</td><td>$+\infty$</td><td>$y > 0$</td></tr> </table>			x	y	Result	Extra conditions	± 0	y	$\pm \infty$	odd $y < 0$	± 0	y	$+\infty$	even $y < 0$	± 0	y	± 0	odd $y > 0$	± 0	y	0	even $y > 0$	± 1	$\pm \infty$	1		1	y	1	any y including NaN	x	± 0	1	any x including NaN	x	y	NaN	finite $x < 0$ and finite non-integer y (see note 2 above)	x	$-\infty$	$+\infty$	$ x < 1$	x	$-\infty$	0	$ x > 1$	x	$+\infty$	0	$ x < 1$	x	$+\infty$	$+\infty$	$ x > 1$	$-\infty$	y	-0	y an odd integer < 0	$-\infty$	y	0	$y < 0$ and not an odd integer	$-\infty$	y	$-\infty$	y an odd integer > 0	$-\infty$	y	$+\infty$	$y > 0$ and not an odd integer	$+\infty$	y	0	$y < 0$	$+\infty$	y	$+\infty$	$y > 0$
x	y	Result	Extra conditions																																																																												
± 0	y	$\pm \infty$	odd $y < 0$																																																																												
± 0	y	$+\infty$	even $y < 0$																																																																												
± 0	y	± 0	odd $y > 0$																																																																												
± 0	y	0	even $y > 0$																																																																												
± 1	$\pm \infty$	1																																																																													
1	y	1	any y including NaN																																																																												
x	± 0	1	any x including NaN																																																																												
x	y	NaN	finite $x < 0$ and finite non-integer y (see note 2 above)																																																																												
x	$-\infty$	$+\infty$	$ x < 1$																																																																												
x	$-\infty$	0	$ x > 1$																																																																												
x	$+\infty$	0	$ x < 1$																																																																												
x	$+\infty$	$+\infty$	$ x > 1$																																																																												
$-\infty$	y	-0	y an odd integer < 0																																																																												
$-\infty$	y	0	$y < 0$ and not an odd integer																																																																												
$-\infty$	y	$-\infty$	y an odd integer > 0																																																																												
$-\infty$	y	$+\infty$	$y > 0$ and not an odd integer																																																																												
$+\infty$	y	0	$y < 0$																																																																												
$+\infty$	y	$+\infty$	$y > 0$																																																																												
Representation	floating point input, floating point output																																																																														
Accuracy	2 ULP under condition that $ y \leq 100$																																																																														

Algorithm	$z_n = x_n^{y_n}, n = 0 \dots N - 1$			
Prototype	void vpowf (float32_t * z, const float32_t * x, const float32_t * y, int N)			
Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	float32_t	y	N	Power values
	int	N		Length of vectors
	Output:			
	float32_t	z	N	Result
Returned value	none			
Restrictions	x, y, z	aligned on a 32-bytes boundary		
	x, y, z	not overlapping		
	N	multiple of 8		
Scalar Versions				
Prototype	float32_t spowf (float32_t x, float32_t y)			
Arguments	Type	Name	Description	
	Input:			
	float32_t	x	Input data	
	float32_t	y	Power value	
Returned value	Result			

2.3.12 Sine/Cosine

Description	These functions compute sine or cosine of input data. Fixed point functions calculate $\sin(\pi \cdot x)$ or $\cos(\pi \cdot x)$ for the number written in the Q15 format. and return result in the Q15 format as well. Notes for non-fast versions: 1. Floating-point functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 2. Floating-point functions require that input value belongs to the closed range $[-102940.0, 102940.0]$, otherwise the respective result is NaN. Input domain for 'fast' versions <code>vfastsinef()</code>, <code>vfastcosf()</code> $x < 804.2477$ The output value is not defined outside of this range or accuracy is degraded
Representation	Q15 input (normalized by π), Q15 output floating point input, floating point output
Accuracy	2 LSB for <code>vsine()</code>, <code>vcos()</code>, <code>ssine()</code>, <code>scos()</code>, 2 ULP for <code>vsinef()</code>, <code>vcosf()</code>, <code>ssinef()</code>, <code>scosf()</code>, 3 ULP for <code>vfastsinef()</code>, <code>vfastcosf()</code>
Algorithm	Fixed point API: $y_n = \sin(\pi x_n), n = \overline{0 \dots N-1} \text{ or }$ $y_n = \cos(\pi x_n), n = \overline{0 \dots N-1}$ Floating point API: $y_n = \sin(x_n), n = \overline{0 \dots N-1} \text{ or }$ $y_n = \cos(x_n), n = \overline{0 \dots N-1}$

Prototypes		<pre>void vsine (int16_t * y, const int16_t * x, int N) void vcos (int16_t * y, const int16_t * x, int N) void vsinef (float32_t * y, const float32_t * x, int N) void vcosf (float32_t * y, const float32_t * x, int N) void vfastsinef (float32_t * y, const float32_t * x, int N) void vfastcosf (float32_t * y, const float32_t * x, int N)</pre>		
Arguments	Type	Name	Size	Description
	Input:			
	int16_t, float32_t	x	N	Input data, Q15 or floating point
	int	N		Length of vectors
	Output:			
	int16_t, float32_t	y	N	Result, Q15 or floating point
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	N	multiple of 16 (vsine,vcos) or 8 (vsinef,vcosf, vfastsinef, vfastcosf)		
Scalar Versions				
Prototypes		<pre>int16_t ssine(int16_t x) int16_t scos(int16_t x) float32_t ssinef (float32_t x) float32_t scosf (float32_t x)</pre>		
Arguments	Type	Name	Description	
	Input:			
	int16_t, float32_t	x	Input data, Q15 or floating point	
Returned Value	Result, Q15 or floating point			

2.3.13 Tangent

Description	These functions compute tangent of input data. Fixed point functions calculate $\tan(\pi \cdot x)$ for the number written in the Q15 format. and return result in the Q15 format as well. Notes for non-fast versions: 1. Fixed-point function result is not defined if input value x is $\pm\pi/2$ (± 8192 in Q15 normalized by π). 2. Floating-point functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 3. Floating-point functions require that input value belongs to the closed range $[-9099.0, 9099.0]$, otherwise the respective result is NaN. Input domain for 'fast' version <code>vfasttanf()</code>: $x < 804.2477$ The output value is not defined outside of this range or accuracy is degraded													
Representation	Q15 input (normalized by π), Q16.15 output floating point input, floating point output													
Accuracy	For the fixed-point functions, accuracy depends on the input value x, as shown in the table below: <table border="1"> <thead> <tr> <th>Range of x</th><th>Absolute error</th><th>Relative error</th></tr> </thead> <tbody> <tr> <td>$[-\pi/4, \pi/4]$</td><td>1 (Q15)</td><td></td></tr> <tr> <td>$[\pi/4, 7\pi/16]$</td><td>15 (Q15)</td><td>$4.6e-4$</td></tr> <tr> <td>$[7\pi/16, 3\pi/4]$</td><td>242 (Q15)</td><td>$1.5e-3$</td></tr> </tbody> </table> 2 ULP for <code>vtanf()</code>, <code>stanf()</code> 3 ULP for <code>vfasttanf()</code>		Range of $ x $	Absolute error	Relative error	$[-\pi/4, \pi/4]$	1 (Q15)		$[\pi/4, 7\pi/16]$	15 (Q15)	$4.6e-4$	$[7\pi/16, 3\pi/4]$	242 (Q15)	$1.5e-3$
Range of $ x $	Absolute error	Relative error												
$[-\pi/4, \pi/4]$	1 (Q15)													
$[\pi/4, 7\pi/16]$	15 (Q15)	$4.6e-4$												
$[7\pi/16, 3\pi/4]$	242 (Q15)	$1.5e-3$												

Algorithm	Fixed point API: $y_n = \tan(\pi x_n), n = 0 \dots N - 1$ Floating point API: $y_n = \tan(x_n), n = 0 \dots N - 1$			
Prototypes	<pre>void vtan (int32_t * y, const int16_t * x, int N) void vtanf (float32_t * y, const float32_t * x, int N) void vfasttanf (float32_t * y, const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t, float32_t	x	N	Input data, Q15 or floating point
	int	N		Length of vectors
	Output:			
	int32_t, float32_t	y	N	Result, Q16.15 or floating point
Returned value	none			
Restrictions	x, y x, y N		aligned on a 32-bytes boundary not overlapping multiple of 16 (vtan) or 8 (vtanf, vfasttanf)	
Scalar Versions				
Prototypes	<pre>int32_t stan (int16_t x) float32_t stanf (float32_t x)</pre>			
Arguments	Type	Name	Description	
	Input:			
	int16_t, float32_t	x	Input data, Q15 or floating point	
Returned Value	Result, Q16.15 or floating point			

2.3.14 Cotangent

Description	These functions compute cotangent of input data. Notes for non-fast versions: 1. Cotangent functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 2. Cotangent functions require that input value belongs to the closed range [-9099.0,9099.0], otherwise the respective result is NaN. Input domain for 'fast' version <code>vfastcotf()</code> : $ x < 804.2477, x \neq 0$ The output value is not defined outside of this range or accuracy is degraded.
Representation	floating point input, floating point output
Accuracy	2 ULP for <code>vcotf()</code> , <code>scotf()</code> 3 ULP for <code>vfastcotf()</code>
Algorithm	$y_n = \cot(x_n), n = 0 \dots N-1$
Prototypes	<pre>void vcotf (float32_t * y, const float32_t * x, int N) void vfastcotf (float32_t * y, const float32_t * x, int N)</pre>

Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	int	N		Length of vectors
	Output:			
	float32_t	y	N	Result
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary not overlapping multiple of 8		
	x, y			
	N			

Scalar Versions

Prototypes float32_t scotf (float32_t x)

Arguments	Type	Name	Description
	Input:		
	int16_t, float32_t	x	Input data
Returned Value	Result		

2.3.15 Arcsine

Description	These functions compute arcsine of input data. Functions output is in radians. Notes for non-fast versions: 1. Arcsine functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 2. Input values should belong to $[-1, 1]$, otherwise functions raise the "invalid" floating-point exception, assign <code>EDOM</code> to <code>errno</code> and set the respective output value to <code>NaN</code> . Input domain for 'fast' version <code>vfastasinf()</code> : $ x \leq 1$ The output value is not defined outside of this range			
Representation	floating point input, floating point output			
Accuracy	2 ULP for <code>vasinf()</code> , <code>sasinf()</code> 3 ULP for <code>vfastasinf()</code>			
Algorithm	$y_n = \arcsin(x_n), n = 0 \dots N - 1$			
Prototypes	<pre>void vasinf (float32_t * y, const float32_t * x, int N) void vfastasinf (float32_t * y, const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	<code>float32_t</code>	<code>x</code>	<code>N</code>	Input data
	<code>int</code>	<code>N</code>		Length of vectors
	Output:			
	<code>float32_t</code>	<code>y</code>	<code>N</code>	Result
Returned value	none			
Restrictions	<code>x, y</code> <code>x, y</code> <code>N</code>	aligned on a 32-bytes boundary not overlapping multiple of 8		
Scalar Versions				
Prototypes	<code>float32_t scotf (float32_t x)</code>			

Arguments	Type	Name	Description
	Input:		
	float32_t	x	Input data
Returned Value	Result		

2.3.16 Arccosine

Description	These functions compute arccosine of input data. Functions output is in radians. Notes for non-fast versions: 1. Arccosine functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 3. Input values should belong to $[-1, 1]$, otherwise functions raise the "invalid" floating-point exception, assign <code>EDOM</code> to <code>errno</code> and set the respective output value to NaN. Input domain for 'fast' version <code>vfastacosf()</code> : $ x \leq 1$ The output value is not defined outside of this range			
Representation	floating point input, floating point output			
Accuracy	2 ULP for <code>vacosf()</code> , <code>sacosf()</code> 3 ULP for <code>vfastacosf()</code>			
Algorithm	$y_n = \arccos(x_n), n = 0 \dots N - 1$			
Prototypes	<pre>void vacosf (float32_t * y, const float32_t * x, int N) void vfastacosf (float32_t * y, const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	int	N		Length of vectors
	Output:			
	float32_t	y	N	Result
Returned value	none			
Restrictions	x, y		aligned on a 32-bytes boundary	
	x, y		not overlapping	
	N		multiple of 8	
Scalar Versions				
Prototypes	<code>float32_t sacosf (float32_t x)</code>			
Arguments	Type	Name	Description	
	Input:			
	int16_t, float32_t	x	Input data	
Returned Value	Result			

2.3.17 Full Arctangent

Description	For fixed point API, these functions calculate four quadrant arctangent of complex argument x and return $\arg(x)/\pi$ in Q15 format. Q15 is also assumed for argument. If real and imaginary components are zero, the result is also zero. For floating point API, these functions calculate full-quadrant arctangent of ratio y/x and output results in radians. Notes for non-fast versions: 1. Full arctangent functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 2. Full arctangent functions assign <code>EDOM</code> to <code>errno</code> whenever $x==0$ and $y==0$. The resulting value depends on signs of x and y, see the Special Cases below. Input domain for 'fast' version: $1.1755e-038 < x < \text{Inf}$ $1.1755e-038 < y < \text{Inf}$ The output value is not defined outside of this range or accuracy is degraded. Special cases: <table><thead><tr><th>y</th><th>x</th><th>Result</th><th>Extra conditions</th><th>Note</th></tr></thead><tbody><tr><td>± 0</td><td>-0</td><td>$\pm \pi$</td><td></td><td rowspan="11">non-fast versions only</td></tr><tr><td>± 0</td><td>$+0$</td><td>± 0</td><td></td></tr><tr><td>± 0</td><td>x</td><td>$\pm \pi$</td><td>$x < 0$</td></tr><tr><td>± 0</td><td>x</td><td>± 0</td><td>$x > 0$</td></tr><tr><td>y</td><td>± 0</td><td>$-\pi/2$</td><td>$y < 0$</td></tr><tr><td>y</td><td>± 0</td><td>$\pi/2$</td><td>$y > 0$</td></tr><tr><td>$\pm y$</td><td>$-\infty$</td><td>$\pm \pi$</td><td>finite $y > 0$</td></tr><tr><td>$\pm y$</td><td>$+\infty$</td><td>± 0</td><td>finite $y > 0$</td></tr><tr><td>$\pm \infty$</td><td>x</td><td>$\pm \pi/2$</td><td>finite x</td></tr><tr><td>$\pm \infty$</td><td>$-\infty$</td><td>$\pm 3\pi/4$</td><td></td></tr><tr><td>$\pm \infty$</td><td>$+\infty$</td><td>$\pm \pi/4$</td><td></td></tr></tbody></table>	y	x	Result	Extra conditions	Note	± 0	-0	$\pm \pi$		non-fast versions only	± 0	$+0$	± 0		± 0	x	$\pm \pi$	$x < 0$	± 0	x	± 0	$x > 0$	y	± 0	$-\pi/2$	$y < 0$	y	± 0	$\pi/2$	$y > 0$	$\pm y$	$-\infty$	$\pm \pi$	finite $y > 0$	$\pm y$	$+\infty$	± 0	finite $y > 0$	$\pm \infty$	x	$\pm \pi/2$	finite x	$\pm \infty$	$-\infty$	$\pm 3\pi/4$		$\pm \infty$	$+\infty$	$\pm \pi/4$	
y	x	Result	Extra conditions	Note																																															
± 0	-0	$\pm \pi$		non-fast versions only																																															
± 0	$+0$	± 0																																																	
± 0	x	$\pm \pi$	$x < 0$																																																
± 0	x	± 0	$x > 0$																																																
y	± 0	$-\pi/2$	$y < 0$																																																
y	± 0	$\pi/2$	$y > 0$																																																
$\pm y$	$-\infty$	$\pm \pi$	finite $y > 0$																																																
$\pm y$	$+\infty$	± 0	finite $y > 0$																																																
$\pm \infty$	x	$\pm \pi/2$	finite x																																																
$\pm \infty$	$-\infty$	$\pm 3\pi/4$																																																	
$\pm \infty$	$+\infty$	$\pm \pi/4$																																																	
Representation	Q15 input, Q15 output floating point input, floating point output																																																		
Accuracy	fixed point: 1 LSB 2 ULP for <code>vatan2f()</code> , <code>satan2f()</code> 3 ULP for <code>vfastatan2f()</code>																																																		
Algorithm	For fixed point API: $z_n = \arg(x_n) / \pi, n = 0 \dots N - 1$ For floating point API: $z_n = \arg(x_n + i \cdot y_n), n = 0 \dots N - 1$																																																		
Prototype	<pre>void vatan2_16 (int16_t * z, const int16_t * x, int N) void vatan2f (float32_t* z, const float32_t* y, const float32_t * x, int N) void vfastatan2f(float32_t* z, const float32_t* y, const float32_t * x, int N)</pre>																																																		
Arguments	Fixed point API: <table><thead><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr></thead><tbody><tr><td colspan="4">Input:</td></tr><tr><td>int16_t</td><td>x</td><td>2*N</td><td>Complex input data. Real and imaginary parts are interleaved and real part goes first.</td></tr><tr><td>int</td><td>N</td><td></td><td>Length of vectors</td></tr><tr><td colspan="4">Output:</td></tr><tr><td>int16_t</td><td>z</td><td>N</td><td>Result</td></tr></tbody></table>	Type	Name	Size	Description	Input:				int16_t	x	2*N	Complex input data. Real and imaginary parts are interleaved and real part goes first.	int	N		Length of vectors	Output:				int16_t	z	N	Result																										
Type	Name	Size	Description																																																
Input:																																																			
int16_t	x	2*N	Complex input data. Real and imaginary parts are interleaved and real part goes first.																																																
int	N		Length of vectors																																																
Output:																																																			
int16_t	z	N	Result																																																

Floating point API:			
Type	Name	Size	Description
Input:			
float32_t	x	N	Real part of input
float32_t	y	N	Imaginary part of input
int	N		Length of vectors
Output:			
int16_t	z	N	Result
Returned value	none		
Restrictions	z, x, y	aligned on a 32-bytes boundary	
	z, x, y	not overlapping	
	N	multiple of 16 (vatan2_16) or 8 (vatan2f, vfastatan2f)	
Scalar Versions			
Prototype	int16_t satan2_16 (int16_t re,int16_t im) float32_t satan2f (float32_t im, float32_t re)		
Arguments	Type	Name	Description
	Input:		
	int16_t, float32_t	re	Real part of input
	int16_t, float32_t	im	Imaginary part of input
Returned value	Result		

2.3.18 Arctangent

Description

These functions compute the principal value of arctangent. For fixed point API, function's input is Q15 format and output is scaled by $\pi/4$.

Notes for non-fast versions:

1. These functions are much faster than full-quadrant arctangent `atan2`, so they are preferable when the full phase is not required.
2. Floating-point functions conform to ANSI C requirements on standard math library functions in respect to treatment of `errno` and floating-point exceptions.

Special cases:

Input	Result	Note
$+\infty$	$\pi/2$	floating point API
$-\infty$	$-\pi/2$	

Input domain for 'fast' version `vfastatanf()`:
 $|x| < \text{Inf}$

The output value is not defined outside of this range.

Representation	Q15 input, Q15 output floating point input, floating point output
Accuracy	1 LSB for fixed point fixed point functions 1 ULP for <code>vatanf()</code>, <code>satanf()</code> 2 ULP for <code>vfastatanf()</code>
Algorithm	For fixed point API: $z_n = \arctan(x_n) / (\pi / 4), n = \overline{0 \dots N - 1}$ For floating point API: $z_n = \arctan(x_n), n = \overline{0 \dots N - 1}$

Prototype	<pre>void vatan16 (int16_t * z, const int16_t * x, int N) void vatanf (float32_t * z, const float32_t * x, int N) void vfastatanf(float32_t * z, const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t, float32_t	x	N	Input data
	int	N		Length of vectors
	Output:			
	int16_t, float32_t	z	N	Results
Returned value	none			
Restrictions	z, x		aligned on a 32-bytes boundary	
	z, x		not overlapping	
	N		multiple of 16 (vatan16) or 8 (vatanf, vfastatanf)	

Scalar Version

Prototype	<pre>int16_t satan16 (int16_t x) float32_t satanf (float32_t x)</pre>		
Arguments	Type	Name	Description
	Input:		
	int16_t, float32_t	x	input data
Returned Value	Result		

2.3.19 Hyperbolic Tangent

Description	These functions compute hyperbolic tangent of input data. Note for non-fast version: Hyperbolic tangent functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. Input domain for 'fast' version <code>vfasttanhf()</code> : $ x < \text{Inf}$ The output value is not defined outside of this range.			
Representation	floating point input, floating point output			
Accuracy	2 ULP for <code>vtanhf()</code> , <code>stanhf()</code> 3 ULP for <code>vfasttanhf()</code>			
Algorithm	$y_n = \tanh(x_n), n = \overline{0 \dots N - 1}$			
Prototypes	<pre>void vtanhf (float32_t * y, const float32_t * x, int N) void vfasttanhf (float32_t * y, const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	<code>float32_t</code>	<code>x</code>	<code>N</code>	Input data
	<code>int</code>	<code>N</code>		Length of vectors
	Output:			
	<code>float32_t</code>	<code>y</code>	<code>N</code>	Result
Returned value	none			
Restrictions	<code>x, y</code> <code>x, y</code> <code>N</code>	aligned on a 32-bytes boundary not overlapping multiple of 8		

Prototypes	float32_t stanhf (float32_t x)		
Arguments	Type	Name	Description
	Input:		
	float32_t	x	Input data
Returned Value	Result		

2.3.20 Hyperbolic Sine

Description	These functions compute hyperbolic sine of input data. Notes for non-fast versions: 1. Hyperbolic sine functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions. 2. Due to limited dynamic range of single precision floating-point format, hyperbolic sine result for an input value <code>x</code> such that <code> x >89.41599</code> is <code>sign(x)*HUGE_VALF</code> . Input domain for 'fast' version <code>vfastsinhf()</code> : <code> x <89.41599</code> The output value is not defined outside of this range.			
Representation	floating point input, floating point output			
Accuracy	2 ULP for <code>vsinhf()</code> , <code>ssinhf()</code> 3 ULP for <code>vfastsinhf()</code>			
Algorithm	$y_n = \sinh(x_n), n = \overline{0 \dots N - 1}$			
Prototypes	<pre>void vsinhf (float32_t * y, const float32_t * x, int N) void vfastsinhf (float32_t * y, const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	<code>float32_t</code>	<code>x</code>	<code>N</code>	Input data
	<code>int</code>	<code>N</code>		Length of vectors
	Output:			
	<code>float32_t</code>	<code>y</code>	<code>N</code>	Result
Returned value	none			
Restrictions	<code>x, y</code> <code>x, y</code> <code>N</code>		aligned on a 32-bytes boundary not overlapping multiple of 8	
Scalar Versions				
Prototypes	<code>float32_t ssinhf (float32_t x)</code>			
Arguments	Type	Name	Description	
	Input:			
	<code>float32_t</code>	<code>x</code>	Input data	
Returned Value	Result			

2.3.21 Hyperbolic Cosine

Description	These functions compute hyperbolic cosine of input data. Notes for non-fast versions: 1. Hyperbolic cosine functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions.
--------------------	---

2. Due to limited dynamic range of single precision floating-point format, hyperbolic cosine result for an input value x such that $|x| > 89.41599$ is `HUGE_VALF`.

Input domain for 'fast' version `vfastcoshf()`:
 $|x| < 89.41599$

The output value is not defined outside of this range.

Representation	floating point input, floating point output			
Accuracy	2 ULP for <code>vcoshf()</code> , <code>scoshf()</code> 3 ULP for <code>vfastcoshf()</code>			
Algorithm	$y_n = \cosh(x_n), n = 0 \dots N - 1$			
Prototypes	<pre>void vcoshf (float32_t * y, const float32_t * x, int N) void vfastcoshf (float32_t * y, const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	<code>float32_t</code>	<code>x</code>	<code>N</code>	Input data
	<code>int</code>	<code>N</code>		Length of vectors
	Output:			
	<code>float32_t</code>	<code>y</code>	<code>N</code>	Result
Returned value	none			
Restrictions	<code>x, y</code> <code>x, y</code> <code>N</code>		aligned on a 32-bytes boundary not overlapping multiple of 8	
Scalar Versions				
Prototypes	<code>float32_t scoshf (float32_t x)</code>			
Arguments	Type	Name	Description	
	Input:			
	<code>int16_t</code> , <code>float32_t</code>	<code>x</code>	Input data	
Returned Value	Result			

2.3.22 Integer to Floating Value Conversion

Description	These functions convert integer input values to floating values and scale them by 2^t .			
Representation	signed 32-bit integer input, floating point output			
Accuracy	exact			
Algorithm	$y_n = 2^t \cdot x_n, n = 0 \dots N-1$			
Prototypes	<code>void vint2float (float32_t * y, const int32_t * x, int t, int N)</code>			
Arguments	Type	Name	Size	Description
	Input:			
	<code>int32_t</code>	<code>x</code>	<code>N</code>	Input data
	<code>int</code>	<code>t</code>		scale factor
	<code>int</code>	<code>N</code>		Length of vectors
	Output:			
	<code>float32_t</code>	<code>y</code>	<code>N</code>	conversion result
Returned value	none			

Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	N	multiple of 8		
	t	Must belong to [-126, 126]		
Scalar Versions				
Prototypes				
float32_t sint2float (int32_t x, int t)				
Arguments	Type	Name	Description	
	Input:			
	float32_t	x	Input data	
	int	t	scale factor	
Returned value				
conversion result				

2.3.23 Floating To Integer Value Conversion

Description	These functions scale input floating input values by 2^{-t} and convert them to integers with saturation.			
Representation	floating point input, signed 32-bit integer output			
Accuracy	exact			
Algorithm	$y_n = 2^{-t} \cdot x_n, n = \overline{0 \dots N - 1}$			
Prototypes	void vfloat2int (int32_t * y, const float32_t * x, int t, int N)			
Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	int	t		scale factor
	int	N		Length of vectors
	Output:			
	int32_t	y	N	conversion result
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	N	multiple of 8		
	t	Must belong to $[-126, 126]$		
Scalar Versions				
Prototypes	int32_t sfloat2int (float32_t x, int t)			
Arguments	Type	Name	Description	
	Input:			
	float32_t	x	Input data	
	int	t	scale factor	
Returned Value	conversion result			

2.3.24 Fixed Point to Half_precision Floating Point Conversion

Description	Function converts 16-bit signed fixed-point data with user-defined point position to 16-bit half-precision floating-point format (IEEE 754-2008 binary16).																														
	NOTES: - IEEE 754-2008 binary16 encoding format provides 1 sign bit, 10 trailing significand bits and 5 exponent bits. - If absolute input value $x \cdot 2^{-q}$ exceeds 65504, the result is saturated to $\pm\text{Inf}$ (0x7C00 or 0xFC00). In particular, if point position is less than or equal to -16 ($q \leq -16$), any nonzero fixed-point value is converted to $\pm\text{Inf}$. - If absolute input value belongs to the closed range of $[2^{-24}, 2^{-15}]$, then the conversion results in a subnormal half-precision floating-point number as defined by IEEE 754-2008. - If input value $x \cdot 2^{-q}$ belongs to the open range of $(-2^{-24}, 2^{-24})$, then it is converted to zero of original sign (0x0000 or 0x8000). Specifically, any fixed-point value is converted to ± 0 when the point position is greater than or equal to 40 ($q \geq 40$). - If input value cannot be exactly represented in half-precision floating-point format, then it is rounded to the next representable value toward zero.																														
Algorithm	see IEEE 754-2008																														
Prototype	<pre>void fix2hf (int16_t* y, const int16_t * x, int16_t q, int N)</pre> <table border="1"> <thead> <tr> <th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr> </thead> <tbody> <tr> <td colspan="4">Input:</td></tr> <tr> <td>int16_t</td><td>x</td><td>N</td><td>input fixed-point data, Q(q)</td></tr> <tr> <td>int16_t</td><td>q</td><td></td><td>input data representation</td></tr> <tr> <td>int</td><td>N</td><td></td><td>size of input/output arrays</td></tr> <tr> <td colspan="4">Output:</td></tr> <tr> <td>int16_t</td><td>y</td><td>N</td><td>output half-precision floating point values</td></tr> </tbody> </table>			Type	Name	Size	Description	Input:				int16_t	x	N	input fixed-point data, Q(q)	int16_t	q		input data representation	int	N		size of input/output arrays	Output:				int16_t	y	N	output half-precision floating point values
Type	Name	Size	Description																												
Input:																															
int16_t	x	N	input fixed-point data, Q(q)																												
int16_t	q		input data representation																												
int	N		size of input/output arrays																												
Output:																															
int16_t	y	N	output half-precision floating point values																												
Returned value	none																														
Restrictions	x, y aligned on a 32-bytes boundary x, y not overlapping N multiple of 16																														

2.3.25 Half-Precision Floating Point to Fixed Point Conversion

Description	Description: Function converts 16-bit half-precision floating-point data (IEEE 754-2008 binary16 format) to 16-bit signed fixed-point format with user-defined point position. NOTES: ▪ $+\text{-Inf}$ is converted to 32767 or -32768, respectively. ▪ NaN values (zero exponent field, non-zero significand field) are treated as $+\text{-Inf}$ depending on the sign bit. ▪ If a finite input value is too large in its magnitude to be represented in the fixed-point format with user-specified point position, then it is saturated to -32768 or 32767 depending on the input sign. ▪ If the input value cannot be exactly represented in fixed-point format with user-specified point position, then it is rounded to the next representable value toward $-\text{Inf}$. ▪ The sensible range for the point position q is $[-15, 39]$. That is, for $q \leq -16$ the result of conversion is always zero or -1 (depending on the input sign), unless the input value is $+\text{-Inf}$ or NaN. For $q \geq 39$, any non-zero floating-point value is converted to either -32768 or 32767, depending on the input sign. ▪ Given an arbitrary floating-point value, the optimum point position for the floating-to-fixed conversion is $q = 14 - e$, where e is the unbiased exponent of the floating-point value. For a normal positive floating-point value, the optimum point position provides a normalized fixed-point result. For a normal negative value on input, the number of redundant sign bits for the conversion result is at most 1.																														
Algorithm	see IEEE 754-2008																														
Prototype	<pre>void hf2fix (int16_t* y, const int16_t * x, int16_t q, int N)</pre> <table border="1"> <thead> <tr> <th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr> </thead> <tbody> <tr> <td colspan="4">Input:</td></tr> <tr> <td>int16_t*</td><td>x</td><td>N</td><td>input half-precision floating point values</td></tr> <tr> <td>int16_t</td><td>q</td><td></td><td>output data representation</td></tr> <tr> <td>int</td><td>N</td><td></td><td>size of input/output arrays</td></tr> <tr> <td colspan="4">Output:</td></tr> <tr> <td>int16_t*</td><td>y</td><td>N</td><td>output fixed-point data, $Q(q)$</td></tr> </tbody> </table>			Type	Name	Size	Description	Input:				int16_t*	x	N	input half-precision floating point values	int16_t	q		output data representation	int	N		size of input/output arrays	Output:				int16_t*	y	N	output fixed-point data, $Q(q)$
Type	Name	Size	Description																												
Input:																															
int16_t*	x	N	input half-precision floating point values																												
int16_t	q		output data representation																												
int	N		size of input/output arrays																												
Output:																															
int16_t*	y	N	output fixed-point data, $Q(q)$																												
Returned value	none																														
Restrictions	x, y aligned on a 32-bytes boundary x, y not overlapping N multiple of 16																														

2.3.26 Floating-point Floor

Description	These functions return the largest integral value that is not greater than an input value. Note: Floor functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions.														
Representation	floating point input, floating point output														
Accuracy	exact														
Algorithm	$y_n = \text{floor}(x_n), n = 0 \dots N - 1$														
Prototypes	<pre>void vffloorf (float32_t * y, const float32_t * x, int N)</pre>														
Arguments	<table border="1"> <thead> <tr> <th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr> </thead> <tbody> <tr> <td colspan="4">Input:</td></tr> <tr> <td>float32_t</td><td>x</td><td>N</td><td>Input data</td></tr> </tbody> </table>	Type	Name	Size	Description	Input:				float32_t	x	N	Input data		
Type	Name	Size	Description												
Input:															
float32_t	x	N	Input data												

	int	N		Length of vectors
	Output:			
	float32_t	y	N	result
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	N	multiple of 8		
Scalar Versions				
Prototypes	float32_t sfloorf (float32_t x)			
Arguments	Type	Name	Description	
	Input:			
	float32_t	x	Input data	
Returned Value	result			

2.3.27 Floating-point Ceil

Description	These functions return the smallest integral value that is not smaller than an input value. Note: Ceil functions conform to ANSI C requirements on standard math library functions in respect to treatment of <code>errno</code> and floating-point exceptions.			
Representation	floating point input, floating point output			
Accuracy	exact			
Algorithm	$y_n = \text{ceil}(x_n), n = 0 \dots N - 1$			
Prototypes	void vceilf (float32_t * y, const float32_t * x, int N)			
Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	int	N		Length of vectors
	Output:			
	float32_t	y	N	result
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	N	multiple of 8		
Scalar Versions				
Prototypes	float32_t sceilf (float32_t x)			
Arguments	Type	Name	Description	
	Input:			
	float32_t	x	Input data	
Returned value	result			

2.3.28 Absolute Value

Description	These functions return absolute value for each input value. Note: In terms of IEEE-754 Std, this is a quiet-computational operation which treats floating-point numbers and NaNs alike, and does not raise any floating-point exceptions.			
-------------	---	--	--	--

Representation	floating point input, floating point output			
Accuracy	exact			
Algorithm	$y_n = x_n , n = 0 \dots N - 1$			
Prototypes	void vabsf (float32_t * y, const float32_t * x, int N)			
Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	int	N		Length of vectors
	Output:			
	float32_t	y	N	result
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	N	multiple of 8		
Scalar Versions				
Prototypes	float32_t sabsf (float32_t x)			
Arguments	Type	Name	Description	
	Input:			
	float32_t	x	Input data	
Returned Value	result			

2.3.29 Copy Sign

Description	These functions copy the sign bit of the second argument to the first argument and return the result. Note: In terms of IEEE-754 Std, this is a quiet-computational operation which treats floating-point numbers and NaNs alike, and does not raise any floating-point exceptions.			
Representation	floating point input, floating point output			
Accuracy	exact			
Algorithm	$z_n = x_n \cdot \text{sgn}(y_n), n = 0 \dots N-1$			
Prototypes	void vcopysignf(float32_t* z, const float32_t* x, const float32_t* y, int N)			
Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	float32_t	y	N	sign value
	int	N		Length of vectors
	Output:			
	float32_t	z	N	result
Returned value	none			
Restrictions	x, y, z x, y, z N		aligned on a 32-bytes boundary not overlapping multiple of 8	
Scalar Versions				
Prototypes	float32_t scopysignf (float32_t x , float32_t y)			

Arguments	Type	Name	Description
	Input:		
	float32_t	x	Input data
	float32_t	y	sign value
Returned value	result		

2.3.30 Clipping

Description	These functions limit the absolute value of inputs. If magnitude of input value is less than the second argument then it is left unchanged. Otherwise it is replaced by absolute value of the second argument or its negation, depending on the sign of the input value.			
Representation	floating point input, floating point output			
Accuracy	exact			
Algorithm	$z_n = \min(\max(x_n, - y_n), y_n), n = 0 \dots N-1$			
Prototypes	void vclipf (float32_t * z, const float32_t * x, float32_t y, int N)			
Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	float32_t	y	N	limiting value
	int	N		Length of vectors
	Output:			
	float32_t	z	N	result
Returned value	none			
Restrictions	x, y, z	aligned on a 32-bytes boundary		
	x, y, z	not overlapping		
	N	multiple of 8		
Scalar Versions				
Prototypes	float32_t sclipf (float32_t x, float32_t y)			
Arguments	Type	Name	Description	
	Input:			
	float32_t	x	Input data	
	float32_t	y	limiting value	
Returned value	result			

2.3.31 Count One Bits in a Word

Description	Functions count the number of one bits in the number or each number of a vector			
Representation	16-bit input 32-bit input			
Accuracy	exact			
Algorithm	$z = \sum_j bit_j(x)$			
Prototypes	void vcountones16 (int16_t *z, const int16_t *x, int N) void vcountones32 (int32_t *z, const int32_t *x, int N)			
Arguments	Type	Name	Size	Description
	Input:			

	int16_t, int32_t	x	N	Input data
	int	N		Length of vectors
	Output:			
	int16_t, int32_t	z	N	number of set bits in each input word
Returned value	none			
Restrictions	x, z x, z N aligned on a 32-bytes boundary not overlapping multiple of 8 for vcountones32() multiple of 16 for vcountones16()			
Scalar Versions				
Prototypes	int countones16(int16_t x) int countones32(int32_t x)			
Arguments	Type	Name	Description	
	Input:			
	int16_t, int32_t	x	Input data	
Returned value	number of set bits			

2.3.32 Average of Two Arguments

Description	These functions compute the average of two arguments. Input domain for 'fast' version <code>vfastavgf()</code> : $ x+y < \text{Inf}$ The output value is not defined outside of this range.			
Representation	floating point input, floating point output			
Accuracy	1 ULP			
Algorithm	$z_n = 0.5 \cdot x_n + 0.5 \cdot y_n, n = \overline{0 \dots N-1}$ for non-fast variant $z_n = 0.5 \cdot (x_n + y_n), n = \overline{0 \dots N-1}$ for fast version			
Prototypes	<pre>void vavgf (float32_t * z, const float32_t * x, const float32_t * y, int N) void vfastavgf(float32_t * z, const float32_t * x, const float32_t * y, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	float32_t	x	N	Input data
	float32_t	y	N	Input data
	int	N		Length of vectors
	Output:			
	float32_t	z	N	result
Returned value	none			
Restrictions	x, y, z x, y, z N		aligned on a 32-bytes boundary not overlapping multiple of 8	
Scalar Versions				
Prototypes	<pre>float32_t savgf (float32_t x, float32_t y)</pre>			

Arguments	Type	Name	Description
	Input:		
	float32_t	x	Input data
	float32_t	y	Input data
Returned value	result		

2.4 Complex Math Functions

Complex math functions include general-purpose complex mathematics. Some complex functions have ‘faster’ versions obeying the same rules as for math functions described in para 2.3.

2.4.1 Complex Division

Description	Fixed-point function returns the fractional and exponential portion of real and imaginary components of the division result. Fixed-point format for the fractional part is 16-bit Q_(15-exp), where exp denotes the respective exponential value. Full division result can be restored in 48-bit Q31 format by sign extending the fractional part to 64 bits and shifting it to the left by 16+exp bit positions Special cases: <table border="1"> <thead> <tr> <th>x</th><th>y</th><th>Result</th><th>API</th><th>Note</th></tr> </thead> <tbody> <tr> <td>±0</td><td>±0</td><td>NaN</td><td>floating point API</td><td></td></tr> <tr> <td>x</td><td>±∞</td><td>0</td><td rowspan="3">floating point API</td><td rowspan="3">for non-fast version only</td></tr> <tr> <td>±∞</td><td>y</td><td>±∞</td></tr> <tr> <td>±∞</td><td>±∞</td><td>NaN</td></tr> <tr> <td>0</td><td>0</td><td>not defined</td><td>fixed point API</td><td></td></tr> </tbody> </table>				x	y	Result	API	Note	±0	±0	NaN	floating point API		x	±∞	0	floating point API	for non-fast version only	±∞	y	±∞	±∞	±∞	NaN	0	0	not defined	fixed point API	
x	y	Result	API	Note																										
±0	±0	NaN	floating point API																											
x	±∞	0	floating point API	for non-fast version only																										
±∞	y	±∞																												
±∞	±∞	NaN																												
0	0	not defined	fixed point API																											
	Note: quotient of infinite number to finite number may have NaN in its real or imaginary part depending on signs of input arguments Input domain for ‘fast’ version is limited by usage of direct formula without normalization of inputs: $z = (x \cdot y') / (y \cdot y')$ so input domain is $real(x) \cdot real(y) < Inf$ $imag(x) \cdot imag(y) < Inf$ $real(y) \cdot real(y) < Inf$ $imag(y) \cdot imag(y) < Inf$ $1.1755e-038 < real(x) \cdot real(y) + imag(x) \cdot imag(y) < Inf$ $1.1755e-038 < real(y) \cdot real(y) + imag(y) \cdot imag(y) < Inf$ The output value is not defined outside of this range or accuracy is degraded.																													
Representation	inputs in Q15, output in mantissa/exponent pairs floating point input, floating point output																													
Accuracy	For fixed point API: 1.5 LSB of fractional part For floating point API: 2 ULP																													
Algorithm	$frac_n \cdot 2^{exp_n} = x_n / y_n, n = 0 \dots N-1$ $z_n = x_n / y_n, n = 0 \dots N-1$																													
Prototype	<pre>void vdividecx (complex_fract16 * fract, int16_t * exp, const complex_fract16 * x, const complex_fract16 * y, int N) void vdividecxf(complex_float * z, const complex_float * x, const complex_float * y, int N) void vfastdividecxf(complex_float * z, const complex_float * x, const complex_float * y, int N)</pre>																													
Arguments	Type	Name	Size	Description																										
	Input:																													
	complex_fract16, complex_float	x	N	Numerator, Q15 or floating point																										
	complex_fract16, complex_float	y	N	denominator, Q15 or floating point																										
	int	N		length of vectors																										
	Output:																													

	complex_fract16, complex_float	fract	N	Fractional part of results, Q(15-exp); if non-zero, then 8192<= fract <=32768, , fixed point API
	int16_t	exp	N	Exponential part of results, -14..16, fixed point API
	complex_fract16, complex_float	z	N	Quotient. floating point API
Returned value	none			
Restrictions	x,y,z,fract,exp		aligned on a 32-bytes boundary	
	x,y,z,fract,exp		not overlapping	
	N		multiple of 8 for vdividecx() and 4 for vdividecxf(), vfastdividecxf()	
Scalar Version				
Prototype	complex_float cdivf(complex_float x, complex_float y)			
Arguments	Type	Name	Description	
	Input:			
	complex_float	x	Numerator	
	complex_float	y	Denominator	
Returned value	Quotient			

2.4.2 Polar to Cartesian Conversion

Description	Function converts a pair of magnitude and phase values into a pair of coordinates on the complex plane. Fixed point position for the output data may be adjusted by the bit shift control argument <i>sh</i>. Note for non-fast version: For the floating-point functions, input phase data should belong to the range [-102940.0, 102940.0], otherwise the conversion result is (0,0). Input domain for 'fast' version vfastpolarf() a <804.2477 The output value is not defined outside of this range or accuracy is degraded.
Representation	Q15 input, Q15 output floating point input, floating point output
Accuracy	9.2*10 ⁻⁵ (3 in Q15). for vpolar() 2 ULP for vpolarf(), spolarf() 3 ULP for vfastpolarf()
Algorithm	Fixed point API: $z_n = \frac{1}{2^{sh}} \cdot r_n \cdot e^{j \cdot \pi \cdot a_n}, n = \overline{0 \dots N-1}$ Floating point API: $z_n = r_n \cdot e^{j \cdot a_n}, n = \overline{0 \dots N-1}$
Prototype	<pre>void vpolar (complex_fract16 * z, const fract16 * r, const fract16 * a, int sh, int N) void vpolarf (complex_float * z, const float32_t * r, const float32_t * a, int N) void vfastpolarf (complex_float * z, const float32_t * r, const float32_t * a, int N)</pre>

Arguments	Type	Name	Size	Description
	Input:			
	int16_t, float32_t	r	N	magnitudes, Q _x or floating point
	int16_t, float32_t	a	N	Phase angles, Q15 or floating point
	int	sh		Right bit shift amount, 0..15 (for fixed point API only)
	int	N		size of input/output arrays
	Output:			
	complex_fract16, complex_float	z	N	Output complex values, Q _(x-sh) of floating point
Returned value	none			
Restrictions	z, r, a z, r, a sh N	aligned on a 32-bytes boundary not overlapping should be in range 0...15 multiple of 16 (vpolar) or 8 (vpolarf)		
Scalar Version				
Prototype	complex_float spolarf(float32_t r, float32_t a)			
Arguments	Type	Name	Description	
	Input:			
	float32_t	r	magnitude	
	float32_t	a	phase angle	
Returned value	Output complex value			

2.4.3 Cartesian to Polar Conversion

Description	These functions convert Cartesian coordinates of points on the complex plane to the polar system (magnitude and phase) Input domain for 'fast' version vfastcartesianf(): $ \text{real}(x) > 1.1755\text{e-}038$ $ \text{imag}(x) > 1.1755\text{e-}038$ $1.1755\text{e-}038 < \text{real}(x) * \text{real}(x) + \text{imag}(x) * \text{imag}(x) < \text{Inf}$ The output value is not defined outside of this range or accuracy is degraded.			
Representation	floating point input, floating point output			
Accuracy	2 ULP for vcartesianf(), scartesianf() 3 ULP for vfastcartesianf()			
Algorithm	$z_n, \varphi_n : x_n \equiv z_n \cdot e^{j\varphi_n}, n = 0 \dots N-1$			
Prototype	<pre>void vcartesianf (float32_t* z, float32_t* phase, const complex_float * x, int N) void vfastcartesianf(float32_t* z, float32_t* phase, const complex_float * x, int N)</pre>			
Arguments	Type	Name	Size	Description
Input:				
	complex_float	x	N	Cartesian coordinates
	int	N		size of input/output arrays
Output:				
	float32_t	z	N	Magnitude data
	float32_t	phase	N	Phase angles
Returned value	none			

Restrictions	x, z, phase	aligned on a 32-bytes boundary	
	x, z, phase	not overlapping	
	N	multiple of 8	
Scalar Version			
Prototype		float32_t scartesianf(complex_float x, float32_t * phase)	
Arguments	Type	Name	Description
	Input:		
	float32_t	x	magnitude
	Output:		
	float32_t	phase[1]	phase angle
Returned Value		Output magnitude	

2.4.4 Complex Vector Magnitude

Description	Function computes the complex vector magnitude. Input domain for 'fast' version <code>vfastcabsf()</code> : $1.1755\text{e-}038 < \text{real}(x) * \text{real}(x) + \text{imag}(x) * \text{imag}(x) < \text{Inf}$ The output value is not defined outside of this range or accuracy is degraded.
Representation	Q15 input, Q15 output floating point input, floating point output
Accuracy	1 LSB for <code>vcabs</code> 2 ULP for <code>vcabsf, scabsf</code> 3 ULP for <code>vfastcabsf</code>
Algorithm	Fixed point API: $z_n = \sqrt{2^{sh} x_n ^2}, n = \overline{0 \dots N - 1}$ Floating point API: $z_n = \sqrt{ x_n ^2}, n = \overline{0 \dots N - 1}$
Prototype	<pre>void vcabs (fract16 * z, const complex_fract16 * x, int sh, int N) void vcabsf (float32_t * z, const complex_float * x, int N) void vfastcabsf (float32_t * z, const complex_float * x, int N)</pre>

Type	Name	Size	Description
Input:			
int16_t, float32_t	x	N	input complex data, Q15 or floating point
int	sh		(vcabs only) bi-directional shift control, an even integer from the range [-2,30]. Positive value corresponds to a left shift
int	N		size of input/output arrays
Output:			
complex_fract16, complex_float	z	N	output magnitude values, Q(15+sh/2) or floating point

NOTE:
fixed point function makes additional left shift BEFORE taking the square root. Thus, user may better control the representation or output data

Returned value	none
----------------	------

Restrictions	z, x	aligned on a 32-bytes boundary	
	z, x	not overlapping	
	sh	>=0	
	N	multiple of 16 (<code>vcabs</code>) or 8 (<code>vcabsf</code> , <code>vfastcabsf</code>)	
Scalar Version			
Prototype		float32_t scabsf(complex_float x)	
Arguments	Type	Name	Description
	Input:		
	complex_float	x	input complex data
Returned Value	Output magnitude		

2.4.5 Conjugate of Complex Number

Description	Function computes the complex vector magnitude.			
Representation	floating point input, floating point output			
Accuracy	exact			
Algorithm	$z_n = x_n^*, n = \overline{0 \dots N-1}$			
Prototype	void vconjf(complex_float * z, const complex_float * x, int N)			
	Type	Name	Size	Description
	Input:			
	complex_float	x	N	input complex data
	int	N		size of input/output arrays
	Output:			
	complex_float	z	N	conjugated results
Returned value	none			
Restrictions	z, x	aligned on a 32-bytes boundary		
	z, x	not overlapping		
	N	multiple of 4		
Scalar Version				
Prototype	complex_float sconjf(complex_float x)			
Arguments	Type	Name	Description	
	Input:			
	complex_float	x	input complex data	
Returned Value	conjugated result			

2.4.6 Phase Angle of Complex Number

Description	These functions compute phase angles of input complex numbers and return the results in radians: $[-\pi, \pi]$		
	Input domain for 'fast' version <code>vfastargf</code> :		
	$1.1755e-038 < \text{real}(x) < \text{Inf}$		
	$1.1755e-038 < \text{imag}(x) < \text{Inf}$		
	The output value is not defined outside of this range or accuracy is degraded.		
Representation	floating point input, floating point output		

Accuracy	2 ULP for <code>vargf</code> , <code>sargf</code> 3 ULP for <code>vfastargf</code>																										
Algorithm	$\varphi_n : x_n \equiv x_n e^{j\varphi_n}, n = \overline{0.., N-1}$																										
Prototype	<pre>void vargf (float32_t * z, const complex_float * x, int N) void vfastargf(float32_t * z, const complex_float * x, int N)</pre>																										
	<table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td colspan="4">Input:</td></tr><tr><td><code>complex_float</code></td><td><code>x</code></td><td><code>N</code></td><td>Complex numbers</td></tr><tr><td><code>int</code></td><td><code>N</code></td><td></td><td>size of input/output arrays</td></tr><tr><td colspan="4">Output:</td></tr><tr><td><code>float32_t</code></td><td><code>z</code></td><td><code>N</code></td><td>Phase angles</td></tr></table>			Type	Name	Size	Description	Input:				<code>complex_float</code>	<code>x</code>	<code>N</code>	Complex numbers	<code>int</code>	<code>N</code>		size of input/output arrays	Output:				<code>float32_t</code>	<code>z</code>	<code>N</code>	Phase angles
Type	Name	Size	Description																								
Input:																											
<code>complex_float</code>	<code>x</code>	<code>N</code>	Complex numbers																								
<code>int</code>	<code>N</code>		size of input/output arrays																								
Output:																											
<code>float32_t</code>	<code>z</code>	<code>N</code>	Phase angles																								
Returned value	none																										
Restrictions	<table><tr><td><code>z, x</code></td><td>aligned on a 32-bytes boundary</td></tr><tr><td><code>z, x</code></td><td>not overlapping</td></tr><tr><td><code>N</code></td><td>multiple of 8</td></tr></table>			<code>z, x</code>	aligned on a 32-bytes boundary	<code>z, x</code>	not overlapping	<code>N</code>	multiple of 8																		
<code>z, x</code>	aligned on a 32-bytes boundary																										
<code>z, x</code>	not overlapping																										
<code>N</code>	multiple of 8																										
Scalar Version																											
Prototype	<code>float32_t sargf(complex_float x)</code>																										
Arguments	<table><tr><th>Type</th><th>Name</th><th>Description</th></tr><tr><td colspan="3">Input:</td></tr><tr><td><code>complex_float</code></td><td><code>x</code></td><td>Complex numbers</td></tr></table>			Type	Name	Description	Input:			<code>complex_float</code>	<code>x</code>	Complex numbers															
Type	Name	Description																									
Input:																											
<code>complex_float</code>	<code>x</code>	Complex numbers																									
Returned Value	phase angle																										

2.4.7 Complex Exponential

Description	These functions compute complex number lying on the unit circle from input phase data. Note for (non-fast versions): For the floating-point functions, input phase data should belong to the range $[-102940.0, 102940.0]$, otherwise the respective result is $0+0j$ Input domain for 'fast' version <code>cfastbexpf()</code> $\text{phase} < 804.2477$ The output value is not defined outside of this range or accuracy is degraded.
Representation	Q15 input, Q15 output floating point input, floating point output
Accuracy	$3.7 \cdot 10^{-4}$ (12 in Q15).for <code>cbexp</code> 2 ULP for <code>cbexpf</code> , <code>sexpf</code> 3 ULP for <code>cfastbexpf</code>
Algorithm	Fixed point API: $z_n = 2^{sh} e^{j \cdot \pi \cdot a_n}, n = \overline{0...N-1}$ Floating point API: $z_n = e^{j \cdot a_n}, n = \overline{0...N-1}$

Prototype	void cbexp (complex_fract16 * z, const int16_t * phase, int sh, int N) void cbexpf (complex_float * z, const float32_t * phase, int N) void cfastbexpf (complex_float * z, const float32_t * phase, int N)																															
	<table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td>Input:</td><td></td><td></td><td></td></tr><tr><td>int16_t, float32_t</td><td>phase</td><td>N</td><td>Phase values, Q15 or floating point</td></tr><tr><td></td><td></td><td></td><td>(cbexp only) left bit shift amount, [0..15]</td></tr><tr><td>int</td><td>N</td><td></td><td>size of input/output arrays</td></tr><tr><td>Output:</td><td></td><td></td><td></td></tr><tr><td>complex_fract16, complex float</td><td>z</td><td>N</td><td>Results, Q15 or floating point</td></tr></table>	Type	Name	Size	Description	Input:				int16_t, float32_t	phase	N	Phase values, Q15 or floating point				(cbexp only) left bit shift amount, [0..15]	int	N		size of input/output arrays	Output:				complex_fract16, complex float	z	N	Results, Q15 or floating point			
Type	Name	Size	Description																													
Input:																																
int16_t, float32_t	phase	N	Phase values, Q15 or floating point																													
			(cbexp only) left bit shift amount, [0..15]																													
int	N		size of input/output arrays																													
Output:																																
complex_fract16, complex float	z	N	Results, Q15 or floating point																													
Returned value	none																															
Restrictions	z, x aligned on a 32-bytes boundary z, x not overlapping N multiple of 16 (cbexp) or 8 (cbexpf, cfastbexpf)																															
Scalar Version																																
Prototype	complex_float sexpf (float32_t phase)																															
Arguments	<table><tr><th>Type</th><th>Name</th><th>Description</th></tr><tr><td>Input:</td><td></td><td></td></tr><tr><td>complex_float</td><td>phase</td><td>Phase value</td></tr></table>	Type	Name	Description	Input:			complex_float	phase	Phase value																						
Type	Name	Description																														
Input:																																
complex_float	phase	Phase value																														
Returned Value	complex result																															

2.4.8 Complex Exponential with Linearly Evolving Phase

Description	These functions compute a series of complex numbers lying on the unit circle, with the phase angle being incremented for each successive complex number. Notes: 1. <code>cbexplin()</code> function computes each complex value from a full-precision linearly evolving angle, thus the function is able to generate long data sequences with no risk of phase errors accumulation. 2. <code>cbexplin_fast()</code> function is approximately twice faster but less accurate because it performs successive rotations by small angle on each iteration. Avoid using this function for long sequences or tiny phase increments. 3. For the non-fast floating-point functions (<code>cbexplinf</code>, <code>cexplinf</code>) , the phase (initial and updated) should belong to the range <code>[-102940.0, 102940.0]</code>, otherwise the respective results is <code>0+0j</code>. 4. Scalar function <code>cexplinf</code> uses the initial phase angle (<code>phase0</code>) to compute a single complex number, and returns the once incremented phase value. Input domain for 'fast' version (<code>cbfastexplinf</code>) $ \text{phase} < 804.2477$ for all initial and updated phases The output value is not defined outside of this range or accuracy is degraded.										
Representation	Q15 input, Q15 output floating point input, floating point output										
Accuracy	<table><tr><td><code>cbexplin</code></td><td>$3.7 \cdot 10^{-4}$ (12 in Q15)</td></tr><tr><td><code>cbexplin_fast</code></td><td>$\sim (\text{floor}(n/8)+1) \cdot 3.7 \cdot 10^{-4}$, $n=0 \dots N-1$</td></tr><tr><td><code>cbexplinf, cexplinf</code></td><td>2 ULP</td></tr><tr><td><code>cbfastexplinf</code></td><td>3 ULP</td></tr></table>			<code>cbexplin</code>	$3.7 \cdot 10^{-4}$ (12 in Q15)	<code>cbexplin_fast</code>	$\sim (\text{floor}(n/8)+1) \cdot 3.7 \cdot 10^{-4}$, $n=0 \dots N-1$	<code>cbexplinf, cexplinf</code>	2 ULP	<code>cbfastexplinf</code>	3 ULP
<code>cbexplin</code>	$3.7 \cdot 10^{-4}$ (12 in Q15)										
<code>cbexplin_fast</code>	$\sim (\text{floor}(n/8)+1) \cdot 3.7 \cdot 10^{-4}$, $n=0 \dots N-1$										
<code>cbexplinf, cexplinf</code>	2 ULP										
<code>cbfastexplinf</code>	3 ULP										

Algorithm	Fixed point API $z_n = e^{j \cdot \pi \cdot (a_0 + n \cdot \Delta)}, n = 0 \dots N - 1$ Floating point API: $z_n = e^{j \cdot (a_0 + n \cdot \Delta)}, n = 0 \dots N - 1$																														
Prototype	<pre>int16_t cbexplin(complex_fract16* z, int16_t phase0, int16_t delta, int N) int16_t cbexplin_fast(complex_fract16* z, int16_t phase0, int16_t delta,int N) float32_t cbexplinf(complex_float* z, float32_t phase0, float32_t delta,int N) float32_t cbfastexplinf(complex_float* z,float32_t phase0,float32_t delta,int N)</pre> <table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td>Input:</td><td></td><td></td><td></td></tr><tr><td>int16_t, float32_t</td><td>phase0</td><td></td><td>initial phase, Q15 or floating point</td></tr><tr><td>int16_t, float32_t</td><td>delta</td><td></td><td>phase increment, Q15 or floating point</td></tr><tr><td>int</td><td>N</td><td></td><td>size of output array</td></tr><tr><td>Output:</td><td></td><td></td><td></td></tr><tr><td>complex_fract16, complex_float</td><td>z</td><td>N</td><td>output complex values, Q15 or floating point</td></tr></table>			Type	Name	Size	Description	Input:				int16_t, float32_t	phase0		initial phase, Q15 or floating point	int16_t, float32_t	delta		phase increment, Q15 or floating point	int	N		size of output array	Output:				complex_fract16, complex_float	z	N	output complex values, Q15 or floating point
Type	Name	Size	Description																												
Input:																															
int16_t, float32_t	phase0		initial phase, Q15 or floating point																												
int16_t, float32_t	delta		phase increment, Q15 or floating point																												
int	N		size of output array																												
Output:																															
complex_fract16, complex_float	z	N	output complex values, Q15 or floating point																												
Returned value	updated phase ($a_0 + N \cdot \Delta$)																														
Restrictions	z, a aligned on a 32-bytes boundary z, a not overlapping N multiple of 16 (cbexplin,cbexplin_fast) or 8 (cbexplinf, cbfastexplinf)																														
Scalar Version																															
Prototype	float32_t cexplinf (complex_float * z, float32_t phase0, float32_t delta)																														
Arguments	<table><tr><th>Type</th><th>Name</th><th>Description</th></tr><tr><td>Input:</td><td></td><td></td></tr><tr><td>float32_t</td><td>phase0</td><td>initial phase</td></tr><tr><td>Output:</td><td></td><td></td></tr><tr><td>complex_float</td><td>z[1]</td><td>output complex value</td></tr></table>	Type	Name	Description	Input:			float32_t	phase0	initial phase	Output:			complex_float	z[1]	output complex value															
Type	Name	Description																													
Input:																															
float32_t	phase0	initial phase																													
Output:																															
complex_float	z[1]	output complex value																													
Returned Value	complex result																														

2.4.9 Correction of Vector Slope

Description	Function adjusts the phase of complex values from input vector by a sequence of linearly evolving angle values. NOTE: Phase adjustment of k -th complex input value is performed through the multiplication by a conjugated rotating factor lying on the unit circle. This factor is calculated from the full-precision phase angle: $a_0 + k \cdot \Delta$, thus the function is able to process long data sequences with no risk of phase errors accumulation.
Representation	Q15 input, Q15 output
Accuracy:	$4.0 \cdot 10^{-4}$ (13 in Q15).
Algorithm	$x_n = x_n e^{-j \cdot \pi \cdot (a_0 + n \cdot \Delta)}, n = 0 \dots N-1$
Prototype	int16_t vslope (complex_fract16 * x, int16_t phase0, int16_t delta, int N)

Arguments	Type	Name	Size	Description
	Input:			
	int16_t	a0		initial phase divided by pi, Q15
	int16_t	d		phase increment divided by pi, Q15
	int	N		size of vector
	Input/Output:			
	complex_fract16	x	N	input/output complex values, Q15
Returned value	updated phase divided by pi ($a_0 + N \cdot d$) modulo 2^{16}			
Restrictions	x	aligned on a 32-bytes boundary		
	N	multiple of 16		

2.5 Vector Operations

This package includes a number of basic operations over real/complex vectors.

2.5.1 Real/Complex Vector Product

Description	These routines take two real vectors and calculate their dot product. Fixed point functions accept 16-bit inputs and the form output as 32-bit number. Accumulation is done in a 40-bit wide accumulator and the output can be scaled down with saturation by <code>rsh</code> bits. Thus, if representation of <code>x</code> input is Q_x, on <code>y</code> input Q_y result will be represented in $Q_{(x+y-rsh)}$ format. Floating point routine makes all computation in floating point arithmetic.			
Representation	Q15 input, Q31 output floating point input, floating point output			
Algorithm	Fixed point API: $r = \frac{1}{2^{rsh}} \sum_{n=0}^{N-1} x_n y_n$ Floating point API: $r = \sum_{n=0}^{N-1} x_n y_n$			
Prototype	<pre>int32_t rvdot (const int16_t * x, const int16_t * y, int rsh, int N) float32_t rvdotf (const float32_t * x, const float32_t * y, int N) complex_fract32 cvdot (const complex_fract16 * x, const complex_fract16 * y, int rsh, int N) complex_float cvdotf (const complex_float* x, const complex_float* y, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t, complex_fract16, float32_t, complex_float	x	N	Input data, Q_x or floating point
	int16_t, complex_fract16, float32_t, complex_float	y	N	Input data, Q_y or floating point
	int	rsh		Right shift of result (for fixed point API only)
	int	N		Length of vectors
Returned Value	Dot product of all data pairs, $Q_{(x+y-rsh)}$ or floating point			
Restrictions	<code>x, y</code>	not overlapping		
	<code>x, y</code>	aligned on a 32-bytes boundary		
	<code>N</code>	multiple of 16 (<code>rvdot</code>), 8 (<code>rvdotf</code> , <code>cvdot</code>) or 4 (<code>cvdotf</code>)		
	<code>rsh</code>	≥ 0		

2.5.2 Real/Complex Vector Sum

Description	These routine make a pair-wise summation of real/complex vectors
Representation	Q15 input, Q15 output floating point input, floating point output
Algorithm	$z_n = x_n + y_n, n = 0 \dots N-1$

Prototype <pre> void rvadd (int16_t * z, const int16_t * x, const int16_t * y, int N) void rvaddf (float32_t * z, const float32_t * x, const float32_t * y, int N) void cvadd (complex_fract16 * z, const complex_fract16 * x, const complex_fract16 * y, int N) void cvaddf (complex_float * z, const complex_float * x, const complex_float * y, int N) </pre>				
Arguments	Type	Name	Size	Description
	Input:			
	int16_t, complex_fract16, float32_t, complex_float	x	N	Input data, Q15 or floating point
	int16_t, complex_fract16, float32_t, complex_float	y	N	Input data, Q15 or floating point
	int	N		Length of vectors
	Output:			
	int16_t, complex_fract16, float32_t, complex_float	z	N	Output data, Q15 or floating point
Returned value	None			
Restrictions	z, x, y	should not overlap		
	x, y, z	aligned on a 32-bytes boundary		
	N	multiple of 16 (rvadd), 8 (rvaddf, cvadd) or 4 (cvaddf)		

2.5.3 Sum of Squares of a Vector

Description	These routines compute the power of vector. Fixed point routine makes accumulation in a 40-bit wide accumulator and the output can be scaled down with saturation by <code>rsh</code> bits. Thus, if representation of <code>x</code> input is Q_x, the result will be represented in $Q(2x-rsh)$ format. Floating point routine makes all computations in floating point.			
Representation	Q15 input, Q31 output floating point input, floating point output			
Algorithm	Fixed point API: $r = \frac{1}{2^{rsh}} \sum_{n=0}^{N-1} x_n ^2$ Floating point API: $r = \sum_{n=0}^{N-1} x_n ^2$			
Prototype	<pre> int32_t vpower (const int16_t * x, int rsh, int N) float32_t vpowerf (const float32_t * x, int N) </pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t, float32_t	x	N	Input data, Q_x or floating point
	int	rsh		Right shift of result (for fixed point API only)
	int	N		Length of vector
Returned Value	Sum of squares of a vector, $Q(2x-rsh)$ or floating point			
Restrictions	x	aligned on a 32-bytes boundary		
	rsh	should not to be negative		
	N	multiple of 16 (vpower) or 8 (vpowerf)		

2.5.4 Square Root of Sum of Squares

Description	These routines compute the square root of the sum of square of a vector Fixed point routine makes accumulation in a 40-bit wide accumulator, output is scaled down with saturation by <code>rsh</code> bits and after this stage the square root is applied. Thus, if representation of <code>x</code> input is Q_x , the result will be represented in $Q_{(x-rsh/2)}$ format. Floating point routine makes all computations in floating point.																							
Representation	Q15 input, Q15 output floating point input, floating point output																							
Algorithm	Fixed point API: $r = \sqrt{\frac{1}{2^{rsh}} \sum_{n=0}^{N-1} x_n^2}$ Floating point API: $r = \sqrt{\sum_{n=0}^{N-1} x_n^2}$																							
Prototype	<pre>int16_t vmag (const int16_t * x, int rsh, int N) float32_t vmagf (const float32_t * x, int N)</pre>																							
Arguments	<table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td>Input:</td><td></td><td></td><td></td></tr><tr><td>int16_t, float32_t</td><td>x</td><td>N</td><td>Input data, Q_x or floating point</td></tr><tr><td>int</td><td>rsh</td><td></td><td>Right shift of result (for fixed point API only)</td></tr><tr><td>int</td><td>N</td><td></td><td>Length of vector</td></tr></table>	Type	Name	Size	Description	Input:				int16_t, float32_t	x	N	Input data, Q_x or floating point	int	rsh		Right shift of result (for fixed point API only)	int	N		Length of vector			
Type	Name	Size	Description																					
Input:																								
int16_t, float32_t	x	N	Input data, Q_x or floating point																					
int	rsh		Right shift of result (for fixed point API only)																					
int	N		Length of vector																					
Returned Value	Square root of sum of squares of a vector, $Q_{(x-rsh/2)}$ or floating point																							
Restrictions	<code>x</code> <code>rsh</code> <code>N</code>	aligned on a 32-bytes boundary should not to be negative multiple of 16 (<code>vmag</code>) or 8 (<code>vmagf</code>)																						

2.5.5 Vector Normalization

Description	This routine makes a shift with saturation of data values in the vector by a given scale factor (degree of 2).			
Representation	Q15 input, Q15 output			
Algorithm	$y_n = x_n \cdot 2^t$			
Prototype	void vnorm (int16_t * y, const int16_t * x, int t, int N)			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t	x	N	Input data
	int	t		Shift count. If positive, it shifts left with saturation, if negative it shifts right
	int	N		Length of vector
	Output:			
	int16_t	y	N	Output data
Returned value	None			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	N	multiple of 16		

2.5.6 Common Exponent

Description	These functions compute base-2 exponent adjustment term needed to normalize data in the input vector. Exact meaning of normalization depends on the data format (see below). For fixed point, these functions determine the number of redundant sign bits for each value (as if it was loaded in a 32-bit register) and return the minimum number over the whole vector. This may be useful for an FFT implementation to normalize data. In floating point, for each finite input value x , functions estimate the integer $E(x)$, such that $0.5 \leq x * 2^{E(x)} < 1$ and $-128 \leq E(x) \leq 148$. The minimum value of $E(x)$ over input data vector is the result.			
	Special cases:			
	Input	Output	Note	
	0	0	Floating point API	
	$\pm\infty$	-129		
	NaN	0		
	0	31	Fixed point API	
	32767	16		
	-32768	16		
	Representation	Q15 input, integer output floating point input, integer output		
Algorithm	Fixed point API:			
	$z = \min_{n=0 \dots N-1} (norm(x_n))$			
	where norm is exponent value (maximum possible shift count) for 32-bit data.			
	Floating point API:			
	$z = -\text{floor}(\log_2(\max_{n=0 \dots N-1}(x_n)))$			
Prototypes	<pre>int vbexp (const int16_t * x, int N) int vbexp_fast (const int16_t * x, int N) int vbexpf (const float32_t * x, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t	x	N	input data, Q15 of floating point
	int	N		length of vectors
Returned value	minimum exponent			
Restrictions	no restrictions applied for non-fast functions <code>vbexp()</code> , <code>vbexpf()</code> .			
	For fast function <code>vbexp_fast()</code> there are some limitations :			
	x	aligned on a 32-bytes boundary		
	N	multiple of 16		
Scalar Versions				
Prototype	<pre>int sbexp (int32_t x) int sbexpf (float32_t x)</pre>			
Arguments	Type	Name	Description	
	Input:			
	int32_t	x	input data, Q15 of floating point	
Returned value	Result			

2.5.7 Mean of Vector Elements

Description	Computes a mean value over all elements of given vector (real or complex)
Representation	Q15 input, Q15 output floating point input, floating point output

Algorithm

$$m = \frac{1}{N} \sum_{n=0}^{N-1} x_n, n = \overline{0 \dots N-1}$$

Prototype

```
void rvmean ( int16_t * m, const int16_t * x, int N )
void rvmeanf( float32_t * m, const float32_t * x, int N )
void cvmean ( complex_fract16 * m, const complex_fract16 * x, int N )
void cvmeanf( complex_float * m, const complex_float * x, int N )
```

Arguments

Type	Name	Size	Description
Input:			
int16_t, float32_t, complex_fract16, complex_float	x	N	input data
int	N		size of input array
Output:			
int16_t, float32_t, complex_fract16, complex_float	m	1	mean value

Returned value

none

Restrictions

x
x, m
N

aligned on a 32-bytes boundary
not overlapping
multiple of 16 (rvmean), 8 (rvmeanf, cvmean), or 4 (cvmeanf)

2.5.8 Dual Peak Search

Description

These functions retrieve the maximum (minimum) and next-to-maximum (next-to-minimum) values of vector elements. They output both the peak values and their indices.

NOTES:

- Each kind of dual-peak search function reserves a special value to maintain internal invariants during the search process. However, it is still legal for the input vector to contain special values among other data. If this is the case, then the following peculiarities should be considered:
 - Input vector elements equal to the special value are ignored.
 - If the total number K of input vector elements distinct from the special value is less than 2 (i.e. $K==0$ or $K==1$), then the last $2-K$ entries of output vector `idx` are assigned zero, and the last $2-K$ elements of output vector `m` are assigned the special value.
 - If the peak value is encountered more than once in the input data vector, then it will be reported twice in `m`, and `idx` will contain indices of the first two occurrences of the peak value.
- For fixed point, special values are -32768 and 32767 for `vmax` and `vmin` respectively. For floating point, they are `-HUGE_VAL` and `HUGE_VAL`.

Representation

Q15 input, Q15 output
floating point input, floating point output

Prototype

```
void vmax ( int16_t * m, int16_t * idx, const int16_t * x, int N )
void vmaxf ( float32_t * m, int16_t * idx, const float32_t * x, int N )
void vmin ( int16_t * m, int16_t * idx, const int16_t * x, int N )
void vminf ( float32_t * m, int16_t * idx, const float32_t * x, int N )
```

Type	Name	Size	Description
Input:			
int16_t, float32_t	x	N	input data
int	N		size of input array
Output:			
int16_t, float32_t	m	2	2 maximum/minimum values is descending (<code>vmax</code> , <code>vmaxf</code>) or ascending (<code>vmin</code> , <code>vminf</code>) order
int16_t	idx	2	indices of 2 above elements; optional

Returned value

none

Restrictions	x	aligned on a 32-bytes boundary
	x, m, idx	not overlapping
	N	multiple of 16 (v_{max} , v_{min}) or 8 (v_{maxf} , v_{minf})

2.5.9 Search for 8 Top Values

Description	Functions search for 8 top values in the input vector. They output both the peak values and respective indices. Notes: 1. The function requires read/write access to the input vector $x[N]$, because it performs some data manipulations on it. Before returning, it always restores original data. 2. Both top-8 search functions reserve a special value to maintain internal invariants during the search process. For v_{max8} this special value is -32768, and for v_{max8f} it is $-HUGE_VALF$. It is still legal for the input vector to contain special values among other data. If this is the case, then the following peculiarities should be considered: ▪ Input vector elements equal to the special value are ignored. ▪ If the total number K of input vector elements distinct from the special value is less than 8 (i.e. $K < 8$), then the last $8-K$ entries of output vector idx are assigned zero, and the last $8-K$ elements of output vector m are assigned the special value. 3. All elements of the input vector x holding the same top value will be reported in search results, unless the total limit of 8 peak elements is exhausted. Example (v_{max8}): Input: $N: 16$ $x[N]: 1, -32768, 1, 2, 3, -32768, -32768, -32768, -32768, -32768, -32768, -32768, -32768, -32768, -32768, -32768$ Note that the number of input value distinct from -32768 is less than 8. Also note the duplicated value at positions 0 and 2. Results: $m[8]: 3, 2, 1, 1, -32768, -32768, -32768, -32768$ $idx[8]: 4, 3, 0, 2, 0, 0, 0, 0$																														
Representation	Q15 input, Q15 output floating point input, floating point output																														
Prototype	<pre>void vmax8 (int16_t* m, int16_t* idx, int16_t * x, int N) void vmax8f (float32_t * m, int16_t * idx, float32_t * x, int N)</pre>																														
	<table border="1"> <thead> <tr> <th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr> </thead> <tbody> <tr> <td colspan="4">Input:</td></tr> <tr> <td>int16_t, float32_t</td><td>x</td><td>N</td><td>input data</td></tr> <tr> <td>int</td><td>N</td><td></td><td>size of input array</td></tr> <tr> <td colspan="4">Output:</td></tr> <tr> <td>int16_t, float32_t</td><td>m</td><td>8</td><td>8 maximum values in descending order</td></tr> <tr> <td>int16_t</td><td>idx</td><td>8</td><td>Maximum value locations in the input vector</td></tr> </tbody> </table>			Type	Name	Size	Description	Input:				int16_t, float32_t	x	N	input data	int	N		size of input array	Output:				int16_t, float32_t	m	8	8 maximum values in descending order	int16_t	idx	8	Maximum value locations in the input vector
Type	Name	Size	Description																												
Input:																															
int16_t, float32_t	x	N	input data																												
int	N		size of input array																												
Output:																															
int16_t, float32_t	m	8	8 maximum values in descending order																												
int16_t	idx	8	Maximum value locations in the input vector																												
Returned value	none																														
Restrictions	x	aligned on a 32-bytes boundary																													
	x, m, idx	not overlapping																													
	N	multiple of 16 (v_{max8}) or 8 (v_{max8f})																													

2.5.10 Find Values above Threshold

Description	Function generates indices of all elements above the threshold.																														
Representation	Q15 input, Q15 output floating point input, floating point output																														
Prototype	<pre>int vthreshold (int16_t* idx, int16_t thr, const int16_t * x, int N) int vthresholdf (int16_t* idx, float32_t thr, const float32_t * x, int N)</pre> <table border="1"> <thead> <tr> <th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr> </thead> <tbody> <tr> <td>Input:</td><td></td><td></td><td></td></tr> <tr> <td>int16_t, float32_t</td><td>thr</td><td></td><td>threshold value</td></tr> <tr> <td>int16_t</td><td>x</td><td>N</td><td>input data</td></tr> <tr> <td>int</td><td>N</td><td></td><td>size of input array</td></tr> <tr> <td>Output:</td><td></td><td></td><td></td></tr> <tr> <td>int16_t, float32_t</td><td>idx</td><td>N+1</td><td>indexes of elements above the threshold NOTE: extra cell should be reserved in the end of array</td></tr> </tbody> </table>			Type	Name	Size	Description	Input:				int16_t, float32_t	thr		threshold value	int16_t	x	N	input data	int	N		size of input array	Output:				int16_t, float32_t	idx	N+1	indexes of elements above the threshold NOTE: extra cell should be reserved in the end of array
Type	Name	Size	Description																												
Input:																															
int16_t, float32_t	thr		threshold value																												
int16_t	x	N	input data																												
int	N		size of input array																												
Output:																															
int16_t, float32_t	idx	N+1	indexes of elements above the threshold NOTE: extra cell should be reserved in the end of array																												
Returned value	number of found elements																														
Restrictions	x, idx aligned on a 32-bytes boundary x, idx not overlapping N multiple of 16 (vthreshold) or 8 (vthresholdf)																														

2.5.11 Move the Elements at Given Indices

Description	Functions move the elements at indices $k \cdot M$ from the input buffer into a separate buffer. Both real and complex data are supported.																																														
Algorithm	$y_k = x_{kM}, k = 0 \dots N-1$																																														
Prototype	<pre>void rmelements2 (int16_t * y, const int16_t * x, int N) void rmelements3 (int16_t * y, const int16_t * x, int N) void rmelements4 (int16_t * y, const int16_t * x, int N) void rmelements6 (int16_t * y, const int16_t * x, int N) void rmelements12 (int16_t * y, const int16_t * x, int N) void rmelements24 (int16_t * y, const int16_t * x, int N) void cmelements2 (complex_fract16 * y, const complex_fract16 * x, int N) void cmelements3 (complex_fract16 * y, const complex_fract16 * x, int N) void cmelements4 (complex_fract16 * y, const complex_fract16 * x, int N) void cmelements6 (complex_fract16 * y, const complex_fract16 * x, int N) void cmelements12 (complex_fract16 * y, const complex_fract16 * x, int N) void cmelements24 (complex_fract16 * y, const complex_fract16 * x, int N)</pre> <table border="1"> <thead> <tr> <th>M</th><th>Data</th><th>Name</th><th>M</th><th>Data</th><th>Name</th></tr> </thead> <tbody> <tr> <td>2</td><td>real</td><td>rmelements2</td><td>6</td><td>real</td><td>rmelements6</td></tr> <tr> <td>2</td><td>complex</td><td>cmelements2</td><td>6</td><td>complex</td><td>cmelements6</td></tr> <tr> <td>3</td><td>real</td><td>rmelements3</td><td>12</td><td>real</td><td>rmelements12</td></tr> <tr> <td>3</td><td>complex</td><td>cmelements3</td><td>12</td><td>complex</td><td>cmelements12</td></tr> <tr> <td>4</td><td>real</td><td>rmelements4</td><td>24</td><td>real</td><td>rmelements24</td></tr> <tr> <td>4</td><td>complex</td><td>cmelements4</td><td>24</td><td>complex</td><td>cmelements24</td></tr> </tbody> </table>					M	Data	Name	M	Data	Name	2	real	rmelements2	6	real	rmelements6	2	complex	cmelements2	6	complex	cmelements6	3	real	rmelements3	12	real	rmelements12	3	complex	cmelements3	12	complex	cmelements12	4	real	rmelements4	24	real	rmelements24	4	complex	cmelements4	24	complex	cmelements24
M	Data	Name	M	Data	Name																																										
2	real	rmelements2	6	real	rmelements6																																										
2	complex	cmelements2	6	complex	cmelements6																																										
3	real	rmelements3	12	real	rmelements12																																										
3	complex	cmelements3	12	complex	cmelements12																																										
4	real	rmelements4	24	real	rmelements24																																										
4	complex	cmelements4	24	complex	cmelements24																																										

Arguments	Type	Name	Size	Description
	Input:			
	int16_t, complex_fract16	x	N*M	input vector
	int	N		size of output vector
	Output:			
	int16_t, complex_fract16	y	N	output vector
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	N	multiple of 16 for real variants or multiple of 8 for complex variants		

2.5.12 Remove the Elements at Given Indices

Description	Function removes the elements at indices $k \cdot M$ from the input buffer (removes holes at those indices)
Representation	Q15 input, Q15 output floating point input, floating point output
Algorithm	$y_k = x_n, k = 0 \dots N-1, n = [k/(M-1)] \cdot M + (k+1 - (M-1) \cdot [k/(M-1)])$

Prototype

```

void rrelements2 (int16_t * y, const int16_t * x, int N )
void rrelements3 (int16_t * y, const int16_t * x, int N )
void rrelements4 (int16_t * y, const int16_t * x, int N )
void rrelements6 (int16_t * y, const int16_t * x, int N )
void rrelements12 (int16_t * y, const int16_t * x, int N )
void rrelements24 (int16_t * y, const int16_t * x, int N )

void crelements2 ( complex_fract16 * y, const complex_fract16 * x, int N )
void crelements3 ( complex_fract16 * y, const complex_fract16 * x, int N )
void crelements4 ( complex_fract16 * y, const complex_fract16 * x, int N )
void crelements6 ( complex_fract16 * y, const complex_fract16 * x, int N )
void crelements12 ( complex_fract16 * y, const complex_fract16 * x, int N )
void crelements24 ( complex_fract16 * y, const complex_fract16 * x, int N )

```

M	Data	Name	M	Data	Name
2	real	rrelements2	6	real	rrelements6
2	complex	crelements2	6	complex	crelements6
3	real	rrelements3	12	real	rrelements12
3	complex	crelements3	12	complex	crelements12
4	real	rrelements4	24	real	rrelements24
4	complex	crelements4	24	complex	crelements24

Arguments	Type	Name	Size	Description
	Input:			
	int16_t, complex_fract16	x	N*M	input vector
	int	N		size of input vector multiplied by M
	Output:			
	int16_t, complex_fract16	y	N*(M-1)	output vector
Returned value	none			

Restrictions	x, y	aligned on a 32-bytes boundary
	x, y	not overlapping
	N	multiple of 16 for real variants or multiple of 8 for complex variants

2.5.13 Angle Unwrapping

Description	Function corrects phase angles to produce smooth phase plots. It adjusts phase angles in a vector input by adding a multiple of 2π when the absolute difference between two consecutive elements of the input vector appears greater than or equal to the jump tolerance. Tolerance values smaller than π (0x4000 in fixed point representation Q14) are equivalent to the tolerance value of π . For fixed point API, input data are normalized by π and represented in Q15. To accommodate the output data format to possible 2π cycles, the fixed-point position is specified through the q argument.																																		
Representation	Q15 input, Q15 output floating point input, floating point output																																		
Algorithm	See equivalent MATLAB function																																		
Prototype	<pre>void aunwrap (int16_t* y, const int16_t * x, int16_t tol, int q, int N) void aunwrapf (float32_t * y, const float32_t * x, float32_t tol, int N)</pre> <table border="1"> <thead> <tr> <th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr> </thead> <tbody> <tr> <td colspan="4">Input:</td></tr> <tr> <td>int16_t, float32_t</td><td>x</td><td>N</td><td>original angles, Q15 or floating point</td></tr> <tr> <td>int16_t, float32_t</td><td>tol</td><td></td><td>jump tolerance (default value is 0x4000 or π), Q14 or floating point</td></tr> <tr> <td>int</td><td>q</td><td></td><td>Fixed-point position for output data, 0..15 (for fixed point API only)</td></tr> <tr> <td>int</td><td>N</td><td></td><td>size of input/output arrays</td></tr> <tr> <td colspan="4">Output:</td></tr> <tr> <td>int16_t, float32_t</td><td>y</td><td>N</td><td>output unwrapped angular data values, Q(q) or floating point</td></tr> </tbody> </table>			Type	Name	Size	Description	Input:				int16_t, float32_t	x	N	original angles, Q15 or floating point	int16_t, float32_t	tol		jump tolerance (default value is 0x4000 or π), Q14 or floating point	int	q		Fixed-point position for output data, 0..15 (for fixed point API only)	int	N		size of input/output arrays	Output:				int16_t, float32_t	y	N	output unwrapped angular data values, Q(q) or floating point
Type	Name	Size	Description																																
Input:																																			
int16_t, float32_t	x	N	original angles, Q15 or floating point																																
int16_t, float32_t	tol		jump tolerance (default value is 0x4000 or π), Q14 or floating point																																
int	q		Fixed-point position for output data, 0..15 (for fixed point API only)																																
int	N		size of input/output arrays																																
Output:																																			
int16_t, float32_t	y	N	output unwrapped angular data values, Q(q) or floating point																																
Returned value	none																																		
Restrictions	x, y	aligned on a 32-bytes boundary																																	
	x, y	not overlapping																																	
	q	in range 0...15 (for fixed point API only)																																	
	N	multiple of 16 (aunwrap) or 8 (aunwrapf)																																	

2.6 Matrix Operations

2.6.1 General Definitions and Matrix Representation

Matrix utility functions process multiple matrices per single invocation. There are two options for the way a few matrices are packed into continuous memory area passed to/from a library function:

- Block order
- Streaming order

Block order is a sequence of matrices coming serially. In the C-language notation, the definition of L block ordered matrices of size $M \times N$ is `int16_t A[L][S][C]` where

L – number of matrices

S – number of elements allocated per matrix ($S \geq M \times N$, see below)

C – 1 for real matrices and 2 for complex

When matrices are stored in the block order, the number of elements allocated per single matrix S doesn't necessarily equal the product $M \times N$. As a general rule, S is rounded in such a way to place the first element of the next matrix to the next aligned address in a memory to cope with memory

alignment issues. If matrix takes less than half a SIMD width, we operate with twice smaller alignment and so on. As an example, for real 16-bit matrices 3×3 S is 16, for complex 16-bit vectors 3×1 S is 4. Exact requirements on L , S and C are specified for each API function that supports the block order for input/output data.

For the streaming order, first $A_{0,0}$ elements of L matrices are followed by the second elements $A_{0,1}$, etc. In the C notation it is represented as `int16_t A[S][L][C]`. L must be a multiple of 4 for complex matrices, or multiple of 8 for real matrices. Consequently, alignment conditions are satisfied automatically so S is always equal to $M \times N$. Streaming order provides considerable performance gain in terms of CPU cycles due to a higher degree of parallelization.

For both block order and streaming order applied to complex matrices, real and imaginary components of a complex number are always stored in adjacent memory locations, with the real part going first.

Some library functions expose additional restrictions on the number of matrices and their dimensions. In particular, it may be required that L , M and N are multiples of small integers mL , mN and mM , respectively.

For either the block order or streaming order, address of the first stored element must be aligned on a 32-bytes boundary.

For fixed point API, functions may work with Q15 matrices as well as with data presented in another Q format, but in the latter case the CPU cycles performance may slightly degrade. All row-by-column multiplies are performed using 40-bit accumulators, with proper scaling applied to the resulting value. If, however, it appears that the result is too large to fit the narrower output storage, then it's saturated.

2.6.2 Real Matrix/Vector Multiply

Description	These functions perform pairwise multiplication of two sequences of real matrices or vectors. Both the block order and streaming order are allowed for input/output matrix sequences.			
Representation	Q15 input, Q15 output floating point input, floating point output			
Algorithm	$z_{m,p}^{(l)} = \sum_{n=0}^{N-1} x_{m,n}^{(l)} \cdot y_{n,p}^{(l)}, m = 0 \dots \overline{M-1}, p = 0 \dots \overline{P-1}, l = 0 \dots \overline{L-1}$			
Prototype	<pre> void funQ (int16_t* z, const int16_t* x, const int16_t* y, int L, int Q) void funQf (float32_t* z, const float32_t* x, const float32_t* y, int L) void funNNQ (int16_t* z, const int16_t* x, const int16_t* y, int N, int L, int Q) void funNNf (float32_t* z, const float32_t* x, const float32_t* y, int N, int L) void funNMQ (int16_t* z, const int16_t* x, const int16_t* y, int N, int M, int L, int Q) void funNMf (float32_t* z, const float32_t* x, const float32_t* y, int N, int M, int L) void funNMQs(void* pScr, int16_t* z, const int16_t* x, const int16_t* y, int N, int L, int Q) void funNMsf(void* pScr, float32_t* z, const float32_t* x, const float32_t* y, int N, int L) void funNNQs(void* pScr, int16_t* z, const int16_t* x, const int16_t* y, int N, int L, int Q) void funNNsf(void* pScr, float32_t* z, const float32_t* x, const float32_t* y, int N, int L) </pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t, float32_t	x	L*Sx	Input matrices x
	int16_t, float32_t	y	L*Sy	Input matrices y
	int	N		Matrix dimension (columnar for $M \times N$)

int	M		Matrix dimension
int	L		Number of matrices
int	Q		Position of fractional point in matrix representation (for fixed point API only)
Output:			
int16_t, float32_t	z	L*Sz	Output matrices z
Temporary:			
void	pScr		scratch memory (required only for $n \times n$, $m \times n$ block order multiplies. Size in bytes is defined by functions matmuln $n \times n$ [f]_getScratchSize(), matmuln $x \times m$ [f]_getScratchSize(), matvmuln $n \times n$ [f]_getScratchSize(), matvmuln $x \times m$ [f]_getScratchSize() respectively

Returned Value None

Restrictions

- x, y, z aligned on a 32-bytes boundary
- x, y, z not overlapping
- Q in range 0...16 (for fixed point API only)
- $L > 0$ the number of matrices must be positive
- other restrictions see in the table below

Function Name	Prototype	Operation	N	M	P	Allocated Space per Matrix			Restriction s	
						Sx	Sy	Sz	mL	mN , mM
Block order, 16-bit data										
matmul2x2n	funQ	2×2 * 2×2	2	2	2	4	4	4	4	
matmul3x3n	funQ	3×3 * 3×3	3	3	3	16	16	16		
matmul4x4n	funQ	4×4 * 4×4	4	4	4	16	16	16		
matmul8x8n	funQ	8×8 * 8×8	8	8	8	64	64	64		
matmul16x16n	funQ	16×16 * 16×16	16	16	16	256	256	256		
matmuln×nn	funNNQs	N×N * N×N	N	N	N	N×N	N×N	N×N		4
matmuln×mn	funNMQs	M×N * N×M	M	N	M	M×N	N×M	M×M		4
matvmul2x2n	funQ	2×2 * 2×1	2	2	1	4	2	2	8	
matvmul3x3n	funQ	3×3 * 3×1	3	3	1	16	4	4	4	
matvmul4x4n	funQ	4×4 * 4×1	4	4	1	16	4	4	4	
matvmul8x8n	funQ	8×8 * 8×1	8	8	1	64	8	8		
matvmul16x16n	funQ	16×16 * 16×1	16	16	1	256	16	16		
matvmuln×nn	funNNQs	N×N * N×1	N	N	1	N×N	N	N	4	4
matvmuln×mn	funNMQs	M×N * N×1	M	N	1	M×N	N	M	4	4
Block order, floating point data										
matmul2x2nf	funf	2×2 * 2×2	2	2	2	4	4	4	2	
matmul3x3nf	funf	3×3 * 3×3	3	3	3	16	16	16		
matmul4x4nf	funf	4×4 * 4×4	4	4	4	16	16	16		
matmul8x8nf	funf	8×8 * 8×8	8	8	8	64	64	64		
matmul16x16nf	funf	16×16 * 16×16	16	16	16	256	256	256		
matmuln×nnf	funNNsf	N×N * N×N	N	N	N	N×N	N×N	N×N		4
matmuln×mnf	funNMsf	M×N * N×M	M	N	M	M×N	N×M	M×M		4
matvmul2x2nf	funf	2×2 * 2×1	2	2	1	4	2	2	4	
matvmul3x3nf	funf	3×3 * 3×1	3	3	1	16	4	4	2	
matvmul4x4nf	funf	4×4 * 4×1	4	4	1	16	4	4	2	
matvmul8x8nf	funf	8×8 * 8×1	8	8	1	64	8	8		
matvmul16x16nf	funf	16×16 * 16×1	16	16	1	256	16	16		
matvmuln×nnf	funNNsf	N×N * N×1	N	N	1	N×N	N	N	2	4
matvmuln×mnf	funNMsf	M×N * N×1	M	N	1	M×N	N	M	2	4
Streaming order, 16-bit data										
matmul2x2s	funQ	2×2 * 2×2	2	2	2	4	4	4	16	
matmul3x3s	funQ	3×3 * 3×3	3	3	3	9	9	9	16	
matmul4x4s	funQ	4×4 * 4×4	4	4	4	16	16	16	16	
matmul8x8s	funO	8×8 * 8×8	8	8	8	64	64	64	16	

Function Name	Prototype	Operation	N	M	P	Allocated Space per Matrix			Restrictions	
						Sx	Sy	Sz	mL	mN, mM
matmul16x16s	funQ	16×16 * 16×16	16	16	16	256	256	256	16	
matmulnxns	funNNQ	N×N * N×N	N	N	N	N×N	N×N	N×N	16	
matmulnxms	funNMQ	M×N * N×M	M	N	M	M×N	N×M	M×M	16	
matvmul2x2s	funQ	2×2 * 2×1	2	2	1	4	2	2	16	
matvmul4x4s	funQ	4×4 * 4×1	4	4	1	16	4	4	16	
matvmul8x8s	funQ	8×8 * 8×1	8	8	1	64	8	8	16	
matvmul16x16s	funQ	16×16 * 16×1	16	16	1	256	16	16	16	
matvmulnxns	funNNQ	N×N * N×1	N	N	1	N×N	N	N	16	
matvmulnxms	funNMQ	M×N * N×1	M	N	1	M×N	N	M	16	
Streaming order, floating point data										
matmul2x2sf	funf	2×2 * 2×2	2	2	2	4	4	4	8	
matmul3x3sf	funf	3×3 * 3×3	3	3	3	9	9	9	8	
matmul4x4sf	funf	4×4 * 4×4	4	4	4	16	16	16	8	
matmul8x8sf	funf	8×8 * 8×8	8	8	8	64	64	64	8	
matmul16x16sf	funf	16×16 * 16×16	16	16	16	256	256	256	8	
matmulnxnsf	funNNf	N×N * N×N	N	N	N	N×N	N×N	N×N	8	
matmulnxmsf	funNMf	M×N * N×M	M	N	M	M×N	N×M	M×M	8	
matvmul2x2sf	funf	2×2 * 2×1	2	2	1	4	2	2	8	
matvmul4x4sf	funf	4×4 * 4×1	4	4	1	16	4	4	8	
matvmul8x8sf	funf	8×8 * 8×1	8	8	1	64	8	8	8	
matvmul16x16sf	funf	16×16 * 16×1	16	16	1	256	16	16	8	
matvmulnxnsf	funNNf	N×N * N×1	N	N	1	N×N	N	N	8	
matvmulnxmsf	funNMf	M×N * N×1	M	N	1	M×N	N	M	8	

NOTES:

In the table above, S_x, S_y, S_z denote per-matrix memory allocation amount for matrices x, y, z in elements of corresponding data type. The last column specifies m_L – a required divisor for L (by default it is equal to 1). For rows with non-blank m_N/m_M value the respective API function expects that both N and M are multiples of the specified value.

2.6.3 Complex Matrix Multiply

Description	These functions perform pairwise multiplication of two sequences of complex matrices or vectors. Both the block order and streaming order are allowed for input/output matrix sequences.
Representation	Q15 input, Q15 output floating point input, floating point output
Algorithm	$z_{m,p}^{(l)} = \sum_{n=0}^{N-1} x_{m,n}^{(l)} \cdot y_{n,p}^{(l)}, m = 0 \dots M-1, p = 0 \dots P-1, l = 0 \dots L-1$
Prototypes	<pre> void cfunQ (complex_fractl6* z, const complex_fractl6* x, const complex_fractl6 * y, int L, int Q) void cfunf (complex_float* z, const complex_float* x, const complex_float * y, int L) void cfunNNQ (complex_fractl6* z, const complex_fractl6* x, const complex_fractl6* y, int N, int L, int Q) void cfunNNf (complex_float * z, const complex_float * x, const complex_float * y, int N, int L) void cfunNMQ (complex_fractl6* z, const complex_fractl6* x, const complex_fractl6* y, int N, int M, int L, int Q) void cfunNMf (complex_float * z, const complex_float * x, const complex_float * y, int N, int M, int L) void cfunNMQs(void* pScr, complex_fractl6* z, const complex_fractl6* x, const complex_fractl6 * y, int N, int L, int Q) void cfunNMsf(void* pScr, complex_float * z, const complex_float * x, const complex_float * y, int N, int L) void cfunNNQs(void* pScr, complex_fractl6* z, const complex_fractl6* x, const complex_fractl6* y, int N, int L, int Q) void cfunNNs(void* pScr, complex_float * z, const complex_float * x, const complex_float * y, int N, int L) </pre>

Arguments	Type	Name	Size	Description
	Input:			
	complex_fract16, complex_float	x	L*Sx	Input matrices x
	complex_fract16, complex_float	y	L*Sy	Input matrices y
	int	N		Matrix dimension (columnar for $M \times N$)
	int	M		Matrix dimension
	int	L		Number of matrices
	int	Q		Position of fractional point in matrix representation (for fixed point API only)
	Output:			
	complex_fract16, complex_float	z	L*Sz	Output matrices z
	Temporary:			
	void	pScr		scratch memory (required only for $n \times n$, $m \times n$ block order multiplies. Size in bytes is defined by functions <code>cmatmulnxnn[f]_getScratchSize()</code> , <code>cmatmulnxmn[f]_getScratchSize()</code> , <code>cmatvmulnxnn[f]_getScratchSize()</code> , <code>cmatvmulnxmn[f]_getScratchSize()</code> respectively
Returned Value	None			
Restrictions	x, y, z aligned on a 32-bytes boundary x, y, z not overlapping Q in range 0...16 (for fixed point API only) $L > 0$ the number of matrices must be positive other restrictions see in the table below			

Function Name	Prototype	Operation	N	M	P	Allocated Space per Matrix			Restrictions	
						Sx	Sy	Sz	mL	mN, mM
Block order, 16-bit data										
cmatmul2x2n	cfunQ	2×2 * 2×2	2	2	2	4	4	4	2	
cmatmul3x3n	cfunQ	3×3 * 3×3	3	3	3	16	16	16		
cmatmul4x4n	cfunQ	4×4 * 4×4	4	4	4	16	16	16		
cmatmul8x8n	cfunQ	8×8 * 8×8	8	8	8	64	64	64		
cmatmul16x16n	cfunQ	16×16 * 16×16	16	16	16	256	256	256		
cmatmulnxnn	cfunNNQs	N×N * N×N	N	N	N	N×N	N×N	N×N		4
cmatmulnxmn	cfunNMQs	M×N * N×M	M	N	M	M×N	N×M	M×M		4
cmatvmul2x2n	cfunQ	2×2 * 2×1	2	2	1	4	2	2	4	
cmatvmul3x3n	cfunQ	3×3 * 3×1	3	3	1	16	4	4	2	
cmatvmul4x4n	cfunQ	4×4 * 4×1	4	4	1	16	4	4	2	
cmatvmul8x8n	cfunQ	8×8 * 8×1	8	8	1	64	8	8		
cmatvmul16x16n	cfunQ	16×16 * 16×1	16	16	1	256	16	16		
cmatvmulnxnn	cfunNNQs	N×N * N×1	N	N	1	N×N	N	N	2	4
cmatvmulnxmn	cfunNMQs	M×N * N×1	M	N	1	M×N	N	M	2	4
Block order, floating point data										
cmatmul2x2nf	cfunf	2×2 * 2×2	2	2	2	4	4	4		
cmatmul3x3nf	cfunf	3×3 * 3×3	3	3	3	12	12	12		
cmatmul4x4nf	cfunf	4×4 * 4×4	4	4	4	16	16	16		
cmatmul8x8nf	cfunf	8×8 * 8×8	8	8	8	64	64	64		
cmatmul16x16nf	cfunf	16×16 * 16×16	16	16	16	256	256	256		
cmatmulnxnnf	cfunNNsf	N×N * N×N	N	N	N	N×N	N×N	N×N		4
cmatmulnxmnf	cfunNMsf	M×N * N×M	M	N	M	M×N	N×M	M×M		4
cmatvmul2x2nf	cfunf	2×2 * 2×1	2	2	1	4	2	2	2	
cmatvmul3x3nf	cfunf	3×3 * 3×1	3	3	1	12	4	4		

Function Name	Prototype	Operation	N	M	P	Allocated Space per Matrix			Restrictions	
						Sx	Sy	Sz	mL	mN, mM
cmatvmul4x4nf	cfunf	4×4 * 4×1	4	4	1	16	4	4		
cmatvmul8x8nf	cfunf	8×8 * 8×1	8	8	1	64	8	8		
cmatvmul16x16nf	cfunf	16×16 * 16×1	16	16	1	256	16	16		
cmatvmulnxxnf	cfunNNsf	N×N * N×1	N	N	1	N×N	N	N		4
cmatvmulnxmnf	cfunNMsf	M×N * N×1	M	N	1	M×N	N	M		4
Streaming order, 16-bit data										
cmatmul2x2s	cfunQ	2×2 * 2×2	2	2	2	4	4	4	8	
cmatmul3x3s	cfunQ	3×3 * 3×3	3	3	3	9	9	9	8	
cmatmul4x4s	cfunQ	4×4 * 4×4	4	4	4	16	16	16	8	
cmatmul8x8s	cfunQ	8×8 * 8×8	8	8	8	64	64	64	8	
cmatmul16x16s	cfunQ	16×16 * 16×16	16	16	16	256	256	256	8	
cmatmulnxxns	cfunNNQ	N×N * N×N	N	N	N	N×N	N×N	N×N	8	
cmatmulnxms	cfunNMQ	M×N * N×M	M	N	M	M×N	N×M	M×M	8	
cmatvmul2x2s	cfunQ	2×2 * 2×1	2	2	1	4	2	2	8	
cmatvmul3x3s	cfunQ	3×3 * 3×1	3	3	1	9	3	3	8	
cmatvmul4x4s	cfunQ	4×4 * 4×1	4	4	1	16	4	4	8	
cmatvmul8x8s	cfunQ	8×8 * 8×1	8	8	1	64	8	8	8	
cmatvmul16x16s	cfunQ	16×16 * 16×1	16	16	1	256	16	16	8	
cmatvmulnxxns	cfunNNQ	N×N * N×1	N	N	1	N×N	N	N	8	
cmatvmulnxms	cfunNMQ	M×N * N×1	M	N	1	M×N	N	M	8	
Streaming order, floating point data										
cmatmul2x2sf	cfunf	2×2 * 2×2	2	2	2	4	4	4	4	
cmatmul3x3sf	cfunf	3×3 * 3×3	3	3	3	9	9	9	4	
cmatmul4x4sf	cfunf	4×4 * 4×4	4	4	4	16	16	16	4	
cmatmul8x8sf	cfunf	8×8 * 8×8	8	8	8	64	64	64	4	
cmatmul16x16sf	cfunf	16×16 * 16×16	16	16	16	256	256	256	4	
cmatmulnxxnsf	cfunNNf	N×N * N×N	N	N	N	N×N	N×N	N×N	4	
cmatmulnxmsf	cfunNMf	M×N * N×M	M	N	M	M×N	N×M	M×M	4	
cmatvmul2x2sf	cfunf	2×2 * 2×1	2	2	1	4	2	2	4	
cmatvmul3x3sf	cfunf	3×3 * 3×1	3	3	1	9	3	3	4	
cmatvmul4x4sf	cfunf	4×4 * 4×1	4	4	1	16	4	4	4	
cmatvmul8x8sf	cfunf	8×8 * 8×1	8	8	1	64	8	8	4	
cmatvmul16x16sf	cfunf	16×16 * 16×1	16	16	1	256	16	16	4	
cmatvmulnxxnsf	cfunNNf	N×N * N×1	N	N	1	N×N	N	N	4	
cmatvmulnxmsf	cfunNMf	M×N * N×1	M	N	1	M×N	N	M	4	

NOTES:

In the table above, S_x , S_y , S_z denote per-matrix memory allocation amount for matrices x, y, z in elements of corresponding data type. The last column specifies m_L – a required divisor for L (by default it is equal to 1). For rows with non-blank m_N/m_M value the respective API function expects that both N and M are multiples of the specified value.

2.6.4 Real Matrix-Complex Matrix/Vector Multiply

Description	These functions perform pairwise multiplication of sequences of real/complex matrices or vectors. Both the block order and streaming order are allowed for input/output matrix sequences.
Representation	Q15 input, Q15 output
Algorithm	$z_{m,p}^{(l)} = \sum_{n=0}^{N-1} x_{m,n}^{(l)} \cdot y_{n,p}^{(l)}, m = 0 \dots M-1, p = 0 \dots P-1, l = 0 \dots L-1$
Prototypes	<pre> void funQ (complex_fract16* z, const int16_t * x, const complex_fract16* y, int L, int Q) void funNNQ (complex_fract16 * z, const int16_t * x, const complex_fract16* y, int N, int L, int Q) void funNMQ (complex_fract16 * z, const int16_t * x, const complex_fract16* y, int N, int M, int L, int Q) void funNMQs(void* pScr, complex_fract16* z, const int16_t * x, const complex_fract16* y, int N, int L, int Q) </pre>

```
void funNNQs(void* pScr, complex_fract16* z,
             const int16_t * x, const complex_fract16* y, int N, int L, int Q)
```

Arguments	Type	Name	Size	Description
	Input:			
	complex_fract16	x	L*Sx	Real input matrices x
	int16_t	y	L*Sy	Complex input matrices y
	int	N		Matrix dimension (columnar for $M \times N$)
	int	M		Matrix dimension
	int	L		Number of matrices
	int	Q		Position of fractional point in matrix representation (for fixed point API only)
	Output:			
	complex_fract16	z	L*Sz	Complex output matrices z
	Temporary:			
	void	pScr		scratch memory (required only for $n \times n, m \times n$ block order multiplies). Size in bytes is defined by functions rcmatmulnxnn_getScratchSize(), rcmatmulnxmn_getScratchSize(), rcmatvmulnxnn_getScratchSize(), rcmatvmulnxmn_getScratchSize() respectively

Returned Value None

Restrictions

- x, y, z aligned on a 32-bytes boundary
- x, y, z not overlapping
- Q in range 0...16
- L>0 the number of matrices must be positive
- other restrictions see in the table below

Function Name	Prototype	Operation	N	M	P	Allocated Space per Matrix			Restrictions	
						Sx	Sy	Sz	mL	mN, mM
Block order, 16-bit data										
rcmatmul2x2n	funQ	2×2 * 2×2	2	2	2	4	4	4	4	
rcmatmul3x3n	funQ	3×3 * 3×3	3	3	3	16	16	16		
rcmatmul4x4n	funQ	4×4 * 4×4	4	4	4	16	16	16		
rcmatmul8x8n	funQ	8×8 * 8×8	8	8	8	64	64	64		
rcmatmul16x16n	funQ	16×16 * 16×16	16	16	16	256	256	256		
rcmatmulnxnn	funNNQs	N×N * N×N	N	N	N	N×N	N×N	N×N		4
rcmatmulnxmn	funNMQs	M×N * N×M	M	N	M	M×N	N×M	M×M		4
rcmatvmul2x2n	funQ	2×2 * 2×1	2	2	1	4	2	2	4	
rcmatvmul3x3n	funQ	3×3 * 3×1	4	4	1	16	4	4	2	
rcmatvmul4x4n	funQ	4×4 * 4×1	4	4	1	16	4	4	2	
rcmatvmul8x8n	funQ	8×8 * 8×1	8	8	1	64	8	8		
rcmatvmul16x16n	funQ	16×16 * 16×1	16	16	1	256	16	16		
rcmatvmulnxnn	funNNQs	N×N * N×1	N	N	1	N×N	N	N	2	4
rcmatvmulnxmn	funNMQs	M×N * N×1	M	N	1	M×N	N	M	2	4
Streaming order, 16-bit data										
rcmatmul2x2s	funQ	2×2 * 2×2	2	2	2	4	4	4	4	
rcmatmul3x3s	funQ	3×3 * 3×3	3	3	3	9	9	9	4	
rcmatmul4x4s	funQ	4×4 * 4×4	4	4	4	16	16	16	4	
rcmatmul8x8s	funQ	8×8 * 8×8	8	8	8	64	64	64	4	
rcmatmul16x16s	funQ	16×16 * 16×16	16	16	16	256	256	256	4	
rcmatmulnxns	funNNQ	N×N * N×N	N	N	N	N×N	N×N	N×N	4	
rcmatmulnxms	funNMQ	M×N * N×M	M	N	M	M×N	N×M	M×M	4	
rcmatvmul2x2s	funQ	2×2 * 2×1	2	2	1	4	2	2	4	

Function Name	Prototype	Operation	N	M	P	Allocated Space per Matrix			Restrictions	
						S _x	S _y	S _z	mL	mN, mM
rcmatvmul3x3s	funQ	3×3 * 3×1	3	3	1	9	3	3	4	
rcmatvmul4x4s	funQ	4×4 * 4×1	4	4	1	16	4	4	4	
rcmatvmul8x8s	funQ	8×8 * 8×1	8	8	1	64	8	8	4	
rcmatvmul16x16s	funQ	16×16 * 16×1	16	16	1	256	16	16	4	
rcmatvmulnxns	funNNQ	N×N * N×1	N	N	1	N×N	N	N	4	
rcmatvmulnxms	funNMQ	M×N * N×1	M	N	1	M×N	N	M	4	

NOTES:

In the table above, S_x, S_y, S_z denote per-matrix memory allocation amount for matrices x, y, z in elements of corresponding data type. The last column specifies mL – a required divisor for L (by default it is equal to 1). For rows with non-blank mN/mM value the respective API function expects that both N and M are multiples of the specified value.

2.6.5 Real Matrix Transpose

Description	These functions transpose the sequence of real matrices written in a block/streaming order.			
Representation	Q15 input, Q15 output floating point input, floating point output			
Algorithm	$y_{m,n}^{(l)} = x_{n,m}^{(l)}, n = 0 \dots N - 1, m = 0 \dots M - 1, l = 0 \dots L - 1$			
Prototypes	<pre>void tran (int16_t* y, const int16_t* x, int L) void tranf (float32_t* y, const float32_t* x, int L) void tranN (int16_t* y, const int16_t* x, int N,int L) void tranNf (float32_t * y, const float32_t* x, int N,int L) void tranNM (int16_t* y, const int16_t* x, int N,int M,int L) void tranNMF(float32_t* y, const float32_t* x, int N,int M,int L)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t, float32_t	x	L*S	Input matrices x
	int	N,M		Matrix dimensions
	int	L		Number of matrices
	Output:			
	int16_t, float32_t	y	L*S	Output matrices y
Returned Value	None			
Restrictions	x,y	aligned on a 32-bytes boundary		
	x,y	not overlapping		
	L>0	the number of matrices must be positive		
	other restrictions see in the table below			

Function Name	Prototype	Operation	N, M	S	Restrictions	
					mL	mN, mM
Block Order, 16-bit data						
mattran2x2n	tran	2×2 -> 2×2	2	4	4	
mattran4x4n	tran	4×4 -> 4×4	4	16		
mattran8x8n	tran	8×8 -> 8×8	8	64		
mattran16x16n	tran	16×16 -> 16×16	16	256		
mattrannxn	tranN	N×N -> N×N	N	N×N		4
mattrannxmn	tranNM	N×M -> M×N	N, M	N×M		4

Function Name	Prototype	Operation	N, M	S	Restrictions	
					mL	mN, mM
Block Order, 16-bit data						
Block Order, floating point data						
mattran2x2nf	tranf	2×2 -> 2×2	2	4	2	
mattran4x4nf	tranf	4×4 -> 4×4	4	16		
mattran8x8nf	tranf	8×8 -> 8×8	8	64		
mattran16x16nf	tranf	16×16 -> 16×16	16	256		
mattrannxn timer	tranNf	N×N -> N×N	N	N×N		4
mattrannxm timer	tranNMf	N×M -> M×N	N, M	N×M		4
Streaming order, 16-bit data						
mattran2x2s	tran	2×2 -> 2×2	2	4	16	
mattran4x4s	tran	4×4 -> 4×4	4	16	16	
mattran8x8s	tran	8×8 -> 8×8	8	64	16	
mattran16x16s	tran	16×16 -> 16×16	16	256	16	
mattrannxns	tranN	N×N -> N×N	N	N×N	16	
mattrannxms	tranNM	N×M -> M×N	N, M	N×M	16	
Streaming order, floating point data						
mattran2x2sf	tranf	2×2 -> 2×2	2	4	8	
mattran4x4sf	tranf	4×4 -> 4×4	4	16	8	
mattran8x8sf	tranf	8×8 -> 8×8	8	64	8	
mattran16x16sf	tranf	16×16 -> 16×16	16	256	8	
mattrannxnsf	tranNf	N×N -> N×N	N	N×N	8	
mattrannxmsf	tranNMf	N×M -> M×N	N, M	N×M	8	

NOTES:

In the table above, *s* denotes per-matrix memory allocation amount for matrices *x*, *y* in elements of corresponding data type. The last column specifies *mL* – a required divisor for *L* (by default it is equal to 1). For rows with non-blank *mN*/*mM* value the respective API function expects that both *N* and *M* are multiples of the specified value.

2.6.6 Complex Matrix Transpose

Description	These functions transpose the sequence of complex matrices written in a block/streaming order. Transpose is done with complex conjugation. NOTE: In the fixed point routines, due to the changing the sign, saturation is always done for imaginary parts.			
Representation	Q15 input, Q15 output floating point input, floating point output			
Algorithm	$y_{m,n}^{(l)} = x_{n,m}^{(l)*}, n = 0 \dots N-1, m = 0 \dots M-1, l = 0 \dots L-1$			
Prototypes	<pre>void ctran (complex_fract16* y, const complex_fract16* x, int L) void ctranf (complex_float* y, const complex_float* x, int L) void ctranN (complex_fract16* y, const complex_fract16* x, int N, int L) void ctranNf (complex_float* y, const complex_float* x, int N, int L) void ctranNM (complex_fract16* y, const complex_fract16* x, int N, int M, int L) void ctranNMf (complex_float* y, const complex_float* x, int N, int M, int L)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	complex_fract16, complex_float	x	L*S	input matrices <i>x</i>
	int	N, M		matrix dimensions
	int	L		number of matrices
	Output:			
	complex_fract16, complex_float	y	L*S	output matrices <i>y</i>
Returned Value	None			

Restrictions	x, y	aligned on a 32-bytes boundary
	x, y	not overlapping
	$L > 0$	the number of matrices must be positive
	other restrictions see in the table below	

Function name	Prototype	Operation	N, M	S	Restrictions	
					mL	mN
Block order, 16-bit data						
cmattran2x2n	ctran	2×2 → 2×2	2	4	2	
cmattran4x4n	ctran	4×4 → 4×4	4	16		
cmattran8x8n	ctran	8×8 → 8×8	8	64		
cmattran16x16n	ctran	16×16 → 16×16	16	256		
cmattrannxnn	ctranN	N×N → N×N	N	N×N		4
cmattrannxmn	ctranNM	N×M → M×N	N, M	N×M		4
Block order, floating point data						
cmattran2x2nf	ctranf	2×2 → 2×2	2	4		
cmattran4x4nf	ctranf	4×4 → 4×4	4	16		
cmattran8x8nf	ctranf	8×8 → 8×8	8	64		
cmattran16x16nf	ctranf	16×16 → 16×16	16	256		
cmattrannxn timer	ctranNf	N×N → N×N	N	N×N		4
cmattrannxm timer	ctranNMf	N×M → M×N	N, M	N×M		4
Streaming order, 16-bit data						
cmattran2x2s	ctran	2×2 → 2×2	2	4	8	
cmattran4x4s	ctran	4×4 → 4×4	4	16	8	
cmattran8x8s	ctran	8×8 → 8×8	8	64	8	
cmattran16x16s	ctran	16×16 → 16×16	16	256	8	
cmattrannxns	ctranN	N×N → N×N	N	N×N	8	
cmattrannxms	ctranNM	N×M → M×N	N, M	N×M	8	
Streaming order, 16-bit data						
cmattran2x2sf	ctranf	2×2 → 2×2	2	4	4	
cmattran4x4sf	ctranf	4×4 → 4×4	4	16	4	
cmattran8x8sf	ctranf	8×8 → 8×8	8	64	4	
cmattran16x16sf	ctranf	16×16 → 16×16	16	256	4	
cmattrannxnsf	ctranNf	N×N → N×N	N	N×N	4	
cmattrannxmsf	ctranNMf	N×M → M×N	N, M	N×M	4	

NOTES:

In the table above, S denote per-matrix memory allocation amount for matrices x, y in elements of corresponding data type. The last column specifies mL – a required divisor for L (by default it is equal to 1). For rows with non-blank mN/mM value the respective API function expects that both N and M are multiples of the specified value.

2.6.7 Matrix Hermitian Product

Description	These functions calculate the product of a matrix with its Hermitian transpose (in other words, the product of a matrix with its complex conjugate transpose) for the sequence of complex matrices written in a block/streaming order. Note that the result will be Hermitian matrix (or self-adjoint matrix). It is a square matrix with complex entries that is equal to its own conjugate transpose – that is, the element in the i -th row and j -th column is equal to the complex conjugate of the element in the j -th row and i -th column, for all indices i and j .
Representation	Q15 input, Q15 output floating point input, floating point output
Algorithm	$y_{n,p}^{(l)} = \sum_{m=0}^{M-1} x_{m,n}^{(l)*} \cdot x_{p,m}^{(l)}, n = 0 \dots N-1, p = 0 \dots N-1, l = 0 \dots L-1$
Prototypes	<pre>void chermQ (complex_fract16* y, const complex_fract16* x, int L, int Q) void chermf (complex_float* y, const complex_float * x, int L) void chermNNQ (complex_fract16* y, const complex_fract16* x, int N, int L, int Q) void chermNNf (complex_float* y, const complex_float * x, int N, int L)</pre>

```

void chermNMQ (complex_fract16* y,
               const complex_fract16* x, int N, int M, int L, int Q)
void chermNMf (complex_float* y, const complex_float * x, int N, int M, int L)
void chermNNQs (void* pScr, complex_fract16* y,
               const complex_fract16* x, int N, int L, int Q)
void chermNNSf (void* pScr, complex_float * y,
               const complex_float * x, int N, int L)
void chermNMQs (void* pScr, complex_fract16* y,
               const complex_fract16* x, int N, int M, int L, int Q)
void chermNMSf (void* pScr, complex_float * y,
               const complex_fract16* x, int N, int M, int L)

```

Arguments	Type	Name	Size	Description
	Input:			
	complex_fract16, complex_float	x	L*Sx	Input matrices x
	int	N		Matrix dimension (columnar for MxN)
	int	M		Matrix dimension
	int	L		Number of matrices
	int	Q		Position of fractional point in matrix representation (0...16), for fixed point API only
	Output:			
	complex_fract16, complex_float	y	L*Sy	Output matrices y
	Temporary:			
	void	pScr		scratch memory (required only for $n \times n, m \times n$ block order multiplies). Size in bytes is defined by functions cmathermnxnn[f]_getScratchSize(), cmathermnxmn[f]_getScratchSize() respectively

Returned value none

Restrictions

- x, y aligned on a 32-bytes boundary
- x, y not overlapping
- Q 0...16 (for fixed point API only)
- L > 0 the number of matrices must be positive

other restrictions see in the table below

Function name	Prototype	Operation	N	M	Sx	Sy	Restrictions	
							mL	mN, mM
Block order, 16-bit data								
cmatherm2x2n	chermQ	2×2 * 2×2	2	2	4	4	2	
cmatherm4x4n	chermQ	4×4 * 4×4	4	4	16	16		
cmatherm4x4n	chermQ	8×8 * 8×8	8	8	64	64		
cmatherm4x4n	chermQ	16×16 * 16×16	16	16	256	256		
cmathermnxnn	chermNNQ	N×N * N×N	N	N	N×N	N×N		4
cmathermnxmn	chermNMQ	N×M * M×N	N	M	N×M	N×N		4
Block order, floating point data								
cmatherm2x2nf	chermf	2×2 * 2×2	2	2	4	4		
cmatherm4x4nf	chermf	4×4 * 4×4	4	4	16	16		
cmatherm4x4nf	chermf	8×8 * 8×8	8	8	64	64		
cmatherm4x4nf	chermf	16×16 * 16×16	16	16	256	256		
cmathermnxnnf	chermNNf	N×N * N×N	N	N	N×N	N×N		4
cmathermnxmnf	chermNMf	N×M * M×N	N	M	N×M	N×N		4
Streaming order, 16-bit data								
cmatherm2x2s	chermQ	2×2 * 2×2	2	2	4	4	8	
cmatherm4x4s	chermQ	4×4 * 4×4	4	4	16	16	8	
cmatherm8x8s	chermQ	8×8 * 8×8	8	8	64	64	8	

Function name	Prototype	Operation	N	M	S _x	S _y	Restrictions	
							mL	mN , mM
Block order, 16-bit data								
cmatherm16x16s	chermQ	16×16 * 16×16	16	16	256	256	8	
cmathermnxns	chermNNQ	N×N * N×N	N	N	N×N	N×N	8	
cmathermnxms	chermNMQ	N×M * M×N	N	M	N×M	N×N	8	
Streaming order, floating point data								
cmatherm2x2sf	chermf	2×2 * 2×2	2	2	4	4	4	
cmatherm4x4sf	chermf	4×4 * 4×4	4	4	16	16	4	
cmatherm8x8sf	chermf	8×8 * 8×8	8	8	64	64	4	
cmatherm16x16sf	chermf	16×16 * 16×16	16	16	256	256	4	
cmathermnxnsf	chermNNf	N×N * N×N	N	N	N×N	N×N	4	
cmathermnxmsf	chermNMf	N×M * M×N	N	M	N×M	N×N	4	

NOTES:

In the table above, S_x and S_y denote per-matrix memory allocation amount for matrices x, y in elements of corresponding data type. The last column specifies m_L – a required divisor for L (by default it is equal to 1). For rows with non-blank m_N/m_M value the respective API function expects that both N and M are multiples of the specified value.

2.6.8 Streaming to Block Order Conversion

Description	These functions convert matrices representing in the streaming order to block (packed) order
Representation	Q15 input, Q15 output floating point input, floating point output
Prototypes	<pre> void rsb (int16_t* y, const int16_t* x, int L) void rsbmn (int16_t* y, const int16_t* x, int M, int N, int L) void rsbf (float32_t* y, const float32_t* x, int L) void rsbmnf (float32_t* y, const float32_t* x, int M, int N, int L) void csb (complex_fract16* y, const complex_fract16* x, int L) void csbmn (complex_fract16* y, const complex_fract16* x, int M, int N, int L) void csbf (complex_float* y, const complex_float* x, int L) void csbmnf (complex_float* y, const complex_float* x, int M, int N, int L) </pre>

Function name	Prototype	N	M	S _x	S _y	Restrictions	
						mL	M*N
Complex 16-bit data							
csb2x2	csb	2	2	4	4	8	
csb3x3	csb	3	3	9	16	8	
csb4x4	csb	4	4	16	16	8	
csb8x8	csb	8	8	64	64	8	
csb2x1	csb	2	1	2	2	8	
csb4x1	csb	4	1	4	4	8	
csb8x1	csb	8	1	8	8	8	
csbm _{xn}	csbm _n	N	M	M*N		8	
Complex floating point data							
csb2x2f	csbf	2	2	4	4	4	
csb3x3f	csbf	3	3	9	16	4	
csb4x4f	csbf	4	4	16	16	4	
csb8x8f	csbf	8	8	64	64	4	
csb2x1f	csbf	2	1	2	2	4	
csb4x1f	csbf	4	1	4	4	4	
csb8x1f	csbf	8	1	8	8	4	
csbm _{xnf}	csbm _{nf}	N	M	M*N		4	
Real 16-bit data							
rsb2x2	rsb	2	2	4	4	16	
rsb3x3	rsb	3	3	9	16	16	
rsb4x4	rsb	4	4	16	16	16	
rsb8x8	rsb	8	8	64	64	16	
rsb2x1	rsb	2	1	2	2	16	
rsb4x1	rsb	4	1	4	4	16	

rsb8x1	rsb	8	1	8	8	16	
rsbmxn	rsbm	N	M	M*N		16	
Real floating point data							
rsb2x2f	rsbf	2	2	4	4	8	
rsb3x3f	rsbf	3	3	9	16	8	
rsb4x4f	rsbf	4	4	16	16	8	
rsb8x8f	rsbf	8	8	64	64	8	
rsb2x1f	rsbf	2	1	2	2	8	
rsb4x1f	rsbf	4	1	4	4	8	
rsb8x1f	rsbf	8	1	8	8	8	
rsbmxf	rsbm	N	M	M*N		8	

In the table above, S_x and S_y denote per-matrix memory allocation amount for matrices x , y . The last column specifies m_L – a required divisor for L . For `rsbmxf`, `rsbmxf` functions, the size S_y of output matrix is selected according to the general rules from para 2.6.1.

Arguments	Type	Name	Size	Description
Input:				
int16_t, float32_t, complex_fract16, complex_float	x		$L \cdot S_x$	Input matrices x represented in the streaming order
int	M			Matrix dimension
int	N			Matrix dimension (columnar for $M \times N$)
int	L			Number of matrices
Output:				
int16_t, float32_t, complex_fract16, complex_float	y		$L \cdot S_y$	Output matrices y represented in the block order
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	L	see parameter m_L in the table above		
	L, N, M	should be >0		

2.6.9 Block to Streaming Order Conversion

Description	These functions convert matrices representing in the block (packed) order to streaming order.
Representation	Q15 input, Q15 output floating point input, floating point output
Prototypes	<pre> void rbs (int16_t* y, const int16_t* x, int L) void rbsm (int16_t* y, const int16_t* x, int M, int N, int L) void rbsf (float32_t* y, const float32_t* x, int L) void rbsmf (float32_t* y, const float32_t* x, int M, int N, int L) void cbs (complex_fract16* y, const complex_fract16* x, int L) void cbsm (complex_fract16* y, const complex_fract16* x, int M, int N, int L) void cbsf (complex_float* y, const complex_float* x, int L) void cbsmf (complex_float* y, const complex_float* x, int M, int N, int L) </pre>

Function name	Prototype	N	M	S _x	S _y	Restrictions	
						mL	M*N
Complex 16-bit data							
cbs2x2	cbs	2	2	4	4	8	
cbs3x3	cbs	3	3	16	9	8	
cbs4x4	cbs	4	4	16	16	8	
cbs8x8	cbs	8	8	64	64	8	
cbs2x1	cbs	2	1	2	2	8	
cbs4x1	cbs	4	1	4	4	8	

cbs8x1	cbs	8	1	8	8	8	
cbsmxn	cbsmn	N	M		M*N	8	
Complex floating point data							
cbs2x2f	cbsf	2	2	4	4	4	
cbs3x3f	cbsf	3	3	16	9	4	
cbs4x4f	cbsf	4	4	16	16	4	
cbs8x8f	cbsf	8	8	64	64	4	
cbs2x1f	cbsf	2	1	2	2	4	
cbs4x1f	cbsf	4	1	4	4	4	
cbs8x1f	cbsf	8	1	8	8	4	
cbsmxnf	cbsmnf	N	M		M*N	4	
Real 16-bit data							
rbs2x2	rbs	2	2	4	4	16	
rbs3x3	rbs	3	3	16	9	16	
rbs4x4	rbs	4	4	16	16	16	
rbs8x8	rbs	8	8	64	64	16	
rbs2x1	rbs	2	1	2	2	16	
rbs4x1	rbs	4	1	4	4	16	
rbs8x1	rbs	8	1	8	8	16	
rbsmxn	rbsmn	N	M		M*N	16	
Real floating point data							
rbs2x2f	rbsf	2	2	4	4	8	
rbs3x3f	rbsf	3	3	16	9	8	
rbs4x4f	rbsf	4	4	16	16	8	
rbs8x8f	rbsf	8	8	64	64	8	
rbs2x1f	rbsf	2	1	2	2	8	
rbs4x1f	rbsf	4	1	4	4	8	
rbs8x1f	rbsf	8	1	8	8	8	
rbsmxnf	rbsmnf	N	M		M*N	8	

In the table above, S_x and S_y denote per-matrix memory allocation amount for matrices x , y . The last column specifies m_L – a required divisor for L . For `cbsmxn`, `rsbm xn` functions, the size S_y of output matrix is selected according to the general rules from para 2.6.1.

Arguments	Type	Name	Size	Description
Input:				
int16_t, float32_t, complex_fract16, complex float	x		$L \cdot S_x$	Input matrices x represented in the block order
int	M			Matrix dimension
int	N			Matrix dimension (columnar for $M \times N$)
int	L			Number of matrices
Output:				
int16_t, float32_t, complex_fract16, complex float	y		$L \cdot S_y$	Output matrices y represented in the streaming order
Returned value	none			
Restrictions	x, y	aligned on a 32-bytes boundary		
	x, y	not overlapping		
	L	see parameter m_L in the table above		
	L, N, M	should be >0		

2.6.10 Interleaving

Description	These functions interleave 2, 3, 4 or 8 streams. Supports real and complex data.
Representation	Q15 input, Q15 output floating point input, floating point output

Prototypes**Complex 16-bit data:**

```

void cinterleave2 ( complex_fract16 * y,
                   const complex_fract16 * x0, const complex_fract16 * x1, int N )
void cinterleave3 ( complex_fract16 * y,
                   const complex_fract16 * x0, const complex_fract16 * x1,
                   const complex_fract16 * x2, int N )
void cinterleave4 ( complex_fract16 * y,
                   const complex_fract16 * x0, const complex_fract16 * x1,
                   const complex_fract16 * x2, const complex_fract16 * x3, int N )
void cinterleave8 ( complex_fract16 * y,
                   const complex_fract16 * x0, const complex_fract16 * x1,
                   const complex_fract16 * x2, const complex_fract16 * x3,
                   const complex_fract16 * x4, const complex_fract16 * x5,
                   const complex_fract16 * x6, const complex_fract16 * x7, int N )

```

Complex floating point data:

```

void cinterleave2f ( complex_float * y,
                   const complex_float * x0, const complex_float * x1, int N )
void cinterleave3f ( complex_float * y,
                   const complex_float * x0, const complex_float * x1,
                   const complex_float * x2, int N )
void cinterleave4f ( complex_float * y,
                   const complex_float * x0, const complex_float * x1,
                   const complex_float * x2, const complex_float * x3, int N )
void cinterleavemf ( complex_float * y, complex_float * x[], int M, int N )

```

Real 16-bit data:

```

void rinterleave2 ( int16_t * y,
                   const int16_t * x0, const int16_t * x1, int N )
void rinterleave3 ( int16_t * y,
                   const int16_t * x0, const int16_t * x1,
                   const int16_t * x2, int N )
void rinterleave4 ( int16_t * y,
                   const int16_t * x0, const int16_t * x1,
                   const int16_t * x2, const int16_t * x3, int N )
void rinterleave8 ( int16_t * y,
                   const int16_t * x0, const int16_t * x1,
                   const int16_t * x2, const int16_t * x3,
                   const int16_t * x4, const int16_t * x5,
                   const int16_t * x6, const int16_t * x7, int N )

```

Real floating point data:

```

void rinterleave2f ( float32_t * y,
                   const float32_t * x0, const float32_t * x1, int N )
void rinterleave3f ( float32_t * y,
                   const float32_t * x0,
                   const float32_t * x1, const float32_t * x2, int N )
void rinterleave4f ( float32_t * y,
                   const float32_t * x0, const float32_t * x1,
                   const float32_t * x2, const float32_t * x3, int N )
void rinterleavemf ( float32_t * y, float32_t * x[], int M, int N )

```

Arguments	Type	Name	Size	Description
	Input:			
	int16_t, float32_t, complex_fract16, complex_float	x, x0...x7	N	input streams
	int	M		Number of streams
	int	N		number of samples in the input arrays
	Output:			
	int16_t, float32_t, complex_fract16, complex_float	y	M*N	Output stream
Returned value	none			

Restrictions	x0...x7, y, x[0..M-1]	aligned on a 32-bytes boundary
	x0...x7, y, x[0..M-1]	not overlapping
	N	multiple of: 16 for real fixed-point data 8 for complex fixed-point data and real floating-point data 4 for complex floating-point data should be >0
	L, N, M	

2.6.11 Deinterleaving

Description	These functions deinterleave stream into 2, 3, 4, 8 or more streams
Representation	Q15 input, Q15 output floating point input, floating point output

Prototypes

Complex 16-bit data:

```
void cdeinterleave2 ( complex_fract16 * y0, complex_fract16 * y1,
                    const complex_fract16 * x, int N )
void cdeinterleave3 ( complex_fract16 * y0, complex_fract16 * y1,
                    complex_fract16 * y2,
                    const complex_fract16 * x, int N )
void cdeinterleave4 ( complex_fract16 * y0, complex_fract16 * y1,
                    complex_fract16 * y2, complex_fract16 * y3,
                    const complex_fract16 * x, int N )
void cdeinterleave8 ( complex_fract16 * y0, complex_fract16 * y1,
                    complex_fract16 * y2, complex_fract16 * y3,
                    complex_fract16 * y4, complex_fract16 * y5,
                    complex_fract16 * y6, complex_fract16 * y7,
                    const complex_fract16 * x, int N )
```

Complex floating point data:

```
void cdeinterleave2f ( complex_float * y0, complex_float * y1,
                    const complex_float * x, int N )
void cdeinterleave3f ( complex_float * y0, complex_float * y1,
                    complex_float * y2,
                    const complex_float * x, int N )
void cdeinterleave4f ( complex_float * y0, complex_float * y1,
                    complex_float * y2, complex_float * y3,
                    const complex_float * x, int N )
void cdeinterleavemf ( complex_float * y[],
                    const complex_float * x, int M, int N )
```

Real 16-bit data:

```
void rdeinterleave2 ( int16_t * y0, int16_t * y1,
                    const int16_t * x, int N )
void rdeinterleave3 ( int16_t * y0, int16_t * y1, int16_t * y2,
                    const int16_t * x, int N )
void rdeinterleave4 ( int16_t * y0, int16_t * y1, int16_t * y2, int16_t * y3,
                    const int16_t * x, int N )
void rdeinterleave8 ( int16_t * y0, int16_t * y1, int16_t * y2, int16_t * y3,
                    int16_t * y4, int16_t * y5, int16_t * y6, int16_t * y7,
                    const int16_t * x, int N )
```

Real floating point data:

```
void rdeinterleave2f ( float32_t * y0, float32_t * y1,
                    const float32_t * x, int N )
void rdeinterleave3f ( float32_t * y0, float32_t * y1, float32_t * y2,
                    const float32_t * x, int N )
void rdeinterleave4f ( float32_t * y0, float32_t * y1,
                    float32_t * y2, float32_t * y3,
                    const float32_t * x, int N )
void rdeinterleavemf ( float32_t * y[],
                    const float32_t * x, int M, int N )
```

Arguments	Type	Name	Size	Description
	Input:			

	int16_t, float32_t, complex_fract16, complex_float	y, y0...y7	M*N	input stream
	int	M		Number of streams
	int	N		number of samples in the input arrays
	Output:			
	int16_t, float32_t, complex_fract16, complex_float	x	N	output streams
Returned value	none			
Restrictions	x0...x7, y, y[0...M-1] x0...x7, y, y[0...M-1] N L, N, M			
		aligned on a 32-bytes boundary not overlapping multiple of: 16 for real fixed-point data 8 for complex fixed-point data and real floating-point data 4 for complex floating-point data should be >0		

2.7 Matrix Decomposition and Inversion Routines

2.7.1 Introduction

Matrix inversion API provides several sets of utility functions:

- QR-based solution to a system of linear equations with full-rank square matrix
- Solution to an overdetermined least squares problem with Cholesky decomposition used to solve the normal equations system
- Cholesky solution for single matrices (only few sizes supported)
- direct 2x2 and 4x4 inversion routines

IMPORTANT NOTE:

Cholesky and QR matrix inversion routines require aligning load store, Vector Divide, Vector Reciprocal Square Root packages for BBE32EP cores

2.7.1.1 QR Housholder Algorithm

QR decomposition of $A_{M \times N}$ results in an orthogonal (unitary for complex $A_{M \times N}$) matrix $Q_{M \times M}$ and an upper triangular matrix $R_{M \times N}$ with necessarily real values on the main diagonal, such that

$$A_{M \times N} = Q_{M \times M} \cdot R_{M \times N}.$$

Making QR decomposition of matrix $A_{M \times N}$ allows to solve linear equation problem for multiple set of right side vectors B , so right side might be expressed as matrix of size $M \times P$ where P is a number of right side vectors. Mathematically, we aim to find a solution for $X_{M \times P}$ to a real- or complex-valued matrix equation: $A_{M \times N} \cdot X_{N \times P} = B_{M \times P}$, where $A_{M \times N}$ is non-singular. Having obtained $Q_{M \times M}$, $R_{M \times N}$ factors of $A_{M \times N}$, we can write:

$$Q \cdot R \cdot X = A \cdot X = B \rightarrow Q^* \cdot Q \cdot R \cdot X = Q^* \cdot B \rightarrow R_{M \times N} \cdot X_{N \times P} = Q_{M \times M}^* \cdot B_{M \times P},$$

where $\langle \dots \rangle^*$ denotes the conjugate transpose of a matrix. Matrix $R_{M \times N}$ is upper triangular, so in order to solve the transformed equation for $X_{M \times P}$ we simply have to apply the back substitution process. Please note that if $P=M=N$ and the right-hand matrix B is assigned the $M \times M$ identity matrix I_M , then the matrix equation $A_{M \times M} \cdot X_{M \times P} = B_{M \times P}$ turns into $A_{M \times M} \cdot X_{M \times M} = I_M$, so that the procedure described above results in $A^{-1}_{M \times M}$ – the inverse matrix for $A_{M \times M}$.

In the practical implementation, the computation of Q matrix might be avoided due to the performance reasons. Instead, QR decomposition returns so called Housholder vectors v which is used in the backward recursion instead of Q matrix in the more effective fasion. So, linear equation problem is resolved by the following phases:

function `cqr_build_rmxns()` makes QR decomposition returning upper triangle matrix R and Housholder vectors v

function `cqr_calc_qbmmps()` computes right side for upper triangle problem: $QB_{M \times P} = Q^*_{M \times M} \cdot B_{M \times P}$ from Housholder vectors v and original right side B

function `cqr_bkwmps()` applies backward recursion process to the matrices R and $QB_{M \times P}$ returning solution $X_{M \times P}$ of original equation $A_{M \times N} \cdot X_{N \times P} = B_{M \times P}$

On the 2x2 matrices, the overhead for computation of Q matrix is smaller than for separate computation of Housholder vectors and computing QB product. So, for this exceptional case, the procedure consists of only 2 stages:

function `cqr2x2s()` makes QR decomposition returning both Q and R matrices

functions `cqr_bkw2mps()` applies backward recursion process to the matrices R and Q , $B_{2 \times P}$ returning solution $X_{2 \times P}$ of original equation $A_{2 \times 2} \cdot X_{2 \times P} = B_{2 \times P}$

2.7.1.2 Cholesky Algorithm

Now let us formulate the least squares problem for an overdetermined system of M complex-valued linear equations in N unknowns: $A_{M \times N} \cdot x_{N \times 1} = b_{M \times 1}$, $M \geq N \geq \text{rank}(A)$. Depending on mutual properties of $A_{M \times N}$ and $b_{M \times 1}$, this system may or may not have a unique solution for $x_{N \times 1}$, so instead of looking for an exact answer we could try to find $\hat{x}_{N \times 1}$ that minimizes the squared Euclidian norm of the residual $r_{M \times 1} = b_{M \times 1} - A_{M \times N} \cdot x_{N \times 1}$:

$$\hat{x}_{N \times 1} = \underset{x_{N \times 1}}{\text{argmin}} (r_{M \times 1}^* \cdot r_{M \times 1}) = \underset{x_{N \times 1}}{\text{argmin}} \|b_{M \times 1} - A_{M \times N} \cdot x_{N \times 1}\|^2.$$

This is accomplished in three steps:

Build a system of **normal equations**:

$$(A^*_{M \times N} \cdot A_{M \times N} + \sigma^2 \cdot I_N) \cdot x_{N \times 1} = A^*_{M \times N} \cdot b_{M \times 1},$$

where $\sigma^2 \cdot I_N$ is the $N \times N$ identity matrix multiplied with the regularization term. The matrix of normal equations system is Hermitian and positive definite, so we can apply the **Cholesky decomposition** to get the unique upper triangular matrix $R_{N \times N}$ with real and positive main diagonal, such that

$R^*_{N \times N} \cdot R_{N \times N} = A^*_{M \times N} \cdot A_{M \times N} + \sigma^2 \cdot I_N$. Now we re-write the system of normal equations:

$$R^*_{N \times N} \cdot R_{N \times N} \cdot x_{N \times 1} = A^*_{M \times N} \cdot b_{M \times 1}$$

Let us denote $R_{N \times N} \cdot x_{N \times 1} = y_{N \times 1}$. The system of normal equations turns into:

$$R^*_{N \times N} \cdot y_{N \times 1} = A^*_{M \times N} \cdot b_{M \times 1},$$

Where $R^*_{N \times N}$ is a lower triangular matrix, so this system can be solved for $y_{N \times 1}$ by the **forward substitution** process.

Now we are back to $R_{N \times N} \cdot x_{N \times 1} = y_{N \times 1}$, this time with $y_{N \times 1}$ being a known vector. As the matrix $R_{N \times N}$ is upper triangular, the system is a subject for the **backward substitution** process that will result in a unique solution for $x_{N \times 1}$.

This procedure is sometimes referred to as the pseudo-inversion of the normal equations system matrix. The proof that the solution for $x_{N \times 1}$ actually equals $\hat{x}_{N \times 1}$ is far beyond the scope of this Reference; mathematical foundation of the described algorithm could be found in [GOLUB]. [GOLUB] Golub G.H., Van Loan C.F. Matrix Computations, 3-rd edition, The Johns Hopkins University Press, 1996

Although the least squares problem was discussed in terms of column-vectors $x_{N \times 1}$ and $b_{M \times 1}$, it may be trivially extended onto matrices $X_{N \times P}$ and $B_{M \times P}$.

Implementation of the pseudo-inversion was splitted into separate stages to provide users with better options for memory allocation and overflow control.

2.7.1.3 Cholesky Algorithm for Banded Matrices

In many applications (i.e. equalization, fitting, etc.), left-side matrices A may have many zeroes outside few positions along main diagonal as shown at the next figure:

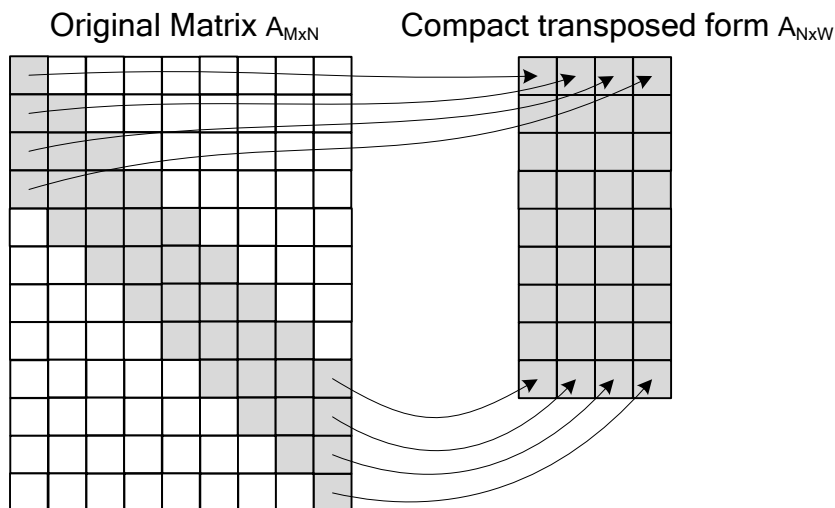


Figure 1. Compact transposed form of banded matrices

These matrices might be easily represented in the compact form and such overdetermined linear equation system may also be decided by Cholesky decomposition. Compact form of representation allows to save computation time and save the memory. General definitions of this algorithm remain the same as described above, but storage format is different – instead of saving all columns, we store only band (of width w) below the ‘diagonal’. As easily seen from the figure $M=N+W-1$. So, the size of each matrix will be $A_{N \times W}$, not $A_{M \times N}$. The unique feature of Cholesky decomposition is that R matrix for such case will be stored in the same format $R_{W \times N}$ and does not require $N \times N$ elements which is substantially bigger for big matrix sizes. All sizes of other matrices (B , y and x) are keeping the same, but these matrices are also written in the transposed form to minimize computational overhead.

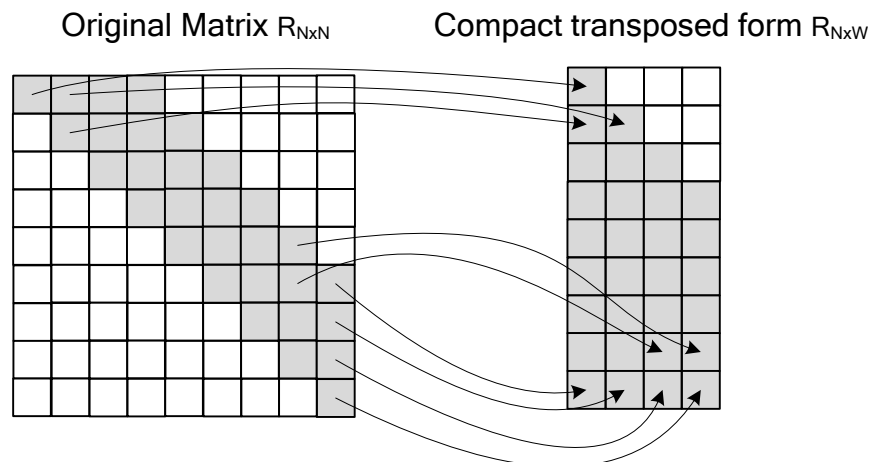


Figure 2. Compact transposed form of R matrices

2.7.1.4 Cholesky Solution for Single Matrices

SIMD nature of BBE32EP architecture is utilized effectively when it operates on matrices written in the streaming order in such a way when 8 or 16 matrices are processed in parallel. Some applications require occasional computation when only one (or few) matrices will be processed. Library includes few routines that apply Cholesky algorithm to single matrices. Those routines are less efficient than routines written for streaming matrix ordering.

2.7.1.5 Direct Matrix Inversion Algorithm

Matrices 2x2 might be inverted directly via determinat computation. For 4x4 matrices, we may use blockwise algorithm by using the following analytic inversion formula:

$$\begin{bmatrix} A & B \\ C & D \end{bmatrix}^{-1} = \begin{bmatrix} A^{-1} + A^{-1}B(D - CA^{-1}B)^{-1}CA^{-1} & -A^{-1}B(D - CA^{-1}B)^{-1} \\ -(D - CA^{-1}B)^{-1}CA^{-1} & (D - CA^{-1}B)^{-1} \end{bmatrix}$$

Where A , B , C , D are 2x2 subblocks of original matrix 4x4. This formal approach does not give good results without slight modification by the obvious reason – good conditioning of 4x4 matrix does not guarantee good conditioning of submatrix A . So, we additionally selecting row/column permutation of original matrix that provides better condition number of upper left submatrix A .

Data layout

Matrix dimensions for linear equations (QR-based methods) and least squares problem (Cholesky methods) are summarized in the table below:

Matrix	QR		Cholesky		Banded Cholesky	
	Number of rows	Number of columns	Number of rows	Number of columns	Number of rows	Number of columns
A	M	M	M	N	W	N
B	M	P	M	P	M	P
Q	M	M				
R	N	N	N	N	W	N
x	N	P	N	P	N	P
y			N	P	N	P

All routines are optimized to utilize all the benefits of BBE32EP core: parallel data loads, SIMD processing and so on. So, they use streaming matrices format when matrix elements with same indexes are grouping together. So, for real data the number of matrices should be a multiple of 16 and for complex matrices it should be a multiple of 8. All data should be aligned on 32-bytes boundary.

All routines require scratch memory for temporary data. The size of that memory depends on matrix dimensions and defined by special macros for each specific function.

Library contains the routines optimized for specific small matrix sizes.

2.7.1.6 Overflow control and fixed point issues

The nature of algorithms does not allow to predict what will be dynamic range of outputs for arbitrary inputs.

Even for well defined matrices, the results of QR/Cholesky decomposition may require bigger dynamic range than original data, so overflow may occurs even in the decomposition stage. Amount of possible data change is measured by growth factor with is formally limited by 2^{M-1} , where M is a matrix size, however in many practical cases it is around $\sqrt{M}$.

So, for well-defined small matrices like 2x2 or 4x4 the 13-bit input dynamic range is a good compromise for most cases.

Right side rotation made by `cqr_calc_qbmxps()` also might cause the overflow. Since this operation is affine projection of original vector **B** in some direction, correct computation is guaranteed if the sum of squares of elements in any column given in Q15 does not exceed 1.

Cholesky decomposition has own specific problem – it may fail due to the round-off errors.

Increasing σ^2 usually helps to minimize the probability of this failure, however, higher values of σ^2 result in less accuracy.

Backward and forward substitution routines determine the output fixed point representation by dynamic range of diagonal elements in the upper-diagonal matrix. This is based on the fact that for well-defined matrices the dynamic range is limited by diagonal data. However, this condition may not be obeyed due to the numerical properties of inputs. So, to minimize potential overflow in the recursion stages that routines are supplied with special parameter that instructs how many extra bits to leave for data outside the main diagonal. Typically, 1 or 2 bits is enough. If recursion routines indicate an overflow, user may increase that number and repeat recursion.

QR/Cholesky decomposition do not change fixed-point representation of data – let say if original data come in Q10, the outputs will be in Q10 as well. QR backward substitution transforms fixed-point representation according to the following formula:

$$Q(X) = Q(B) - Q(A) + 10$$

where

$Q(X)$	representation of result X
$Q(A)$	representation of original matrix A
$Q(B)$	representation of right side B

In the Cholesky, user should select manually representation of data at the each stage.

2.7.2 QR decomposition by Householder reflections

Description	QR decomposition for real (complex) matrices by Householder reflections. For a real (complex) matrix $A_{M \times N}$, the decomposition results in an orthogonal (unitary) matrix $Q_{M \times M}$ and an upper triangular matrix $R_{M \times N}$, such that $A_{M \times N} = Q_{M \times M} \cdot R_{M \times N}$. Upper triangular matrix R computed for a matrix A always have real numbers on its main diagonal. Instead of direct computation of Q matrix, these functions produce a set of $M-1$ Householder vectors V for each of input matrices A. This approach allow us to save CPU cycles and memory when solving a system of linear equations: it is cheaper to perform $M-1$ elementary reflections for a right-hand side vector if compared to explicit multiplication of that vector by matrix Q. NOTES: Due to round-off errors and limited dynamic range of the underlying fixed-point arithmetic, small magnitude numbers may appear below the main diagonal of matrix R. Those elements are actually computed and included in the output data so the user may use them to estimate the quality of a particular result. The decomposition algorithm is applied to a few matrices per single call, with input/output matrix sequences being stored in the streaming order (refer to 2.6.1 for details). Data processing is performed in-place, i.e. output matrices R replace the input matrices A. For fixed point API, Housholder vectors V are calculated in Q14 fixed-point representation. For the upper triangular matrix R the fixed-point format is the same as for the input matrix.				
Representation	16-bit fixed point floating point				
Algorithm	$[Q_{M \times M}^{(l)}, R_{M \times N}^{(l)}] = qr_hs(A_{M \times N}^{(l)}): Q_{M \times M}^{(l)} \cdot R_{M \times N}^{(l)} \cong A_{M \times N}^{(l)}, l = 0 \dots L-1$				
Prototypes (streaming ordered matrices)	<pre> void cfunqr (void* pScr, complex_fract16 * Q, complex_fract16 * R, int L) void funqr (void* pScr, int16_t * Q, int16_t * R, int L) void cfun (void* pScr, complex_fract16* V, complex_fract16* R, int L) void cfunf (void* pScr, complex_float* V, complex_float* R, complex_float* D, int L); void rfun (void* pScr, int16_t* V, int16_t* R, int L) void rfunf (void* pScr, float32_t* V, float32_t * R, float32_t * D, int L) void cfunmn (void* pScr, complex_fract16* V, complex_fract16* R, int M, int N, int L) void cfunmnf(void* pScr, complex_float * V, complex_float * R, complex_float *D, int M, int N, int L) void rfunmn (void* pScr, int16_t* V, int16_t* R, int M, int N, int L) void rfunmnf(void* pScr, float32_t* V, float32_t* R, int M, int N, int L) </pre>				
Function Name	Prototype	M	N	mL	Scratch size, s
Fixed point, complex-valued QR decomposition (R + Housholder vectors)					
cqr2x2s	cfunqr	2	2	8	cqr2x2s_getScratchSize
cqr_build_r3x3s	cfun	3	3	8	cqr_build_r3x3s_getScratchSize
cqr_build_r4x2s	cfun	4	2	8	cqr_build_r4x2s_getScratchSize
cqr_build_r4x4s	cfun	4	4	8	cqr_build_r4x4s_getScratchSize
cqr_build_r5x5s	cfun	5	5	8	cqr_build_r5x5s_getScratchSize
cqr_build_r6x6s	cfun	6	6	8	cqr_build_r6x6s_getScratchSize
cqr_build_r7x7s	cfun	7	7	8	cqr_build_r7x7s_getScratchSize
cqr_build_r8x4s	cfun	8	4	8	cqr_build_r8x4s_getScratchSize
cqr_build_r8x8s	cfun	8	8	8	cqr_build_r8x8s_getScratchSize
cqr_build_r16x8s	cfun	16	8	8	cqr_build_r16x8s_getScratchSize
cqr_build_r16x16s	cfun	16	16	8	cqr_build_r16x16s_getScratchSize
cqr_build_rmxns	cfunmn			8	cqr_build_rmxns_getScratchSize
Floating point, complex-valued QR decomposition (R + Housholder vectors)					
cqr_build_r2x2sf	cfunf	2	2	4	cqr_build_r2x2sf_getScratchSize
cqr_build_r3x3sf	cfunf	3	3	4	cqr_build_r3x3sf_getScratchSize
cqr_build_r4x4sf	cfunf	4	4	4	cqr_build_r4x4sf_getScratchSize
cqr_build_rmxnsf	cfunmnf			4	cqr_build_rmxnsf_getScratchSize
Fixed point, real-valued QR decomposition (R + Housholder vectors)					
qr2x2s	funqr	2	2	16	qr2x2s_getScratchSize
qr_build_r4x4s	rfun	4	4	16	qr_build_r4x4s_getScratchSize
qr_build_r8x8s	rfun	8	8	16	qr_build_r8x8s_getScratchSize
qr_build_rmxns	rfunmn			16	qr_build_rmxns_getScratchSize

Floating point, real-valued QR decomposition (R + Housholder vectors)					
qr_build_r2x2sf	rfunf	2	2	8	qr_build_r2x2sf_getScratchSize
qr_build_r3x3sf	rfunf	3	3	8	qr_build_r3x3sf_getScratchSize
qr_build_r4x4sf	rfunf	4	4	8	qr_build_r4x4sf_getScratchSize
qr_build_rmxnsf	rfunmnf			8	qr_build_rmxnsf_getScratchSize
Arguments	Type	Name	Size		Description
	Input:				
	int	M			Matrix dimension (number of rows for $M \times N$)
	int	N			Matrix dimension (number of columns for $M \times N$)
	int	L			Number of matrices
	Input/Output:				
	complex_fract16, complex_float, float32_t, int16_t	R	[M*N] [L]		Sequence of L input matrices A_i ; upon completion is replaced with L upper triangular matrices R
	Output:				
	complex_fract16, complex_float, float32_t, int16_t	V	[]		Sequence of L Housholder vectors v . Size of Housholder vector storage is $QR[F]_{_SIZE_V}(M, N, L)$ bytes for real data and $CQR[F]_{_SIZE_V}(M, N, L)$ bytes for complex matrices
	complex_float, float32_t	D	[N*L]		reciprocals of main diagonal written in a special format (for floating point API only)
	complex_fract16, int16_t	Q	[M*M] [L]		output rotation matrices (L matrices of size $M \times M$) – only for dedicated 2x2 fixed point APIs
	Temporary:				
	void*	pScr			Scratch memory area of S bytes, see table above. See 1.12.4 for an example of scratch area allocation.
Restrictions	V, R, pScr must not overlap V, R, pScr must be aligned on a 32-bytes boundary Number of matrices L must be a multiple of mL (see table above) $M \geq N$ M, N, L should be bigger than 1				
Returned value	none				

**Prototypes
(block ordered
matrices)**

```

void cfun (void *pScr,
           complex_fract16* A, complex_fract16* V, complex_fract16* D, int L);
void cfunf (void *pScr,
            complex_float* A, complex_float* V, complex_float * D, int L);
void rfunf (void *pScr,
            float32_t* A, float32_t* V, float32_t* D, int L)
void cfunmn(void *pScr,
            complex_fract16* A, complex_fract16* V, complex_fract16* D,
            int M, int N, int L)
void cfunmnf(void *pScr,
             complex_float* A, complex_float* V, complex_float * D,
             int M, int N, int L)
void rfunmnf(void *pScr,
             float32_t* A, float32_t* V, float32_t* D,
             int M, int N, int L)

```

Function Name	Prototype	M	N	mN	Scratch size, s
Fixed point, complex-valued QR decomposition (R + Housholder vectors)					
cqr_build_r8x8n	cfun	8	8		cqr_build_r8x8n_getScratchSize
cqr_build_r16x16n	cfun	16	16		cqr_build_r16x16n_getScratchSize
cqr_build_r32x32n	cfun	32	32		cqr_build_r32x32n_getScratchSize
cqr_build_rmxnn	cfunmn			4	cqr_build_rmxnn_getScratchSize
Floating point, complex-valued QR decomposition (R + Housholder vectors)					
cqr_build_r8x8nf	cfunf	8	8		cqr_build_r8x8nf_getScratchSize
cqr_build_r16x16nf	cfunf	16	16		cqr_build_r16x16nf_getScratchSize
cqr_build_rmxnnf	cfunmnf			4	cqr_build_rmxnnf_getScratchSize
Floating point, real-valued QR decomposition (R + Housholder vectors)					
qr_build_r8x8nf	rfunf	8	8		qr_build_r8x8nf_getScratchSize
qr_build_r16x16nf	rfunf	16	16		qr_build_r16x16nf_getScratchSize
qr_build_rmxnn	rfunmnf			4	qr_build_rmxnnf_getScratchSize

Arguments	Type	Name	Size	Description
	Input:			
	int	M		Matrix dimension (number of rows for $M \times N$)
	int	N		Matrix dimension (number of columns for $M \times N$)
		SA, SV, SD		Matrix sizes SA, SV, SD are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for complex_fract16, float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float or, if it is less, to the closest bigger multiple of degree of 2: SA=size (M*N) , SV=size ((2*M-N+1) *N/2+N) *L) , SD=size (N)
	int	L		Number of matrices
	Input/Output:			
	complex_fract16, complex_float, float32_t	A	[L] [SA]	at the input it is the sequence of L complex matrices A. at the end of the process, matrices R are replaced inputs. In case of non-square matrices ($N \neq M$), only $N \times N$ elements of each matrix will be valid.
	Output:			
	complex_fract16, complex_float, float32_t	V	[SV]	sequence of L Housholder and rotation vectors
	complex_fract16, float32_t, complex_float	D	[L*SD]	reciprocal of main diagonal in the special format
	Temporary:			
	void*	pScr		Scratch memory area of s bytes, see table above. See 1.12.4 for an example of scratch area allocation.
Restrictions	A, V, pScr must not overlap All matrices and the scratch should not overlap and be aligned on 32-bytes boundary M, N, L should be bigger than 1 M and N should be a multiple of mN (4) Number of columns for input matrices A must not exceed the number of rows: $N \leq M$.			
Returned value	none			

2.7.3 Rotation by Housholder Vectors

Description	These functions apply Householder reflections to L $M \times P$ matrices B in the course of solving a set of linear problems $A \cdot X = B$ through the QR decomposition of matrices A: $A \cdot X = B$, $A = Q \cdot R \Rightarrow Q \cdot R \cdot X = B \Rightarrow R \cdot X = Q' \cdot B$. Instead of direct multiplication of each matrix B by conjugate transpose of the corresponding matrix Q, we use a set of vectors v to perform a sequence of Householder reflections (see QR decomposition routines). Fixed point representation of output matrices is the same as for input matrices. NOTES: 1. Data layout for matrices is selected as for other matrices written in a stream order. 2. Data transform is performed in-place.
Representation	16-bit fixed point floating point

Algorithm

$$[QB_{M \times P}^{(l)}] = Q_{M \times M}^{(l)*} \cdot B_{M \times P}^{(l)}, l = 0 \dots L-1$$

Prototypes
(streaming
ordered
matrices)

```
void cfun(void *pScr, complex_fract16 *B, const complex_fract16 *V, int L);
void cfunmnp(void *pScr, complex_fract16 *B, const complex_fract16 *V, int M, int N,
int P, int L);
void rfun(void *pScr, int16_t *B, const int16_t *V, int L);
void rfunmnp(void *pScr, int16_t *B, const int16_t *V, int M, int N, int P, int L);
void cfunmnpf(void *pScr, complex_float* B, const complex_float* V, int M, int N, int
P, int L);
void cfunf(void *pScr, complex_float* B, const complex_float* V, int L);
void rfunmnpf(void *pScr, float32_t* B, const float32_t* V, int M, int N, int P, int L);
void rfunf(void *pScr, float32_t* B, const float32_t* V, int L);
```

Function Name	Prototype	M	N	P	mL	Scratch size, s
Fixed point, complex-valued QB transformation						
cqr_calc_qb3x3x1s	cfun	3	3	1	8	cqr_calc_qb3x3x1s_getScratchSize
cqr_calc_qb3x3x3s	cfun	3	3	3	8	cqr_calc_qb3x3x3s_getScratchSize
cqr_calc_qb4x2x1s	cfun	4	2	1	8	cqr_calc_qb4x2x1s_getScratchSize
cqr_calc_qb4x4x1s	cfun	4	4	1	8	cqr_calc_qb4x4x1s_getScratchSize
cqr_calc_qb4x4x4s	cfun	4	4	4	8	cqr_calc_qb4x4x4s_getScratchSize
cqr_calc_qb5x5x1s	cfun	5	5	1	8	cqr_calc_qb5x5x1s_getScratchSize
cqr_calc_qb5x5x5s	cfun	5	5	5	8	cqr_calc_qb5x5x5s_getScratchSize
cqr_calc_qb6x6x1s	cfun	6	6	1	8	cqr_calc_qb6x6x1s_getScratchSize
cqr_calc_qb6x6x6s	cfun	6	6	6	8	cqr_calc_qb6x6x6s_getScratchSize
cqr_calc_qb7x7x1s	cfun	7	7	1	8	cqr_calc_qb7x7x1s_getScratchSize
cqr_calc_qb7x7x7s	cfun	7	7	7	8	cqr_calc_qb7x7x7s_getScratchSize
cqr_calc_qb8x4x1s	cfun	8	4	1	8	cqr_calc_qb8x4x1s_getScratchSize
cqr_calc_qb8x8x1s	cfun	8	8	1	8	cqr_calc_qb8x8x1s_getScratchSize
cqr_calc_qb8x8x8s	cfun	8	8	8	8	cqr_calc_qb8x8x8s_getScratchSize
cqr_calc_qb16x8x1s	cfun	16	8	1	8	cqr_calc_qb16x8x1s_getScratchSize
cqr_calc_qb16x16x1s	cfun	16	16	1	8	cqr_calc_qb16x16x1s_getScratchSize
cqr_calc_qb16x16x16s	cfun	16	16	16	8	cqr_calc_qb16x16x16s_getScratchSize
cqr_calc_qbmnxnmps	cfunmnp	M	N	P	8	cqr_calc_qbmnxnmps_getScratchSize
Floating point, complex-valued QB transformation						
cqr_calc_qb2x2x1sf	cfunf	2	2	1	4	cqr_calc_qb2x2x1sf_getScratchSize
cqr_calc_qb3x3x1sf	cfunf	3	3	1	4	cqr_calc_qb3x3x1sf_getScratchSize
cqr_calc_qb4x4x1sf	cfunf	4	4	1	4	cqr_calc_qb4x4x1sf_getScratchSize
cqr_calc_qbmnxnpsf	cfunf	M	N	P	4	cqr_calc_qbmnxnpsf_getScratchSize
Fixed point, real-valued QB transformation						
qr_calc_qb4x4x1s	rfun	4	4	1	16	qr_calc_qb4x4x1s_getScratchSize
qr_calc_qb4x4x4s	rfun	8	4	4	16	qr_calc_qb4x4x4s_getScratchSize
qr_calc_qb8x8x1s	rfun	8	8	8	16	qr_calc_qb8x8x1s_getScratchSize
qr_calc_qb8x8x8s	rfun	8	8	1	16	qr_calc_qb8x8x8s_getScratchSize
qr_calc_qbmnxnmps	rfunmnp	M	N	P	16	qr_calc_qbmnxnmps_getScratchSize
Floating point, complex-valued QB transformation						
qr_calc_qb2x2x1sf	rfunf	2	2	1	8	qr_calc_qb2x2x1sf_getScratchSize
qr_calc_qb3x3x1sf	rfunf	3	3	1	8	qr_calc_qb3x3x1sf_getScratchSize
qr_calc_qb4x4x1sf	rfunf	4	4	1	8	qr_calc_qb4x4x1sf_getScratchSize
qr_calc_qbmnxnpsf	rfunmnp	M	N	P	8	qr_calc_qbmnxnpsf_getScratchSize

Arguments	Type	Name	Size	Description
	Input:			
	int	M		Matrix dimension (number of rows for $M \times N$)
	int	P		Matrix dimension (number of columns for $M \times P$)
	int	L		Number of matrices
	complex_fract16, complex_float, int16_t, float32_t	V	[]	Sequence of L Housholder vectors V. Size of Housholder vector storage is $QR[F]_{_SIZE_V}(M, N, L)$ bytes for real data and $CQR[F]_{_SIZE_V}(M, N, L)$ bytes for complex matrices
	Input/Output:			
	complex_fract16, complex_float, int16_t, float32_t	B	[M*P][L]	On the input: matrices B (L matrices of size $M \times P$) On the output: Matrices $Q^* \cdot B$ (L matrices of size $M \times P$)
	Temporary:			
	void*	pScr		Scratch memory area of s bytes. See 1.12.4 for an example of scratch area allocation.

Restrictions

V, R, pScr must not overlap
V, R, pScr must be aligned on a 32-bytes boundary
Number of matrices L must be a multiple of mL (see Table above).
M, N, L should be bigger than 1

Returned value none

Prototypes
(block ordered matrices)

```
void cfun (void *pScr, complex_fract16* B, const complex_fract16* V, int L);
void cfunf (void *pScr, complex_float* B, const complex_float* V, int L);
void rfunf (void *pScr, float32_t* B, const float32_t* V, int L);
void cfunmnp (void *pScr,
    complex_fract16* B, const complex_fract16* V, int M, int N, int P, int L);
void cfunmnpf (void *pScr,
    complex_float* B, const complex_float* V, int M, int N, int P, int L);
void rfunmnpf (void *pScr, float32_t* B, const float32_t* V, int M, int N, int P, int L)
```

Function Name	Prototype	M	N	P	mN	Scratch size, s
Fixed point, complex-valued QB transformation						
cqr_calc_qb8x8x1n	cfun	8	8	1		cqr_calc_qb8x8x1n_getScratchSize
cqr_calc_qb8x8x8n	cfun	8	8	8		cqr_calc_qb8x8x8n_getScratchSize
cqr_calc_qb16x16x1n	cfun	16	16	1		cqr_calc_qb16x16x1n_getScratchSize
cqr_calc_qb16x16x16n	cfun	16	16	16		cqr_calc_qb16x16x16n_getScratchSize
cqr_calc_qb32x32x1n	cfun	32	32	1		cqr_calc_qb32x32x1n_getScratchSize
cqr_calc_qb32x32x32n	cfun	32	32	32		cqr_calc_qb32x32x32n_getScratchSize
cqr_calc_qbmnxnpxn	cfunmnp				4	cqr_calc_qbmnxnpxn_getScratchSize
Floating point, complex-valued QB transformation						
cqr_calc_qb8x8x1nf	cfunf	8	8	1		cqr_calc_qb8x8x1nf_getScratchSize
cqr_calc_qb16x16x1nf	cfunf	16	16	1		cqr_calc_qb16x16x1nf_getScratchSize
cqr_calc_qbmnxnpxnf	cfunmnpf				4	cqr_calc_qbmnxnpxnf_getScratchSize
Floating point, real-valued QB transformation						
qr_calc_qb8x8x1nf	rfunf	8	8	1		qr_calc_qb8x8x1nf_getScratchSize
qr_calc_qb16x16x1nf	rfunf	16	16	1		qr_calc_qb16x16x1nf_getScratchSize
qr_calc_qbmnxnpxnf	rfunmnpf				4	qr_calc_qbmnxnpxnf_getScratchSize

Arguments	Type	Name	Size	Description
	Input:			
	int	M		Matrix dimension (number of rows for $M \times N$)
	int	P		Matrix dimension (number of columns for $M \times P$)
		SB, SV		Matrix sizes SB, SV are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for complex_fract16, float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float or, if it is less, to the closest bigger multiple of degree of 2: $SB = \text{size}(M \times P)$, $SV = \text{size}(((2 \times M - N + 1) \times N / 2 + N) \times L)$
	int	L		Number of matrices
	complex_fract16, complex_float, float32_t	V	[SV]	sequence of L Householder and rotation vectors
	Input/Output:			
	complex_fract16, complex_float, float32_t	B	[L][SB]	On the input: matrices B (L matrices of size $M \times P$) On the output: Matrices $Q' \times B$ (L matrices of size $M \times P$)
	Temporary:			
	void*	pScr		Scratch memory area of S bytes. See 1.12.4 for an example of scratch area allocation.
Restrictions	B, V, pScr must not overlap All matrices and the scratch should not overlap and be aligned on 32-bytes boundary M, N, L should be bigger than 1 M and N should be a multiple of mN (4) $N \leq M$			
Returned value	none			

2.7.4 QR-based Back Substitution

Description	Provided with QR factors of a square matrix $A_{M \times M}$, perform the back substitution to find the solution to a real- or complex-valued matrix equation: $A_{M \times N} \times X_{N \times P} = B_{M \times P}$. Last stage of solving a set of L linear problems $A_{M \times M} \times X_{M \times P} = B_{M \times P}$ through the QR decomposition by Householder reflections: back substitution process for L systems of linear equations $R_{M \times N} \times X_{N \times P} = Q B_{M \times P}$, where R is an $M \times N$ upper-triangular matrix, $X_{N \times P}$ is an $N \times P$ matrix of unknowns, $Q B_{M \times P}$ is an $M \times P$ matrix resulting from Householder reflections being applied to the right side matrix $B_{M \times P}$ of the original linear problem: $Q B_{M \times P} = Q_{M \times M} \times B_{M \times P}$. Fixed-point representation for output data is a function of fixed-point format of input data: $Q(X) = Q(QB) - Q(R) + 10$, where $Q(x)$ stands for the Fixed-Point Position of data item x. NOTES: 1. Data layout for matrices is selected as for other matrices written in a stream order. 2. Data transform is performed in-place
Representation	16-bit fixed point floating point

Algorithm

Given L systems of matrix equations:

$$Q_{M \times M}^{(l)} \cdot R_{M \times N}^{(l)} \cdot X_{N \times P}^{(l)} = A_{M \times N}^{(l)} \cdot X_{N \times P}^{(l)} = B_{M \times P}^{(l)}, l = \overline{0 \dots L-1},$$

find a solution for each $X_{N \times P}^{(l)}$.

**Prototypes
(streaming
ordered matrices)**

```

void cfun (void* pScr, complex_fract16* B, const complex_fract16* R,
           int L);
void cfunf (void* pScr, complex_float* B, const complex_float* R,
            const complex_float* D, int L);
void cfunnpf (void* pScr, complex_float* B, const complex_float* R,
              const complex_float* D, int N, int P, int L);
void rfun (void* pScr, int16_t* B, const int16_t* R, int L);
void rfunf (void* pScr, float32_t* B, const float32_t* R,
            const float32_t* D, int L);
void cfunnnp (void* pScr, complex_fract16* B, const complex_fract16* R,
              int N, int P, int L);
void rfunnp (void* pScr, int16_t* B, const int16_t* R,
              int N, int P, int L);
void rfunnnpf (void* pScr, float32_t* B, const float32_t* R,
               const float32_t* D, int N, int P, int L);
void cfun2xP (void* pScr, complex_fract16* B, const complex_fract16* Q,
              const complex_fract16* R, int L);
void rfun2xP (void* pScr, int16_t* B, const int16_t* Q,
              const int16_t* R, int L);

```

Function Name	Proto	N	P	mL	Scratch size, s
Fixed point, complex-valued back substitution					
cqr bkw2x2x1s	cfun2xP	2	1	8	cqr bkw2x2x1s_getScratchSize
cqr bkw2x2x2s	cfun2xP	2	2	8	cqr bkw2x2x2s_getScratchSize
cqr bkw2x1s	cfun	2	1	8	cqr bkw2x1s_getScratchSize
cqr bkw3x1s	cfun	3	1	8	cqr bkw3x1s_getScratchSize
cqr bkw3x3s	cfun	3	3	8	cqr bkw3x3s_getScratchSize
cqr bkw4x1s	cfun	4	1	8	cqr bkw4x1s_getScratchSize
cqr bkw4x4s	cfun	4	4	8	cqr bkw4x4s_getScratchSize
cqr bkw5x1s	cfun	5	1	8	cqr bkw5x1s_getScratchSize
cqr bkw5x5s	cfun	5	5	8	cqr bkw5x5s_getScratchSize
cqr bkw6x1s	cfun	6	1	8	cqr bkw6x1s_getScratchSize
cqr bkw6x6s	cfun	6	6	8	cqr bkw6x6s_getScratchSize
cqr bkw7x1s	cfun	7	1	8	cqr bkw7x1s_getScratchSize
cqr bkw7x7s	cfun	7	7	8	cqr bkw7x7s_getScratchSize
cqr bkw8x1s	cfun	8	1	8	cqr bkw8x1s_getScratchSize
cqr bkw8x8s	cfun	8	8	8	cqr bkw8x8s_getScratchSize
cqr bkw16x1s	cfun	16	1	8	cqr bkw16x1s_getScratchSize
cqr bkw16x16s	cfun	16	18	8	cqr bkw16x16s_getScratchSize
cqr bkwnxps	cfunnnp			8	cqr bkwnxps_getScratchSize
Floating point, complex-valued back substitution					
cqr bkw2x1sf	cfunf	2	1	4	cqr bkw2x1sf_getScratchSize
cqr bkw3x1sf	cfunf	3	1	4	cqr bkw3x1sf_getScratchSize
cqr bkw4x1sf	cfunf	4	1	4	cqr bkw4x1sf_getScratchSize
cqr bkwnxpsf	cfunnnpf			4	cqr bkwnxpsf_getScratchSize
Fixed point, real-valued back substitution					
qr bkw2x2x1s	rfun2xP	2	1	16	qr bkw2x2x1s_getScratchSize
qr bkw2x2x2s	rfun2xP	2	2	16	qr bkw2x2x2s_getScratchSize
qr bkw4x1s	rfun	4	1	16	qr bkw4x1s_getScratchSize
qr bkw4x4s	rfun	4	4	16	qr bkw4x4s_getScratchSize
qr bkw8x1s	rfun	8	1	16	qr bkw8x1s_getScratchSize
qr bkw8x8s	rfun	8	8	16	qr bkw8x8s_getScratchSize
qr bkwnxps	rfun			16	qr bkwnxps_getScratchSize
Floating point, real-valued back substitution					
qr bkw2x1sf	rfunf	2	1	8	qr bkw2x1sf_getScratchSize
qr bkw3x1sf	rfunf	3	1	8	qr bkw3x1sf_getScratchSize
qr bkw4x1sf	rfunf	4	1	8	qr bkw4x1sf_getScratchSize
qr bkwnxpsf	rfunnnpf			8	qr bkwnxpsf_getScratchSize

Arguments	Type	Name	Size	Description
Input:				
int	M			Matrix dimension (number of rows for $M \times N$)
int	P			Matrix dimension (number of columns for $M \times P$)
int	L			Number of matrix equations
complex_fract16, complex_float, int16_t, float32_t	R		$[M \times N] [L]$	Sequence of L upper triangle input matrices R .
complex_float, float32_t	D		$[N \times L]$	reciprocals of main diagonal written in a special format (for floating point API only)
Input/Output:				
complex_fract16, complex_float, int16_t, float32_t	QB		$[M \times P] [L]$	Sequence of L input matrices $QB = Q' * B$; upon completion is replaced with L solution matrices X . Since real size of solution matrix x is $N \times P$, not $M \times P$, last $(M-N) * P$ elements will be ignored.
Temporary:				
void*	pScr			Scratch memory area of s bytes

Restrictions

$R, B, pScr$ must not overlap

$R, B, pScr$ must be aligned on a 32-bytes boundary

Number of matrix equations L must be a multiple of mL (see table above)

M, N, L should be bigger than 1

Returned value none

Prototypes
(block ordered matrices)

```
void cfun (void *pScr, complex_fract16* X, const complex_fract16* R,
const complex_fract16* D, int qABX, int L)
void cfunf (void *pScr, complex_float* X, const complex_float* R,
const float32_t* D, int L)
void rfunf (void *pScr, float32_t* X, const float32_t* R,
const float32_t* D, int L)
void cfunnp (void *pScr, complex_fract16* X, const complex_fract16* R,
const complex_fract16* D, int qABX, int M, int N, int P, int L)
void cfunnpf (void *pScr, complex_float* X, const complex_float* R,
const float32_t* D, int M, int N, int P, int L)
void rfunnpf (void *pScr, float32_t* X, const float32_t* R,
const float32_t* D, int M, int N, int P, int L)
```

Function Name	Protot ype	N	P	mN	Scratch size, s
Fixed point, complex-valued back substitution					
cqr_bkw8x8x1n	cfun	8	1		cqr_bkw8x8x1n_getScratchSize
cqr_bkw8x8x8n	cfun	8	8		cqr_bkw8x8x8n_getScratchSize
cqr_bkw16x16x1n	cfun	16	1		cqr_bkw16x16x1n_getScratchSize
cqr_bkw16x16x16n	cfun	16	16		cqr_bkw16x16x16n_getScratchSize
cqr_bkw32x32x1n	cfun	32	1		cqr_bkw32x32x1n_getScratchSize
cqr_bkw32x32x32n	cfun	32	32		cqr_bkw32x32x32n_getScratchSize
cqr_bkwmnxnpxn	cfunnp			4	cqr_bkwmnxnpxn_getScratchSize
Floating point, complex-valued back substitution					
cqr_bkw8x8x1nf	cfunf	8	1		cqr_bkw8x8x1nf_getScratchSize
cqr_bkw16x16x1nf	cfunf	16	1		cqr_bkw16x16x1nf_getScratchSize
cqr_bkwmnxnpxnf	cfunnpf			4	cqr_bkwmnxnpxnf_getScratchSize
Floating point, real-valued back substitution					
qr_bkw8x8x1nf	rfunf	8	1		qr_bkw8x8x1nf_getScratchSize
qr_bkw16x16x1nf	rfunf	16	1		qr_bkw16x16x1nf_getScratchSize
qr_bkwmnxnpxnf	rfunnpf			4	qr_bkwmnxnpxnf_getScratchSize

Arguments	Type	Name	Size	Description
	Input:			
	int	M		Matrix dimension (number of rows for $M \times N$)
	int	P		Matrix dimension (number of columns for $M \times P$)
		SA, SB		Matrix sizes SA, SB are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for complex_fract16, float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float or, if it is less, to the closest bigger multiple of degree of 2: SA=size(M*N), SB=size(M*P), SD=size(N)
	int	L		Number of matrix equations
	int	qABX		qA-qB+qX where qA, qB, qX - fixed point representations of matrices A, B, X (for fixed point API only)
	complex_fract16, complex_float, float32_t	R	[L][SA]	Sequence of L upper triangle input matrices R (only $N \times N$ elements of each matrix are used)
	complex_fract16, float32_t	D	[L][SD]	reciprocal of main diagonal (mantissa, exponent) in the special format
	Input/Output:			
	complex_fract16, complex_float, float32_t	X	[M*P][L]	at the input it is the sequence of L updated right parts $Z=Q'B$. They will be replaced with MMSE solution vectors X (only $N \times P$ elements are used)
	Temporary:			
	void*	pScr		Scratch memory area of S bytes
Restrictions	X, R, pScr must not overlap All matrices and the scratch should not overlap and be aligned on 32-bytes boundary M, N, L should be bigger than 1 M and N should be a multiple of mN (4) Number of columns for input matrices R must not exceed the number of rows: $N \leq M$.			
Returned value	none			

2.7.5 Cholesky MMSE Solver

Description	Compute the MMSE solution for a system of linear equations $A^*x=B$, where A is an $M \times N$ real (complex) matrix with $M \geq N$ and $\text{rank}(A) = N$, x is an $N \times 1$ vector of unknowns, and B is an $M \times 1$ right hand side vector. This task is accomplished in 3 steps: - Cholesky decomposition is applied to the matrix of normal equations system, which results in an upper triangular matrix $R_{N \times N}$ with real and positive numbers on the main diagonal, such that $R_{N \times N}^* \cdot R_{N \times N} \cong A_{M \times N}^* \cdot A_{M \times N} + \sigma^2 I_{N \times N}$ - Forward recursion step: solve $R_{N \times N}^* \cdot y_{N \times P} = A_{M \times N}^* B_{M \times P}$ for $N \times 1$ vector y. - Backward recursion step: solve the system $R_{N \times N} \cdot x_{N \times P} = y_{N \times P}$ for the $N \times 1$ vector of unknowns x. For a single $M \times N$ matrix A, these 3 steps may be done simultaneously for P variants of $M \times 1$ right hand side column vectors b gathered into an $M \times P$ input matrix B. MMSE solution is computed independently for each of P columns, with resulting column vectors forming the solution matrix x of size $N \times P$. Functions process L pairs of $M \times N$ matrices A and $M \times P$ matrices B in a single call, which results in L solution matrices x of size $N \times P$. L matrices of each kind are stored as stream ordered sequences (refer to 2.6.1 for details).
Representation	floating point
Algorithm	find $R_{N \times N}^{(l)} = \text{chol}(A_{M \times N}^{(l)*} \cdot A_{M \times N}^{(l)} + \sigma^2 I_{N \times N})$: $R_{N \times N}^{(l)*} \cdot R_{N \times N}^{(l)} \cong A_{M \times N}^{(l)*} \cdot A_{M \times N}^{(l)} + \sigma^2 I_{N \times N}$ solve $R_{N \times N}^{(l)*} \cdot y_{N \times P}^{(l)} = A_{M \times N}^{(l)*} B_{M \times P}^{(l)}$ solve $R_{N \times N}^{(l)} \cdot x_{N \times P}^{(l)} = y_{N \times P}^{(l)}$ $l = 0 \dots L - 1$

Prototypes

```

void cfunf (void * pScr, complex_float * x, const complex_float* A,
            const complex_float* B, const float32_t* sigma2, int L)
void cfunMNPf (void * pScr, complex_float * x, const complex_float * A,
               const complex_float * B, const float32_t* sigma2, int M, int N, int P, int L)
void rfunf (void * pScr, float32_t * x, const float32_t * A,
            const float32_t * B, const float32_t * sigma2, int L)
void rfunfMN (void * pScr, float32_t * x, const float32_t * A,
              const float32_t * B, const float32_t * sigma2, int M, int N, int P, int L)

```

Function Name	Prototype	M	N	mL	mN	Scratch allocation function
Floating point, complex data, streaming order						
cholmmse2x2x1sf	cfunf	2	2	4		cholmmse2x2x1sf_getScratchSize
cholmmse3x3x1sf	cfunf	3	3	4		cholmmse3x3x1sf_getScratchSize
cholmmse4x4x1sf	cfunf	4	4	4		cholmmse4x4x1sf_getScratchSize
cholmmse mxnxpsf	cfunMNPf	M	N	4		cholmmsemnxpsf_getScratchSize
Floating point, real data, streaming order						
rcholmmse2x2x1sf	rfunf	2	2	8		rcholmmse2x2x1sf_getScratchSize
rcholmmse3x3x1sf	rfunf	3	3	8		rcholmmse3x3x1sf_getScratchSize
rcholmmse4x4x1sf	rfunf	4	4	8		rcholmmse4x4x1sf_getScratchSize
rcholmmse mxnxpsf	rfunMNPf	M	N	8		rcholmmsemnxpsf_getScratchSize
Floating point, complex data, block order						
cholmmse8x8x1nf	cfunf	8	8			cholmmse2x2x1nf_getScratchSize
cholmmse16x16x1nf	cfunf	16	16			cholmmse3x3x1nf_getScratchSize
cholmmse mxnxpnf	cfunMNPf	M	N		4	cholmmsemnxpnf_getScratchSize
Floating point, real data, block order						
rcholmmse8x8x1nf	rfunf	8	8			rcholmmse2x2x1nf_getScratchSize
rcholmmse16x16x1nf	rfunf	16	16			rcholmmse3x3x1nf_getScratchSize
rcholmmse mxnxpnf	rfunMNPf	M	N		4	rcholmmsemnxpnf_getScratchSize

Arguments for streaming order

Type	Name	Size	Description
Input:			
int	M		Dimensional parameter: number of rows in input matrices A
int	N		Dimensional parameter: number of columns in input matrices A
int	P		Number of columns in right-side matrices B
int	L		Number of matrices
float32_t	sigma2	[L]	Regularization term
complex_float float32_t	A	[M*N] [L]	Sequence of L complex matrices A
complex_float float32_t	B	[M*P] [L]	Original right-side matrices B
Output:			
complex_float, float32_t	x	[N*P] [L]	Decision matrices x
Temporary:			
pScr			Scratch data area, size in bytes is requested by corresponding scratch allocation function

Arguments for block order	Type	Name	Size	Description
Arguments for block order	Input:			
	int	M		Dimensional parameter: number of rows in input matrices A
	int	N		Dimensional parameter: number of columns in input matrices A
	int	P		Number of columns in right-side matrices B
		SA, SB, SX		Matrix sizes SA, SR, SD are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for complex_fract16, float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float or, if it is less, to the closest bigger multiple of degree of 2: SA=size(M*N), SB=size(M*P), SX=size(N*P)
	int	L		Number of matrices
	float32_t	sigma2	[L]	Regularization term
	complex_float float32_t	A	[L] [SA]	Sequence of L complex matrices A
	complex_float float32_t	B	[L] [SB]	Original right-side matrices B
	Output:			
	complex_float, float32_t	x	[L] [SX]	Decision matrices x
	Temporary:			
	pScr			Scratch data area, size in bytes is requested by corresponding scratch allocation function

Restrictions All matrices should not overlap and be aligned on 32-bytes boundary
 $M \geq N$
 N, M, L should be bigger than 1
 N, M should be a multiple of mN (see Table above)
Number of matrix equations L must be a multiple of mL (see table above)

Returned value none

2.7.6 Cholesky Decomposition for Pseudo-inversion

Description	Apply the Cholesky decomposition to the matrix of normal equations system associated with a complex-valued least squares problem: $A_{M \times N} \cdot X_{N \times P} = B_{M \times P}$, $M \geq N$. The decomposition results in an upper triangular complex matrix $R_{N \times N}$ with real and positive numbers on the main diagonal, such that $R^* \cdot R = A^* \cdot A + \sigma^2 \cdot I_N$, where $\langle \dots \rangle^*$ denotes the conjugate transpose of a matrix, and $\sigma^2 \cdot I_N$ is the $N \times N$ identity matrix multiplied with the regularization term. NOTES: The decomposition algorithm is applied to a few matrices per single call, with input/output matrix sequences being stored in the streaming order (refer to 2.6.1 for details). Fixed-point data type of upper triangular matrices R is the same as the data type of input matrices A. Fixed point position for the regularization term σ^2 must match the scale of product $A^* \cdot A$. If, for instance, matrix A is represented as Q15, then Q30 is expected for σ^2. In order to find the solution to the above-mentioned least squares problem $A_{M \times N} \cdot X_{N \times P} = B_{M \times P}$, one has to follow the Cholesky decomposition with forward and backward substitution procedures. The API allows to apply procedure to the usual matrices (written in streaming order) and for banded matrices (written in block order) where matrix A contains W non-zero elements on the main diagonal and below. In that case, matrices A_t and R_t are written in the transposed compact form and dimensions are $W \times N$ for both.
Representation	16-bit fixed point floating point

Algorithm $R_{N \times N}^{(l)} = \text{chol}(A_{M \times N}^{(l)*} \cdot A_{M \times N}^{(l)} + \sigma^2 I_{N \times N})$: $R_{N \times N}^{(l)*} \cdot R_{N \times N}^{(l)} \cong A_{M \times N}^{(l)*} \cdot A_{M \times N}^{(l)} + \sigma^2 I_{N \times N}$,
 $l = 0 \dots L - 1$

**Prototypes
(streaming order)**

```
void cfun (complex_fract16 *R, complex_fract16 * D,
          const complex_fract16 * A, const int32_t* sigma2, int L)
void cfunMN (complex_fract16 *R, complex_fract16 * D,
            const complex_fract16 * A, const int32_t* sigma2, int M, int N, int L)
void cfunf (complex_float *R, complex_float * D,
           const complex_float * A, const float32_t* sigma2, int L)
void cfunfMN (complex_float *R, complex_float * D,
             const complex_float * A, const float32_t* sigma2, int M, int N, int L)
void rfunf (float32_t *R, float32_t * D,
           const float32_t * A, const float32_t * sigma2, int L)
void rfunfMN (float32_t *R, float32_t * D,
             const float32_t * A, const float32_t * sigma2, int M, int N, int L)
```

Function Name	Prototype	M	N	mL
Fixed point, complex data				
chol2x2s	cfun	2	2	8
chol4x4s	cfun	4	4	8
chol8x8s	cfun	8	8	8
cholmxns	cfunMN	M	N	8
Floating point, complex data				
chol2x2sf	cfunf	2	2	4
chol3x3sf	cfunf	3	3	4
chol4x4sf	cfunf	4	4	4
cholmxnsf	cfunMNf	M	N	4
Floating point, real data				
rchol2x2sf	rfunf	2	2	8
rchol3x3sf	rfunf	3	3	8
rchol4x4sf	rfunf	4	4	8
rcholmxnsf	rfunMNf	M	N	8

Arguments

Type	Name	Size	Description
Input:			
int	M		Dimensional parameter: number of rows in input matrices A
int	N		Dimensional parameter: number of columns in input matrices A , number of rows and columns in upper triangular matrices R
int	L		Number of matrices
int32_t, float32_t	sigma2	[L]	Regularization term. For fixed point API, fixed point position is twice the number of fractional bits for matrices A, R
complex_fract16, float32_t, complex_float,	A	[M*N] [L]	Sequence of L complex matrices A (streaming format)
Output:			
complex_fract16, float32_t, complex_float,	R	[N*N] [L]	Sequence of L upper triangular complex matrices R (streaming format)
complex_fract16	D	[L*N*2]	reciprocals of the main diagonal written in a special format (fixed point API)
float32_t, complex_float	D	[L*N]	reciprocals of the main diagonal written in a special format (floating point API)

Restrictions	All matrices should not overlap and be aligned on 32-bytes boundary Number of matrix equations L must be a multiple of mL (see table above) $M \geq N$ N, M, L should be bigger than 1																																															
Prototypes (banded matrices in block order)	<pre>int funn (complex_fract16 * Rt, complex_fract16 * D, const complex_fract16 * At, const int32_t * sigma2,int L) int funnWN (complex_fract16 * Rt, complex_fract16 * D, const complex_fract16 * At, const int32_t * sigma2, int W, int N, int L) ;</pre>																																															
	<table><tr><th>Function Name</th><th>Prototype</th><th>M</th><th>W</th><th>N</th></tr><tr><td>bchol4x16n</td><td>funn</td><td>19</td><td>4</td><td>16</td></tr><tr><td>bchol8x16n</td><td>funn</td><td>23</td><td>8</td><td>16</td></tr><tr><td>bchol4x32n</td><td>funn</td><td>35</td><td>4</td><td>32</td></tr><tr><td>bchol8x32n</td><td>funn</td><td>39</td><td>8</td><td>32</td></tr><tr><td>bchol16x32n</td><td>funn</td><td>47</td><td>16</td><td>32</td></tr><tr><td>bcholwxnn</td><td>funnWN</td><td></td><td></td><td></td></tr></table>	Function Name	Prototype	M	W	N	bchol4x16n	funn	19	4	16	bchol8x16n	funn	23	8	16	bchol4x32n	funn	35	4	32	bchol8x32n	funn	39	8	32	bchol16x32n	funn	47	16	32	bcholwxnn	funnWN															
Function Name	Prototype	M	W	N																																												
bchol4x16n	funn	19	4	16																																												
bchol8x16n	funn	23	8	16																																												
bchol4x32n	funn	35	4	32																																												
bchol8x32n	funn	39	8	32																																												
bchol16x32n	funn	47	16	32																																												
bcholwxnn	funnWN																																															
Arguments	<table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td colspan="4">Input:</td></tr><tr><td>int</td><td>M</td><td></td><td>Dimensional parameter: number of rows in input matrices A</td></tr><tr><td>int</td><td>W</td><td></td><td>Dimensional parameter: the band width</td></tr><tr><td>int</td><td>N</td><td></td><td>Dimensional parameter: number of columns in input matrices A, number of rows and columns in upper triangular matrices R</td></tr><tr><td>int</td><td>L</td><td></td><td>Number of matrices</td></tr><tr><td>int32_t</td><td>sigma2</td><td>[L]</td><td>Regularization term; fixed point position is twice the number of fractional bits for matrices A_t, R_t</td></tr><tr><td>complex_fract16</td><td>A_t</td><td>[L] [N*W]</td><td>Sequence of L complex matrices A written in transposed compact form (block order)</td></tr><tr><td colspan="4">Output:</td></tr><tr><td>complex_fract16</td><td>R_t</td><td>[L] [N*W]</td><td>Sequence of L upper triangular complex matrices R_t written in transposed compact form in block order</td></tr><tr><td>complex_fract16</td><td>D</td><td>[L] [N']</td><td>reciprocals of the main diagonal, N' is computed as for complex block ordered matrices of size N</td></tr></table>	Type	Name	Size	Description	Input:				int	M		Dimensional parameter: number of rows in input matrices A	int	W		Dimensional parameter: the band width	int	N		Dimensional parameter: number of columns in input matrices A , number of rows and columns in upper triangular matrices R	int	L		Number of matrices	int32_t	sigma2	[L]	Regularization term; fixed point position is twice the number of fractional bits for matrices A_t , R_t	complex_fract16	A_t	[L] [N*W]	Sequence of L complex matrices A written in transposed compact form (block order)	Output:				complex_fract16	R_t	[L] [N*W]	Sequence of L upper triangular complex matrices R_t written in transposed compact form in block order	complex_fract16	D	[L] [N']	reciprocals of the main diagonal, N' is computed as for complex block ordered matrices of size N			
Type	Name	Size	Description																																													
Input:																																																
int	M		Dimensional parameter: number of rows in input matrices A																																													
int	W		Dimensional parameter: the band width																																													
int	N		Dimensional parameter: number of columns in input matrices A , number of rows and columns in upper triangular matrices R																																													
int	L		Number of matrices																																													
int32_t	sigma2	[L]	Regularization term; fixed point position is twice the number of fractional bits for matrices A_t , R_t																																													
complex_fract16	A_t	[L] [N*W]	Sequence of L complex matrices A written in transposed compact form (block order)																																													
Output:																																																
complex_fract16	R_t	[L] [N*W]	Sequence of L upper triangular complex matrices R_t written in transposed compact form in block order																																													
complex_fract16	D	[L] [N']	reciprocals of the main diagonal, N' is computed as for complex block ordered matrices of size N																																													
Restrictions	All matrices should not overlap and be aligned on 32-bytes boundary W, N should be a multiples of 4 W should be 16 or less $M \geq N$ W, N, M, L should be bigger than 1																																															

Prototypes (block order)

```

void cfun (void *pScr, complex_fract16 * R, complex_fract16* D,
          const complex_fract16 * A, int L)
void cfunMN (void *pScr, complex_fract16 * R, complex_fract16* D,
            const complex_fract16 * A, int M, int N, int L)
void cfunf (void *pScr, complex_float * R, float32_t * D,
            const complex_float * A, const float32_t* sigma2, int L)
void cfunMNf(void *pScr, complex_float * R, float32_t * D,
             const complex_float * A, const float32_t* sigma2, int M, int N, int L)
void rfunf (void *pScr, float32_t * R, float32_t * D,
            const float32_t * A, const float32_t* sigma2, int L)
void rfunMNf(void *pScr, float32_t * R, float32_t * D,
             const float32_t * A, const float32_t* sigma2, int M, int N, int L)

```

Function Name	Prototype	M	N	mN	Scratch allocation function
Fixed point complex data					
chol8x8n	cfun	8	8		chol8x8n_getScratchSize
chol16x16n	cfun	16	16		chol16x16n_getScratchSize
chol32x32n	cfun	32	32		chol32x32n_getScratchSize
cholmxnn	cfunMN	M	N	4	cholmxnn_getScratchSize
Floating point complex data					
chol8x8nf	cfunf	8	8		chol8x8nf_getScratchSize
chol16x16nf	cfunf	16	16		chol16x16nf_getScratchSize
chol32x32nf	cfunf	32	32		chol32x32nf_getScratchSize
cholmxnnf	cfunMNf	M	N	4	cholmxnnf_getScratchSize
Floating point real data					
rchol8x8nf	rfunf	8	8		rchol8x8nf_getScratchSize
rchol16x16nf	rfunf	16	16		rchol16x16nf_getScratchSize
rchol32x32nf	rfunf	32	32		rchol32x32nf_getScratchSize
rcholmxnnf	rfunMNf	M	N	4	rcholmxnnf_getScratchSize

Arguments	Type	Name	Size	Description
	Input:			
	int	M		Dimensional parameter: number of rows in input matrices A
	int	N		Dimensional parameter: number of columns in input matrices A , number of rows and columns in upper triangular matrices R
		SA, SR, SD		Matrix sizes SA, SR, SD are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of $BBE_SIMD_WIDTH/2==8$ elements for <code>complex_fract16</code>, <code>float32_t</code> $BBE_SIMD_WIDTH/4==4$ elements for <code>complex_float</code> or, if it is less, to the closest bigger multiple of degree of 2: $SA=size(M*N), SR=size((N+1)*N)/2, SD=size(N)$
	int	L		Number of matrices
	<code>complex_fract16</code> , <code>complex_float</code> , <code>float32_t</code>	A	[L] [SA]	Sequence of L matrices A (block format)
	<code>float32_t</code>	sigma2	[L]	regularization term for each matrix
	Output:			
	<code>complex_fract16</code> , <code>complex_float</code> , <code>float32_t</code>	R	[L] [SR]	Sequence of L upper triangular matrices R
	<code>complex_fract16</code> , <code>float32_t</code>	D	[L] [SD]	reciprocals of the main diagonal
	Temporary:			
	void	pScr		Scratch size in bytes is defined by the scratch allocation function
Restrictions	All matrices and scratch memory should not overlap and be aligned on 32-bytes boundary M, N should be a multiples of mN (4) specified in the table above $M \geq N$ N, M, L should be bigger than 1			
Returned value	none			

2.7.7 Cholesky Forward Recursion for Pseudo-inversion

Description	These functions make forward recursion stage of pseudo-inversion. They use Cholesky decomposition $R_{N \times N}$ of original matrices $A_{M \times N}$. NOTES: The API allows to apply procedure to the usual matrices (written in streaming order) and for banded matrices (written in block order) where matrix A contains W non-zero elements on the main diagonal and below. In that case, matrices A_t and R_t are written in the transposed compact form and dimensions are $W \times N$ for both.
Algorithm	Solution of equations $R_{N \times N}^{(l)*} \cdot y_{N \times P}^{(l)} = A_{M \times N}^{(l)*} B_{M \times P}^{(l)}, l = \overline{0 \dots L-1}$

**Prototypes
(streaming order)**

```

void cfun( complex_fract16* y, const complex_fract16* R, const complex_fract16* D,
          const complex_fract16* A, const complex_fract16* B,
          int qA,int qB,int qY, int L)
void cfunMNP (complex_fract16* y, const complex_fract16* R,
             const complex_fract16* D, const complex_fract16* A,
             const complex_fract16* B, int qA,int qB,int qY,
             int M,int N, int P, int L)
void cfunf (complex_float * y, const complex_float * R, const complex_float * D,
           const complex_float * A, const complex_float * B, int L )
void cfunMNPf (complex_float * y, const complex_float * R,const complex_float * D,
              const complex_float * A, const complex_float * B,
              int M, int N, int P, int L )
void rfunf (float32_t * y, const float32_t * R, const float32_t * D,
           const float32_t * A, const float32_t * B, int L )
void rfunMNPf (float32_t * y, const float32_t * R, const float32_t * D,
              const float32_t * A, const float32_t * B, int M, int N, int P, int L )

```

Function Name	Prototype	M	N	P	mL
Fixed point, complex data					
chol_fwd2x2x1s	cfun	2	2	1	8
chol_fwd4x4x1s	cfun	4	4	1	8
chol_fwd8x8x1s	cfun	8	8	1	8
chol_fwdmxnxps	cfunMNP	M	N	P	8
Floating point, complex data					
chol_fwd2x2x1sf	cfunf	2	2	1	4
chol_fwd3x3x1sf	cfunf	3	3	1	4
chol_fwd4x4x1sf	cfunf	4	4	1	4
chol_fwdmxnxpsf	cfunMNPf	M	N	P	4
Floating point, real data					
rchol_fwd2x2x1sf	rfunf	2	2	1	8
rchol_fwd3x3x1sf	rfunf	3	3	1	8
rchol_fwd4x4x1sf	rfunf	4	4	1	8
rchol_fwdmxnxpsf	rfunMNPf	M	N	P	8

Arguments	Type	Name	Size	Description																																			
Input:																																							
int		M		Matrix dimension (number of rows in matrices A)																																			
int		N		Matrix dimension (number of columns in matrices R)																																			
int		P		Number of columns in right-side matrices B																																			
int		L		Number of matrices																																			
int		qA		fixed point representation of matrices A and R (for fixed point API only)																																			
int		qB		fixed point representation of matrix B (for fixed point API only)																																			
int		qY		fixed point representation of matrix y (for fixed point API only)																																			
complex_fract16, complex_float float32_t		R	[N*N] [L]	Cholesky upper triangular matrices R (streaming format)																																			
complex_fract16, complex_float float32_t		A	[M*N] [L]	Original left-side matrices A (streaming format)																																			
complex_fract16, complex_float float32_t		B	[M*P] [L]	Original right-side matrices B (streaming format)																																			
complex_fract16		D	[L*N*2]	reciprocals of the main diagonal written in a special format (fixed point API)																																			
float32_t, complex_float		D	[L*N]	reciprocals of the main diagonal written in a special format (floating point API)																																			
Output:																																							
complex_fract16, complex_float float32_t		y	[N*P] [L]	Decision matrices y																																			
Restrictions	All matrices should not overlap and be aligned on 32-bytes boundary Number of matrix equations L must be a multiple of m_L (see table above) N, M, L should be bigger than 1 $M \geq N$																																						
Prototypes (banded matrices in block order)	<pre>int bfun (complex_fract16* y, const complex_fract16* Rt, const complex_fract16* D, const complex_fract16* At, const complex_fract16 Bt, int qB,int qY,int L) int bfunWNP (complex_fract16* y, const complex_fract16* Rt, const complex_fract16* D, const int16_t* At, const complex_fract16* Bt, int qB,int qY, int W, int N, int P, int L)</pre>																																						
<table><tr><th>Function Name</th><th>Prototype</th><th>M</th><th>W</th><th>N</th></tr><tr><td>bcholfwd4x16x1n</td><td>bfun</td><td>19</td><td>4</td><td>16</td></tr><tr><td>bcholfwd8x16x1n</td><td>bfun</td><td>23</td><td>8</td><td>16</td></tr><tr><td>bcholfwd4x32x1n</td><td>bfun</td><td>35</td><td>4</td><td>32</td></tr><tr><td>bcholfwd8x32x1n</td><td>bfun</td><td>39</td><td>8</td><td>32</td></tr><tr><td>bcholfwd16x32x1n</td><td>bfun</td><td>47</td><td>16</td><td>32</td></tr><tr><td>bcholfwdwxnxpn</td><td>bfunWNP</td><td></td><td></td><td></td></tr></table>					Function Name	Prototype	M	W	N	bcholfwd4x16x1n	bfun	19	4	16	bcholfwd8x16x1n	bfun	23	8	16	bcholfwd4x32x1n	bfun	35	4	32	bcholfwd8x32x1n	bfun	39	8	32	bcholfwd16x32x1n	bfun	47	16	32	bcholfwdwxnxpn	bfunWNP			
Function Name	Prototype	M	W	N																																			
bcholfwd4x16x1n	bfun	19	4	16																																			
bcholfwd8x16x1n	bfun	23	8	16																																			
bcholfwd4x32x1n	bfun	35	4	32																																			
bcholfwd8x32x1n	bfun	39	8	32																																			
bcholfwd16x32x1n	bfun	47	16	32																																			
bcholfwdwxnxpn	bfunWNP																																						

Arguments	Type	Name	Size	Description
	Input:			
	int	M		Matrix dimension (number of rows in matrices A)
	int	W		Dimensional parameter: the band width
	int	N		Matrix dimension (number of columns in matrices R)
	int	P		Number of columns in right-side matrices B
	int	L		Number of matrices
	int	qB		fixed point representation of matrix B
	int	qY		fixed point representation of matrix y
	complex_fract16	Rt	[L] [W*N]	Cholesky upper-triangle matrices R represented in the compact form (saved only elements on the main diagonal and above in such a way that diagonal elements are in the last row)
	complex_fract16	At	[L] [W*N]	Original left-side matrices A represented in the compact form (only band)
	complex_fract16	Bt	[L] [SB]	Original right-side matrices B . SB is computed as for complex block ordered matrices of size $(W+N-1) * P$
	complex_fract16	D	[L] [N']	sequence of L reciprocals of main diagonal A represented in block floating point (mantissa and exponent). N' is computed as for complex block ordered matrices of size N
	Output:			
	complex_fract16	Yt	[N*P] [L]	Decision matrices y
Restrictions	All matrices should not overlap and be aligned on 32-bytes boundary Matrix sizes should be bigger than 1 W, N should be a multiples of 4 W should be 16 or less $M \geq N$ W, N, M, L should be bigger than 1			
Returned value	not defined			

Prototypes (block order)

```

void cfun (void *pScr, complex_fract16* y,
           const complex_fract16* R, const complex_fract16* D,
           const complex_fract16* A, const complex_fract16* B,
           int qB, int qY, int L)
void cfunMNP (void *pScr, complex_fract16* y,
              const complex_fract16* R, const complex_fract16* D,
              const complex_fract16* A, const complex_fract16* B,
              int qB, int qY, int M, int N, int P, int L);
void cfunf (void *pScr, complex_float * y,
            const complex_float * R, const complex_float * D,
            const complex_float * A, const complex_float * B,
            int L );
void cfunMNPf (void *pScr, complex_float * y,
               const complex_float * R, const complex_float * D,
               const complex_float * A, const complex_float * B,
               int M, int N, int P, int L )
void rfunf (void *pScr, float32_t * y,
            const float32_t * R, const float32_t * D,
            const float32_t * A, const float32_t * B, int L )
void rfunMNPf (void *pScr, float32_t * y,
               const float32_t * R, const float32_t * D,
               const float32_t * A, const float32_t * B, int M, int N, int P, int L )

```

Function Name	Prototype	M	N	P	mN	Scratch allocation function
Fixed point, complex matrices						
chol_fwd8x8x1n	cfun	8	8	1		chol_fwd8x8x1n_getScratchSize
chol_fwd16x16x1n	cfun	16	16	1		chol_fwd16x16x1n_getScratchSize
chol_fwd32x32x1n	cfun	32	32	1		chol_fwd32x32x1n_getScratchSize
chol_fwdmxnpxn	cfunMNP	M	N	P	4	chol_fwdmxnpxn_getScratchSize
Floating point, complex matrices						
chol_fwd8x8x1nf	cfunf	8	8	1		chol_fwd8x8x1nf_getScratchSize
chol_fwd16x16x1nf	cfunf	16	16	1		chol_fwd16x16x1nf_getScratchSize
chol_fwd32x32x1nf	cfunf	32	32	1		chol_fwd32x32x1nf_getScratchSize
chol_fwdmxnpxnf	cfunMNPf	M	N	P	4	chol_fwdmxnpxnf_getScratchSize
Floating point, real matrices						
rchol_fwd8x8x1nf	rfunf	8	8	1		rchol_fwd8x8x1nf_getScratchSize
rchol_fwd16x16x1nf	rfunf	16	16	1		rchol_fwd16x16x1nf_getScratchSize
rchol_fwd32x32x1nf	rfunf	32	32	1		rchol_fwd32x32x1nf_getScratchSize
rchol_fwdmxnpxnf	rfunMNPf	M	N	P	4	rchol_fwdmxnpxnf_getScratchSize

Arguments	Type	Name	Size	Description
	Input:			
	int	M		Matrix dimension (number of rows in matrices A)
	int	N		Matrix dimension (number of columns in matrices R)
	int	P		Number of columns in right-side matrices B
		SA, SR, SD, SY		Matrix sizes SA, SR, SD, SY are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for complex_fract16, float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float SA=size (M*N) SR=size (((N+1) *N) /2) SD=size (N) SB=size (M*P) SY=size (N*P)
	int	L		Number of matrices
	int	qB		fixed point representation of matrix B (for fixed point API only)
	int	qY		fixed point representation of matrix Y (for fixed point API only)
	complex_fract16, complex_float, float32_t	R	[L] [SR]	sequence of L upper triangular matrices R
	complex_fract16, complex_float, float32_t	A	[L] [SA]	sequence of L matrices A
	complex_fract16, complex_float, float32_t	B	[L] [SB]	sequence of L original right-side matrices B
	complex_fract16, float32_t, complex_float	D	[L] [SD]	sequence of L reciprocals of main diagonal
	Output:			
	complex_fract16, complex_float, float32_t	y	[L] [SY]	Decision matrices y
	Temporary:			
	void	pScr		Scratch size in bytes is defined by scratch allocation functions, see above
Restrictions	All matrices and scratch memory should not overlap and be aligned on 32-bytes boundary N , M , L should be bigger than 1 M , N should be a multiples of mN (4)			
Returned value	none			

2.7.8 Cholesky Backward Recursion for Pseudo-inversion

Description	These functions make backward recursion stage of pseudo-inversion. They use Cholesky decomposition of original matrices and results of forward recursion. NOTES: The API allows to apply procedure to the usual matrices (written in streaming order) and for banded matrices (written in block order) where matrix A contains W non-zero elements on the main diagonal and below. In that case, matrices A_t and R_t are written in the transposed compact form and dimensions are $W \times N$ for both.
Algorithm	Solution of equations $R_{N \times N}^{(l)} \cdot x_{N \times P}^{(l)} = y_{N \times P}^{(l)}, l = \overline{0 \dots L-1}$

**Prototypes
(streaming order)**

```

void fun ( complex_fract16* x, const complex_fract16* R,
           const complex_fract16* D, const complex_fract16* y,
           int qA, int qY, int qX, int L);
void funNP (complex_fract16* x, const complex_fract16* R,
            const complex_fract16* D, const complex_fract16* y,
            int qA, int qY, int qX, int N, int P, int L)
void cfunf (complex_float * x, const complex_float * R,
            const complex_float * D, const complex_float * y, int L );
void cfunNPf (complex_float * x, const complex_float * R,
              const complex_float * D, const complex_float * y, int N, int P, int L)
void rfunf (float32_t * x, const float32_t * R,
            const float32_t * D, const float32_t * y, int L )
void rfunNPf (float32_t * x, const float32_t * R,
              const float32_t * D, const float32_t * y, int N, int P, int L)

```

Function Name	Proto	N	P	mL
Fixed point complex matrices				
cholbkw2x1s	cfun	2	1	8
cholbkw4x1s	cfun	4	1	8
cholbkw8x1s	cfun	8	1	8
cholbkwnxps	cfunNP	N	P	8
Floating point complex matrices				
cholbkw2x1sf	cfunf	2	1	4
cholbkw4x1sf	cfunf	3	1	4
cholbkw8x1sf	cfunf	4	1	4
cholbkwnxpsf	cfunNPf	N	P	4
Floating point real matrices				
rcholbkw2x1sf	rfunf	2	1	8
rcholbkw4x1sf	rfunf	3	1	8
rcholbkw8x1sf	rfunf	4	1	8
rcholbkwnxpsf	rfunNPf	N	P	8

Arguments

Type	Name	Size	Description
Input:			
int	N		Matrix dimension (number of columns in matrices R)
int	P		Number of columns in right-side matrices B
int	L		Number of matrices
int	qA		fixed point representation of matrices A and R (for fixed point API only)
int	qX		fixed point representation of matrix x (for fixed point API only)
int	qY		fixed point representation of matrix y (for fixed point API only)
complex_fract16, complex_float, float32_t	R	[N*N] [L]	Cholesky upper triangular matrices R (streaming format)
complex_fract16, complex_float, float32_t	y	[N*P] [L]	Results of forward recursion stage (streaming format)
complex_fract16	D	[L*N*2]	reciprocals of the main diagonal written in a special format (fixed point API)
float32_t, complex_float	D	[L*N]	reciprocals of the main diagonal written in a special format (floating point API)
Output:			
complex_fract16, complex_float, float32_t	x	[N*P] [L]	Decision matrices x (streaming format)

Restrictions

All matrices should not overlap and be aligned on 32-bytes boundary
 Number of matrix equations L must be a multiple of mL (see table above)
 N , L should be bigger than 1

**Prototypes
(banded matrices
in block order)**

```
int bfunWNP (complex_fract16* Xt, const complex_fract16* Rt,
             const complex_fract16* D, const complex_fract16* Yt,
             int qA, int qY, int qX, int W, int N, int P, int L)
int bfun     (complex_fract16* Xt, const complex_fract16* Rt,
             const complex_fract16* D, const int16_t* Yt,
             int qA, int qY, int qX, int L)
```

Function Name	Prototype	M	W	N
bcholbkw4x16x1n	bfun	19	4	16
bcholbkw8x16x1n	bfun	23	8	16
bcholbkw4x32x1n	bfun	35	4	32
bcholbkw8x32x1n	bfun	39	8	32
bcholbkw16x32x1n	bfun	47	16	32
bcholbkwxxnpxn	bfunWNP			

Arguments

Type	Name	Size	Description
Input:			
int	W		Dimensional parameter: the band width
int	N		Matrix dimension (number of columns in matrices R_t)
int	P		Number of columns in right-side matrices B
int	L		Number of matrices
int	qA		fixed point representation of matrices A_t and R_t
int	qX		fixed point representation of matrix X
int	qY		fixed point representation of matrix Y
complex_fract16	R_t	[L] [N*W]	Cholesky upper-triangle matrices R represented in the compact form (saved only elements on the main diagonal and above in such a way that diagonal elements are in the last row)
complex_fract16	Y_t	[L] [SY]	Results of forward recursion stage. S_Y is computed as for complex block ordered matrices of size $N*P$
complex_fract16	D	[L] [N']	sequence of L reciprocals of main diagonal R represented in the block floating point (mantissa and exponent). N' is computed as for complex block ordered matrices of size N
Output:			
complex_fract16	X_t	[L] [SX]	Decision matrices X . S_X is computed as for complex block ordered matrices of size $N*P$

Restrictions

All matrices should not overlap and be aligned on 32-bytes boundary
 W , N should be a multiples of 4
 W should be 16 or less
 $M \geq N$
 W , M , N , L should be bigger than 1

Returned value

not defined

Prototypes (block order)

```

void cfun ( void *pScr, complex_fract16* x,
            const complex_fract16* R, const complex_fract16* D,
            const complex_fract16* y, int qA, int qY, int qX, int L)
void cfunNP ( void *pScr, complex_fract16* x,
              const complex_fract16* R, const complex_fract16* D,
              const complex_fract16* y, int qA, int qY, int qX, int N, int P, int L)
void cfunf (void * pScr, complex_float * x, const complex_float* R,
            const complex_float_t * D, const complex_float* y, int L)
void cfunNPf (void * pScr, complex_float * x, const complex_float * R,
              const complex_float_t * D, const complex_float * y, int N, int P,
              int L )
void rfunf (void * pScr, float32_t * x, const float32_t * R,
            const float32_t * D, const float32_t * y, int L )
void rfunNPf (void * pScr, float32_t * x, const float32_t * R,
              const float32_t * D, const float32_t * y, int N, int P, int L )

```

Function Name	Prototype	M	P	mN	scratch allocation functions
Fixed point complex matrices					
cholbkw8x1n	cfun	8	1		cholbkw8x1n_getScratchSize
cholbkw16x1n	cfun	16	1		cholbkw16x1n_getScratchSize
cholbkw32x1n	cfun	32	1		cholbkw32x1n_getScratchSize
cholbkwnxpn	cfunNP			4	cholbkwnxpn_getScratchSize
Floating point complex matrices					
cholbkw8x1nf	cfunf	8	1		cholbkw8x1nf_getScratchSize
cholbkw16x1nf	cfunf	16	1		cholbkw16x1nf_getScratchSize
cholbkw32x1nf	cfunf	32	1		cholbkw32x1nf_getScratchSize
cholbkwnxpnf	cfunNPf			4	cholbkwnxpnf_getScratchSize
Floating point real matrices					
rcholbkw8x1nf	rfunf	8	1		rcholbkw8x1nf_getScratchSize
rcholbkw16x1nf	rfunf	16	1		rcholbkw16x1nf_getScratchSize
rcholbkw32x1nf	rfunf	32	1		rcholbkw32x1nf_getScratchSize
rcholbkwnxpnf	rfunNPf			4	rcholbkwnxpnf_getScratchSize

Arguments	Type	Name	Size	Description
	Input:			
	int	N		Matrix dimension (number of columns in matrices A)
	int	P		Number of columns in right-side matrices B
		SR, SD, SY		Matrix sizes SR, SD, SY are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for complex_fract16, float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float SR=size((N+1)*N)/2, SD=size(N), SX=size(N*P), SY=size(N*P)
	int	L		Number of matrices
	int	qA		fixed point representation of matrices A and R (for fixed point API only)
	int	qX		fixed point representation of matrix x (for fixed point API only)
	int	qY		fixed point representation of matrix y (for fixed point API only)
	complex_fract16, complex_float, float32_t	R	[L][SD]	Cholesky upper-triangle matrices R
	complex_fract16, complex_float, float32_t	y	[L][SY]	Results of forward recursion stage
	complex_fract16, float32_t, complex_float	D	[L][SD]	sequence of L reciprocals of main diagonal R
	Output:			
	complex_fract16, complex_float, float32_t	x	[L][SX]	Decision matrices x .
Restrictions	All matrices and scratch should not overlap and be aligned on 32-bytes boundary N should be a multiple of mN (4) N, L should be bigger than 1			
Returned value	none			

2.7.9 Direct Matrix Inversion

Description	These functions make matrix inversion for 2x2, 3x3 and 4x4 matrices. For 2x2, they use direct determinant solution, for 3x3, 4x4 they use modified blockwise method (with sub-optimal row/column permutation preventing from bad conditioning of upper sub-block). API defines number of functions for block and streaming representation of input matrices (see details in 2.6.1) so some restrictions to the alignment and number of matrices will be applied. NOTES: 1. routines makes in-place computations i.e. inverted matrix replaces original one. 2. Functions <code>chermmatinv2x2n()</code>, <code>chermmatinv4x4n()</code> assume Hermitian matrices on input. Functions <code>cmatinv2x2n()</code>, <code>cmatinv4x4n()</code> work with arbitrary matrices.
Representation	16-bit fixed point floating point
Algorithm	$B^{(l)} = A^{(l)-1}, l = 0 \dots L-1$

Prototypes

```

void cfunQ (void * pScr, complex_fract16 * A, int qA, int L)
void cfunf (void * pScr, complex_float * A, int L)
void funf (void * pScr, float32_t * A, int L)

```

Function Name	Protot ype	Size	s	mL	Scratch allocation function
Block order, 16-bit complex data					
cmatinv2x2n	cfunQ	2×2	4	2	cmatinv2x2n_getScratchSize
cmatinv4x4n	cfunQ	4×4	16		cmatinv4x4n_getScratchSize
chermmatinv2x2n	cfunQ	2×2	4	2	chermmatinv2x2n_getScratchSize
chermmatinv4x4n	cfunQ	4×4	16		chermmatinv4x4n_getScratchSize
Block order, floating point real data					
matinv2x2nf	funf	2×2	4	2	matinv2x2nf_getScratchSize
matinv3x3nf	funf	3×3	16		matinv3x3nf_getScratchSize
matinv4x4nf	funf	4×4	16		matinv4x4nf_getScratchSize
Block order, floating point complex data					
cmatinv2x2nf	cfunf	2×2	4		cmatinv2x2nf_getScratchSize
cmatinv3x3nf	cfunf	3×3	12		cmatinv3x3nf_getScratchSize
cmatinv4x4nf	cfunf	4×4	16		cmatinv4x4nf_getScratchSize
Streaming order, floating point real data					
matinv2x2sf	funf	2×2	4	8	matinv2x2sf_getScratchSize
matinv3x3sf	funf	3×3	9	8	matinv3x3sf_getScratchSize
matinv4x4sf	funf	4×4	16	8	matinv4x4sf_getScratchSize
Streaming order, floating point complex data					
cmatinv2x2sf	cfunf	2×2	4	4	cmatinv2x2sf_getScratchSize
cmatinv3x3sf	cfunf	3×3	9	4	cmatinv3x3sf_getScratchSize
cmatinv4x4sf	cfunf	4×4	16	4	cmatinv4x4sf_getScratchSize

Arguments

Type	Name	Size	Description
	N		Dimension (2,3,4)
	S		Allocated space per matrix, in elements of given data type
Input:			
int	L		Number of matrices
int	qA		fixed point representation of matrices A (for fixed point API only)
Input/Output:			
complex_fract16, complex_float, float32_t	A	L*S	Input/output complex matrices
Temporary:			
void*	pScr		Scratch memory area of requested by scratch allocation function

Restrictions

All matrices and the scratch should not overlap and be aligned on 32-bytes boundary
L should be a multiple of mL parameter specified in the table above

Returned value

none

Notes

1. Avoid inverting of ill-conditioned matrices
2. In the fixed point API, 4x4 inversion process may cause the overflows in the intermediate computations. To avoid it, limit dynamic range of input matrices by 11...13 bits.

2.7.10 Matrix Inversion by Gauss-Jordan Algorithm

Description Functions perform in-place inversion of real/complex matrices by Gauss-Jordan elimination method, with full pivoting. Inversion result is not defined for a close to singular input matrix.
API defines number of functions for block and streaming representation of input matrices (see details in 2.6.1) so some restrictions to the alignment, matrix size and number of matrices will be applied.

NOTES:

1. routines makes in-place computations i.e. inverted matrix replaces original one.

Representation floating point

Algorithm $B^{(l)} = A^{(l)-1}, l = 0 \dots L-1$

Prototypes

```
void funf ( void * pScr, float32_t * A, int L );
void cfunf ( void * pScr, complex_float * A, int L );
void funnf ( void * pScr, float32_t * A, int N, int L );
void cfunnf ( void * pScr, complex_float * A, int N, int L );
```

Function Name	Prototype	Size	SA	mL	mN	Scratch allocation function
Block order, floating point complex data						
cmatinvgj8x8nf	cfunf	8×8	64			cmatinvgj8x8nf_getScratchSize
cmatinvgj16x16nf	cfunf	16×16	256			cmatinvgj16x16nf_getScratchSize
cmatinvgjnxnnf	cfunnf	N×N	N*N		4	cmatinvgjnxnnf_getScratchSize
Block order, floating point real data						
matinvgj8x8nf	funf	8×8	64			matinvgj8x8nf_getScratchSize
matinvgj16x16nf	funf	16×16	256			matinvgj16x16nf_getScratchSize
matinvgjnxnnf	funnf	N×N	N*N		4	matinvgjnxnnf_getScratchSize
Streaming order, floating point complex data						
cmatinvgj2x2sf	cfunf	2×2	4	4		cmatinv2x2sf_getScratchSize
cmatinvgj3x3sf	cfunf	3×3	9	4		cmatinv3x3sf_getScratchSize
cmatinvgj4x4sf	cfunf	4×4	16	4		cmatinv4x4sf_getScratchSize
cmatinvgjnxnsf	cfunnf	N×N	N*N	4		cmatinvgjnxnsf_getScratchSize
Streaming order, floating point real data						
matinvgj2x2sf	funf	2×2	4	8		matinv2x2sf_getScratchSize
matinvgj3x3sf	funf	3×3	9	8		matinv3x3sf_getScratchSize
matinvgj4x4sf	funf	4×4	16	8		matinv4x4sf_getScratchSize
matinvgjnxnsf	funnf	N×N	N*N	8		matinvgjnxnsf_getScratchSize

Arguments	Type	Name	Size	Description
		N		Dimension (2, 3, 4, N)
		SA		Allocated space per matrix, in elements of given data type
Input:				
int		L		Number of matrices
int		qA		fixed point representation of matrices A (for fixed point API only)
Input/Output:				
complex_fract16, complex_float, float32_t		A	L*SA	Input/output complex matrices
Temporary:				
void*		pScr		Scratch memory area of requested by scratch allocation function

Restrictions All matrices and the scratch should not overlap and be aligned on 32-bytes boundary
L should be a multiple of mL parameter specified in the table above
N should be a multiple of mN parameter specified in the table above

Returned value none

2.7.11 LU Decomposition

Description	Functions compute LU decomposition of a square matrix A using partial pivoting with row interchanges: $P \cdot A = L \cdot U$ where P is the permutation matrix, L is the lower triangular factor (with diagonal elements equal to 1), U is the upper triangular factor. Algorithm is applied in-place to a sequence of real/complex matrices stored in block order. For each input matrix A the resulting factor U replaces the upper triangle of input matrix A, and subdiagonal elements of factor L replace the lower triangle of matrix A. Ones on the main diagonal of resulting factor L are discarded. Row ordering used by LU decomposition algorithm for input matrix A is stored to the vector of permutation indices P: i-th row of the matrix was interchanged with row $P[i]$ Decomposition result is not defined for a close to singular input matrix.
Representation	floating point
Algorithm	$L_{N \times N}^{(l)}, U_{N \times N}^{(l)} : P^{(l)} A_{N \times N}^{(l)} \equiv L_{N \times N}^{(l)} U_{N \times N}^{(l)}$ $A_{N \times N}^{(l)} = L_{N \times N}^{(l)} - I_{N \times N} + U_{N \times N}^{(l)}, l = 0 \dots L - 1$ where : $L_{N \times N}^{(l)}$ is lower triangle matrix of size $N \times N$ $U_{N \times N}^{(l)}$ is upper triangle matrix of size $N \times N$
Prototypes	<pre>void rfunf (void * pScr, float32_t * A, int16_t * P, int L); void rfunnf (void * pScr, float32_t * A, int16_t * P, int N, int L); void cfunf (void * pScr, complex_float * A, int16_t * P, int L); void cfunnf (void * pScr, complex_float * A, int16_t * P, int N, int L);</pre>

Function Name	Proto	Size	mL	mN	Scratch allocation function
Floating point, complex data, stream ordering					
clu2x2sf	cfunf	2×2	4		clu2x2sf_getScratchSize
clu3x3sf	cfunf	3×3	4		clu3x3sf_getScratchSize
clu4x4sf	cfunf	4×4	4		clu4x4sf_getScratchSize
clunxnsf	cfunnf	N×N	4		clunxnsf_getScratchSize
Floating point, real data, stream ordering					
lu2x2sf	rfunf	2×2	8		lu2x2sf_getScratchSize
lu3x3sf	rfunf	3×3	8		lu3x3sf_getScratchSize
lu4x4sf	rfunf	4×4	8		lu4x4sf_getScratchSize
lunxnsf	rfunnf	N×N	8		lunxnsf_getScratchSize
Floating point, complex data, stream ordering					
clu8x8nf	cfunf	8×8			clu8x8nf_getScratchSize
clu16x16nf	cfunf	16×16			clu16x16nf_getScratchSize
clunxnnf	cfunnf	N×N		4	clunxnnf_getScratchSize
Floating point, complex data, stream ordering					
lu8x8nf	rfunf	8×8			lu8x8nf_getScratchSize
lu16x16nf	rfunf	16×16			lu16x16nf_getScratchSize
lunxnnf	rfunnf	N×N		4	lunxnnf_getScratchSize

Arguments (stream ordered matrices)	Type	Name	Size	Description
	Input:			
	int	L		Number of matrices
	int	N		Dimension
	Input/Output:			
	complex_float, float32_t	A	[N*N] [L]	Input matrices, packed L and U factors on output
	Output:			
	int16_t	P	[N] [L]	Permutation index vectors
	Temporary:			
	void*	pScr		Scratch memory area of requested by scratch allocation function
Arguments (block ordered matrices)	Type	Name	Size	Description
	Input:			
	int	L		Number of matrices
	int	N		Dimension
		SA, SP		Matrix sizes SA, SP are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float SA=size (N*N) , SP=size (N)
	Input/Output:			
	complex_float, float32_t	A	[L] [SA]	Input matrices, packed L and U factors on output
	Output:			
	int16_t	P	[L] [SP]	Permutation index vectors
	Temporary:			
	void*	pScr		Scratch memory area of requested by scratch allocation function
Restrictions	All matrices and the scratch should not overlap and be aligned on 32-bytes boundary L should be a multiple of mL parameter specified in the table above N should be a multiple of mN parameter specified in the table above			
Returned value	none			

2.7.12 Matrix Determinant

Description	Functions compute determinant of matrix using previously taken LU decomposition
Representation	floating point
Algorithm	$L_{N \times N}^{(l)}, U_{N \times N}^{(l)} : P^{(l)} A_{N \times N}^{(l)} \equiv L_{N \times N}^{(l)} U_{N \times N}^{(l)}$ $\det(A_{N \times N}^{(l)}) = \prod_{n=0}^{N-1} U_{n,n}^{(l)}$ where : $L_{N \times N}^{(l)}$ is lower triangle matrix of size N×N $U_{N \times N}^{(l)}$ is upper triangle matrix of size N×N

Prototypes

```

void rfunf ( float32_t * D, const float32_t * LU, int L )
void rfunnf ( float32_t * D, const float32_t * LU, int N, int L )
void cfunf ( complex_float * D, const complex_float * LU, int L )
void cfunnf ( complex_float * D, const complex_float * LU, int N, int L )

```

Function Name	Proto	Size	mL	mN
Floating point, complex data, stream ordering				
cdet2x2sf	cfunf	2×2	4	
cdet3x3sf	cfunf	3×3	4	
cdet4x4sf	cfunf	4×4	4	
cdetnxnsf	cfunnf	M×N	4	
Floating point, real data, stream ordering				
det2x2sf	rfunf	2×2	8	
det3x3sf	rfunf	3×3	8	
det4x4sf	rfunf	4×4	8	
detnxnsf	rfunnf	M×N	8	
Floating point, complex data, stream ordering				
det8x8nf	cfunf	8×8		
det16x16nf	cfunf	16×16		
detnxnnf	cfunnf	M×N		4
Floating point, complex data, stream ordering				
det8x8nf	rfunf	8×8		
det16x16nf	rfunf	16×16		
detnxnnf	rfunnf	M×N		4

Arguments
(stream ordered
matrices)

Type	Name	Size	Description
Input:			
int	L		Number of matrices
int	N		Dimension
Input:			
complex_float, float32_t	LU	[N*N] [L]	packed L and U factors
Output:			
complex_float, float32_t	D	[L]	determinant values

Arguments (block
ordered matrices)

Type	Name	Size	Description
	N, M		Dimensions
Input:			
int	L		Number of matrices
int	N		Dimension
	SA		Matrix sizes SA is selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float SA=size (N*N)
Input:			
complex_float, float32_t	LU	[L] [SA]	packed L and U factors
Output:			
complex_float, float32_t	D	[L]	determinant values

Restrictions

All matrices and the scratch should not overlap and be aligned on 32-bytes boundary
L should be a multiple of mL parameter specified in the table above
N should be a multiple of mN parameter specified in the table above

Returned value

none

2.7.13 Singular Value Decomposition

Description	Singular value decomposition (SVD) of matrix A is $A = U \cdot S \cdot V^H$ where S is an $N \times N$ diagonal matrix with $s[0..N-1]$ on its main diagonal, and V^H is the (conjugate) transpose of V. For a real (complex) matrix A of size $M \times N$ with $M \geq N$, SVD functions compute: - the ordered sequence of N singular values $s[N], s[0] \geq s[1] \geq \dots \geq s[N-1]$ N orthogonal (orthonormal) left-singular vectors comprising the $M \times N$ column space matrix U N orthogonal (orthonormal) right-singular vectors comprising the $N \times N$ row space matrix V Left- and right-singular vectors form orthonormal (orthogonal) bases of the column (U) and row (V) spaces of matrix A. These bases, together with the singular values sequence, constitute the Thin Singular Value. SVD API operates with number of matrices at a time and allows to work with stream and block ordered matrices with some restrictions on input arguments for each case. NOTES: - SVD implementation may perform in-place transformations of input matrices, so INPUT DATA MAY APPEAR DAMAGED after the call. - Once U or V matrix is not required, set the corresponding output pointer parameter to zero to allow for a lower complexity implementation of the SVD algorithm.
Representation	floating point
Algorithm	TBD
Prototypes	<pre>void cfunf (void * pScr, complex_float* U, float32_t* s, complex_float* V, complex_float * A, int L) void cfunmnf (void * pScr, complex_float * U, float32_t* s, complex_float* V, complex_float * A, int M, int N, int L) void rfunf (void * pScr, float32_t * U, float32_t* s, float32_t* V, float32_t * A, int L) void rfunmnf (void * pScr, float32_t * U, float32_t * s, float32_t * V, float32_t * A, int M, int N, int L)</pre>

Function Name	Proto	Size	mL	mN	Scratch allocation function
Floating point, complex data, stream ordering					
svd2x2sf	cfunf	2x2	8		svd2x2sf_getScratchSize
svd3x3sf	cfunf	3x3	8		svd3x3sf_getScratchSize
svd4x4sf	cfunf	4x4	8		svd4x4sf_getScratchSize
svdmxnsf	cfunmnf	MxN	8		svdmxnsf_getScratchSize
Floating point, real data, stream ordering					
rsvd2x2sf	rfunf	2x2	8		rsvd2x2sf_getScratchSize
rsvd3x3sf	rfunf	3x3	8		rsvd3x3sf_getScratchSize
rsvd4x4sf	rfunf	4x4	8		rsvd4x4sf_getScratchSize
rsvdmxnsf	rfunmnf	MxN	8		rsvdmxnsf_getScratchSize
Floating point, complex data, stream ordering					
svd8x8nf	cfunf	8x8			svd8x8nf_getScratchSize
svd16x16nf	cfunf	16x16			svd16x16nf_getScratchSize
svdmxnfnf	cfunmnf	MxN		4	svdmxnfnf_getScratchSize
Floating point, complex data, stream ordering					
rsvd8x8nf	rfunf	8x8			rsvd8x8nf_getScratchSize
rsvd16x16nf	rfunf	16x16			rsvd16x16nf_getScratchSize
rsvdmxnfnf	rfunmnf	MxN		4	rsvdmxnfnf_getScratchSize

Arguments (stream ordered matrices)	Type	Name	Size	Description
	Input:			
	int	L		Number of matrices
	int	N, M		Dimensions
	Input/Output:			
	complex_float, float32_t	A	[M*N] [L]	Input matrices, streaming format. DAMAGED after the call due to the in-place operation
	Output:			
	complex_float, float32_t	U	[M*N] [L]	MxN matrices comprised of M left-singular column vectors. If NULL, this argument is assumed to be optional and will not be computed
	float32_t	s	[N] [L]	Nx1 vectors of singular values in descending order
	complex_float, float32_t	V	[N*N] [L]	NxN matrices comprised of N right-singular column vectors. If NULL, this argument is assumed to be optional and will not be computed
Arguments (block ordered matrices)	Temporary:			
	void*	pScr		Scratch memory area of requested by scratch allocation function
	Type	Name	Size	Description
	Input:			
	int	L		Number of matrices
	int	N, M		Dimensions
		SA Ss SV		Matrix sizes SA, Ss, SV are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float SA=size (M*N), Ss=size (N), SV=size (N*N)
	Input/Output:			
	complex_float, float32_t	A	[L] [SA]	Input matrices, streaming format. DAMAGED after the call due to the in-place operation
	Output:			
	complex_float, float32_t	U	[L] [SA]	MxN matrices comprised of M left-singular column vectors. If NULL, this argument is assumed to be optional and will not be computed
	float32_t	s	[L] [N]	Nx1 vectors of singular values in descending order
	complex_float, float32_t	V	[L] [SV]	NxN matrices comprised of N right-singular column vectors. If NULL, this argument is assumed to be optional and will not be computed
	Temporary:			
	void*	pScr		Scratch memory area of requested by scratch allocation function
All matrices and the scratch should not overlap and be aligned on 32-bytes boundary				
Restrictions	L should be a multiple of mL parameter specified in the table above			
	N should be a multiple of mN parameter specified in the table above			
	N must be greater than 1			
	M must be greater than or equal to N			
Returned value	none			

2.7.14 Eigenvalues and Eigenvectors

Description	For a complex (real) matrix A of size $N \times N$, <code>eigen</code> (<code>reigen</code>) functions compute: N eigenvalues $e[N]$, with possible repetitions N right eigenvectors $v[N]$, normalized such that the Euclidian norm of an eigenvector is 1 Right eigenvectors satisfy the equation $A \cdot v[n] = s[n] \cdot v[n]$, $n=0..N-1$. For complex matrices, eigenvalues and eigenvectors are also complex. For real matrices, eigenvalues and corresponding eigenvectors are either real, or come in conjugate pairs. This API operates with number of matrices at a time and allows to work with stream and block ordered matrices with some restrictions on input arguments for each case. NOTES: - Functions may perform in-place transformations of input matrices, so that INPUT DATA MAY APPEAR DAMAGED after the call. - Once the eigenvectors are not required, set the corresponding output pointer v to zero, so that a lower complexity algorithm will be used.																																																																																																																		
Representation	floating point																																																																																																																		
Algorithm	TBD																																																																																																																		
Prototypes	<pre>void cfunf (void * pScr, complex_float * e, complex_float * V, complex_float * A, int L); void cfunnf (void * pScr, complex_float * e, complex_float * V, complex_float * A, int N, int L); void rfunf (void * pScr, complex_float * e, complex_float * V, float32_t * A, int L); void rfunnf (void * pScr, complex_float * e, complex_float * V, float32_t * A, int N, int L);</pre>																																																																																																																		
<table><tr><th>Function Name</th><th>Proto</th><th>Size</th><th>mL</th><th>mN</th><th>Scratch allocation function</th></tr><tr><td colspan="6">Floating point, complex data, stream ordering</td></tr><tr><td><code>eigen2x2sf</code></td><td><code>cfunf</code></td><td><code>2x2</code></td><td>4</td><td></td><td><code>eigen2x2sf_getScratchSize</code></td></tr><tr><td><code>eigen3x3sf</code></td><td><code>cfunf</code></td><td><code>3x3</code></td><td>4</td><td></td><td><code>eigen3x3sf_getScratchSize</code></td></tr><tr><td><code>eigen4x4sf</code></td><td><code>cfunf</code></td><td><code>4x4</code></td><td>4</td><td></td><td><code>eigen4x4sf_getScratchSize</code></td></tr><tr><td><code>eigennxnsf</code></td><td><code>cfunnf</code></td><td><code>NxN</code></td><td>4</td><td></td><td><code>eigennxnsf_getScratchSize</code></td></tr><tr><td colspan="6">Floating point, real data, stream ordering</td></tr><tr><td><code>reigen2x2sf</code></td><td><code>rfunf</code></td><td><code>2x2</code></td><td>8</td><td></td><td><code>reigen2x2sf_getScratchSize</code></td></tr><tr><td><code>reigen3x3sf</code></td><td><code>rfunf</code></td><td><code>3x3</code></td><td>8</td><td></td><td><code>reigen3x3sf_getScratchSize</code></td></tr><tr><td><code>reigen4x4sf</code></td><td><code>rfunf</code></td><td><code>4x4</code></td><td>8</td><td></td><td><code>reigen4x4sf_getScratchSize</code></td></tr><tr><td><code>reigennxnsf</code></td><td><code>rfunnf</code></td><td><code>NxN</code></td><td>8</td><td></td><td><code>reigennxnsf_getScratchSize</code></td></tr><tr><td colspan="6">Floating point, complex data, stream ordering</td></tr><tr><td><code>eigen8x8nf</code></td><td><code>cfunf</code></td><td><code>8x8</code></td><td></td><td></td><td><code>eigen8x8nf_getScratchSize</code></td></tr><tr><td><code>eigen16x16nf</code></td><td><code>cfunf</code></td><td><code>16x16</code></td><td></td><td></td><td><code>eigen16x16nf_getScratchSize</code></td></tr><tr><td><code>eigennxnnf</code></td><td><code>cfunnf</code></td><td><code>NxN</code></td><td></td><td>4</td><td><code>eigennxnnf_getScratchSize</code></td></tr><tr><td colspan="6">Floating point, complex data, stream ordering</td></tr><tr><td><code>reigen8x8nf</code></td><td><code>rfunf</code></td><td><code>8x8</code></td><td></td><td></td><td><code>reigen8x8nf_getScratchSize</code></td></tr><tr><td><code>reigen16x16nf</code></td><td><code>rfunf</code></td><td><code>16x16</code></td><td></td><td></td><td><code>reigen16x16nf_getScratchSize</code></td></tr><tr><td><code>reigennxnnf</code></td><td><code>rfunnf</code></td><td><code>NxN</code></td><td></td><td>4</td><td><code>reigennxnnf_getScratchSize</code></td></tr></table>		Function Name	Proto	Size	mL	mN	Scratch allocation function	Floating point, complex data, stream ordering						<code>eigen2x2sf</code>	<code>cfunf</code>	<code>2x2</code>	4		<code>eigen2x2sf_getScratchSize</code>	<code>eigen3x3sf</code>	<code>cfunf</code>	<code>3x3</code>	4		<code>eigen3x3sf_getScratchSize</code>	<code>eigen4x4sf</code>	<code>cfunf</code>	<code>4x4</code>	4		<code>eigen4x4sf_getScratchSize</code>	<code>eigennxnsf</code>	<code>cfunnf</code>	<code>NxN</code>	4		<code>eigennxnsf_getScratchSize</code>	Floating point, real data, stream ordering						<code>reigen2x2sf</code>	<code>rfunf</code>	<code>2x2</code>	8		<code>reigen2x2sf_getScratchSize</code>	<code>reigen3x3sf</code>	<code>rfunf</code>	<code>3x3</code>	8		<code>reigen3x3sf_getScratchSize</code>	<code>reigen4x4sf</code>	<code>rfunf</code>	<code>4x4</code>	8		<code>reigen4x4sf_getScratchSize</code>	<code>reigennxnsf</code>	<code>rfunnf</code>	<code>NxN</code>	8		<code>reigennxnsf_getScratchSize</code>	Floating point, complex data, stream ordering						<code>eigen8x8nf</code>	<code>cfunf</code>	<code>8x8</code>			<code>eigen8x8nf_getScratchSize</code>	<code>eigen16x16nf</code>	<code>cfunf</code>	<code>16x16</code>			<code>eigen16x16nf_getScratchSize</code>	<code>eigennxnnf</code>	<code>cfunnf</code>	<code>NxN</code>		4	<code>eigennxnnf_getScratchSize</code>	Floating point, complex data, stream ordering						<code>reigen8x8nf</code>	<code>rfunf</code>	<code>8x8</code>			<code>reigen8x8nf_getScratchSize</code>	<code>reigen16x16nf</code>	<code>rfunf</code>	<code>16x16</code>			<code>reigen16x16nf_getScratchSize</code>	<code>reigennxnnf</code>	<code>rfunnf</code>	<code>NxN</code>		4	<code>reigennxnnf_getScratchSize</code>
Function Name	Proto	Size	mL	mN	Scratch allocation function																																																																																																														
Floating point, complex data, stream ordering																																																																																																																			
<code>eigen2x2sf</code>	<code>cfunf</code>	<code>2x2</code>	4		<code>eigen2x2sf_getScratchSize</code>																																																																																																														
<code>eigen3x3sf</code>	<code>cfunf</code>	<code>3x3</code>	4		<code>eigen3x3sf_getScratchSize</code>																																																																																																														
<code>eigen4x4sf</code>	<code>cfunf</code>	<code>4x4</code>	4		<code>eigen4x4sf_getScratchSize</code>																																																																																																														
<code>eigennxnsf</code>	<code>cfunnf</code>	<code>NxN</code>	4		<code>eigennxnsf_getScratchSize</code>																																																																																																														
Floating point, real data, stream ordering																																																																																																																			
<code>reigen2x2sf</code>	<code>rfunf</code>	<code>2x2</code>	8		<code>reigen2x2sf_getScratchSize</code>																																																																																																														
<code>reigen3x3sf</code>	<code>rfunf</code>	<code>3x3</code>	8		<code>reigen3x3sf_getScratchSize</code>																																																																																																														
<code>reigen4x4sf</code>	<code>rfunf</code>	<code>4x4</code>	8		<code>reigen4x4sf_getScratchSize</code>																																																																																																														
<code>reigennxnsf</code>	<code>rfunnf</code>	<code>NxN</code>	8		<code>reigennxnsf_getScratchSize</code>																																																																																																														
Floating point, complex data, stream ordering																																																																																																																			
<code>eigen8x8nf</code>	<code>cfunf</code>	<code>8x8</code>			<code>eigen8x8nf_getScratchSize</code>																																																																																																														
<code>eigen16x16nf</code>	<code>cfunf</code>	<code>16x16</code>			<code>eigen16x16nf_getScratchSize</code>																																																																																																														
<code>eigennxnnf</code>	<code>cfunnf</code>	<code>NxN</code>		4	<code>eigennxnnf_getScratchSize</code>																																																																																																														
Floating point, complex data, stream ordering																																																																																																																			
<code>reigen8x8nf</code>	<code>rfunf</code>	<code>8x8</code>			<code>reigen8x8nf_getScratchSize</code>																																																																																																														
<code>reigen16x16nf</code>	<code>rfunf</code>	<code>16x16</code>			<code>reigen16x16nf_getScratchSize</code>																																																																																																														
<code>reigennxnnf</code>	<code>rfunnf</code>	<code>NxN</code>		4	<code>reigennxnnf_getScratchSize</code>																																																																																																														

Arguments (stream ordered matrices)	Type	Name	Size	Description
	Input:			
	int	L		Number of matrices
	int	N		Dimensions
	Input/Output:			
	complex_float, float32_t	A	[N*N] [L]	Input matrices, streaming format. DAMAGED after the call due to the in-place operation
	Output:			
	complex_float	e	[N] [L]	Nx1 vectors of eigenvalues
	complex_float	V	[N*N] [L]	NxN matrices comprised of N column eigenvectors. If NULL, this argument is assumed to be optional and will not be computed
	Temporary:			
	void*	pScr		Scratch memory area of requested by scratch allocation function
Arguments (block ordered matrices)	Type	Name	Size	Description
	Input:			
	int	L		Number of matrices
	int	N		Dimensions
		SA Se		Matrix sizes SA, Se are selected as usual for block ordered matrix sequences of corresponding datatype, i.e. total size is rounded up to the closest bigger multiple of - BBE_SIMD_WIDTH/2==8 elements for float32_t - BBE_SIMD_WIDTH/4==4 elements for complex_float SA=size(N*N), Se=size(N)
	Input/Output:			
	complex_float, float32_t	A	[L] [SA]	Input matrices, streaming format. DAMAGED after the call due to the in-place operation
	Output:			
	complex_float	e	[L] [Se]	Nx1 vectors of eigenvalues
	complex_float	V	[L] [SA]	NxN matrices comprised of N column eigenvectors. If NULL, this argument is assumed to be optional and will not be computed
	Temporary:			
	void*	pScr		Scratch memory area of requested by scratch allocation function
Restrictions				
All matrices and the scratch should not overlap and be aligned on 32-bytes boundary				
L should be a multiple of mL parameter specified in the table above				
N should be a multiple of mN parameter specified in the table above				
N must be greater than 1				
Returned value				
none				

2.8 Communications

2.8.1 CRC Calculation

Description	This function calculates 8-, 16-, 24- or 32-bit CRC checksum:
Prototypes	Object allocation: size_t crc_alloc(int order, uint32_t poly)

This returns the size of memory in bytes to be allocated

Object initialization:

```
crc_handle_t crc_init(void * objmem, int order, uint32_t poly);
```

This returns a handle to the object

CRC processing:

```
uint32_t crc_process(crc_handle_t handle, uint32_t reg, const uint8_t*
bitstream, int N);
```

Arguments	Type	Name	Size	Description
	Input:			
	int	order		Order of polynomial (8,16,24 or 32)
	uint32	t_poly		Polynomial without higher degree, for example: $x^{16} + x^{12} + x^5 + 1$ corresponds to 0x1021
	void *	objmem		Allocated memory block
	uint8_t*	bitstream		Bitstream formatted as 8 bits per word
	int	N		Number of bytes
	crc_handle	handle		Handle to CRC
	uint32	reg		Original CRC register value
Returned Value	updated CRC register value			
Restrictions	bistream should be aligned on 2-byte boundary			

2.8.2 Slicer Hard: QPSK, QAM16, QAM64

Description	These functions make demapping signal points from predefined constellations to bits forming hard decisions. Demapping is done for fixed signal representation Q10 for all constellations. Supported constellations: QAM16,QAM64 and QPSK from [LTE] ETSI TS 136 211 V10.0.0 (2011-01); LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Physical channels and modulation (3GPP TS 36.211 version 10.0.0 Release 10)			
Algorithm	$s_n : \min_{m=0 \dots M-1} S(m) - x_n , n = 0 \dots N-1, S - \text{constellation}$			
Prototypes	<pre>void sliceh_lte_qpsk (int16_t * b, const int16_t * s, int N) void sliceh_lte_16qam(int16_t * b, const int16_t * s, int N) void sliceh_lte_64qam(int16_t * b, const int16_t * s, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int16_t	x	2*N	Complex signal points, Q10
	int	N		Number of signal points
	int	K		$\log_2$ of constellation size
	Output:			
	int16_t	d	N	bitstream formatted as sequence of 16-bit words each containing K-tuples
Returned Value	None			
Restrictions	s, b	must be aligned on a 32-bytes boundary		
	N	must be a multiple of 8		

2.8.3 QAM Mapper: QPSK, QAM16, QAM64

Description	These functions translate a number of binary K-tuples (where K is 2 for QPSK, 4 for QAM16 or 6 for QAM64) into an array of complex points. Hereafter we will refer to the following standards: [LTE] ETSI TS 136 211 V10.0.0 (2011-01); LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Physical channels and modulation (3GPP TS 36.211 version 10.0.0 Release 10)							
Algorithm	$x_n = S(d_n), n = 0 \dots \overline{N-1}, S - \text{constellation}$							
Prototypes	<pre>void qammap_qpsk(int16_t * r, const int16_t * d, int N, int type) void qammap_qam(int16_t * r, const int16_t * d, int N)</pre>							
Arguments	Constellation		Function Name		Prototype		Reference	
	QPSK		qammap_lte_qpsk		qammap_qpsk		[LTE] 7.1.2	
	LTE QAM16		qammap_lte_qam16		qammap_qam		[LTE] 7.1.3	
	LTE QAM64		qammap_lte_qam64		qammap_qam		[LTE] 7.1.4	
	Type		Name		Size		Description	
	Input:							
	int16_t		d		N		Binary K-tuples, $K = \log_2(P)$, where P is the number of signal points for the selected constellation. Each n-tuple occupies n LSBs of a 16-bit word; 16-n MSBs must be zero.	
	int		N				Number of K-tuples	
int		K				$\log_2$ of constellation size		
Output:								
int16_t		r		2*N		Complex signal points, Q5.10. In-phase (I) and quadrature (Q) phase components are interleaved with the I-component going first (at even indices).		
Returned Value	None							
Restrictions	N d, r		multiple of 16 must be aligned on a 32-bytes boundary					

2.8.4 Universal Grey Coding/QAM Mapping

Description	These functions make flexible mapping of input bits to the complex signal points. Constellations are represented as I/Q levels. Functions map data n-tuples ($n=1 \dots 16$) onto a user-defined QAM2ⁿ constellation. The user provides a constellation by passing an array s of 16-bit complex numbers, where the correspondence between a QAM point s and its related data item m is established through the array index: $s_I = S[2*m+0]$ (in-phase component), $s_Q = S[2*m+1]$ (quadrature phase component).
Algorithm	$r_n = S(d_n), n = 0 \dots \overline{N-1}, S - \text{constellation}$

Object Initialization

Type	Name	Size	Description
Input:			
int16_t	I	16	I levels
int16_t	Q	16	Q levels
int	M		Size of constellation

Returns: Handle to the object

Prototype

```
void gcode(void* pScr, int16_t * r, const int16_t * d, const int16_t * S, int N)
```

Arguments

Type	Name	Size	Description
Temporary:			
int16_t	pScr		Scratch memory for storing temporary data. Allocated space should be at least <code>GCODE_SCRATCH()</code> bytes
Input:			
int16_t	S	$2 * (2^n)$	Constellation, n is number of bits in a n -tuple
int16_t	d	N	Bitstream formatted as 8 bits per word
int	N		Number of signal points
Output:			
int16_t	r	$2 * N$	Output complex signal points

Returned Value

None

Restrictions

r, d aligned on a 32-bytes boundary
 N multiple of 16

Bitstream is not masking, so it is the user's responsibility to guarantee that the constellation map contains a complex number for each data item, otherwise the operation is undefined

2.8.5 Soft Demapper for QPSK and QAM Constellations

Description

Soft demapper algorithm converts equalized complex data into the sequence of log-likelihood ratios (LLRs). It supports QAM/QPSK constellations from LTE standard.

For detailed information on mapping bits onto constellations, please refer to [LTE] ETSI TS 136 211 V10.0.0 (2011-01); LTE; Evolved Universal Terrestrial Radio Access (E-UTRA); Physical channels and modulation (3GPP TS 36.211 version 10.0.0 Release 10); Soft decoding algorithm is run over a block of I/Q samples. For each input sample s , the demapper computes approximations of sub-optimal simplified log-likelihood ratios (LLR) for K bit positions, where K equals the base-2 logarithm of the constellation size. Let $S_{0,k,s}$ denote the constellation point with 0 at k -th bit position which is closest to s , while $S_{1,k,s}$ is the closest point with 1 at k -th bit position, $k=0 \dots K-1$. Then the value of sub-optimal LLR is defined as:

$$LLR(s, k) = (\min |s - S_{0,k,s}|^2 - \min |s - S_{1,k,s}|^2),$$

where σ^2 - scaling divisor

Prototypes

```
void softdemap (int16_t * llr, const int16_t * s, int N,)
```

Constellation	Function name	representation	Reference
LTE			
QPSK	softdemap_lte_qpsk	Q10	LTE QPSK [LTE] 7.1.2
QAM16	softdemap_lte_16qam	Q10	LTE 16QAM [LTE] 7.1.3
QAM64	softdemap_lte_64qam	Q10	LTE 64QAM [LTE] 7.1.4

Type	Name	Size	Description
Output:			
int16_t*	llr	K*N	LLR estimations, Q10. K equals the base-2 logarithm of the constellation size. llr[0] holds the LLR value for the left-most bit position of a K-tuple that matches s[0]+j*s[1]
Input:			
int16_t	s	2*N	input complex samples. I and Q components are interleaved with the real part going first (at even indices). Fixed point position depends on the constellation size, as described above
int	N		number of samples

Returned Value None

Restrictions llr, s must be aligned on a 32-bytes boundary
N multiple of 8

2.8.6 Bit Segmentation

Description Bit segmentation, bitstream of 1, 2, 3, 4, 6, 8 bits to 16 bits. This function takes as input a raw bitstream of length $M*N/8$ passed as a pointer to uint8_t type and then decomposes it and produces at the output N words, where a word is a 16-bit short. The choice for bits per word, M, is 1, 2, 3, 4, 6, 8, or 16.

Functions use precoded M value, for example, bsegmnt3() uses M=3.

Prototype

```
void bsegmnt1 (int16_t* s, const uint8_t* b, int N)
void bsegmnt2 (int16_t* s, const uint8_t* b, int N)
void bsegmnt3 (int16_t* s, const uint8_t* b, int N)
void bsegmnt4 (int16_t* s, const uint8_t* b, int N)
void bsegmnt6 (int16_t* s, const uint8_t* b, int N)
void bsegmnt8 (int16_t* s, const uint8_t* b, int N)
```

Arguments

Type	Name	Size	Description
Input:			
uint8_t	b	[N*M/8]	Unformatted bitstream
int	N		Number of symbols
int	M		Number of bits per symbol
Output:			
int16_t	s	N	Bitstream formatted as M bits per word

Returned Value None

Restrictions M should be 1, 2, 3, 4, 6 or 8
s, b must be aligned on a 32-bytes boundary
N must be a multiple of 16

2.8.7 Bit Packing

Description	Bit packing, formatted bitstream to raw bitstream. This function takes a formatted input bitstream stored in N words of M -bits per word, where a word is a short and M can be 1, 2, 3, 4, 6, 8 or 16. Its output is a raw stream of $N*M/8$ bits indexed by a pointer to <code>uint8_t</code> type.			
Prototype	<pre>void bpack1 (uint8_t* b, const int16_t* s, int N) void bpack2 (uint8_t* b, const int16_t* s, int N) void bpack3 (uint8_t* b, const int16_t* s, int N) void bpack4 (uint8_t* b, const int16_t* s, int N) void bpack6 (uint8_t* b, const int16_t* s, int N) void bpack8 (uint8_t* b, const int16_t* s, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	<code>int16_t</code>	<code>s</code>	<code>N</code>	Bitstream formatted as M bits per word
	<code>int</code>	<code>N</code>		Number of symbols
	<code>int</code>	<code>M</code>		Number of bits per symbol
	Output:			
	<code>uint8_t</code>	<code>b</code>	<code>[N*M/8]</code>	Unformatted bitstream
Returned Value	none			
Performance Restrictions	<code>M</code>	should be 1, 2, 3, 4, 6 or 8		
	<code>s, b</code>	must be aligned on a 32-bytes boundary		
	<code>N</code>	must be a multiple of 16		

2.8.8 Convolution Encoder

Description	Allows encoding of data with arbitrary selected polynomials with a constraint length from 3 to 9 and the code rate up to 1/4. Processing functions use precoded <code>R</code> value, for example, <code>convenc_process3()</code> uses <code>R=3</code> .																								
Object allocation	<code>size_t convenc_alloc()</code> Returns: size of memory in bytes to be allocated																								
Object Initialization	<code>convenc_handle_t convenc_init(void * objmem, int K, int R, const int16_t * poly)</code> <table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td>Input:</td><td></td><td></td><td></td></tr><tr><td><code>void*</code></td><td><code>objmem</code></td><td></td><td>Allocated memory block</td></tr><tr><td><code>int</code></td><td><code>K</code></td><td></td><td>Constraint length (3...9)</td></tr><tr><td><code>int</code></td><td><code>R</code></td><td></td><td>Code rate (2...4)</td></tr><tr><td><code>int16_t</code></td><td><code>poly</code></td><td><code>[R]</code></td><td>Polynomials in octal form</td></tr></table> Returns: handle to the object	Type	Name	Size	Description	Input:				<code>void*</code>	<code>objmem</code>		Allocated memory block	<code>int</code>	<code>K</code>		Constraint length (3...9)	<code>int</code>	<code>R</code>		Code rate (2...4)	<code>int16_t</code>	<code>poly</code>	<code>[R]</code>	Polynomials in octal form
Type	Name	Size	Description																						
Input:																									
<code>void*</code>	<code>objmem</code>		Allocated memory block																						
<code>int</code>	<code>K</code>		Constraint length (3...9)																						
<code>int</code>	<code>R</code>		Code rate (2...4)																						
<code>int16_t</code>	<code>poly</code>	<code>[R]</code>	Polynomials in octal form																						
Encoding	<code>int16_t convenc_process2 (convenc_handle_t handle, int16_t* e, const int16_t* s, int N) int16_t convenc_process3 (convenc_handle_t handle, int16_t* e, const int16_t* s, int N) int16_t convenc_process4 (convenc_handle_t handle, int16_t* e, const int16_t* s, int N)</code>																								

Arguments	Type	Name	Size	Description
	Input:			
	int16_t	s	N/16	unformatted bitstream (16 bits per word)
	int	N		Number of input words
	int16_t	state		State of encoder
	Output:			
	int16_t	e	(N*R)/16	unformatted bitstream
Returned Value	Updated state of encoder			
Restrictions	K	should be in range 3...9		
	R	should be in range 2...4		
	s, e	must be aligned on a 32-bytes boundary		

2.8.9 Zadoff-Chu Sequence

Description	Functions generate a complex exponential with some properties. Implement the base algorithms from LTE standard. There are 2 flavors of Zadov-Chu generation <code>zcseq_gen()</code> generates ZC per para 5.5.1.1, 3GPP TS 36.211 V8.8.0 (2009-09). It forms M outputs for $u=0...29$ and $v=0...1$. <code>zcseq_gen2()</code> calculates “raw sequence” $\exp(-j\pi u \cdot n \cdot (n+1) / N_{zc})$ defined in the para 5.7.2 of above recommendation with N_{zc} outputs and two possible predefined values of N_{zc} and u in range $1...N_{zc}-1$			
Accuracy	9 ($2.7 \cdot 10^{-4}$) for <code>zcseq_gen()</code> 43 ($1.3 \cdot 10^{-3}$) for <code>zcseq_gen2()</code>			
Prototype (first flavor)	void zcseq_gen (int16_t* r, int16_t u, int16_t v, int16_t M)			
	Type	Name	Size	Description
	Output:			
	int16_t	r	2*M	output complex Zadoff-Chu sequence, Q15
	Input:			
	int16_t	u		group number, 0...29
	int16_t	v		base sequence number within the group, 0...1
	int	M		size of sequence (36...1320 in steps of 12)
Returned Value	none			
Algorithm	see para 5.5.1.1, 3GPP TS 36.211 V8.8.0 (2009-09) Accuracy: $2 \cdot 10^{-4}$			
Restrictions	r should be aligned on 32-bytes boundary			
Prototype (second flavor)	void zcseq_gen2 (int16_t * r, const uint16_t u, uint16_t Nzc)			
	Type	Name	Size	Description
	Output:			
	int16_t	r	2*Nzc	output complex Zadoff-Chu sequence, Q15
	Input:			
	uint16_t	u		Root no. to be used for ZC sequence generation ($1...N_{zc}-1$)
	uint16_t	Nzc		Length of ZC sequence (839 or 139)
Returned Value	none			
Algorithm	This function generates u -th root Zadoff-Chu(ZC) sequence as defined in section 5.7.2 of 3GPP 36.211 V8.4.0(2008-09) $X_{u,n} = \exp(-j\pi u \cdot n \cdot (n+1) / N_{zc})$, where $0 \leq n \leq N_{zc}-1$			
Accuracy	for $N_{zc}=839$: $1.8 \cdot 10^{-3}$ for $N_{zc}=139$: $1.4 \cdot 10^{-3}$			
Restrictions	r should be aligned on 32-bytes boundary			

2.8.10 LTE Pseudo-random Sequence Generation and Scrambling

Description	Functions generate pseudo-random sequence (PRS) defined in the LTE standard (para 7.2) with given length and codeword size. Supported sizes are 1, 2, 4, 6, 8, 10 and 16 bits, with each codeword packed in a separate 16-bit word. In addition to pseudo-random sequence generation, the scrambler functions also perform bitwise modulo 2 summation between the generated sequence and input codewords. Function <code>lteprs_reset()</code> prepares shift registers and fills them with proper values derived from initial values (1 for first register and <code>cinit</code> for second one). Functions appear in the two forms: generic version allow to generate arbitrary number of output codewords, fast versions expose some additional restrictions in the arguments.																																				
Algorithm	See para 7.2 of 3GPP TS 36.211 V8.8.0 (2009-09)																																				
Prototypes	<pre>reset generator void lteprs_reset (uint32_t* r, uint32_t cinit) generate PRS void lteprs_gen (uint32_t* r, uint16_t *c, int N) scramble by PRS void lteprs_scramble (uint32_t* r, uint16_t *c, const uint16_t *b, int N)</pre>																																				
<table><tr><th>Codeword length, M</th><th>PRS generation Function name</th><th>Faster version</th></tr><tr><td>1</td><td><code>lteprs_gen1</code></td><td><code>lteprs_gen1_fast</code></td></tr><tr><td>2</td><td><code>lteprs_gen2</code></td><td><code>lteprs_gen2_fast</code></td></tr><tr><td>4</td><td><code>lteprs_gen4</code></td><td><code>lteprs_gen4_fast</code></td></tr><tr><td>6</td><td><code>lteprs_gen6</code></td><td><code>lteprs_gen6_fast</code></td></tr><tr><td>8</td><td><code>lteprs_gen8</code></td><td><code>lteprs_gen8_fast</code></td></tr><tr><td>10</td><td><code>lteprs_gen10</code></td><td><code>lteprs_gen10_fast</code></td></tr><tr><td>16</td><td><code>lteprs_gen16</code></td><td><code>lteprs_gen16_fast</code></td></tr></table>		Codeword length, M	PRS generation Function name	Faster version	1	<code>lteprs_gen1</code>	<code>lteprs_gen1_fast</code>	2	<code>lteprs_gen2</code>	<code>lteprs_gen2_fast</code>	4	<code>lteprs_gen4</code>	<code>lteprs_gen4_fast</code>	6	<code>lteprs_gen6</code>	<code>lteprs_gen6_fast</code>	8	<code>lteprs_gen8</code>	<code>lteprs_gen8_fast</code>	10	<code>lteprs_gen10</code>	<code>lteprs_gen10_fast</code>	16	<code>lteprs_gen16</code>	<code>lteprs_gen16_fast</code>												
Codeword length, M	PRS generation Function name	Faster version																																			
1	<code>lteprs_gen1</code>	<code>lteprs_gen1_fast</code>																																			
2	<code>lteprs_gen2</code>	<code>lteprs_gen2_fast</code>																																			
4	<code>lteprs_gen4</code>	<code>lteprs_gen4_fast</code>																																			
6	<code>lteprs_gen6</code>	<code>lteprs_gen6_fast</code>																																			
8	<code>lteprs_gen8</code>	<code>lteprs_gen8_fast</code>																																			
10	<code>lteprs_gen10</code>	<code>lteprs_gen10_fast</code>																																			
16	<code>lteprs_gen16</code>	<code>lteprs_gen16_fast</code>																																			
<table><tr><th>Codeword length, M</th><th>Scrambling Function Name</th><th>Faster version</th></tr><tr><td>1</td><td><code>lteprs_scramble1</code></td><td><code>lteprs_scramble1_fast</code></td></tr><tr><td>2</td><td><code>lteprs_scramble2</code></td><td><code>lteprs_scramble2_fast</code></td></tr><tr><td>4</td><td><code>lteprs_scramble4</code></td><td><code>lteprs_scramble4_fast</code></td></tr><tr><td>6</td><td><code>lteprs_scramble6</code></td><td><code>lteprs_scramble6_fast</code></td></tr><tr><td>8</td><td><code>lteprs_scramble8</code></td><td><code>lteprs_scramble8_fast</code></td></tr><tr><td>10</td><td><code>lteprs_scramble10</code></td><td><code>lteprs_scramble10_fast</code></td></tr><tr><td>16</td><td><code>lteprs_scramble16</code></td><td><code>lteprs_scramble16_fast</code></td></tr></table>		Codeword length, M	Scrambling Function Name	Faster version	1	<code>lteprs_scramble1</code>	<code>lteprs_scramble1_fast</code>	2	<code>lteprs_scramble2</code>	<code>lteprs_scramble2_fast</code>	4	<code>lteprs_scramble4</code>	<code>lteprs_scramble4_fast</code>	6	<code>lteprs_scramble6</code>	<code>lteprs_scramble6_fast</code>	8	<code>lteprs_scramble8</code>	<code>lteprs_scramble8_fast</code>	10	<code>lteprs_scramble10</code>	<code>lteprs_scramble10_fast</code>	16	<code>lteprs_scramble16</code>	<code>lteprs_scramble16_fast</code>												
Codeword length, M	Scrambling Function Name	Faster version																																			
1	<code>lteprs_scramble1</code>	<code>lteprs_scramble1_fast</code>																																			
2	<code>lteprs_scramble2</code>	<code>lteprs_scramble2_fast</code>																																			
4	<code>lteprs_scramble4</code>	<code>lteprs_scramble4_fast</code>																																			
6	<code>lteprs_scramble6</code>	<code>lteprs_scramble6_fast</code>																																			
8	<code>lteprs_scramble8</code>	<code>lteprs_scramble8_fast</code>																																			
10	<code>lteprs_scramble10</code>	<code>lteprs_scramble10_fast</code>																																			
16	<code>lteprs_scramble16</code>	<code>lteprs_scramble16_fast</code>																																			
<table><tr><th>Type</th><th>Name</th><th>Size</th><th>Description</th></tr><tr><td>Input/Output:</td><td></td><td></td><td></td></tr><tr><td><code>uint32_t</code></td><td><code>r</code></td><td>2</td><td>32-bit LTEPRS states</td></tr><tr><td>Output:</td><td></td><td></td><td></td></tr><tr><td><code>uint16_t*</code></td><td><code>c</code></td><td>N</td><td>output codewords</td></tr><tr><td>Input:</td><td></td><td></td><td></td></tr><tr><td><code>uint16_t*</code></td><td><code>b</code></td><td>N</td><td>input codewords</td></tr><tr><td><code>uint32_t</code></td><td><code>cinit</code></td><td></td><td>initialization value for second shift register</td></tr><tr><td><code>int</code></td><td><code>N</code></td><td></td><td>size of sequence</td></tr></table>		Type	Name	Size	Description	Input/Output:				<code>uint32_t</code>	<code>r</code>	2	32-bit LTEPRS states	Output:				<code>uint16_t*</code>	<code>c</code>	N	output codewords	Input:				<code>uint16_t*</code>	<code>b</code>	N	input codewords	<code>uint32_t</code>	<code>cinit</code>		initialization value for second shift register	<code>int</code>	<code>N</code>		size of sequence
Type	Name	Size	Description																																		
Input/Output:																																					
<code>uint32_t</code>	<code>r</code>	2	32-bit LTEPRS states																																		
Output:																																					
<code>uint16_t*</code>	<code>c</code>	N	output codewords																																		
Input:																																					
<code>uint16_t*</code>	<code>b</code>	N	input codewords																																		
<code>uint32_t</code>	<code>cinit</code>		initialization value for second shift register																																		
<code>int</code>	<code>N</code>		size of sequence																																		
Returned Value	none																																				
Restrictions	none for generic version For all fast versions: <code>c, b</code> must be aligned on a 32-bytes boundary <code>N</code> must be a multiple of 16																																				

2.8.11 SFD

Description	Alamouti SFD. Function calculates maximum likelihood outputs by Alamouti formulas for given Tx/Rx diversity. See [SFD1] 4x4 version is the SFD for combination of FSTD and SFBC technologies. In fact, this function calculates two independent <code>sfd2x2</code>. [SFD1] A Simple Transmit Diversity Technique for Wireless Communications. Siavash M. Alamouti, IEEE JOURNAL ON SELECT AREAS IN COMMUNICATIONS, VOL. 16, NO. 8, OCTOBER 1998 [SFD2] LTE - The UMTS Long Term Evolution: From Theory to Practice. Stefania Sesia, Issam Toufik, Matthew Baker
Algorithm	For <code>sfd2x2</code>: see [SFD1], formula (15) $s_0^{(n)} = h_0^{(n)*} \cdot r_0^{(n)} + h_1^{(n)} \cdot r_1^{(n)*} + h_2^{(n)*} \cdot r_2^{(n)} + h_3^{(n)} \cdot r_3^{(n)*}$ $s_1^{(n)} = h_1^{(n)*} \cdot r_0^{(n)} - h_0^{(n)} \cdot r_1^{(n)*} + h_3^{(n)*} \cdot r_2^{(n)} - h_2^{(n)} \cdot r_3^{(n)*}$ $n = 0 \dots N - 1$ For <code>sfd4x4</code>: see [SFD2] $s_0^{(n)} = h_0^{(n)*} \cdot r_0^{(n)} + h_1^{(n)} \cdot r_1^{(n)*} + h_2^{(n)*} \cdot r_2^{(n)} + h_3^{(n)} \cdot r_3^{(n)*}$ $s_1^{(n)} = h_1^{(n)*} \cdot r_0^{(n)} - h_0^{(n)} \cdot r_1^{(n)*} + h_3^{(n)*} \cdot r_2^{(n)} - h_2^{(n)} \cdot r_3^{(n)*}$ $s_2^{(n)} = h_4^{(n)*} \cdot r_4^{(n)} + h_5^{(n)} \cdot r_5^{(n)*} + h_6^{(n)*} \cdot r_6^{(n)} + h_7^{(n)} \cdot r_7^{(n)*}$ $s_3^{(n)} = h_5^{(n)*} \cdot r_4^{(n)} - h_4^{(n)} \cdot r_5^{(n)*} + h_7^{(n)*} \cdot r_6^{(n)} - h_6^{(n)} \cdot r_7^{(n)*}$ $n = 0 \dots N - 1$
Prototypes	<pre>void sfd2x2(int16_t * s0, int16_t * s1, const int16_t * r0, const int16_t * r1, const int16_t * r2, const int16_t * r3, const int16_t * h, int N, int rsh) void sfd4x4(int16_t * s0, int16_t * s1, int16_t * s2, int16_t * s3, const int16_t * r0, const int16_t * r1, const int16_t * r2, const int16_t * r3, const int16_t * r4, const int16_t * r5, const int16_t * r6, const int16_t * r7, const int16_t * h, int N, int rsh)</pre>

Type	Name	Size	Description
Input:			
int16_t	r0	2*N	input signals, complex vectors of N elements, fixed point presentation Q _r
int16_t	r1	2*N	
int16_t	r2	2*N	
int16_t	r3	2*N	
int16_t	r4	2*N	
int16_t	r5	2*N	
int16_t	r6	2*N	
int16_t	r7	2*N	
int16_t	h	[] [N] [2]	impulse response, sequence of N 2x2 (<code>sfd2x2</code>) or of 4x2 (<code>sfd4x4</code>) complex matrices stored in the streaming order (see 2.6.1), fixed point presentation Q _h
int	N		size of vectors
int	rsh		additional shift right of outputs
Output:			
int16_t*	s0	2*N	output signals, complex vectors of N elements, fixed point presentation Q _r +Q _h ·rsh
int16_t*	s1	2*N	
int16_t*	s2	2*N	
int16_t*	s3	2*N	

Returned value	none
Restrictions	arrays must not overlap arrays must be aligned on a 32-bytes boundary N must be a multiple of 8

2.9 Fast Fourier Transforms

2.9.1 General Definitions and Notes

Fixed point FFT functions make 16-bit scaling fast Fourier transforms for complex/real data. Floating point FFTs operate with floating point data and do not make scaling. FFTs use bit-reversal permutations so spectral data appear in the usual order. They normally use in-place transformations so **input data may be damaged**.

In the FFT with auto scaling, the omission of overflow is guaranteed if dynamic range of input data (`bexp` parameter) is set properly. In that case, the routine detects the dynamic range at the output of each internal stage and makes proper scaling at the next stage. This kind of FFT utilizes dedicated FFT ISA available on the BBE32EP core.

All the routines return total shift count due to the scaling. Floating point functions do not make additional scaling so they just return zero.

2.9.2 FFT on Complex Data

Description	These functions make FFT on complex data of various lengths: 2^n, $n=4..15$ mixed-radix fixed point FFTs: $12 \cdot 2^m \cdot 3^n \cdot 5^p$ in range 12...1200 and 1536 These functions make forward FFT on complex data. Fixed point FFT functions with <code>_norm</code> suffix expect input data to be normalized, i.e. the minimum number of redundant sign bits over <code>x[]</code> (a.k.a the common block exponent) should be zero. Neglecting to normalize data leads to significant loss in transform quality. On the contrary, regular variants with no <code>_norm</code> suffix allow for non-zero common block exponent, but they appear slightly slower due to internal data normalization. <code>tffft</code> family functions make FFT on complex data of power of 2 sizes: 2^n, $n=13..15$. As opposed to regular FFT routines, these use smaller twiddle factor tables but work a bit slower. NOTES: Bit-reversing permutation is done here. FFT runs in-place algorithm, so INPUT DATA WILL APPEAR DAMAGED after the call For mixed-radix FFTs, input and output data arrays size (<code>s</code>) might be bigger than FFT size (<code>N</code>) to provide effective data aligned data access. This size is calculated by <code>CNFFT_BUF_SIZE(N)</code>
Representation:	16-bit input, 16-bit output, automatic data scaling at each stage floating point input, floating point output:
Algorithm	$y = FFT(x)$
Prototypes	Radix2 FFT with auto scaling <pre>int cfftas (complex_fract16 * y, complex_fract16 * x, int bexp); int cfftas_norm (complex_fract16 * y, complex_fract16 * x);</pre> Floating point FFT <pre>int cfftff (complex_float * y, complex_float * x);</pre> Mixed radix FFT with auto scaling <pre>int cnfftas(int16_t * pScr, complex_fract16 * y, complex_fract16 * x, int bexp);</pre>

Name	Size, N	Name	Size, N	Prototype
fixed point FFT, auto scaling, power of 2				
cfftas16	16	cfftas1024	1024	cfftas
cfftas32	32	cfftas2048	2048	
cfftas64	64	cfftas4096	4096	
cfftas128	128	cfftas8192	8192	
cfftas256	256	cfftas16384	16384	
cfftas512	512	cfftas32768	32768	
Floating point FFT, power of 2				
cfft16f	16	cfft1024f	1024	cfft1f
cfft32f	32	cfft2048f	2048	
cfft64f	64	cfft4096f	4096	
cfft128f	128	cfft8192f	8192	
cfft256f	256	cfft16384f	16384	
cfft512f	512	cfft32768f	32768	
Fixed point FFT with reduced twiddle tables				
tfftas8192	8192	tfftas32768	32768	cfftas
tfftas16384	16384			
Floating point FFT with reduced twiddle tables				
tfft8192f	8192	tfft32768f	32768	cfft1f
tfft16384f	16384			
Mixed radix fixed point FFT with auto scaling				
cnfft12	12	cnfft384	384	cnfftas
cnfft24	24	cnfft432	432	
cnfft36	36	cnfft480	480	
cnfft48	48	cnfft540	540	
cnfft60	60	cnfft576	576	
cnfft72	72	cnfft600	600	
cnfft96	96	cnfft648	648	
cnfft108	108	cnfft720	720	
cnfft120	120	cnfft768	768	
cnfft144	144	cnfft864	864	
cnfft180	180	cnfft900	900	
cnfft192	192	cnfft960	960	
cnfft216	216	cnfft972	972	
cnfft240	240	cnfft1080	1080	
cnfft288	288	cnfft1152	1152	
cnfft300	300	cnfft1200	1200	
cnfft324	324	cnfft1536	1536	
cnfft360	360			
auto scaling, normalized input				
cfftas16_norm	16	cfftas1024_norm	1024	cfftas_norm
cfftas32_norm	32	cfftas2048_norm	2048	
cfftas64_norm	64	cfftas4096_norm	4096	
cfftas128_norm	128	cfftas8192_norm	8192	
cfftas256_norm	256	cfftas16384_norm	16384	
cfftas512_norm	512	cfftas32768_norm	32768	

Arguments	Type	Name	Size	Description
		N		FFT size
		S		Allocated space for input/output data, CNFFT_BUF_SIZE(N)
Input:				
complex_fract16, complex_float	x	S		Complex input signal
int	bexp			Common block exponent (for fixed point API only), that is the minimum number of redundant sign bits over input data x[]
Output:				
complex_fract16, complex_float	y	S		Output spectrum
Scratch:				

	int16_t	pScr		Scratch memory for storing temporary data (for mixed radix FFT only), Size is defined by <code>CNFFT_SCRATCH_SIZE(N)</code>
Returned value	Total shift amount applied throughout the transform to scale the data. Total shift is bi-directional, with positive numbers corresponding to the right shift.			
Restrictions	$x, y, pScr$ should not overlap $x, y, pScr$ aligned on a 32-bytes boundary			

2.9.3 FFT on Real Data

Description	These functions make FFT on real data of length 2^n, $n=4..15$. The algorithm exploits the symmetry properties of the FFT: first, a complex FFT of half the original size is applied to input data, then the resulting spectrum undergoes a postprocessing procedure which results in complex spectrum of real input data. NOTES: 1. Bit-reversing reordering is done here. 2. FFT runs in-place algorithm so INPUT DATA WILL APPEAR DAMAGED after the call.
Representation	16-bit input, 16-bit output, automatic data scaling at each stage floating point input, floating point output
Algorithm	$y = FFT(real(x))$
Prototype	Fixed point FFT : <pre>int rfft (complex_fract16 * y, int16_t * x, int bexp)</pre> Floating point FFT <pre>int rfftf (complex_float* y, float32_t * x)</pre>

Name	Size, N	Name	Size, N	Prototype
Fixed point FFT				
rfft16	16	rfft1024	1024	rfft
rfft32	32	rfft2048	2048	
rfft64	64	rfft4096	4096	
rfft128	128	rfft8192	8192	
rfft256	256	rfft16384	16384	
rfft512	512	rfft32768	32768	
Floating point FFT				
rfft16f	16	rfft1024f	1024	rfftf
rfft32f	32	rfft2048f	2048	
rfft64f	64	rfft4096f	4096	
rfft128f	128	rfft8192f	8192	
rfft256f	256	rfft16384f	16384	
rfft512f	512	rfft32768f	32768	

Arguments	Type	Name	Size	Description
	Input:			
	int16_t, float32_t	x	N	Input signal
	int	bexp		Common block exponent (for fixed point API only), that is the minimum number of redundant sign bits over input data $x[]$
	Output:			
	complex_fract16, complex_float	y	(N/2+1)	Output spectrum (positive side)
Returned value	Total right shift amount applied throughout the transform stages to scale the data.			
Restrictions	x, y should not overlap x, y aligned on a 32-bytes boundary			

2.9.4 Inverse FFT on Complex Data

Description	These functions make inverse FFT on complex data of various lengths: 2^n, $n=4..15$ mixed-radix fixed point IFFTs: $12 \cdot 2^m \cdot 3^n \cdot 5^p$ in range 12...1200 and 1536 These functions make forward IFFT on complex data. Fixed point IFFT functions with <code>_norm</code> suffix expect input data to be normalized, i.e. the minimum number of redundant sign bits over <code>x[]</code> (a.k.a the common block exponent) should be zero. Neglecting to normalize data leads to significant loss in transform quality. On the contrary, regular variants with no <code>_norm</code> suffix allow for non-zero common block exponent, but they appear slightly slower due to internal data normalization. <code>tffft</code> family functions make IFFT on complex data of power of 2 sizes: 2^n, $n=13..15$. As opposed to regular IFFT routines, these use smaller twiddle factor tables but work a bit slower. NOTES: Bit-reversing permutation is done here. IFFT runs in-place algorithm, so INPUT DATA WILL APPEAR DAMAGED after the call For mixed-radix IFFTs, input and output data arrays size (<code>s</code>) might be bigger than IFFT size (<code>N</code>) to provide effective data aligned data access. This size is calculated by <code>CINFFT_BUF_SIZE(N)</code>
Representation:	16-bit input, 16-bit output, automatic data scaling at each stage floating point input, floating point output:
Algorithm	$y = FFT^{-1}(x)$
Prototypes	Radix2 FFT with auto scaling <pre>int ciffstas (complex_fract16 * y, complex_fract16 * x, int bexp); int ciffstas_norm (complex_fract16 * y, complex_fract16 * x);</pre> Floating point FFT <pre>int ciffstf (complex_float * y, complex_float * x);</pre> Mixed radix FFT with auto scaling <pre>int cinnstas(int16_t * pScr, complex_fract16 * y, complex_fract16 * x,int bexp)</pre>

Name	Size, N	Name	Size, N	Prototype
fixed point FFT, auto scaling, power of 2				
cifftas16	16	cifftas1024	1024	cifftas
cifftas32	32	cifftas2048	2048	
cifftas64	64	cifftas4096	4096	
cifftas128	128	cifftas8192	8192	
cifftas256	256	cifftas16384	16384	
cifftas512	512	cifftas32768	32768	
Floating point FFT, power of 2				
cifft16f	16	cifft1024f	1024	cifftf
cifft32f	32	cifft2048f	2048	
cifft64f	64	cifft4096f	4096	
cifft128f	128	cifft8192f	8192	
cifft256f	256	cifft16384f	16384	
cifft512f	512	cifft32768f	32768	
Fixed point FFT with reduced twiddle tables				
tifftas8192	8192	tifftas32768	32768	cifftas
tifftas16384	16384			
Floating point FFT with reduced twiddle tables				
tifft8192f	8192	tifft32768f	32768	cifftf
tifft16384f	16384			
Mixed radix fixed point FFT with auto scaling				
cinfft12	12	cinfft384	384	cnfftas
cinfft24	24	cinfft432	432	
cinfft36	36	cinfft480	480	

	cinfft48	48	cinfft540	540	
	cinfft60	60	cinfft576	576	
	cinfft72	72	cinfft600	600	
	cinfft96	96	cinfft648	648	
	cinfft108	108	cinfft720	720	
	cinfft120	120	cinfft768	768	
	cinfft144	144	cinfft864	864	
	cinfft180	180	cinfft900	900	
	cinfft192	192	cinfft960	960	
	cinfft216	216	cinfft972	972	
	cinfft240	240	cinfft1080	1080	
	cinfft288	288	cinfft1152	1152	
	cinfft300	300	cinfft1200	1200	
	cinfft324	324	cinfft1536	1536	
	cinfft360	360			
auto scaling, normalized input					
	cifftas16 norm	16	cifftas1024 norm	1024	cifftas_norm
	cifftas32 norm	32	cifftas2048 norm	2048	
	cifftas64 norm	64	cifftas4096 norm	4096	
	cifftas128 norm	128	cifftas8192 norm	8192	
	cifftas256 norm	256	cifftas16384 norm	16384	
	cifftas512 norm	512	cifftas32768 norm	32768	

Arguments	Type	Name	Size	Description
		N		FFT size
		S		Allocated space for input/output data, CINFFT_BUF_SIZE(N)
Input:				
complex_fract16, complex_float	x	S		Input spectrum. Real and imaginary data are interleaved; real data goes first
int	bexp			Common block exponent (for fixed point API only), that is the minimum number of redundant sign bits over input data x[]
Output:				
complex_fract16, complex_float	y	S		Complex output signal. Real and imaginary data are interleaved; real data goes first
Scratch:				
int16_t	pScr			Scratch memory for storing temporary data (for mixed radix IFFT only), Size is defined by CINFFT_SCRATCH_SIZE(N)
Returned value	Total shift amount applied throughout the transform to scale the data. Total shift is bi-directional, with positive numbers corresponding to the right shift.			
Restrictions	x, y, pScr should not overlap x, y, pScr aligned on a 32-bytes boundary			

2.9.5 Inverse FFT Forming Real Data

Description	These functions make inverse FFT forming real data of length 2^n, $n=4..15$. The algorithm exploits the symmetry properties of the IFFT: the input spectrum is modified in such a way that a complex-valued inverse FFT of half the original size that is applied to the transformed spectrum actually results in real data. NOTES: 1. Bit-reversing reordering is done here. 2. FFT runs in-place algorithm so INPUT DATA WILL APPEAR DAMAGED after the call.
Representation	16-bit input, 16-bit output, automatic data scaling at each stage floating point input, floating point output
Algorithm	$y = \text{real}(\text{FFT}^{-1}(x))$

Prototype

Fixed point FFT :

int rffft (int16_t * y, complex_fract16 * x, int bexp)

Floating point FFT

int rffftf (float32_t * y, complex_float * x)

Name	Size, N	Name	Size, N	Prototype
Fixed point FFT				
riffft16	16	riffft1024	1024	riffft
riffft32	32	riffft2048	2048	
riffft64	64	riffft4096	4096	
riffft128	128	riffft8192	8192	
riffft256	256	riffft16384	16384	
riffft512	512	riffft32768	32768	
Floating point FFT				
riffft16f	16	riffft1024f	1024	riffft f
riffft32f	32	riffft2048f	2048	
riffft64f	64	riffft4096f	4096	
riffft128f	128	riffft8192f	8192	
riffft256f	256	riffft16384f	16384	
riffft512f	512	riffft32768f	32768	

Arguments	Type	Name	Size	Description
	Input:			
	complex_fract16, complex_float	x	(N/2+1)	Input spectrum
	int	bexp		Common block exponent (for fixed point API only), that is the minimum number of redundant sign bits over input data x[]
	Output:			
	int16_t, float32_t	y	N	Real output signal
Returned value	Total right shift amount applied throughout the transform stages to scale the data.			
Restrictions	x, y	should not overlap		
	x, y	aligned on a 32-bytes boundary		

2.9.6 Blockwise FFT on Complex Data

Description	These functions make forward FFT on L blocks. Fixed point functions work with no scaling so It is user's responsibility to pre-scale input data in such a way that FFT calculation overflows are avoided. NOTES: 1. Bit-reversing permutation is done here. 2. FFT runs an in-place algorithm, so INPUT DATA WILL APPEAR DAMAGED after the call. 3. For mixed-radix FFTs, input and output data arrays size (s) might be bigger than FFT size (N) to provide effective data aligned data access. This size is calculated by <code>BCNFFT_BUF_SIZE(N)</code>. These functions also require additional scratch memory for storage of intermediate data.
Representation	16-bit input, 16-bit output, automatic data scaling at each stage floating point input, floating point output
Algorithm	$y = FFT(x)$
Prototypes	<pre>int bcfft (complex_fract16 * y, complex_fract16 * x, int L) int bcfft (complex_float * y, complex_float * x, int L) int bcnfft (int16_t * pScr, complex_fract16* y, complex_fract16 * x, int L)</pre>

Name	Size, N	Name	Size, N	Prototype
Fixed point FFT				
bcfft16	16	bcfft64	64	bcfft

	bcfft32	32	bcfft128	128	
	Floating point FFT				
	bcfft16f	16	bcfft64f	64	bcfftf
	bcfft32f	32	bcfft128f	128	
	Fixed point mixed radix FFT				
	bcnfft12	12	bcnfft72	72	bcnfft
	bcnfft24	24	bcnfft96	96	
	bcnfft36	36	bcnfft108	108	
	bcnfft48	48	bcnfft120	120	
	bcnfft60	60			

Arguments	Type	Name	Size	Description
		N		FFT size
		S		N for radix2 FFT, BCNFFT_BUF_SIZE(N) for mixed radix FFT
	Input:			
	complex_fract16, complex_float	x	L*S	Complex input signal. Real and imaginary data are interleaved; real data goes first.
	int	L		number of data blocks
	Output:			
	complex_fract16, complex_float	y	L*S	Output spectrum
	Temporary:			
	int16_t	pScr		scratch size (for mixed radix fixed point API) in 16-bit words defined by BCNFFT_SCRATCH_SIZE(N)

Returned value	Total shift amount applied throughout the transform to scale the data. Total shift is bi-directional, with positive numbers corresponding to the right shift.		
----------------	---	--	--

Restrictions	x, y, pScr	should not overlap
	x, y, pScr	aligned on a 32-bytes boundary
	L	should be bigger than zero

2.9.7 Blockwise Inverse FFT on Complex Data

Description	These functions make inverse FFT on L blocks. Fixed point functions work with no scaling so It is user's responsibility to pre-scale input data in such a way that IFFT calculation overflows are avoided. NOTES: 1. Bit-reversing permutation is done here. 2. FFT runs an in-place algorithm, so INPUT DATA WILL APPEAR DAMAGED after the call. 3. For mixed-radix FFTs, input and output data arrays size (s) might be bigger than IFFT size (N) to provide effective data aligned data access. This size is calculated by <code>BCINFFT_BUF_SIZE(N)</code>. These functions also require additional scratch memory for storage of intermediate data.
Representation	16-bit input, 16-bit output, automatic data scaling at each stage floating point input, floating point output
Algorithm	$y = FFT^{-1}(x)$
Prototypes	<pre>int bcifft (int16_t * y, int16_t * x, int L) int bcifftf (float32_t * y, float32_t * x, int L) int bcinfft (int16_t * pScr, int16_t * y, int16_t * x, int L)</pre>

Name	Size, N	Name	Size, N	Prototype
Fixed point FFT				
bcifft16	16	bcifft64	64	bcifft
bcifft32	32	bcifft128	128	
Floating point FFT				
bcifft16f	16	bcifft64f	64	bcifftf
bcifft32f	32	bcifft128f	128	

Fixed point mixed radix FFT				
bcinfft12	12	bcinfft72	72	bcinfft
bcinfft24	24	bcinfft96	96	
bcinfft36	36	bcinfft108	108	
bcinfft48	48	bcinfft120	120	
bcinfft60	60			

Arguments	Type	Name	Size	Description
		N		FFT size
		S		N for radix2 FFT, BCINFFT_BUF_SIZE(N) for mixed radix IFFT
Input:				
complex_fract16, complex_float	x	L*S		Complex spectrum
int	L			number of data blocks
Output:				
complex_fract16, complex_float	y	L*S		Output input signal
Temporary:				
int16_t	pScr			scratch size (for mixed radix fixed point API) in 16-bit words defined by BCINFFT_SCRATCH_SIZE(N)

Returned value	Total shift amount applied throughout the transform to scale the data. Total shift is bi-directional, with positive numbers corresponding to the right shift.			
Restrictions	x, y, pScr	should not overlap		
	x, y, pScr	aligned on a 32-bytes boundary		
	L	should be bigger than zero		

2.9.8 Blockwise FFT on Real Data

Description	These functions make forward FFT on real data on L blocks. FFT implementation for real signal exploits the symmetry properties of the Fourier Transform: first, a blockwise complex FFT of half the original size is applied to input data, and then the resulting spectrum undergoes a conversion procedure which results in complex spectrum of real input data. NOTES: 1. Bit-reversing permutation is done here. 2. FFT runs an in-place algorithm, so INPUT DATA WILL APPEAR DAMAGED after the call. 3. As the complex spectrum of a real FFT is conjugate symmetric about the midpoint, the <code>brfft</code> functions generate only the first $(N/2) + 1$ points of the FFT, with the first point being the DC component, and the last point – the Nyquist frequency component. For real signal these two spectral components have zero imaginary part, thus they can be packed in a single “complex” value. Finally, the m-th row of the output matrix will contain the following values: ▪ DC component of the signal in <code>y[m][0].re</code> ▪ Nyquist frequency component in <code>y[m][0].im</code> ▪ first half of the complex spectrum in <code>y[m][1]...y[m][(N/2)-1]</code> These data are used to reconstruct N values that are stored into the m-th row of real output matrix <code>y[m][N]</code>. The reconstruction is accomplished through a special conversion of input spectrum, such that subsequent invocation of blockwise complex inverse FFT of size $N/2$ actually produces the desired real signal.			
Representation	floating point input, floating point output			
Algorithm	$y = FFT(real(x))$			
Prototypes	<pre>int brfft (float32_t * y, float32_t * x, int L)</pre>			
	Name	Size, N	Name	Size, N
	brfft16f	16	bfft64f	64
	brfft			

	brfft32f	32	bfft128f	128	
--	----------	----	----------	-----	--

Arguments	Type	Name	Size	Description
		N		FFT size
Input:				
float32_t	x		L*N	Real input signal
int	L			number of data blocks
Output:				
float32_t	y		L*N	Output spectrum. Real and imaginary data are interleaved; real data goes first, see Note 3 above

Returned value	zero
----------------	------

Restrictions	x, y	should not overlap
	x, y	aligned on a 32-bytes boundary
	L	should be bigger than zero

2.9.9 Blockwise Inverse FFT Forming Real Data

Description	These functions make inverse FFT forming real data on L blocks. FFT implementation for real signal exploits the symmetry properties of the Fourier Transform: first, a blockwise complex FFT of half the original size is applied to input data, and then the resulting spectrum undergoes a conversion procedure which results in complex spectrum of real input data. NOTES: 1. Bit-reversing permutation is done here. 2. IFFT runs an in-place algorithm, so INPUT DATA WILL APPEAR DAMAGED after the call. 3. Spectrum of real signal is conjugate-symmetric about the midpoint, thus real inverse FFT functions require only the first half of spectrum and Nyquist frequency bin (altogether $N/2+1$ complex points) to reconstruct the time domain signal. Moreover, the DC component and Nyquist frequency component have zero imaginary part, thus they can be packed in a single “complex” value in input data. So, the m-th row of the input matrix shall contain the following values: ▪ DC component of the signal in $x[m][0].re$ ▪ Nyquist frequency component in $x[m][0].im$ ▪ first half of the complex spectrum in $x[m][1]...x[m][(N/2)-1]$ These data are used to reconstruct N values that are stored into the m-th row of real output matrix $y[m][N]$. The reconstruction is accomplished through a special conversion of input spectrum, such that subsequent invocation of blockwise complex inverse FFT of size $N/2$ actually produces the desired real signal.														
Representation	floating point input, floating point output														
Algorithm	$y = real(FFT^{-1}(x))$														
Prototypes	<pre>int brifftf (float32_t * y, float32_t * x, int L)</pre> <table><tr><th>Name</th><th>Size, N</th><th>Name</th><th>Size, N</th><th>Prototype</th></tr><tr><td>brifft16f</td><td>16</td><td>brifft64f</td><td>64</td><td rowspan="2">brifftf</td></tr><tr><td>brifft32f</td><td>32</td><td>brifft128f</td><td>128</td></tr></table>	Name	Size, N	Name	Size, N	Prototype	brifft16f	16	brifft64f	64	brifftf	brifft32f	32	brifft128f	128
Name	Size, N	Name	Size, N	Prototype											
brifft16f	16	brifft64f	64	brifftf											
brifft32f	32	brifft128f	128												

Arguments	Type	Name	Size	Description
		N		FFT size
Input:				
float32_t	x		L*N	Input spectrum. Real and imaginary data are interleaved; real data goes first, see Note 3 above
int	L			number of data blocks
Output:				

float32_t	y	L*N	Output real signal
-----------	---	-----	--------------------

Returned value	zero		
Restrictions	x, y	should not overlap	
	x, y	aligned on a 32-bytes boundary	
	L	should be bigger than zero	

2.10 Fitting and interpolation routines

2.10.1 Polynomial Fitting

Polynomial fitting finds the coefficients of a polynomial p of degree N that fits the data in M points y_m , $m = \overline{0..M-1}$, in a least squares sense. The resulted p is a vector of length $N+1$ containing the polynomial coefficients in descending powers. For the best accuracy in the fixed point environment the library uses centered representation and exact resulted formula is

$$y(x) = \sum_{n=0}^N p_n x^{N-n}$$

The process is organized in 3 phases:

at the first stage algorithm calculates Vandermonde matrices V , R for given input grid. This stage does not depend on the values in the grid points

at the second stage, Vandermonde matrices are applied to the vector y calculating polynomial coefficients

at the third stage user may interpolate arbitrary data using given polynomial coefficients

NOTES:

If input grid data are known in advance, first stage may be omitted and V , R matrices would be precalculated.

For better performance on the BBE32EP, the matrices V and R are stored in the transposed form and should be aligned on 32-bytes boundary. Additionally, each row is padded with needed number of zeros forming $8 \times M'$ and $8 \times N'$ matrices. For fixed point routines M' is a closest bigger multiple of 16, N' is 16. For floating point routines M' is a closest bigger multiple of 8, N' is 8.

For fixed point API, library uses following representations:

Grid points, data values	Q15
Polynomial coefficients	Q8.23
Vandermonde matrix V	Q15
Upper triangle decomposition of Vandermonde matrix R	Q11

That selection guaranties no overflow and reasonable accuracy in most cases. However, for better accuracy of polyfit routines it is recommended to scale input grid in such a way to maximize the span.

Fixed-point nature of BBE32EP core also affects to the overall accuracy of fitting. Library uses iterative enhancement in the `pfit_process()` stage to compensate possible accuracy degradation

which especially observing at bigger polynomial degrees or small dynamic range of input grid points. User may control the number of iteration to find reasonable compromise between accuracy and performance. Also, it would be noted that maximum number of points in the input grid is limited by the dynamic range of matrix R (Q11) and 40-bit width of BBE32EP registers.

Polynomial degree, N	Maximum number of grid points, M
2	230
3	256
4	256
5	256
6	256

Description	the <code>pfit_XXX</code> functions fit (in least squares sense) a degree N polynomial to input data sampled at a points grid of length M , and use that polynomial to interpolate data at arbitrary query points. Namely, the <code>pfitN_grid()</code> functions compute the Vandermonde matrix for the sample points grid and perform the Cholesky decomposition of that matrix, the <code>pfitN_process()</code> functions calculate the least squares solution for the polynomial coefficients. Finally, the <code>pfitN_eval()</code> functions evaluate the polynomial at query points.
Representation	16-bit input, 16-bit output floating point input, floating point output
Algorithm	see description above

Prototypes

Fixed point API:

preprocessing functions:

```
void fngrid(void * pScr, int16_t * V, int16_t * R, const int16_t *x, int M) ;
```

Scratch allocation:

```
size_t pfit_grid_getScratchSize( int M, int N )
```

calculate coefficients:

```
void fnprocess ( void * pScr, int32_t * p, const int16_t * V, const int16_t * R,
                 const int16_t * y, int M, int maxIter) ;
```

Scratch allocation:

```
size_t pfit_process_getScratchSize( int M, int N )
```

interpolation using given coefficients:

```
void fneval (void* pScr,int16_t* yi,const int16_t *xi, const int32_t * p,int P) ;
```

Polynomial degree, N	Preprocessing	Calculate coefficients	Interpolation
1	<code>pfit_grid1</code>	<code>pfit_process1</code>	<code>pfit_eval1</code>
2	<code>pfit_grid2</code>	<code>pfit_process2</code>	<code>pfit_eval2</code>
3	<code>pfit_grid3</code>	<code>pfit_process3</code>	<code>pfit_eval3</code>
4	<code>pfit_grid4</code>	<code>pfit_process4</code>	<code>pfit_eval4</code>
5	<code>pfit_grid5</code>	<code>pfit_process5</code>	<code>pfit_eval5</code>
6	<code>pfit_grid6</code>	<code>pfit_process6</code>	<code>pfit_eval6</code>

Floating point API:

preprocessing functions:

```
void fngridf(void * pScr, float32_t * V, float32_t * R, const float32_t *x, int M) ;
```

Scratch allocation:

```
size_t pfitf_grid_getScratchSize( int M, int N )
```

calculate coefficients:

```
void fnprocessf ( void * pScr, float32_t * p, const float32_t * V, const float32_t
                 * R, const float32_t * y, int M, int maxIter) ;
```

Scratch allocation:

```
size_t pfitf_process_getScratchSize( int M, int N )
```

interpolation using given coefficients:

```
void fnevalf (void* pScr, float32_t * yi,const float32_t *xi, const float32_t *
p,int P) ;
```

Polynomial degree, N	Preprocessing	Calculate coefficients	Interpolation
1	Pfitf_grid1	pfitf_process1	pfitf_eval1
2	pfitf_grid2	pfitf_process2	pfitf_eval2
3	Pfitf_grid3	pfitf_process3	pfitf_eval3
4	Pfitf_grid4	pfitf_process4	pfitf_eval4
5	Pfitf_grid5	pfitf_process5	pfitf_eval5
6	pfitf_grid6	pfitf_process6	pfitf_eval6

Arguments	Type	Name	Size	Description
Input:				
int	N			Degree of polynomial, 1..6
int	M			number of grid points
int	P			Number of query points
int	maxIter			Number of least squares solution enhancement iterations. Right choice depends on required accuracy, the ad-hoc value is $(N+1)/2$
int16_t, float32_t	x	M		grid points, Q15 or floating point
int16_t, float32_t	y	M		data values in the grid points, Q15 or floating point
int16_t, float32_t	xi	P		interpolating points, Q15 or floating point
		M'		(M+7) & ~7 for floating point API (M+15) & ~15 for fixed point API
		N'		8 for floating point API 16 for fixed point API
Intermediate				
int16_t, float32_t	V	M' * 8		Vandermonde matrix, Q15 or floating point
int16_t, float32_t	R	N' * 8		upper triangle Vandermonde matrix, Q11 or floating point
Output:				
int16_t, float32_t	yi	M		interpolated values in xi, Q15 or floating point
int32_t, float32_t	p	N+1		polynomial coefficients p, Q8.23 or floating point
Temporary:				
void*	pScr			Scratch memory area, required size (in bytes) is defined by functions pfit_XXX_getScratchSize(), see above.
Restrictions	N should be in range 2...6 M should be not less than N+1. pScr should be aligned on a 32-bytes boundary V, R should be aligned on a 32-bytes boundary			
Returned Value	none			

2.10.2 Interpolation

Generic interpolation may be done by 2 methods: neighbor and linear.

Given an array $y[M]$ of the target function values ($y[2*M]$ if complex-valued) sampled at M control points (grid), find values of the target function $y_i[N]$ ($y_i[2*N]$) at points $x_i[N]$. Desired points $x[N]$ must be located inside the grid points: $x[0] \leq x_i[0..N-1] \leq x[M-1]$.

There are two options for the interpolation method:

- for each of x_i , locate the nearest control point (a neighbor) and return the corresponding value of the target function

- perform the linear interpolation between the two closest control points

Neighbour locating and linear interpolation methods are implemented for both the real- and complex-valued functions in a real argument.

All interpolation methods strictly require ascending ordering of input and output grid points. The best accuracy is achieved if grid points are equally spaced.

Description	These functions interpolate the data given on grid to another points.			
Representation	16-bit input, 16-bit output			
Algorithm	see description above			
Prototypes	Neighbour method, real-valued target function: <pre>void vinterp_neib(void * scr, int16_t * yi, const int16_t * xi, const int16_t * y, const int16_t * x, int M, int N)</pre> Neighbour method, complex-valued target function: <pre>void vinterp_cneib(void * scr, int16_t * yi, const int16_t * xi, const int16_t * y, const int16_t * x, int M, int N)</pre> Linear fit method, real-valued target function: <pre>void vinterp_lin(void * scr, int16_t * yi, const int16_t * xi, const int16_t * y, const int16_t * x, int M, int N)</pre> Linear fit method, complex-valued target function: <pre>void vinterp_clin(void * scr, int16_t * yi, const int16_t * xi, const int16_t * y, const int16_t * x, int M, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int	M		size of input grid
	int	N		number of output points
	int16_t	x		grid points (should be sorted in the ascending order)
	int16_t	xi	N	output points (should be sorted in the ascending order)
	int16_t	y	M*C	data values in the grid points; c is 1 for real data, or 2 for complex data. For complex data, real and imaginary components are interleaved, with the real part going first (at even indices).
	Output:			
	int16_t	yi	N*C	interpolated data values in xi; c is 1 for real data, or 2 for complex data.
	Temporary:			
	void*	scr		Scratch memory area; required size (in bytes) is defined by functions: vinterp_neib_getScratchSize(), vinterp_cneib_getScratchSize(), vinterp_lin_getScratchSize(), vinterp_clin_getScratchSize() respectively
Restrictions	All arrays should not overlap Input grid points in x and xi should be sorted in the ascending order Distance between two successive grid points x must be less than 16384 (0.5 in Q15) and greater than 0 Output points (xi) should be inside [x[0], x[M-1]] Scratch pScr, x[M], xi[N], yi[C*N] - must be aligned on a 32-bytes boundary For complex-valued variants y, yi must be aligned on a 4-bytes boundary M should be >=1 N should be a multiple of 16			

Returned value	none
----------------	------

2.10.3 Tap-based Interpolation

These two routines allow making 2 or 4-point interpolation of all elements from given vectors.

Description	2 and 4-tap interpolators.			
Representation	16-bit input, 16-bit output			
Algorithm	$y_k = t_0x_{1_k} + t_1x_{2_k}$ or $y_k = t_0x_{1_k} + t_1x_{2_k} + t_2x_{3_k} + t_3x_{4_k}, \; k = \overline{0...N-1}$			
Prototypes	2-tap interpolation: <pre>void interp_2tap(int16_t * y, const int16_t * x1, const int16_t * x2, const int16_t * taps, int N)</pre> 4-tap interpolation: <pre>void interp_4tap(int16_t * y, const int16_t * x1, const int16_t * x2, const int16_t * x3, const int16_t * x4, const int16_t * taps, int N)</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	int	N		size of complex vectors
	int16_t	x1, x2, x3, x4	N*2	input complex vectors of length N, Q15
	int16_t	taps	2 or 4	taps,Q15
	Output:			
	int16_t	y	N*2	output complex vector of length N,Q15
Restrictions	All arrays should not overlap x1, x2, x3, x4, y should be aligned on 32-bytes boundary N should be a multiple of 8			
Returned value	none			

2.10.4 Find Roots of a Polynomial

Description	For given input array $c[N+1]$ containing $N+1$ complex/real coefficients of a degree N polynomial, find its N roots (possibly with repetitions) and store results to the output array $r[N]$.
Algorithm	Roots of a degree N polynomial are computed as eigenvalues of $N \times N$ companion matrix associated with the polynomial: $r = eig \left(\begin{bmatrix} 0 & 0 & \dots & 0 & \frac{c_N}{c_0} \\ 1 & 0 & \dots & 0 & \frac{c_{N-1}}{c_0} \\ 0 & 1 & \dots & \dots & \dots \\ \dots & \dots & & 0 & \dots \\ 0 & \dots & 0 & 1 & \frac{c_1}{c_0} \end{bmatrix} \right)$

Prototypes	<pre>void prootf (void * pScr, complex_float * r, const complex_float * c, int N); void rprootf(void * pScr, complex_float * r, const float32_t * c, int N);</pre>			
Arguments	Type	Name	Size	Description
	Input:			
	complex_float, float32_t	c	N+1	polynomial coefficients in descending powers order
	int	N		polynomial degree
	Output:			
	complex_float	r	N	
	Temporary:			
	void*	pScr		scratch size in bytes, returned by prootf_getScratchSize(), rprootf_getScratchSize() respectively
Restrictions	<code>r</code>, <code>c</code>, <code>pScr</code> should not overlap and should be aligned on 32-bytes boundary <code>N</code> must be greater than zero <code>c[0]</code> the leading coefficient must be non-zero			
Returned value	none			

2.11 Identification Routines

2.11.1 Library Version Request

Description	This function returns library version information.			
Prototype	void NatureDSP_Baseband_get_library_version(char *version_string);			
Arguments	Type	Name	Size	Description
	Output:			
	char	version_string	>=30	Buffer to store version information
Returned Value	None			
Restrictions	version_string must point to a buffer large enough to hold up to 30 characters			

2.11.2 Library API Version Request

Description	This function returns library API version information.			
Prototype	void NatureDSP_Baseband_get_library_api_version(char *version_string);			
Arguments	Type	Name	Size	Description
	Output:			
	char	version_string	>=30	Buffer to store version information
Returned Value	None			
Restrictions	version_string must point to a buffer large enough to hold up to 30 characters			

2.11.3 Library API Capability Request

Description	This function returns non-zero if given function (by its address) is supported by specific processor capabilities.			
--------------------	--	--	--	--

NOTE:

in the gcc/xcc environment, calls of this function are not necessary - if function pointer is non-zero it means it is supported. VisualStudio linker does not support section removal so this function might be used for running library under MSVC environment

Very few library functions may disable their capabilities dynamically (only for particular combination of input parameters). Behavior for such situation is defined in the description of those functions.

Prototype

```
int NatureDSP_Baseband_isPresent(NatureDSP_Baseband_funptr fun)
```

Arguments

Type	Name	Description
Input:		
NatureDSP_Baseband_funptr	fun	one of library functions

Returned Value

non-zero if function is supported by library

3 Test Environment and Examples

3.1 Supported Use Environment, Configurations, and Targets

This Library supports any BBE32EP core configuration.

3.2 Building the Library under Xplorer

3.2.1 Importing the workspaces in Xtensa Xplorer

NatureDSP Library is provided as two workspaces:

- Lib workspace
This contains the library project bbe32ep_lib. This contains all the bbe32ep library optimized and reference C routines, built as a single library.
- Demo workspace
This contains the demo testbench project bbe32ep_demoperf.

Import the xws in XX as “Xtensa Xplorer workspace”.

Make sure that the library workspace is imported first. This is because the project in the demo workspace has a dependency on the library project, and the dependency is not correctly set if the library project is not present when the demo workspace is imported

3.2.2 Building and running tests

To build the library: In XX, select the bbe32ep_lib project, and Debug or Release target, and build.

To build the test-bench: In XX, select the bbe32ep_demoperf project, select Debug or Release target, and build.

To run the test-bench, select bbe32ep_demoperf project, and Run.

This will execute a cycles performance test for each routine in the bbe32ep library.

3.3 Other platforms supported for building the NatureDSP Baseband Library and the Testdriver

Library and testdriver project can also be built under the following platforms:

OS	Language	Environment	Tool
Linux	C	xcc	make
Linux	C++	Gcc	make
Windows	C	xcc	xt-make
Windows	C++	MSVC(2013)	Visual Studio

4 Customer Support

If you have questions, want to report problems or suggestions regarding the **NatureDSP Baseband** library or want to port this library to another platforms, contact **IntegrIT** Ltd. at support@integrity.com. Visit www.integrity.com to get more information about products and services.